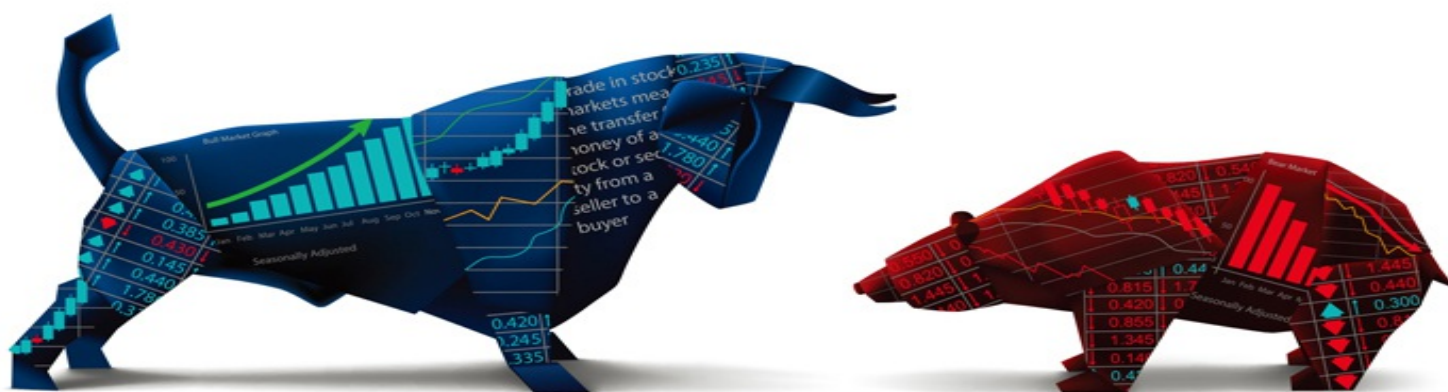




HZ BOOKS
华章IT

解密 金融数据



Unlocking Financial Data

A Practical Guide to Technology for Equity and Fixed Income Analysts

Justin Pauley 著
朱轩彤 董宁 译

 机械工业出版社
China Machine Press

目录

[O'Reilly Media, Inc.介绍](#)

[业界评论](#)

[本书赞誉](#)

[译者序](#)

[前言](#)

[第1章 简介](#)

[1.1 概览](#)

[1.1.1 第一部分：获取金融数据](#)

[1.1.2 第二部分：金融数据分析](#)

[1.1.3 第三部分：创建金融报告](#)

[1.2 金融市场](#)

[1.2.1 股票](#)

[1.2.2 企业贷款（银行贷款、杠杆贷款）](#)

[1.2.3 企业债券](#)

[1.3 三种途径](#)

[1.3.1 途径1：Microsoft Excel](#)

[1.3.2 途径2：Microsoft Access](#)

[1.3.3 途径3：C#](#)

[1.4 线上文件](#)

[1.5 本章小结](#)

[第2章 组织金融数据](#)

[2.1 途径1：Excel](#)

[2.1.1 Excel区域与Excel表](#)

[2.1.2 增加引用列](#)

[2.1.3 数据验证](#)

[2.2 途径2和3：Access中的表格](#)

[2.3 本章小结](#)

[第3章 彭博](#)

[3.1 确定字段](#)

[3.1.1 鼠标滑动](#)

[3.1.2 FLDS屏幕](#)

[3.1.3 彭博函数构造器和在Excel中发现字段](#)

[3.1.4 如果其他都失败了.....](#)

[3.2 Excel案例](#)

[3.2.1 提取单一字段（BDP）](#)

[3.2.2 提取批量数据（BDS）](#)

[3.2.3 提取历史数据 \(BDH\)](#)

[3.3 用于比较的有价证券](#)

[3.3.1 指数](#)

[3.3.2 对等有价证券](#)

[3.3.3 关于有价证券](#)

[3.4 途径1和2：Excel和Access](#)

[3.4.1 企业债券、贷款和指数](#)

[3.4.2 公司工作表](#)

[3.4.3 引用和重写](#)

[3.5 途径3：彭博C#API](#)

[3.5.1 设置Microsoft Access以用于C#](#)

[3.5.2 彭博C#API](#)

[3.5.3 基本引用示例](#)

[3.5.4 基本历史示例](#)

[3.5.5 填入Access数据库](#)

[3.6 本章小结](#)

[第4章 IHS Markit：大企业数据](#)

[4.1 企业贷款](#)

[4.1.1 数据请求](#)

[4.1.2 贷款信息](#)

[4.1.3 贷款定价、金融和分析](#)

[4.2 企业和主权债券](#)

[4.3 途径1：在Excel中存储Markit信息](#)

[4.4 途径2：将Markit数据导入Microsoft Access](#)

[4.5 途径3：用C#导入Markit数据](#)

[4.6 本章小结](#)

[第5章 金融数据分析](#)

[5.1 数据真实性](#)

[5.1.1 检查数据](#)

[5.1.2 样本规模](#)

[5.1.3 异常值](#)

[5.2 投资组合](#)

[5.2.1 投资组合工作表](#)

[5.2.2 投资组合数据库表](#)

[5.3 连接Excel工作表和Microsoft Access](#)

[5.4 留存记录](#)

[5.4.1 途径1：Excel](#)

[5.4.2 途径2：Microsoft Access](#)

[5.4.3 途径3：C#](#)

[5.5 本章小结](#)

[第6章 相对价值分析](#)

[6.1 途径1：Excel](#)

[6.1.1 Excel中的相关性与回归](#)

[6.1.2 对等组](#)

[6.1.3 评级](#)

[6.1.4 统计工作表](#)

[6.1.5 并排比较](#)

[6.1.6 指数](#)

[6.1.7 加权z值](#)

[6.2 途径2：Access](#)

[6.2.1 Access中的相关性和回归](#)

[6.2.2 Access中的中位数](#)

[6.3 途径3：C#](#)

[6.3.1 相关性和回归](#)

[6.3.2 对等组](#)

[6.3.3 评级](#)

[6.3.4 统计表](#)

[6.3.5 并排比较](#)

[6.3.6 加权z值](#)

[6.4 本章小结](#)

[第7章 组合风险分析](#)

[7.1 途径1：Excel](#)

[7.1.1 方差、波动性和标准差](#)

[7.1.2 带历史或预期收益的夏普比率](#)

[7.1.3 投资组合分解](#)

[7.1.4 警告标志](#)

[7.2 途径2：Access](#)

[7.2.1 投资组合分解](#)

[7.2.2 警告标志](#)

[7.3 途径3：C#](#)

[7.3.1 历史或预测收益的夏普比率](#)

[7.3.2 投资组合分解和警告标志](#)

[7.4 本章小结](#)

[第8章 市场分析](#)

[8.1 途径1：Excel](#)

[8.1.1 新发放贷款分析](#)

[8.1.2 再融资](#)

[8.1.3 价格历史](#)

[8.2 途径2和3：Access和C#](#)

[8.2.1 新发放贷款分析](#)

[8.2.2 再融资](#)

[8.2.3 价格历史](#)

[8.2.4 更进一步](#)

[8.3 本章小结](#)

[第9章 创建报告](#)

[9.1 途径1：Excel](#)

[9.2 途径2：Microsoft Access](#)

[9.3 途径3：C#和SSRS](#)

[9.4 本章小结](#)

[第10章 投资组合报告](#)

[10.1 监测业绩和风险](#)

[10.2 途径1：Microsoft Excel](#)

[10.2.1 计算收益](#)

[10.2.2 投资组合报告](#)

[10.3 途径3：C#和SSRS](#)

[10.4 本章小结](#)

[第11章 结论](#)

[附录A 表格参考](#)

O' Reilly Media , Inc.介绍

O' Reilly Media通过图书、杂志、在线服务、调查研究和会议等方式传播创新知识。自1978年开始，O' Reilly一直都是前沿发展的见证者和推动者。超级极客们正在开创着未来，而我们关注真正重要的技术趋势——通过放大那些“细微的信号”来刺激社会对新科技的应用。作为技术社区中活跃的参与者，O' Reilly的发展充满了对创新的倡导、创造和发扬光大。

O' Reilly为软件开发人员带来革命性的“动物书”；创建第一个商业网站（GNN）；组织了影响深远的开放源代码峰会，以至于开源软件运动以此命名；创立了Make杂志，从而成为DIY革命的主要先锋；公司一如既往地通过多种形式缔结信息与人的纽带。O' Reilly的会议和峰会集聚了众多超级极客和高瞻远瞩的商业领袖，共同描绘出开创新产业的革命性思想。作为技术人士获取信息的选择，O' Reilly现在还将先锋专家的知识传递给普通的计算机用户。无论是通过书籍出版，在线服务或者面授课程，每一项O' Reilly的产品都反映了公司不可动摇的理念——信息是激发创新的力量。

业界评论

“O' Reilly Radar博客有口皆碑。”

——Wired

“O' Reilly凭借一系列（真希望当初我也想到了）非凡想法建立了数百万美元的业务。”

——Business 2.0

“O’ Reilly Conference是聚集关键思想领袖的绝对典范。”

——CRN

“一本O’ Reilly的书就代表一个有用、有前途、需要学习的主题。”

——Irish Times

“Tim是位特立独行的商人，他不光放眼于最长远、最广阔的视野并且切实地按照Yogi Berra的建议去做了：‘如果你在路上遇到岔路口，走小路（岔路）。’回顾过去Tim似乎每一次都选择了小路，而且有几次都是一闪即逝的机会，尽管大路也不错。”

——Linux Journal

本书赞誉

只有最佳运用信息的金融分析师才能超越同行。在《解密金融数据》一书中，本书作者手把手教你如何获得这种宝贵的优势。

——Thomas Majewski , Eagle Point Credit Management公司合伙人

《解密金融数据》是在金融领域进行海量信息分析的有效工具，为想运用当今技术加强决策并在市场中占据优势地位的专业人士提供了无价指导。本书作者是结构化金融领域的领军人物。他的洞见来源于他利用当今科技分析经验数据的能力。

——David Trepanier , CFA , 美国某著名金融机构的CLO/结构化产品总监

真正有天赋的金融分析师是很少的，只有很小一部分才懂得利用科技。贾斯汀·保利是在两方面都非常出色的少数人之一。他在有效获取和管理数据方面的能力超群，还能利用这些数据进行清晰简明的分析。

——David Preston , CFA , Wells Fargo Securities公司董事总经理、CLO和商业ABS研究总监

在进行金融数据分析时，技术工具的重要性不言而喻。贾斯汀·保利是结构化产品市场的领军人物，很大程度上是因为他有深厚的技术功底。在《解密金融数据》一书中，作者讲述的获取金融数据的方法是其他地方都没有讲过的，因此很有价值。本书几乎能使任何分析师受益，不管你有没有技术基础。

——Dylan Ross , Brigade Capital Management公司合伙人

译者序

21世纪，大数据挖掘和处理已经成为时代的潮流。信息属性强大的金融数据能够为金融分析预测提供强有力的支持。然而由于数据挖掘处理涉及很多计算机应用技术，很多金融行业从业者或者相关人士已经感觉到自己跟不上数据时代的脚步，特别是非计算机技术专业的人士对挖掘、处理大数据感到十分挠头。甚至在一些金融企业中，购买的金融数据库可能只在后台支持部门使用，前台业务部门的人由于不知道如何挖掘处理数据而放弃了使用。因此，大家迫切需要简单实用的辅导材料，帮助他们走出困境。

本书就是一本内容系统、浅显易懂、关注实操的书。非计算机专业的人士即使只有比较初步的Microsoft Excel或Access知识，在阅读学习本书后也能很容易地掌握在彭博和IHS Markit等金融数据系统中搜索有用信息并进行分析汇总的方法。针对有计算机技术基础的人，本书还提供了C#编码，以此满足更多读者的需求。

本书具有以下特点：

1. 全书讲解方式简单直接，内容和语言没有晦涩难懂的感觉，读者完全不用担

心自己的计算机技术基础，非计算机技术专业的人士阅读学习没有压力。

2. 实际操作性极强，方便自学，从如何导入数据到如何编码，本书都给出了具体的操作步骤和实例，可以直接拿来使用。

3. 内容全面丰富，涉及数据搜索、分析到编制相关报告，深度和广度兼顾，是一本“工具红宝书”。

从金融科技的角度看，科技将服务金融业务、引领金融创新、驱动金融变革。但是归根结底，科技是要为人服务的，包括从事金融业的人，不能用烦琐的科技束缚了金融从业者或相关人士的手脚。我们的目标是为你提供一本简单易学的金融数据挖掘处理“工具红宝书”，希望这本书能够帮助你从金融数据“小白”成长为行家里手！

译者

前言

如果你在书店打开了这本书，正在犹豫这本书是否适合你，那么让我来帮你分析：

· 本书能帮助你将来自于彭博和IHS Markit的金融数据与自己的观点结合起来进行分析，并用Microsoft Excel制作专业的分析报告，而不需要进行编程或者IT部门的协助。

· 本书会一步一步地指导你快速制作出专业化的市场信息报告，其中包括历史财务信息、比较分析和相对价值。

· 对于资产组合经理来说，本书展示了如何制作一份专业的资产组合总结报告，其中对资产组合的业绩、成长性、风险调整后收益和组成等进行了高度概括。

· 如果你是程序员，或者可能成为程序员，本书还介绍了如何使用C#实现相同的功能。

· 我并非学术人员，所以关注“实战”。你需要的基础知识仅仅是金融知识和

Excel知识。你不需要有任何编程、VBA或高等数学基础。

我一直喜欢计算机科学，所以不奇怪我的第一份工作是在Wachovia（现为Wells Fargo）公司做程序员。不过在我的职业生涯早期，我就意识到了有些重要的东西会指引我走向完全不同的职业道路。我认识到我运用科技获取和分析数据的能力在IT部门以外的其他领域（例如金融领域）能发挥更大的作用。

做了三年程序员之后，我转入银行研究部门，用我的计算机技术挖掘财富市场上的海量信息，并与内部消息相结合。这些技术使我比其他分析师有更大的优势，我成为苏格兰皇家银行的策略总监，之后我又成为位于纽约的Brigade Capital Management公司的高级结构化信贷分析师。在Brigade公司，我用计算机技术对投资进行估值，并体现了很大优势。

在我的职业生涯中，我不时地会教许多同事一些有用的计算机技术。在教他们的过程中，我了解到即使你不懂编程，也能够深入彭博屏幕的背后做很多分析工作。你需要的基础知识不过是金融行业雇佣的大多数人已经掌握的知识、分析思维和使用Excel的经验（当然，还需要本书）。本书还照顾到程序员的需求，本书没有费时费力地讲解API文档，而是用许多有用的C#案例展示如何从彭博和IHS Markit获取市场数据，用Math.Net开源库进行金融分析，用SQL Server Reporting Services（SSRS）自动生成专业化的报告。

我写本书的目的就是教更多人使用这些技术，帮助人们使自己的简历在厚厚的简历堆中脱颖而出。

本书排版约定

 本书中的不少代码都被拆成几行写，这是由于印刷页面的限制。如果你在系统中运行代码就无须分行。

斜体字（Italic）

表示新的术语、链接、电子邮箱地址、文件名和文件扩展名。

等宽字体（Constant width）

用于程序清单，也用于在段落中引用程序元素，例如变量名、函数名、数据库、数据类型、环境变量、程序语句和关键词。

加粗等宽字体 (Constant width bold)

表示应该由用户输入的命令或者其他文字信息。

斜体的等宽字体 (Constant width italic)

表示此处应该替换为由用户提供的数值，或者根据上下文确定的数值。

 这个图标表示一个通用的注释。

 这个图标表示警告。

使用代码示例

可以在http://bit.ly/unlockFD_examples上下载辅助资料（代码示例、练习等）

本书的目标是切实好用，帮助你完成工作。原则上，如果本书提供了代码，你就可以在你的程序和文档中使用。除非你要大范围复制代码，否则不用与我们联系要求获得许可。例如，使用本书中的几大段代码不需要许可。售卖O’ Reilly出版社的书的案例光盘需要许可。回答问题时，引用本书或本书中的案例不需要许可。在你的产品文档中大量使用本书中的示例代码需要许可。

我们赞赏但不要求注明引用。引用通常包括书名、作者、出版商和ISBN。例如 “Unlocking Financial Data by Justin Pauley (O’ Reilly) .Copyright 2018Justin Pauley , 978-1-491-97325-7” 。

如果你觉得自己使用代码示例的范围超出公平使用或上述许可的范围，请联系 permissions@oreilly.com。

Safari在线图书

Safari Books Online针对企业、政府、教育机构和个人提供了不同的购买计划，你可根据实际需求进行选购。

用户可以访问上千种图书、培训视频、学习路径、互动教材和专业的播放列表，这些内容来自超过250个出版商，包括O'Reilly Media、哈佛商业评论、Prentice Hall Professional、Addison-Wesley Professional、Microsoft Press、Sams、Que、Peachpit Press、Adobe、Focal Press、Cisco Press、John Wiley & Sons、Syngress、Morgan Kaufmann、IBM Redbooks、Packt、Adobe Press、FT Press、Apress、Manning、New Riders、McGraw-Hill、Jones & Bartlett和Course Technology等。关于Safari在线图书的更多信息，请访问<http://oreilly.com/safari>。

联系方式

美国：

O'Reilly Media, Inc.

1005 Gravenstein Highway North

Sebastopol, CA 95472

中国：

北京市西城区西直门南大街2号成铭大厦C座807室（100035）

奥莱利技术咨询（北京）有限公司

我们有个关于本书的网页，上面有勘误表，示例和所有的附加信息。可以通过以下链接访问：http://bit.ly/unlockingFinancialData_1e。

关于本书的评论和技术问题，请发邮件给bookquestions@oreilly.com。

关于本书的更多信息，如教程、会议、新闻，请参见网站：

<http://www.oreilly.com>

致谢

本书献给我的妻子Emily Pauley。特别感谢对本书贡献了宝贵时间和支持的人；没有你们我无法完成本书（排名不分先后）：Alex Belgrade、Scott Moore、Jeana Curro、Dan Bleicher、Matt Perkal、Sumit Sablok和Tom O' Shea。

第1章 简介

金融分析师的工作简单说就是找出有价值证券的价值，有价值证券包括股票、贷款、债券，甚至房地产贷款的证券化投资组合等。换句话说，金融分析师需要确定他买卖一种有价值证券的价格。确定价格没有统一的方法、模型或者公式，因为每笔投资都是不同的。然而，估值过程通常包括以下两个重要步骤：

- 分析师需要详细、彻底地了解每笔潜在投资，分辨出各种投资方式的细微差别，以及赚钱和赔钱的概率。

- 风险调整后的收益结果与其他的潜在投资进行比较，以确定给定投资的估值到底是高了、低了，还是合理。

不是简单地对每个有价值证券进行一次估值；投资必须进行持续估值。在经济高度全球化的今天，投资风险持续变化，大宗商品价格变动、政府监管措施、自然灾害、消费者敏感程度以及其他数不清的因素都可能对投资风险产生影响。同理，即使有些因素并没有直接影响一个有价值证券，但是它们可能使一项投资与其他投资相比看上去更便宜或者更昂贵。

对有价值证券进行估值的过程是复杂的，耗费大量时间，还需要分析大量金融数据。大部分金融数据是由发行有价值证券的公司或者相关银行提供的，因此并不总是能提供完整、准确的情况。更重要的是，大部分公司都有海量金融数据，这些数据虽然能帮助估值，但是技术部门以外的人不知道如何获取数据，甚至不知道数据的存在。

本书将带你超越彭博终端，解密彭博数据的真实潜力。本书仅仅使用Excel（而不用任何编程技术）展示如何获取彭博屏幕背后的数据，按照你的意愿组织数据，并服务于你对投资的决策考虑。本书还介绍如何从IHS Markit获取金融信息，IHS Markit是企业债券和贷款数据（特别是定价）的最佳数据源之一。许多银行、资管经理、对冲基金订购了Markit数据，但是大部分数据都用于支撑内部或后台系统，却很少到达分析师手中。

除了展示如何获取金融信息外，本书第二部分介绍如何进行数据分析。该部分教你利用金融数据，通过分析贷款、保险和再融资的价格变动趋势，确定相对价值，测量投资组合风险，计算有价值证券之间的相关性并研判市场趋势。

最后，我们将把数据、分析与你的观点结合起来，根据个人意愿创建定制报告。与千篇一律的报告不同，你的报告会将一个公司的业绩与你自己设定的对等组、你认为有价值的对应基准指数或者你认为有价值的金融字段和计算进行比较。

尽管这些内容（数据获取、分析和报告）都可以用简单的方法自动实现、维护和更新，不需要进行编程，不过本书还是为程序员或者潜在程序员提供了使用C#编程语言的方法。数据和信息在所有的行业都是宝贵的，在金融行业尤甚，程序开发人员写代码、查询数据库的能力可以使他比分析师更有价值。本书中有很多案例，告诉你如何获取金融信息，对大型数据集进行不同种类的简单金融分析，并使用SSRS（不用SQL Server）创建金融报告。如果你对介于Excel和C#之间的方法感兴趣，我们还介绍了如何使用Microsoft Access数据库。

本章具体介绍本书的各个部分，特别是理念和目标。并根据投资类型（股票、债券和贷款），分别介绍如何使用本书。

1.1 概览

本书分为三大部分：获取金融数据、金融数据分析、创建金融报告。每一部分都有明确且实用的目标。

1.1.1 第一部分：获取金融数据

第一部分（从第2章到第4章）介绍如何用彭博（Bloomberg）和Markit获取、

存储金融数据，如股票、指数、债券和银行贷款等。从彭博和Markit系统中获取这些信息，你将可以：

- 获取彭博屏幕上看不到的额外信息。
- 创建包含企业数据的表，以便一起比较多个公司、债券或贷款。
- 在大部分可交易企业债券和贷款市场中查询每日价格和机构信息。
- 根据个人偏好，仅显示重要的内容。
- 使用其他来源，重写错误或缺失信息。
- 根据个人观点，对公司进行分类，建立合适的对等组（peer groups）。
- 把彭博和Markit数据与你的计算和观点相结合。

学完该部分后，你能灵活分解和运用金融数据，灵活性远超使用彭博终端。此外，第3章的技巧能保证你用易于维护的方式存储数据，并与其他数据集相结合。

1.1.2 第二部分：金融数据分析

第二部分（从第5章到第8章）将第一部分收集到的金融数据放入具体的背景环境中进行金融分析。第二部分先将某个有价值证券与对等有价值证券进行比较，然后检测投资组合的风险水平，最后与更广阔的市场趋势相结合。使用该部分介绍的技术，你将可以：

- 用相关性和回归确定两个有价值证券（或指数）之间的关系。
- 将各个有价值证券的业绩与有类似风险和收益性质的有价值证券对等组进行比较。
- 用加权Z值（Z-score）对有价值证券的相对业绩进行排序。

·通过计算方差、标准差和夏普比率（Sharpe ratio），测量投资组合风险调整后的收益。

- 把投资组合“划分”成几个组，以关注看不见的内容和趋势。

- 用不同的标准建立投资组合门槛，以关注风险。

- 用Markit数据发现对价格、新发放息差和再融资有意义的趋势。

学完该部分后，你能把第一部分介绍的数据库加以应用，进行观点、风险和趋势分析。此外，第6章中描述的方法将告诉你如何保存历史，发现用第一部分的知识获取到的数据库存在的特殊问题。

1.1.3 第三部分：创建金融报告

第三部分（第9章和第10章）结合了第一部分和第二部分的内容，教你如何对各个公司和投资组合创建分析报告。在该部分中，我们将学习把前面的章节中用到的数据和分析，按照你的意愿进行展示并绘制图表。该部分将展示如何：

- 为每个公司创建一份两页长的分析报告（活页，tear sheet），其中内容包括重要的历史金融数据、自定义备注、与对等公司的价值比较和分析师研究出来的价格趋势等。

- 用一个投资组合或一个指数的历史收益，计算时间加权（几何）投资组合收益、年化投资组合收益、年化投资组合标准差和夏普比率。

- 创建一份两页长的投资组合摘要报告，其中包括高度概括的投资组合业绩、增长、风险调整后的收益和投资产品组成。

学完该部分后，你就能从下拉菜单选择一个公司名，然后马上就能创建一个定制专业公司报告，而不用进行编程。此外，你还能针对投资组合的业绩绘制图表，并将其业绩与基准进行风险和收益比较。

1.2 金融市场

本书关注三个重要金融市场：股票、企业债券和企业贷款。尽管如此，你可以将在本书中学到的内容直接应用到其他市场，比如结构化产品、市政债券等。

1.2.1 股票

本书中的很大篇幅都关注股票市场。本书第一部分展示如何用彭博数据创建一个有基本数据和技术数据（转换成使用单一货币）的表。因为是在Excel里，所以你可以很容易地用自己的输入内容对彭博数据进行补充。例如，通过增加自己的类型列，你可以对相似的公司进行正确的分组，而非依靠传统的行业分类信息（有可能是误导信息）。此外，彭博有海量金融企业数据。本书教你如何找到想寻找的确切信息，而不用对着每个公司数十屏幕的资料狂翻一气。下面仅是部分字段：

- 领域、行业、子行业。
- 流通股中短期利率所占的比重。
- 标准普尔和穆迪评级。
- 市值、企业价值、EPS。
- 总债务、净债务、总债务/EBITDA。
- 股息收益率、十二个月总收益。
- 三个月和今年迄今价格变动（%）。
- 利息备付、自由现金流（FCF），自由现金流/总债务。
- 信用违约交换息差。
- 买入/卖出/持有建议。
- 毛利、滚动市盈率EBITDA。
- 历史金融数据。

本书第二部分教你如何计算两个公司之间或者一个公司与一个对标指数（基准指数）之间的关系（相关性和Beta）。还介绍如何对Excel公司表中每个自定义的分类添加业绩数据中位数。你可以使用业绩数据中位数，将每个公司的业绩与其对等组进行比较。

本书第三部分教你如何设计公司报告，而不是依靠第三方提供的报告。定制报

告的优势就是可以使用自己想用的字段、备注和计算。

1.2.2 企业贷款（银行贷款、杠杆贷款）

与股票一样，本书第一部分从彭博提取一系列有用的字段，存入Excel（或Access），包括：

- 利差、下限、指数。
- 到期日。
- 穆迪和标准普尔机构评级。
- 今年迄今和三个月价格变化。
- 贴现差额、收益率。
- 下一个提前赎回日、赎回价格。

然而和股票不一样的是，本书第一部分展示如何从IHS Markit获取贷款数据，其中的贷款信息可以说是丰富的。此外，Markit数据涵盖了大部分可交易的贷款，而不是少量数据。每笔贷款的每日贷款价格，结合每笔贷款的机构信息，使我们能够了解整体贷款趋势，例如某个行业的贷款趋势。如下是一些Markit贷款机构数据：

- 发行人、行业、机构类型。
- 发起人、牵头行、管理行。
- 规模、利差、下限、OID。
- CUSIP。
- 留置权类型（第一留置权/第二留置权等）。
- 不包含借方保护条款的高风险贷款标示（Cov-Lite Flag）。

- 穆迪和标准普尔机构评级。

- 发行日期、交割日期、到期日期。

在第二部分，贷款分类取决于母公司的分类以及风险和收益性质，例如“Short CCC”代表评级为CCC的短期贷款。根据对等组的中位数对贷款进行排序。除了机构层面的信息和价格，Markit提供本书第二部分用到的再融资信息判断趋势，例如分行业的息差收窄或扩大。

1.2.3 企业债券

和股票一样的是，在彭博中有大量企业债券数据可以将其抽取出来放入Excel（或者Access）表。一些字段包括：

- 票面利息。

- 到期日。

- 穆迪和标准普尔机构评级。

- YAS利差和YAS收益率（参见彭博YAS屏幕）。

- 今年迄今和三个月价格变化。

- 可赎回标示、下一个提前赎回日、下一次赎回价格。

和贷款一样，债券也归到第二部分。每个分类的中位数值结果之后都用作该分类每个债券的基准值。

1.3 三种途径

本书的理念有三种不同的实现方法（途径），从技术难度角度说，从易到难的排序是：Excel、Microsoft Access和C#。由于各章的理念相互依存，所以要做好实践。本书会用“途径”专门指明某种方法，便于你找到对应方法。

由于大多数人熟悉Excel，因此本书大部分章节使用Excel作为主要数据处理手

段。Microsoft Access途径是建立在Excel途径基础之上的，要先把数据库连接到Excel工作簿，然后在Access数据库中查询数据，这可避免使用复杂的Excel公式。C#途径用代码从第三方系统（彭博和Markit）提取数据，在Microsoft Access中存储数据并分析，使用SSRS生成报告。

每章都会采用三种途径，有两个原因。第一个原因是本书的目标读者不同。使用Excel很多年的金融分析师可能从来没使用过数据库，或者没写过一行代码。而对于程序员来说，有很多获取、操作和存储数据的方法比Excel更直接。第二个原因是鼓励从没有使用过数据库或者没写过代码的人试一试。尽管本书不教授如何查询数据库或者编写程序，你可以结合本书与教授如何查询数据库或者编写程序的书（例如Jennifer Greene和Andrew Stellman写的《Head First C#》（O’ Reilly））一起深度学习。此外，初次涉及编程的人员可以使用Excel实例以便更好地理解C#代码。下面列出了每种途径的优缺点。

1.3.1 途径1：Microsoft Excel

无论你是分析师、程序员、科学家、会计或者其他职业，只要你和数字打交道，都可能用到Excel。Excel是市场上最强大、灵活、受欢迎的分析程序之一。它除了拥有大量实用函数之外，Excel的界面也适合大众使用。然而，它与Access和C#相比还有一些不足：

- 尽管Excel处理几千行数据没问题，但是如果数据量很大，计算效率就不高了。数据的行数或者列数过多可能导致效率下降，甚至宕机。

- Excel公式可能很快就会变得复杂冗长，难以阅读。仅仅进行最基本的运算就可能用到多个内函数和数组函数。

- 尽管Excel公式是动态的，但是很难维持。随便插入一行就可能使某些公式出现错误而难以察觉。

- 在出问题的时候，动态的长公式难以进行调试。

1.3.2 途径2：Microsoft Access

尽管Microsoft Access就是Microsoft的企业版数据库软件SQL Server的简化版，但是几乎所有人都可以不必求助于IT部门就能用上Access。Access是一个很棒的数据库应用，能方便地连接到Excel，导入逗号分隔值（CSV）文件，或者你自己的、长达数百万行的表（最大约2GB）。如果你从来没使用过数据库，也不会觉得Access比Excel更难。和Excel一样，数据存储在类似于Excel工作表的数据库表格里，但是数据库的列更有条理（定义为日期、文本、数字等）。然而，Access用一种很简单的语言分割处理数据，而非使用很长的Excel公式，那种语言被命名为结构化查询语言（Structured Query Language，SQL），既方便使用，又方便读取。你可以用SQL从多个表连接数据，而且可以包括许多与Excel相同的集合函数（平均值、最大值、最小值、标准差等）。查询2017年出版的书籍数量，可用如下简单的查询方式：

```
SELECT COUNT(*) FROM Books WHERE Year(PublicationDate) = 2017
```

在本书中，用途径2的人需要先用Excel从彭博获取数据，然后把Excel工作表连接到Access再进行查询。能够简便查询数据的能力使Access很强大。然而，Access也有缺点：

- 其内置报告函数比Excel弱很多。除了不直观之外，还缺少Excel具备的画图函数。

- Access没有Excel那么灵活。使用Access无法直接在单元格中输入公式，也不能在数据表旁边直接插入图表。

- 查询前一行的值可能比想象得更复杂。

- 查询结果不像Excel那样容易排格式。

- 尽管Access可以容易地连接到Excel表格获取彭博数据，但是它却不能连接到彭博。

- SQL查询尽管可以阅读，但也可能变得冗长复杂。

1.3.3 途径3：C#

C# (读音为 “C sharp”) 是使用Microsoft.NET Framework的一种编程语言。尽管C#函数强大，但也能简单易学。此外，还有Microsoft的Visual Studio集成开发环境 (Integrated Development Environment , IDE) ，这是一种写代码的工具，可以用它创建漂亮的应用。与Access和Excel不同，C#不受表、行、列的限制。它可以从Excel、Access、网站、email、彭博、文本文件或者差不多任何东西里获取数据。C#可以接入无穷多的数学、科学和金融函数。可将数据和计算传送给SSRS，用几行代码就可以产生漂亮的报告，然后转换成PDF或Excel文件。最后，因为C#创建应用，所以可以用它更新数据、创建报告，甚至发送邮件。尽管如此，C#也有其缺点：

- C#需要的前台工作比Excel多。用C#从彭博获取数据需要编写多行代码，而Excel只需要一个简单的公式。

- SSRS是与C#结合使用的报告工具，功能很强大；然而，它没有Excel那么简单灵活。

- 在工作电脑上安装C# (.NET) 可能需要IT部门协助并获得特殊许可。

- 尽管C#是比较简单的编程语言，但是学习来也要经历学习曲线。和其他编程语言一样，C#也有自己的语法和细微差别。

尽管我们鼓励你探索新技术，不过概念比实现更重要；用最适合你、最契合你的实际情况的技术就好了。

1.4 线上文件

你可以从O’ Reilly出版社的网站上下载全套Excel工作表、Access数据库和C#代码，然后根据具体情况修改使用。这样你就不用全部重新开始。然而，你还是需要明白每个步骤的意义，这样才能不犯错误。而且由于版权问题，本书也不能提供所有数据（特别是Markit的数据），因此有些数据并非真实的。

可以从http://bit.ly/unlockFD_examples下载文件。

1.5 本章小结

在最终作出投资决策时，最重要的是分析师的信念。没人能教会你信念，信念来自于多年的经验——挣钱（有时也赔钱）。要知道一些数字是否有意义，或者有些公司的CFO是否确切地回答关于受益的问题，是需要经验的。世界上所有的数据和分析加起来也不能铸成信念，本书的目的是帮你在指端收集更多信息，以支持你铸成信念，发现更多的投资想法。

第2章 组织金融数据

数据！数据！数据！没有黏土做不了砖。

——福尔摩斯

尽管马上就从彭博等来源获取金融数据很有吸引力，先坐下来想一想你打算把数据存在哪里，这样做绝对是“磨刀不误砍柴工”。做个规划或者方案是重要的一步，因为杂乱无章的数据很快就会难以更新，或者容易出错。例如，一个用于追踪学生、教师和课程的Microsoft Excel工作簿或Access数据库。如果每个班的老师都是用姓名作标识，然后老师结婚改了姓，那么与她的名字相关的所有信息都得更新。如果出现一点小状况，就可能出现大问题。如果学校足够大，就可能有两位老师重名，排课和发工资都可能出现问題。

为了解决这个问题，需要把数据存储所谓的“第三范式”。简而言之，我们要为每套数据创建一个数据库或者一张Excel表（例如学生们一张表、老师们一张表、各个课程一张表等）。这些工作表（worksheet）或表（table）中的列只包含分属于各自实体的属性。这称为“有”（has-a）关系。这样，所有信息和属性就不会重复了。例如，一个教师的全名只会出现在Teacher表中，因为教师“有”姓名。Class表包含关于课程（例如生物、数学等）的信息，不应包含教师姓名列，因为这将导致信息重复。

然而，这就产生了如何把教师分派到课程的问题，因为每个课程“有”教师。当你需要一张表引用另一表中的一个实体（教师、学生等），就要使用主关键字（Primary Key）。主关键字是一个永远不会改变的唯一标识符，例如教师证号码（Teacher ID）或者社保号码，它标示出表中某个独一无二的行。因为唯一标识符永远不会改变，我们可以用它在Class表中作为引用（reference），也称为外关键

字 (Foreign Key) , 以引用Teacher表的一个教师。因为Class表用主关键字引用教师, 如果教师改名的话, 只更新Teacher表就行了。尽管这听起来可能有点复杂, 你需要记住不应把同样的数据存储在多个地方。

本章展示如何在Excel和Access中创建这些列表, 并带你学习如何正确地存储和获取信息。

2.1 途径1: Excel

2.1.1 Excel区域与Excel表

Excel是一个很棒、很灵活的工具, 因为可以把数据和公式放在工作表中的任何地方。然而, 如果不增加一些结构, 公式可能会难以阅读, 而且还会产生不易发觉的错误。有人就因为模型中的毛病损失了钱财或丢了工作。避免这些问题的一种方法是将Excel区域 (range) 转换为Excel表 (table) 。

在一张Excel工作表中, 任何相互连接的单元格群组都是一个Excel区域。Excel区域所受的限制很少; 它既可以是一个单元格, 也可以是整张工作表。另一方面, Excel表是在Excel区域上增加了列标题, 并提供命名引用。例如, 如果在Excel工作簿上随便一个位置存放了一个学生表单, 那么数学生个数的公式如下所示:

```
=COUNTA(A1:A14)
```

如果列移动了或者增加了新的行, 可能公式就不能查出正确的行数。然而, 如果把Excel区域转换成Excel表, 命名为 "Student", 公式如下所示:

```
=COUNTA(Student[StudentID])
```

在这种情况下, 你可以完全确信表里包括了所有学生。用下面的步骤填入Excel区域, 并转换为Excel表:

1. 第2行的A至E列分别命名为: StudentID、FirstName、LastName、DateOfBirth和Sex。

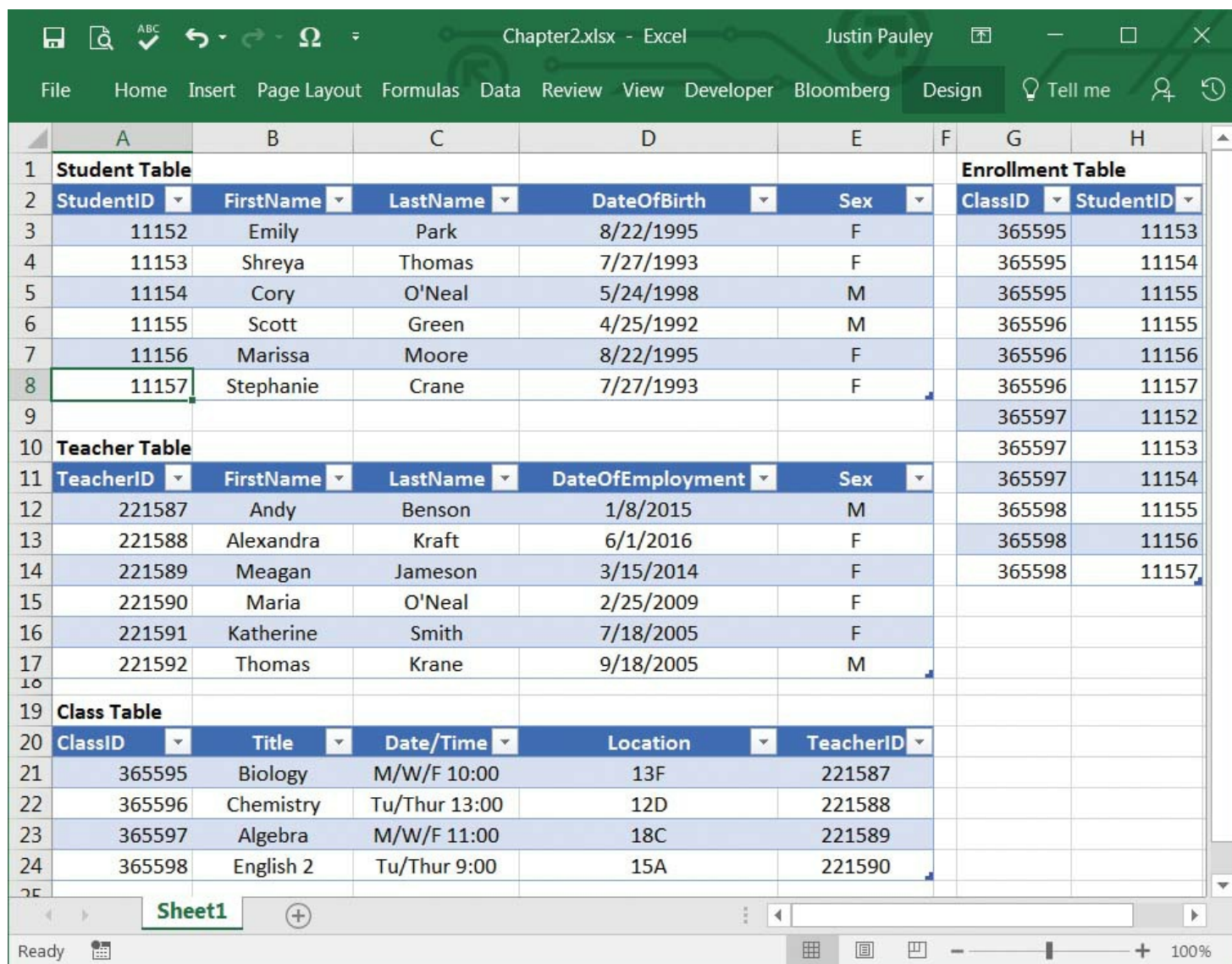
2.从第3行到第8行，增加与列标题对应的学生数据。

3.选择第1步和第2步创建的Excel区域中的每一个单元格。

4.在Excel功能区上，在Home键上，选择“Format as Table”（或者按Ctrl+T组合键）。确保在“My table has headers”框上打钩，然后点击OK。

图2-1显示的是几个Excel表，我们用这些表讨论第三范式和存储数据的正确方式。

把Excel区域转换为Excel表，除了增加彩色格式之外，还使每个列标题旁边的下拉式控件变得可用。此外，右击Excel表并从Table选项中选择Totals Row，就能在Excel表中添加一个合计行（Total Row），你就能为每列选择一个聚合函数功能了（见图2-2）。



Student Table						Enrollment Table	
StudentID	FirstName	LastName	DateOfBirth	Sex		ClassID	StudentID
11152	Emily	Park	8/22/1995	F		365595	11153
11153	Shreya	Thomas	7/27/1993	F		365595	11154
11154	Cory	O'Neal	5/24/1998	M		365595	11155
11155	Scott	Green	4/25/1992	M		365596	11155
11156	Marissa	Moore	8/22/1995	F		365596	11156
11157	Stephanie	Crane	7/27/1993	F		365596	11157
						365597	11152
						365597	11153
Teacher Table						365597	11154
TeacherID	FirstName	LastName	DateOfEmployment	Sex		365598	11155
221587	Andy	Benson	1/8/2015	M		365598	11156
221588	Alexandra	Kraft	6/1/2016	F		365598	11157
221589	Meagan	Jameson	3/15/2014	F			
221590	Maria	O'Neal	2/25/2009	F			
221591	Katherine	Smith	7/18/2005	F			
221592	Thomas	Krane	9/18/2005	M			
Class Table							
ClassID	Title	Date/Time	Location	TeacherID			
365595	Biology	M/W/F 10:00	13F	221587			
365596	Chemistry	Tu/Thur 13:00	12D	221588			
365597	Algebra	M/W/F 11:00	18C	221589			
365598	English 2	Tu/Thur 9:00	15A	221590			

图2-1：Excel表示例

Student Table					
StudentID	FirstName	LastName	DateOfBirth	Sex	
11152	Emily	Park	8/22/1995	F	
11153	Shreya	Thomas	7/27/1993	F	
11154	Cory	O'Neal	5/24/1998	M	
11155	Scott	Green	4/25/1992	M	
11156	Marissa	Moore	8/22/1995	F	
11157	Stephanie	Crane	7/27/1993	F	
Total					6

Teacher Table			
TeacherID	FirstName	LastName	DateOfEmployment

图2-2：Total行的下拉式菜单

命名表格是很重要的。方法是点击表中任意单元格，然后在函数区上，点击 Design，然后在左上角的Table Name文本框中输入一个名字（如Student）。将图2-1中的表命名为：Student、Teacher、Class和Enrollment。命名表格之所以很重要，因为这让引用每个Excel表的公式变得更加便于读写。在本章中的后续部分，将会介绍这个内容。

2.1.2 增加引用列

根据图2-1，示例中的数据分成四个表格：Student、Teacher、Class和Enrollment。Student表包含每个学生的所有属性，包括唯一标识符（StudentID）作为主关键字。同理，Teacher表包含每个教师的所有属性，包括唯一标识符和主关键字（TeacherID）。

Class表开头很像Student和Teacher表，包含每个课程的属性，包括主关键字（ClassID）。然而，因为一个课程“有”教师，所以它包括Teacher表的主关键字

列 (TeacherID)，以引用该课程的教师。包含来自于另一个表的主关键字的列，称为外关键字 (Foreign Key)。Enrollment表 (包括每个课程学生的表单) 有两个外关键字：ClassID (确认学生课程的班级) 和StudentID (确认参加课程的学生)。此外，因为根据定义Enrollment表每行都是唯一的 (一个学生不能两次在同一个课程)，可以让ClassID和StudentID二者共同作为主关键字，以标识一个唯一的行。当两列或更多列共同组成一个主关键字，称为复合关键字 (Composite Key) 或者复合主关键字 (Composite Primary Key)。

然而在Excel中，如果不得不引用好几张表才能确认教师、课程和学生名，就有点恼人了。所以，我们采用INDEX和MATCH函数，用主关键字和外关键字从一张表到另一个表增加列。重要的是，我们并非向其他表增加属性本身，这样会产生数据冗余；我们增加的是引用或连接。如果更新原始表中的数据，所有对该数据的引用也会自动更新。

尽管许多Excel用户熟悉VLOOKUP函数，传统上它用一个数据集里的标识符“查询”另一个数据集里的信息，但是VLOOKUP的功能其实不如INDEX和MATCH函数结合使用。VLOOKUP的主要缺陷是依赖于列的顺序和位置。使用VLOOKUP时，你想查找的信息必须位于lookup值的右侧。此外，如果增加、删除、移动列，就可能导致VLOOKUP偏好错误的单元格，由此导致重大错误。这些错误在使用INDEX和MATCH函数时不会出现。向TeacherID外关键字与Teacher表的TeacherID主关键字相匹配的Class表增加教师姓名的公式，如下所示：

```
=INDEX(Teacher[FirstName],MATCH([TeacherID],Teacher[TeacherID],0),
```

把公式分解一下，INDEX函数有两个参数：列名和行号。INDEX函数返回该行该列交叉的那个单元格的值。MATCH函数有三个参数：查询值、要搜索的列和匹配类型 (匹配类型应当一直为零)。MATCH函数返回包含查询值的行号。在Class表TeacherID列与Teacher表TeacherID列相匹配的前提下，INDEX函数返回Teacher表FirstName列的内容。你还可以把它与姓相结合：

```
=INDEX(Teacher[FirstName],MATCH([TeacherID],Teacher[TeacherID],0),0)&" "&INDEX(Teacher[LastName],MATCH([TeacherID],Teacher[TeacherID],0))
```

本公式向Enrollment表中添加一列，用ClassID展示Class表中的Title：

```
=INDEX(Class[Title],MATCH([ClassID],Class[ClassID],0))
```

本公式向Enrollment表添加学生的全名：

```
=INDEX(Student[FirstName],MATCH([StudentID],Student[StudentID],0),  
&" "&  
INDEX(Student[LastName],MATCH([StudentID],Student[StudentID],0)))
```

本公式向Enrollment表添加学生的性别：

```
=INDEX(Student[Sex],MATCH([StudentID],Student[StudentID],0))
```

注意，如果你改变了任何源信息（学生或教师姓名），使用INDEX和MATCH引用的单元格会自动更新。尽管这看上去有点复杂，你越用就会越熟练。此外，Excel有很棒的自动补全功能，边打字边可以从字段表单中进行选择。

除了向Excel表增加引用列之外，有时增加包含总结式信息的列也是有用的。例如，这可能包括向Class表增加每班学生数量列，或者向Teacher表增加每班教师数量列。用命名的Excel表和列，向Class表增加学生数量列非常容易，具体如下：

```
=COUNTIF(Enrollment[ClassID],[@ClassID])
```

或者再进一步，计算每班女学生的数量：

```
=COUNTIFS(Enrollment[ClassID],[ClassID],Enrollment[Student's  
Sex],"F")
```

注意，在Excel表中复制粘贴而来的公式是不同的。在Excel区域中，复制和粘贴会导致公式被更新，引用的列会根据粘贴位置而相对变化。而在Excel表中，复制和粘贴单元格会导致使用确切的公式。要锁定某单元的公式用于拖动（而非在公式中用\$），你需要像Class[[ClassID]:[ClassID]]这样引用单元格，而非单纯用[ClassID]。

2.1.3 数据验证

要保证主关键字和外关键字是正确的，这很重要。如果一个课程里有某个TeacherID并不存在于Teacher表中，就很糟糕了。幸运的是，Excel有一种数据验证 (Data Validation) 工具，能够帮助维护这些关系。Excel的数据验证工具强制使一个单元格包含来自于某个表单的值，还能关注到表单发生变化以及外关键字不再匹配的情况。

然而不幸的是，数据验证工具只适用于已命名的区域，而不适用于表列名。用下面的步骤添加一个已命名的区域，连接到Teacher表中的TeacherID列。

- 1.在函数区上，点击Formulas选项卡。在Defined Names组中，点击Define Name。

- 2.在Name旁边输入TeacherIDs，在“Refers to” 旁边输入=Teacher[TeacherID]，然后点击OK。

然后，用下面的步骤为Class表TeacherID外关键字列增加数据验证。

- 3.在Class Excel表中，选择TeacherID列对应的那几行。

- 4.在函数区上点击Data选项卡，在Data Tools组中点击Data Validation，然后选择Data Validation。

- 5.选择Allow下拉菜单下面的List。取消勾选Ignore blank。把Source设置为=TeacherIDs，然后点击OK。

结果是Class表中每个TeacherID单元格中有一个下拉框，这样你就只能选择Teacher表中存在的TeacherID。此外，点击Data Validation下拉菜单中Circle Invalid Data，Excel会在不正确的外关键字上画一个红色的圈。图2-3展示了红色的圈和下拉式数据验证功能。

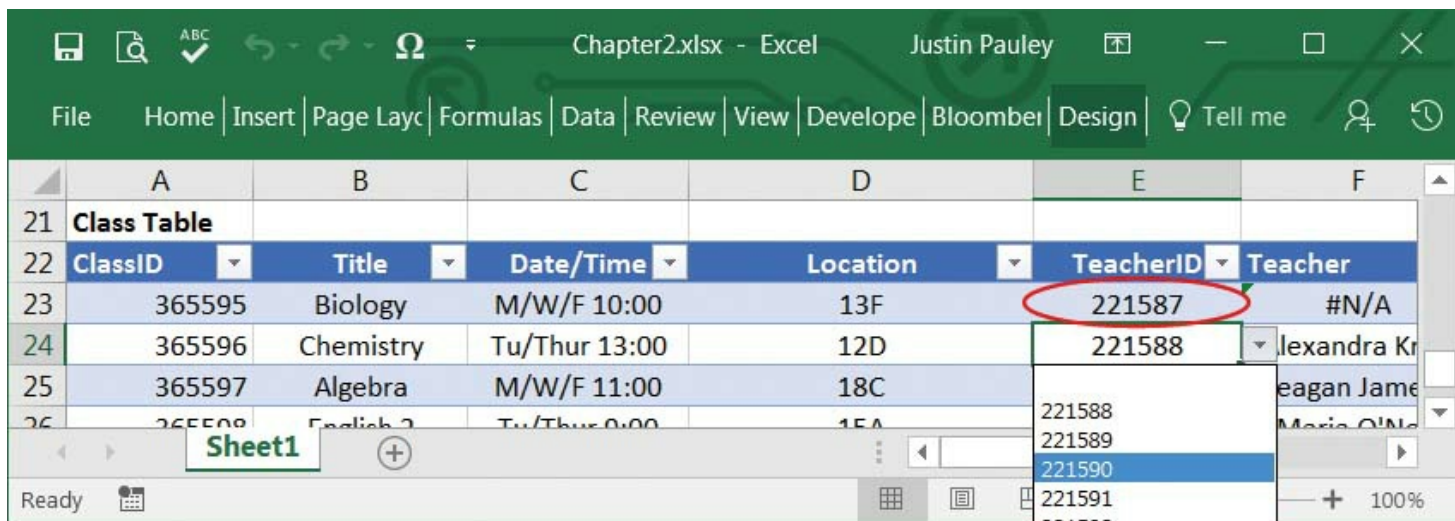


图2-3：Excel数据验证功能

2.2 途径2和3：Access中的表格

在涉及数据结构和表格之间的关系时，Microsoft Access比Excel严谨得多。本节展示如何在Access中创建表，在表之间建立联系，并用查询把数据结合起来。本节使用的部分例子与2.1节的Excel例子相同，推荐你也阅读2.1节。

第一步是与图2-1一样的图表。在Design模式下，为每张表增加列，把主关键字列（StudentID、TeacherID等）的数据类型设置为Number。右击Primary Key列，然后选择要建立的主关键字，使这些列成为各个对应表的主关键字。不需要对外关键字（如Class表中的TeacherID）进行任何处理。

然后在表中填上数据，在函数区上，点击Database Tools，然后点击Relationships。把外关键字从一张表拖到其他表的对应主关键字处（如将Class表中的TeacherID拖到Teacher表中的TeacherID处）。在Edit Relationships对话框（见图2-4）中，选择Enforce Referential Integrity，以确保外关键字与主关键字匹配。

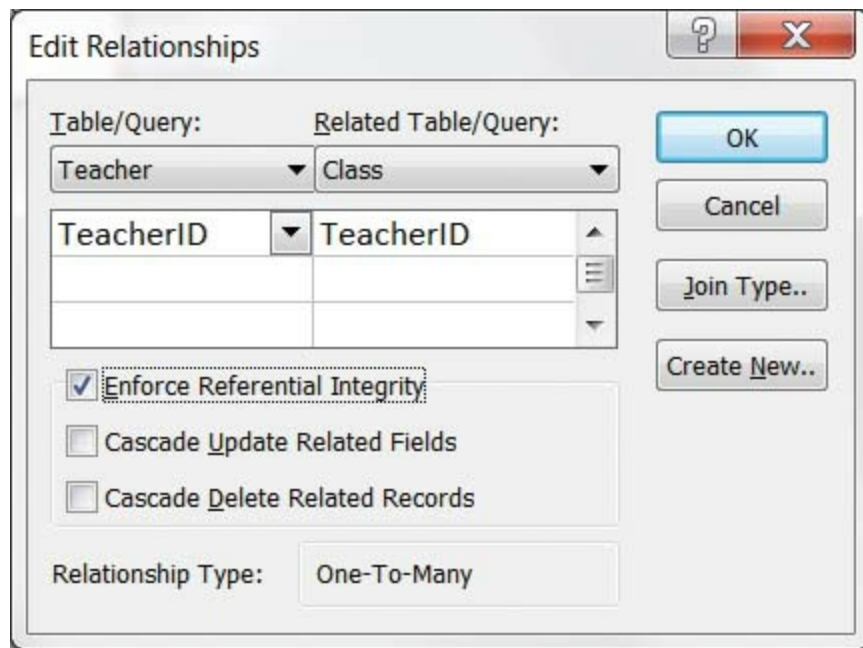


图2-4：Access中的Edit Relationships对话框

当所有的关系都建立起来之后，Relationships选项卡应该看起来像图2-5这样。如果你试图向Class表添加一行，其中的TeacherID却并不存在于Teacher表中，那么这些关系会导致错误。

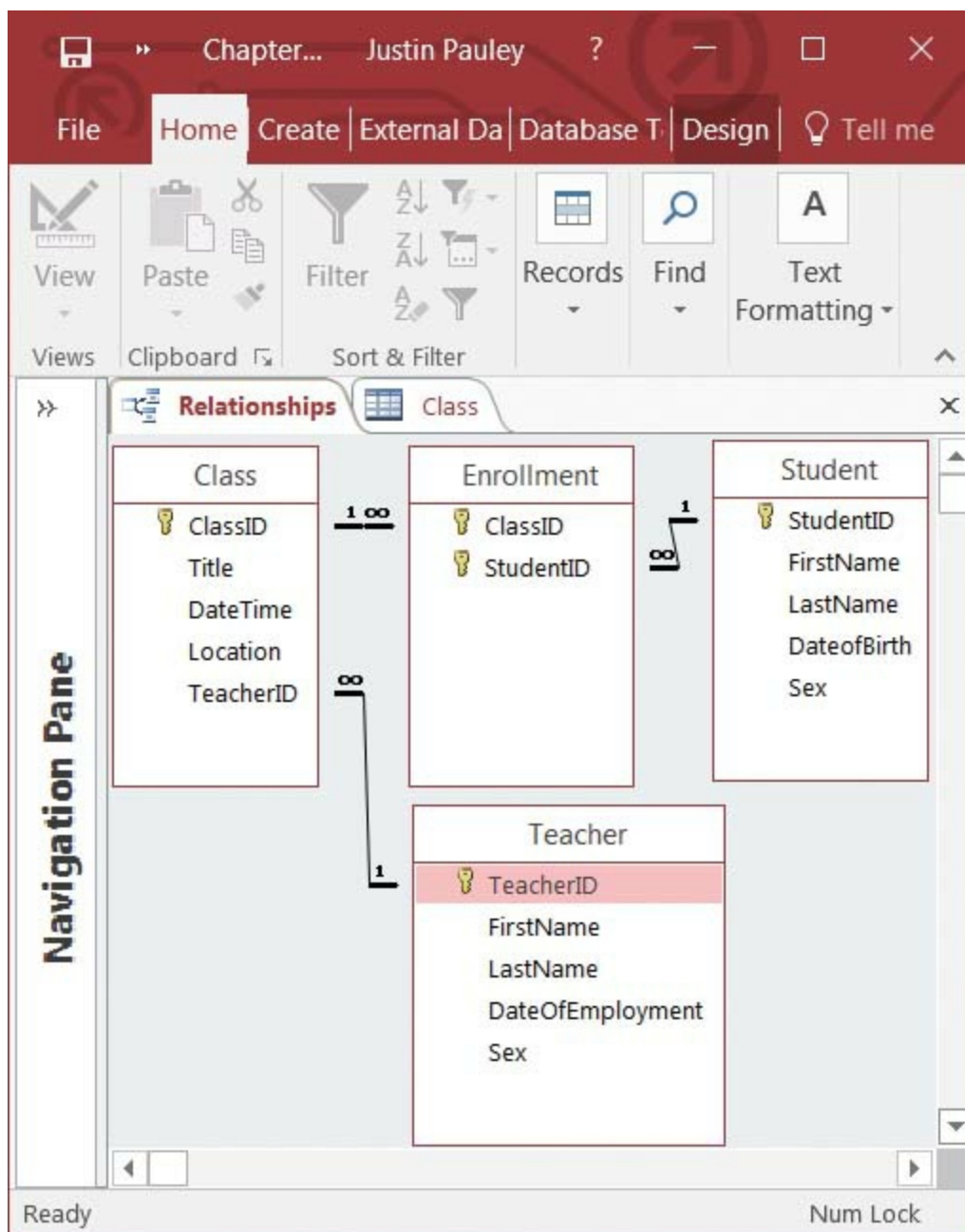


图2-5：Access中的表格关系

用查询连接数据

对于不熟悉SQL的人来说，听说要学习SQL有点吓人，实际上它比你想象的简单得多；甚至比在本书中的一些复杂的Excel公式还简单。尽管本书简要介绍了如何使用SQL，仍推荐阅读Thomas Nield写的《Getting Started with SQL》一书（O'Reilly出版社）。如下是SQL查询基本语法：

```
SELECT Column1, Column2, Column3 from TableName;
```


该查询将展示TableName表中列1至列3每行的内容。还可以添加一个WHERE子句，进一步过滤一些结果。例如，如下将从Enrollment表返回ClassId为365595的所有StudentID：

```
SELECT StudentID from Enrollment WHERE ClassId=365595
```

你还可以使用JOIN语句把多张表连接起来。例如，修改前一个查询，以包含Student表中该学生的姓和名，Student表的StudentID外关键字与Student表的主关键字匹配，用如下带内连接（INNER JOIN）的查询：

```
SELECT E.StudentID, S.FirstName as [Student's First Name],  
S.LastName as [Student's Last Name] From Enrollment E INNER JOIN  
Student S on S.StudentID=E.StudentID WHERE ClassId=365595
```

SQL查询对于回答复杂问题是很有力的工具。尽管SQL查询可能有点复杂，但是读着就比较容易理解的，不像有些Excel公式那样令人费解。如下查询将Class表与Teacher表连接起来，展示教师姓名和课程信息。此外，用子查询提取每班的学生数和女学生数：

```
SELECT C.ClassID, C.Title, C.DateTime, C.Location, T.FirstName  
as [Teacher's First Name], T.Lastname as [Teacher's Last Name],  
(Select count(*) from Enrollment E where E.ClassID=C.ClassID) as [#  
of Students], (Select count(*) from Enrollment E inner join Student  
S on S.StudentID=E.StudentID where E.ClassID=C.ClassID and  
S.Sex='F' ) as [# of Female Students] from Class C INNER JOIN  
Teacher T on T.TeacherID=C.TeacherID
```

2.3 本章小结

在本章中，我们讲到了预先对数据架构进行规划是很重要的，规划能够让你的数据组织合理，避免未来出现让人头痛的麻烦。一个好的架构应当用一个唯一标识符（主关键字）来标示一个有价证券及其属性，并引用该唯一标识符，而非重复其属性（“第三范式”）。为金融数据找到唯一标识符可能并不太容易；最好是大部分时间都用该标识符，保持一致。无论你用什么途径，Excel还是Microsoft

Access，所用的架构都是相同的。在第3章中，我们将介绍如何提取彭博数据，存入Excel和Access中。

第3章 彭博

Bloomberg Professional（也称为彭博终端）是金融分析师最有用的工具之一。然而，尽管彭博终端本身有许多功能，但是其灵活程度和分析能力不及Microsoft Excel和Access。在本章中，我们用彭博提供的金融数据结合Excel、Access的灵活性，创建一个强大的分析工具。此外，彭博有Excel Add-in函数，还有完整的.NET API，方便获取其金融数据。

我们将探索Excel Add-in和.NET API的常用特性；对于其他特性请查阅DAPI<GO>上的彭博文档。

从彭博获取的数据是供个人使用的。扩散这些数据可能违反服务使用规则。此外，每天或每月能从彭博获取的有价值证券和字段数量是有限制的。想获得更多信息，请参见彭博文档（DAPI<GO>）或咨询彭博业务代表（BREP<GO>）。

在开始前需要发出一点警告：尽管彭博公司会甄别验证数据，但是仍然有些彭博信息可能是不正确的。然而，这不能成为你的分析不正确的理由。在本书后面，我们将讨论从大数据集合中识别和剔除不良数据的技术，但是只能手动审查数据以保证准确性，没有别的办法。如果你认为一些彭博数据不正确，请联系Bloomberg Help，以便该公司为你和其他用户修补数据。尽管如此，不要让数据可能不正确的想法吓坏你，不正确数据是小概率事件，而且在处理金融数据时，如果你想事事完美，那么可能一事无成。

3.1 确定字段

将彭博数据放入Excel或其API的第一步是，确定你想获取什么字段。幸运的是，确定字段有几个简单的方法。

3.1.1 鼠标滑动

在许多彭博屏幕中，你可以滑动鼠标，展示对应彭博字段的工具提示。例如，运行AAPL US Equity DES<GO>，提取Apple公司的描述屏幕，然后把光标指向“52Wk H”（52周以来高股价）；会出现一个工具框，提示你相应的Excel字段ID是HIGH_52WEEK，如图3-1所示。



图3-1：展示Excel字段ID的工具提示

🐭 标签可能不太明确，所以需要FLDS<GO>命令再检查一下定义，详见下节。

3.1.2 FLDS屏幕

彭博允许用户用FLDS屏幕搜索字段。要打开FLDS屏幕，首先挑选一个有价值证券（例如Nokia FH Equity<GO>），然后用FLDS<GO>。打开FLDS屏幕之后，用查询文本框搜索一个字段（例如搜索“market cap”（市值）），如图3-2所示。这个方法比鼠标滑动法更好，因为字段数量众多，查询能够提取出的字段比在工具提示框

找到的更有用。例如，在描述屏幕（Excel字段ID：CUR_MKT_CAP）上把鼠标滑动到Mkt Cap上，不会显示Currency Adjusted Market Cap字段（Excel字段ID：CRNCY_ADJ_MKT_CAP），该字段可以用不同的货币提取市值（见图3-2）。如果你要比较的两个公司属于不同地区，调整货币单位就是很重要的。此外，FLDS屏幕显示每个字段的现值，使你更容易找到想要的字段。FLDS屏幕的另一个优点是能从彭博直接把字段拖倒你的Excel工作表中去（需要在Options下选用该功能）。

FLDS 3-BLOOMBERG

GO F1 F2 F3 F4 PRINT HELP MEMO SEARC QUOTE QUOTE MONIT NEWS MSG MENU PG BA PG FW

< > NOKIA OYJ Equity FLDS Related Functions Menu Message ★ ✎ ⚙ ?

CLIP

Use the left mouse button to click and drag an area of the desktop.

NOKIA FH Equity Source Calcrt 98 Save 99 Options Page 1/9 Field Search

market cap View Ranked Filter Equity Field Type All

	ID	Mnemonic	Description	Ovrd	Value
1)	RR902	CUR_MKT_CAP	Current Market Cap		29,761.94
2)	RR910	CRNCY_ADJ_MKT_CAP	Currency Adj Market Cap		29,761.94
3)	RR250	HISTORICAL_MARKET_CAP	Historical Market Cap		29,463.86
4)	FD115	FUND_MKT_CAP_FOCUS	Market Cap Focus		
5)	RR930	MSCI_MKT_CAP	MSCI Market Cap		N.A.
6)	IX219	TOPIX_MARKET_CAP_SEGM...	TOPIX Market Cap Segment2		
7)	RX066	MKT_CAP_LAST_TRD	Market Cap - Last Trade		29,761.94
8)	RR263	DEBT_TO_MKT_CAP	Debt To Market Cap Ratio		0.13
9)	IX218	TOPIX_MARKET_CAP_SEGM...	TOPIX Market Cap Segment1		
10)	RR963	CURRENT_EV_TO_MKT_CAP	EV to Market Cap		0.77
11)	FD103	FUND_MEDIAN_MKT_CAP	Fund Median Market Cap		
12)	FD102	FUND_AVG_MKT_CAP	Fund Average Market Cap		
13)	RR480	EV_TO_MKT_CAP	Periodic EV to Market Cap		0.66
14)	RX017	MKT_CAP_TO_REVENUE	Market Cap To Revenue		1.68
15)	RX669	CASH_%_CURRENT_MARKET...	Cash % Current Market Cap		23.50
16)	FD327	HB_MARKET_CAP_FOCUS	Holdings Based Market Cap Focus		
17)	IA362	MSCI_SEC_INIT_FULL_MKT...	MSCI Security Initial Full Market Cap ...		

图3-2：FLDS屏幕

更重要的是，在FLDS屏幕中，你可以点击一个字段，查看更详细的描述和可选重写（overrides）（见图3-3）。在进行分析之前，阅读每个字段的描述是很重要的。例如，你可以用EQY_FUND_CRNCY（见图3-3）在请求CRNCY_ADJ_MKT_CAP时重写货币。在本章后面将探讨重写问题。

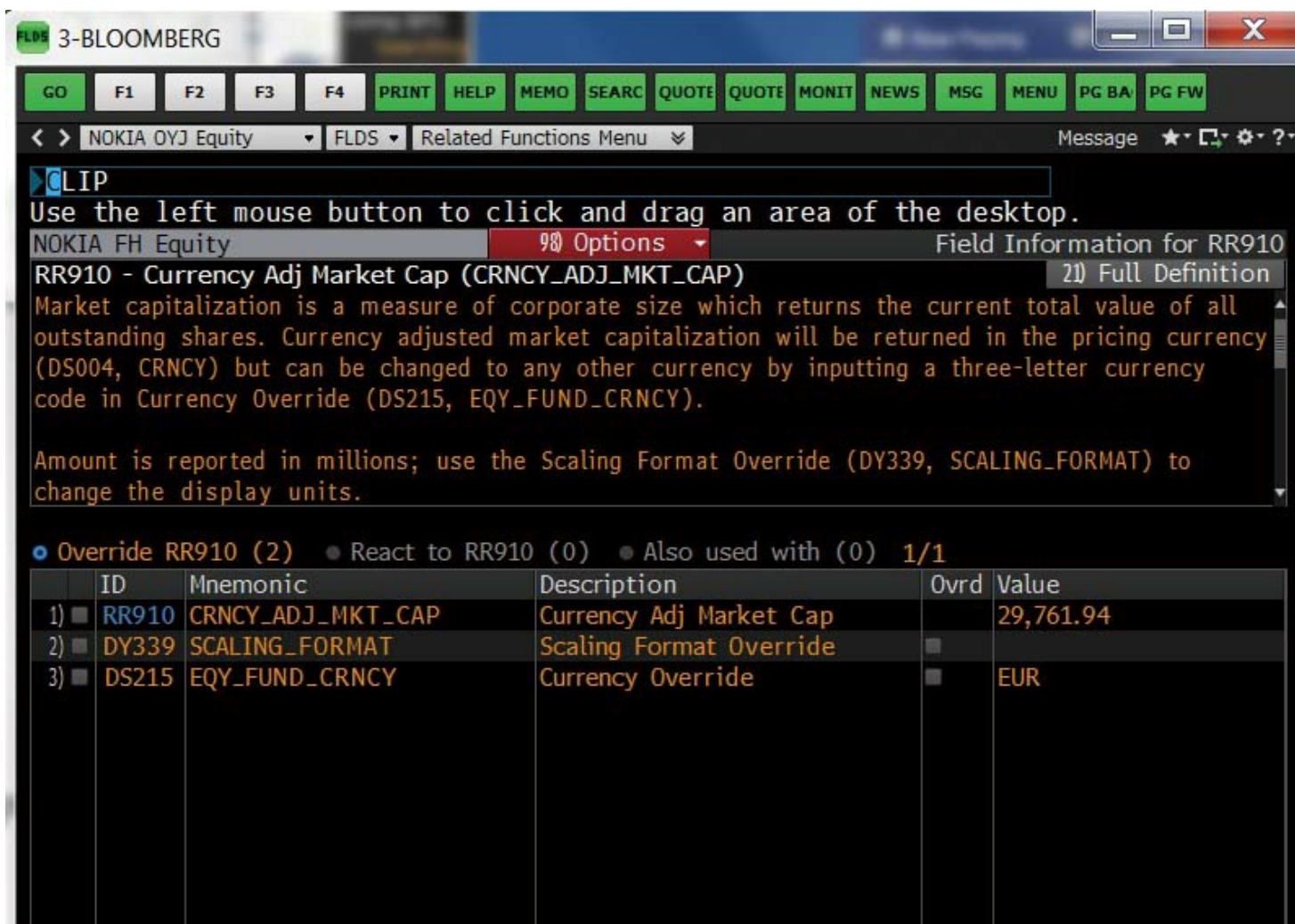


图3-3：彭博字段描述

3.1.3 彭博函数构造器和在Excel中发现字段

用彭博Office工具，你能用直观的菜单从Excel直接获取FLDS屏幕的功能。在Excel的函数区上，点击彭博选项卡，然后在Create group中，点击Function Builder，如图3-4所示。

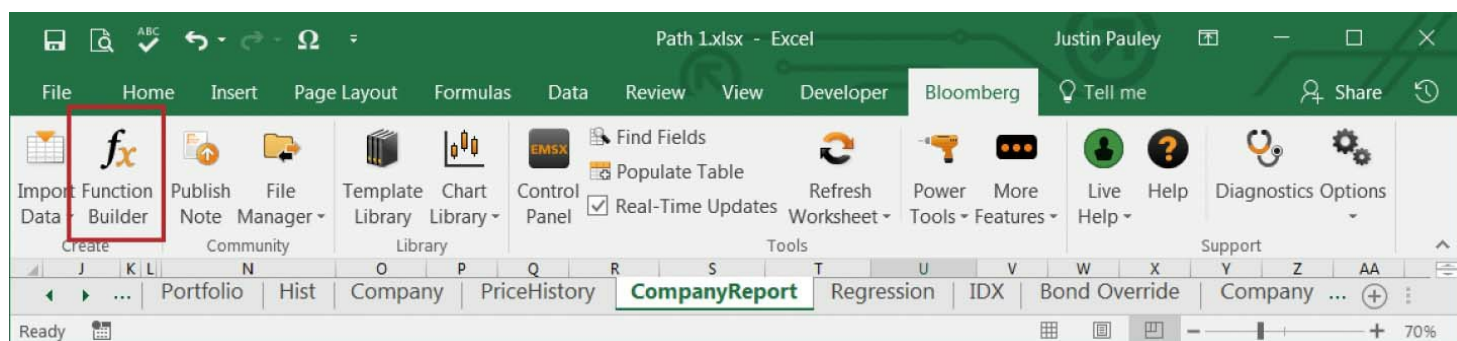


图3-4：Excel功能区上的彭博控件

函数构造器 (Function Builder) 有许多有用的函数，但是这里仅关注 Bloomberg Data Point (BDP) 函数。在本章后面会更详细地介绍BDP函数，它从彭博中提取一个单一数据点。选择BDP之后，你可以用Function Builder搜索一个有价证券和字段，见图3-5。

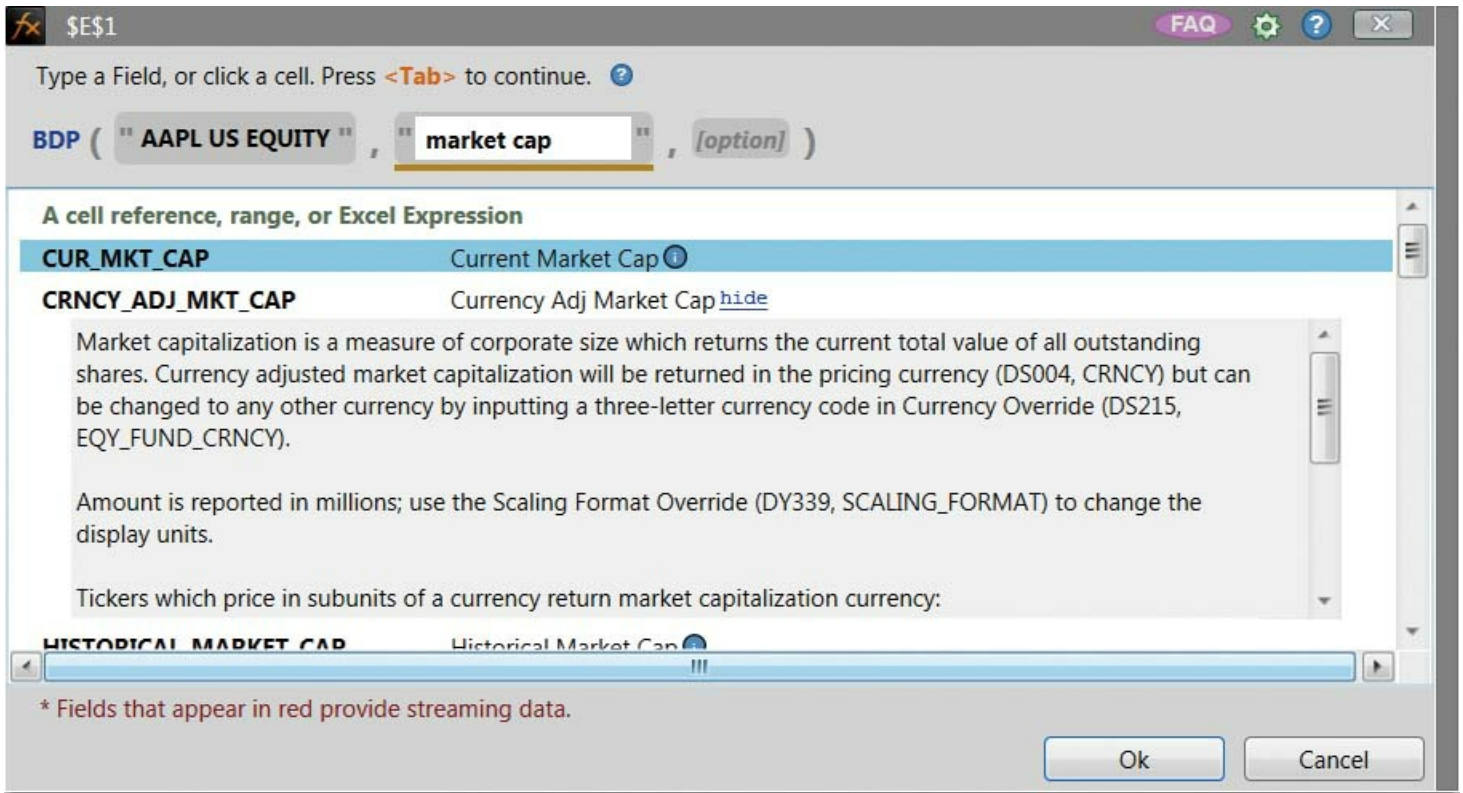


图3-5：Excel中的彭博函数构造器

或者用彭博字段搜索 (Field Search) 浏览字段库，在Excel的Bloomberg控件上，在Tools群中，选择Find Fields按钮（见图3-6）。尽管可以按类别下拉，但是用FLDS屏幕更方便查询，因为它将可用字段按热门程度进行排序。

现在你明白使用Field Search和Function Builder很容易，但是不要直接把本章的其余部分越过去。下面几节展示几个不用将彭博字段硬编码成Excel公式，就能提取数据的简单方式。这将保证你的工作表和数据库表更容易维护，可以用于第5章和第9章。

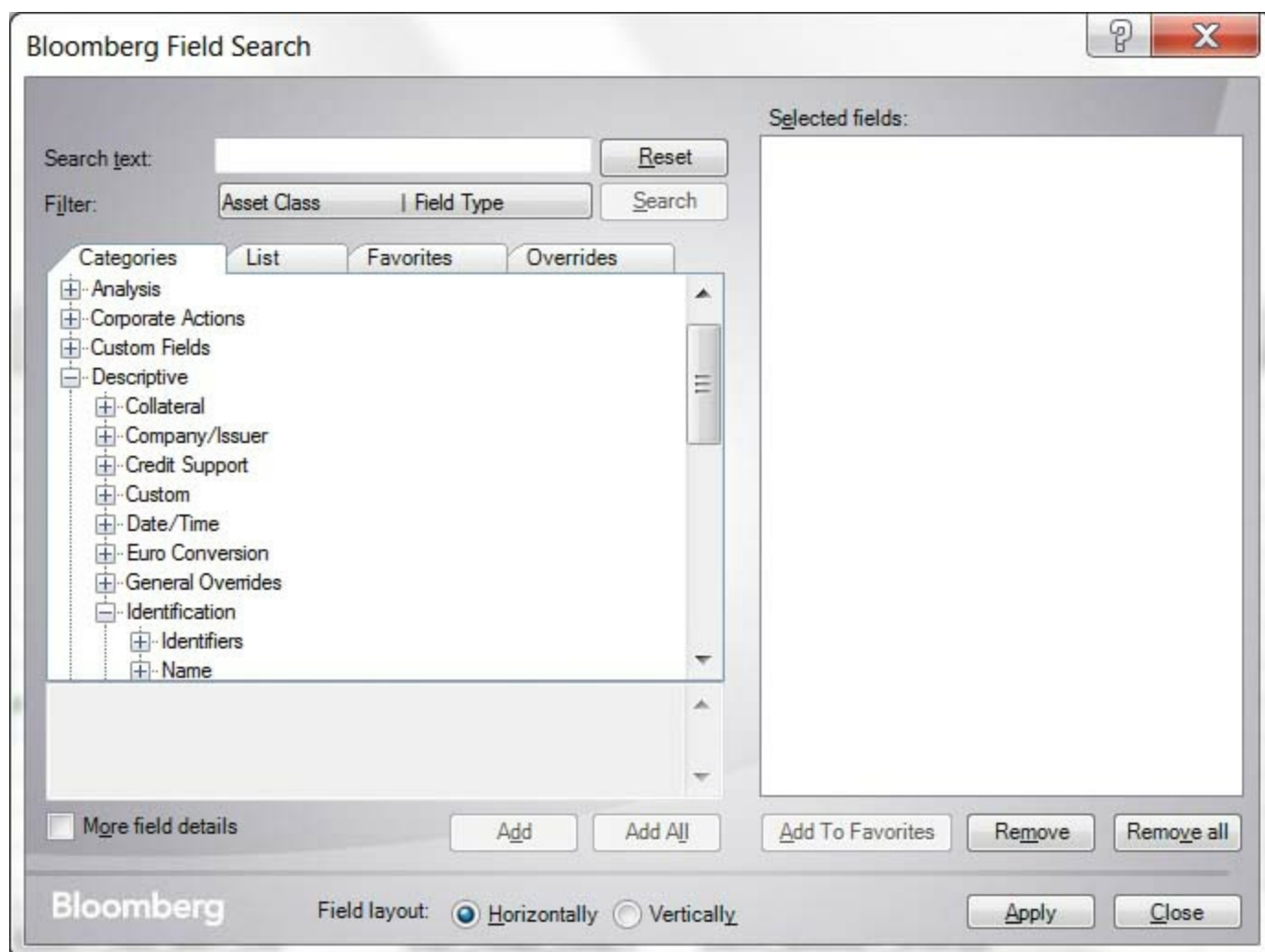


图3-6：Excel中的彭博字段搜索对话框

3.1.4 如果其他都失败了.....

如果在Excel你还不能找到你要寻找的字段，那么就在Bloomberg选项卡上点击Live Help，向彭博的在线帮助团队寻求帮助。你还可以在彭博中点击HELP<GO>屏幕上的Live Help按钮。

3.2 Excel案例

在处理工作表和数据库表之前，先看看如何将数据提取到Excel的过程。即使你只打算用C#获取彭博数据，用Excel也是理解多种彭博函数的有效办法。

3.2.1 提取单一字段（BDP）

如前所述，可以使用彭博的BDP函数，用一个彭博字段获取某数据点，例如价格或者评级。BDP函数的参数简单明了：

```
=BDP("Security","Field","Option 1", "Option 2",..."Option N")
```

可以对"Security"实参用不同标识符。例如，可以用企业债券的ISIN、CUSIP或者Bloomberg ID Number，例如 "US103186AA06Corp" "103186AA0Corp" 或者 "EK1711978Corp"。

用Open Text Corporation (OTC CN Equity<GO>) 公司作为一个例子，首先用如下Excel公式提取当前市值：

```
=BDP("OTC CN Equity", "CUR_MKT_CAP")
```

该Excel公式会返回相同的数字，在DES<GO>屏幕上将OTC CN Equity命名为Mkt Cap（不过DES彭博屏幕返回的市值是以百万计，而Excel公式返回完整、未经格式处理的数字）。根据CUR_MKT_CAP的FLDS屏幕描述，市值按照公司的计价货币显示。可以用如下Excel公式提取货币：

```
=BDP("OTC CN Equity","QUOTED_CRNCY")
```

本公式返回加拿大元兑换代码CAD。可以在彭博上用FXTF<GO>找到一个FX tickers表单。可以用美元提取Open Text公司的市值，或者提供一个重写（override）字段，或者用汇率。调整货币单位所使用的override字段应为EQY_FUND_CRNCY，见CRNCY_ADJ_MKT_CAP字段描述屏幕（图3-3）。要使用该重写字段，你可以向BDP函数添加两个实参（第一个实参是我们正在改写的字段，第二个字段是值），或者使用格式OVERRIDE_FIELD=OVERRIDE_VALUE。例如，要在Excel中用美元显示Open Text公司的市值，下面两种方式都可以：

```
=BDP("OTC CN Equity", "CRNCY_ADJ_MKT_CAP", "EQY_FUND_CRNCY",  
"USD")
```

或者

```
=BDP("OTC CN Equity", "CRNCY_ADJ_MKT_CAP",
```


"EQY_FUND_CRNCY=USD")

如CRNCY_ADJ_MKT_CAP的字段描述屏幕所述，返回的数值单位是百万。如果你想改变返回市值的格式，用表单中列出的其他override字段（见图3-3），如SCALING_FORMAT。在字段描述屏幕点击SCALING_FORMAT，显示可用选项。下面的Excel公式将格式转换为基准单位（UNT），而不是百万：

```
=BDP("OTC CN Equity", "CRNCY_ADJ_MKT_CAP",  
"EQY_FUND_CRNCY=USD", "SCALING_FORMAT=UNT")
```

或者可以在Excel中提取“CAD to USD”汇率，并调整市值，而不用override字段。做法是用如下公式提取CADUSD Spot Exchange Rate（是彭博中的另一个有价值证券，用CADUSD Curncy<GO>）的PX_LAST字段：

```
=BDP("CADUSD Curncy", "PX_LAST")
```

你可能已经注意到有两个结果不匹配（一是CRNCY_ADJ_MKT_CAP公式，二是CUR_MKT_CAP的结果和加拿大元兑美元汇率）。这可能是因为定价源不同。

 彭博为有价值证券提供多种定价源。你需要明白你在使用哪种或哪些定价源。

你可以到PCS<GO>浏览定价源或者用BDP公式提取PRICING_SOURCE字段。此外，QFX<GO>为不同的货币定价源提供说明，PCSS<GO>为其他定价源提供说明。

重写默认定价源，并用彭博综合利率（Composite Rate，CMP）对伦敦货币市场（CMPL）提取CADUSD汇率，要用如下公式，在有价值证券实参上增加CMPL：

```
=BDP("CADUSD CMPL Curncy", "PX_LAST")
```

对于其他有价值证券，例如企业债券，在有价值证券实参内用“@”符号重写默认定价源：

```
=BDP("US103186AA06@BVAL Corp", "PX_BID")
```

表3-1包括彭博的PCSS屏幕的基本总结。

表3-1：定价源摘要

定价源	描述
BGN	基于来自多个消息渠道的实时复合报价
BVAL	结合了直接市场观察和定量定价模型
MSG1	彭博从你的收件箱挖掘价格
TRAC	TRACE 是美国金融业监管局（FINRA）提供的企业和中介机构债券价格发布服务。获取实时债券价格信息需要许可并缴费，否则只能在四天之后查看价格信息

最后一个需要讲解的BDP可选实参是"Fill"。如果请求某个有价证券没有的字段，就默认返回"#N/A N/A"。"Fill"实参将不能完成的请求的返回值修改为你提供的值。例如，请求一个非上市企业（如Dell）的CUR_MKT_CAP将返回"#N/A N/A"，但你可以用如下公式将它改为一个简单的破折号（"-"）：

```
=BDP("DELL US Equity","CUR_MKT_CAP","Fill=-")
```

为了返回一个空的单元格，将实参设为B（"Fill=B"）。

3.2.2 提取批量数据（BDS）

你可能已经注意到FLDS屏幕有时在Value列显示Show Bulk Data；这表示该字段将返回多个值。例如，Microsoft（MSFT US Equity<GO>）公司的FLDS屏幕中在BLOOMBERG_PEERS旁边显示Show Bulk Data。点击Show Bulk Data显示彭博专有算法得到多个公司名称。用BDS函数把该表单提取到Excel中，BDS函数与BDP函数语法相似：

```
=BDS("Security","Field","Option 1", "Option 2",..."Option N")
```

与BDP函数不同，BDS函数将提取多个行或列的数据，还有几个不同的选项。

 BDS返回的表单将覆盖现有单元格中的数据，所以确保你把函数放入空白区域。

可以用DIRECTION参数指明返回值的显示是水平的还是垂直的（默认为垂直的）。要按照水平格式返回Microsoft公司的BLOOMBERG_PEERS，用如下公式：

=BDS("MSFT US Equity","BLOOMBERG_PEERS","DIRECTION=HORIZONTAL")

SORTASC (升序)、SORTDESC (降序) 参数控制返回数据的顺序。你可以把这些参数设为列数或列名 (把HEADERS参数设为Y)。例如，要按字母顺序返回BLOOMBERG_PEERS数据用

=BDS("MSFT US Equity","BLOOMBERG_PEERS","SORTASC=1")

或者

=BDS("MSFT US Equity","BLOOMBERG_PEERS","SORTASC=Peer Ticker")

STARTROW、ENDROW、STARTCOL和ENDCOL参数限制返回的数据。如果把STARTROW设为3，前两行就从返回数据中排除了。同理，如果把ENDROW设为4，第四行之后的内容就从返回数据中排除了。结合使用这些参数，例如把STARTROW设为2，ENDROW设为2，将只获取表单中的第二个元素。你可以将这些参数与SORTASC和SORTDESC参数结合使用。例如，要获取前两个BLOOMBERG_PEERS，按字母顺序排列，用如下公式：

=BDS("MSFT US Equity","BLOOMBERG_PEERS","SORTASC=1","ENDROW=2")

 在本段里，主参数用加粗字体强调。参见附录表A-2。

AGGREGATE参数将BDS结果合并至一个单元格中。你可以将它与SEPARATOR参数结合使用。SEPARATOR参数的有效值包括：B (空白)、C (逗号) 或SC (分号)。例如，如下公式在一个单元格内返回前两个BLOOMBERG_PEERS，用分号分隔：

=BDS("MSFT US Equity","BLOOMBERG_PEERS","ENDROW=2","AGGREGATE=Y", "SEPARATOR=SC")

要把Microsoft的最后三次分红提取到一个单元格中，首先用如下公式显示Microsoft的分红历史：

=BDS("MSFT US Equity","DVD_HIST_ALL","HEADERS=Y")

图3-7显示结果。

Declared Date	Ex-Date	Record Date	Payable Date	Dividend Amount	Dividend Frequency	Dividend Type
6/14/2016	8/16/2016	8/18/2016	9/8/2016	0.36	Quarter	Regular Cash
3/15/2016	5/17/2016	5/19/2016	6/9/2016	0.36	Quarter	Regular Cash
12/2/2015	2/16/2016	2/18/2016	3/10/2016	0.36	Quarter	Regular Cash
9/15/2015	11/17/2015	11/19/2015	12/10/2015	0.36	Quarter	Regular Cash
6/9/2015	8/18/2015	8/20/2015	9/10/2015	0.31	Quarter	Regular Cash
3/10/2015	5/19/2015	5/21/2015	6/11/2015	0.31	Quarter	Regular Cash
12/3/2014	2/17/2015	2/19/2015	3/12/2015	0.31	Quarter	Regular Cash
9/16/2014	11/18/2014	11/20/2014	12/11/2014	0.31	Quarter	Regular Cash
6/10/2014	8/19/2014	8/21/2014	9/11/2014	0.28	Quarter	Regular Cash
3/11/2014	5/13/2014	5/15/2014	6/12/2014	0.28	Quarter	Regular Cash
11/19/2013	2/18/2014	2/20/2014	3/13/2014	0.28	Quarter	Regular Cash
9/17/2013	11/19/2013	11/21/2013	12/12/2013	0.28	Quarter	Regular Cash
6/12/2013	8/13/2013	8/15/2013	9/12/2013	0.23	Quarter	Regular Cash
3/11/2013	5/14/2013	5/16/2013	6/13/2013	0.23	Quarter	Regular Cash
11/28/2012	2/19/2013	2/21/2013	3/14/2013	0.23	Quarter	Regular Cash
9/18/2012	11/13/2012	11/15/2012	12/13/2012	0.23	Quarter	Regular Cash
6/13/2012	8/14/2012	8/16/2012	9/13/2012	0.20	Quarter	Regular Cash
3/13/2012	5/15/2012	5/17/2012	6/14/2012	0.20	Quarter	Regular Cash
12/14/2011	2/14/2012	2/16/2012	3/8/2012	0.20	Quarter	Regular Cash
9/20/2011	11/15/2011	11/17/2011	12/8/2011	0.20	Quarter	Regular Cash

图3-7：用BDS显示分红历史

第二，添加"SortDESC=Record Date"，确保结果排序正确：

```
=BDS("MSFT US Equity","DVD_HIST_ALL","HEADERS=Y","SortDESC=Record Date")
```

第三，删除所有行，仅保留前三行，增加"ENDROW=3"。还要删除HEADERS参数，因为那也会被包含在行数统计中：

```
=BDS("MSFT US Equity","DVD_HIST_ALL","SortDESC=Record Date","ENDROW=3")
```

第四，用Startcol和Endcol分离Dividend Amount列（第五列）：

```
=BDS("MSFT US Equity","DVD_HIST_ALL","SortDESC=Record Date","ENDROW=3", "STARTCOL=5","ENDCOL=5")
```

最后，用AGGREGATE和SEPARATOR参数将数据合并到一个单元格中，用逗号分隔：

```
=BDS("MSFT US Equity","DVD_HIST_ALL","SORTDESC=Record  
Date","ENDROW=3",  
"STARTCOL=5","ENDCOL=5","AGGREGATE=Y","SEPARATOR=C")
```

要用Excel SUM函数把最后三个收益加起来，用ARRAY参数把结果转换成Excel数组，这样可以使用Excel聚合函数，例如AVERAGE或MEDIAN。与其他Excel数组函数一样，必须在输入后同时按Ctrl+Shift+Enter键：

```
=SUM(BDS("MSFT US Equity","DVD_HIST_ALL","SORTDESC=Record  
Date","ENDROW=3", "STARTCOL=5","ENDCOL=5","ARRAY=TRUE"))
```

最后一个要讲的参数是PCS，它改变定价源。例如，要把定价源设为BGN，就把"PCS=BGN"添加到BDS公式中去。

3.2.3 提取历史数据 (BDH)

除了提取一个有价证券最新的金融数据，你可以用Excel BDH函数提取历史数据。如果是按字段描述屏幕标示的，历史值就可以获得。与BDP和BDS函数一样，BDH的前两个实参是SECURITY和FIELD。要提取某一天一个字段的值，唯一可以添加的实参是你请求的日期。例如，用如下公式提取Apple公司2015年7月1日的股票价格：

```
=BDH("AAPL US Equity","PX_LAST","7/1/2015")
```

然而，要提取某一段时间的历史数据，请提供一个起始日期和结束日期。例如，用如下公式提取Apple公司2014年6月2日到2014年6月30日之间的股票价格：

```
=BDH("AAPL US Equity","PX_LAST","6/2/2014","6/30/2014")
```

清除BDH公式下面的所有数据是个好主意，以保证不误写或者错误地合并任何内容。

一个特定日期的格式（例如在上一个例子中）应当与你的电脑默认设置的一致（美国一般用MM/DD/YYYY，欧洲一般用DD/MM/YYYY）。结束日期用空字符串（""）就默认为今天。

日期实参是灵活的，除了特定日期，该实参还允许用不同的日期类型和相对日期。共有三种不同的日期类型：财政年度（Fiscal，F）、日历年度（Calendar，C）或者实际年度（Actual，A），可以参见表3-2。你还可以用这些日期类型表示特定的相对日期，例如"-1AW"表示一周以前。

表3-2：彭博日期类型

类型	每日	每周	每月	每季度	每半年	每年
Fiscal	—	—	—	FQ	FS	FY
Calendar	CD	CW	CM	CQ	CS	CY
Actual	AD	AW	AM	AQ	AS	AY

Fiscal代表某个公司的财政年度。例如，用如下公式提取Plantronics公司PLT US Equity<GO>从2016年6月30日开始的2016财年第一季度历史股票价格，而很多公司的财年始于三月：

```
=BDH("PLT US Equity","PX_LAST","FQ1 2016", "")
```

Calendar和Actual日期类型有所不同。对于Actual类型，"-1AW"代表恰好七天之前；"-1AY"代表恰好一年之前，以此类推。而另一方面对于Calendar类型，"-1CW"代表上周开始的前一个活跃日，"-1CY"代表去年开始的前一天。

作为一个例子，如下公式会在2016年6月13日开始至2016年9月13日的三个月或一个季度。

```
=BDH("AAPL US Equity","PX_LAST","-1AQ","9/13/2016")
```

而如下公式会在2016年3月31日开始，也就是前一个季度之前那天（9月13日在第三季度，4月在第二季度）。

```
=BDH("AAPL US Equity","PX_LAST","-1CQ","9/13/2016")
```

BDH会默认只返回交易日的值，由此跳过市场关闭的假日。然而，并不是所有市场都是用相同的节假日日历，彭博会使用与证券交易所对应的日历。例如，Apple（AAPL US Equity<GO>）公司从2016年7月1日到7月10日的历史价格不包括

7月4日，因为当天是美国的非交易日。然而，Lloyds Banking Group (LLOY LN Equity<GO>) 会显示7月4日的值，因为当天是伦敦的交易日。想要按照美国的交易日显示Lloyds公司的历史价格，按照如下方法添加CDR参数：

```
=BDH("LLOY LN  
Equity","PX_LAST","7/1/2016","7/10/2016","CDR=US")
```

完整日历代码参见CDR<GO>。或者，要指明返回的日子（不管是不是假日）用DAYS参数。除了把交易日的默认值设为T，你还可以把DAYS参数在周末设为W，在全年所有日期设为A。例如，如下公式将展示Apple公司从7月1日到7月10日的股票历史价格，包括周末和节日（7月4日）：

```
=BDH("AAPL US  
Equity","PX_LAST","7/1/2016","7/10/2016","DAYS=A")
```

因为市场在周末和节假日关闭，BDH会自动使用上个交易日的值。要调整该做法，就把Fill参数换成如下之一：

C、P或Previous

与默认一样，用前面最后一个可用日期的值。

N、E或Error

显示错误信息。

F

用下一个可用日期的值。

B或Blank

显示一个空白单元格。

NA

返回一个Excel N/A错误。

PNA

用前一日期的值（如有），否则返回一个Excel N/A错误。

或者输入文本，例如“Market Closed”，如下所示：

```
=BDH("AAPL US  
Equity","PX_LAST","7/1/2016","7/10/2016","DAYS=A", "FILL=Market  
Closed")
```

在本节中，主要参数都用加粗字体表示。还可参见附录中的表A-4。

默认数据显示频率是每日，不过如果想调整显示频率，就设置PERIOD参数。你可以把PERIOD参数设置成表3-2中列出的日期类型。例如，下面的公式展示Apple公司从2016年1月到2016年9月每月15日的股票收盘价格。

```
=BDH("AAPL US  
Equity","PX_LAST","1/15/2016","9/15/2016","PERIOD=AM")
```

你可以将PERIOD参数与相对日期结合，显示Apple公司从今年年初以来每月月末的股票价格：

```
=BDH("AAPL US Equity","PX_LAST","-CY","", "PERIOD=CM")
```

此外，请求价格字段时，用QUOTE参数显示一个特定时段的每日平均价格，而非收盘价，例如：

```
=BDH("AAPL US Equity","PX_LAST","-CY","", "PERIOD=CM", "QUOTE=A")
```

2014年6月9日周一，Apple公司的股价在1拆7之后从645.57美元降到92.70美元。然而，BDH显示Apple公司在2014年6月9日的收盘股价记录是92.2243美元。这是因为彭博对公司剥离、股票分拆/合并、股票分红和附权发行自动进行调整。如果你不想要自动调整，用DPDF<GO>屏幕重新设置参数，或者把CAPCHG参数设为N，如下所示：

```
=BDH("AAPL US
```


Equity","PX_LAST","6/1/2014","6/10/2014","CAPCHG=N")

此外，DPDF<GO>屏幕有调整Normal Cash Dividends和Abnormal Cash Dividends的选项。要在BDH函数中重写这些功能，将CSHADJNORMAL或CSHADJABNORMAL设置为Y或N。或者把USEDPDF参数设为N，忽略所有的DPDF偏好。

要调整返回值的货币，把FX参数设为货币代码，例如"FX=USD"或"FX=CAD"。

彭博还包括几个参数，可用于修改显示值的方式。POINTS控制返回天数的最大值，例如"POINTS=5"只返回五天。默认日期显示是升序排列，不过如果想让日期显示降序排列，就将SORT参数设置为D（代表descending，降序）。BDH函数与BDS函数一样，也可用DIRECTION、ARRAY和PCS参数。然而在使用APPAY参数时，你可能想将它与DATES参数结合使用。设置"Dates=HIDE"将返回的值不包括日期列。最后，在使用PX_LAST时，一些固定收益产品显示收益而非价格；想要改成PRICE的话，将QUOTETYPE改为P，代表价格。

3.3 用于比较的有价证券

评价有价证券的业绩并不是孤立的过程，需要放入一定的背景中进行评估。换句话说，要了解一个债券或股票的业绩，必须与其同类进行多维度的比较。准确选择进行对比的同类名单很重要，尽管有很多现成的方案，但是自动选择也过于片面了。根据你的分析情况，可将Apple公司的股票业绩与NASDAQ指数、S&P 500指数，或者你自己选定的某个企业组的指数进行比较。此外，你的样本集中的有价证券数量也很重要，证券数量过少可能导致结果不明显；证券数量过多可能导致错误结果。

本节介绍多种指数及如何找到可比较的有价证券的方法。

3.3.1 指数

彭博上可以查看知名股票指数，例如标准普尔500指数（SPX Index<GO>）、道琼斯工业平均指数（INDU Index<GO>）、NASDAQ综合指数（CCMP Index<GO>）和罗素2000指数（RTY Index<GO>）。每个指数都不同，其中包含的企业数量从30个到5000个不等。例如，Wilshire 5000指数（W5000Index<GO>）包含了几乎所有

总部在美国的公开上市企业。此外，还有几个仅包含部分公司的“子指数”，其中一些是彭博创造的。例如，罗素3000科技指数（RGUST Index<GO>）是罗素3000指数中科技公司的资本化加权指数。

除了常见指数，2016年8月24日，彭博收购了Barclays Risk Analytics and Index Solutions有限公司，这样彭博客户就能获取大量以彭博和巴克莱（Barclays）共同命名的指数数据。要浏览不同指数，用IN<GO>。例如，Bloomberg Barclays US Corporate High Yield Bond Index包含2152只以美元标价的高收益固定利率企业债券。

对于一个更有针对性的指数，可尝试进行搜索。简单搜索“Bloomberg HY Technology”可能返回Bloomberg USD High Yield Corporate Bond Index Technology（BUHYTE Index<GO>），这是按照一定规则设计的、由科技公司公开发行的高收益固定利率美元企业债券的市值加权指数。

或者，几家大银行和金融机构在彭博客户端为客户提供定制指数。美林银行的客户可以获得该银行的US High Yield Technology Index（H0TY Index<GO>）以及其他很多指数信息。

有两种简易方法把指数的成分股名单放入Excel。方法一是用MEMB<GO>，并在Output选项下选择Excel；方法二是用BDS公式提取INDX_MEMBERS字段：

```
=BDS("BUHYTE Index","INDX_MEMBERS")
```

有时，Excel导出工具或者BDS函数返回的证券标识符会与你的分析所用到的标识符有所不同。例如，前面用于提取指数成员的公式例子返回了一个彭博唯一标识符（例如“COLW3497804”）组成的表单，而不是ISIN组成的表单。要保持一致性，用BDP函数提取你更喜欢的标识符。尽管并不总是如此，有时返回的标识符有前缀，必须在传递给BDP函数之前删除掉。在这种情况下，必须用Excel SUBSTITUTE函数，删除每个标识符中的“CO”，再增加“Corp”，使其成为一个有效的标识符。如下公式将正确的Security字段传递给BDP函数，以获取ISIN（ID_ISIN）：

```
=BDP(SUBSTITUTE(N4,"CO","",1) & " Corp","ID_ISIN") 注意： 单元格
```

3.3.2 对等有价值证券

之前讨论过如何使用BDS函数获取某有价值证券的对等组，彭博专有算法用下面的公式：

```
=BDS("MSFT US Equity","BLOOMBERG_PEERS")
```

在确定可比较有价值证券方面，算法还远远不完美。幸运的是，彭博提供一个屏幕，显示有价值证券之间的相关性。图3-8显示Microsoft (MSFT US Equity<GO>) 公司的Peer Correlation (PC) 屏幕 (PC<GO>)。PC屏幕是一种有用的方法，用来发现有一定相关性的有价值证券和指数。

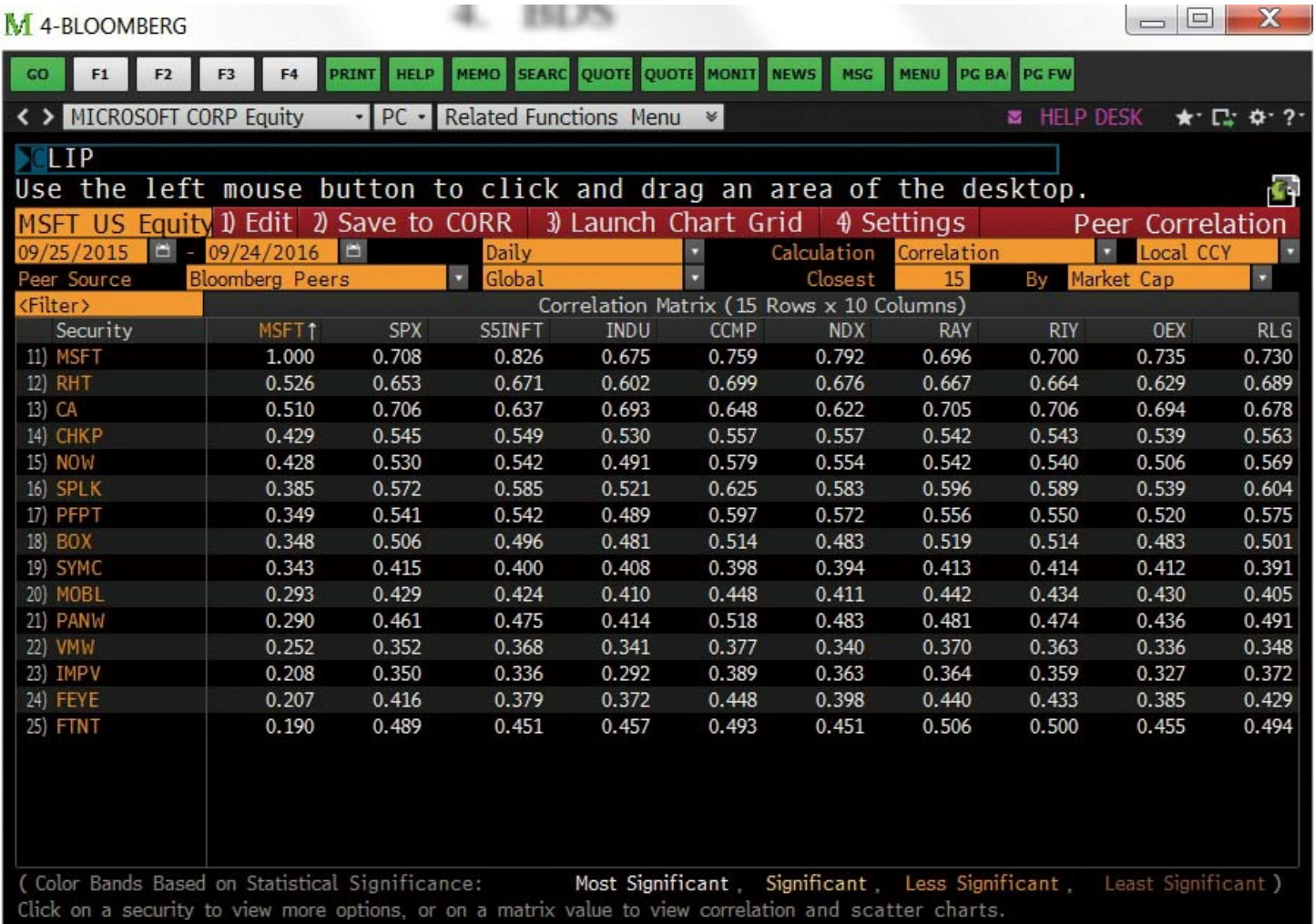


图3-8：Microsoft公司的Peer Correlation屏幕

3.3.3 关于有价值证券

Related Securities (RELS<GO>) 屏幕提供了关于选中的有价值证券的发行商的详细概览。如First Data Corp公司的Related Securities屏幕所示，该公司有好几个债券（见图3-9）。

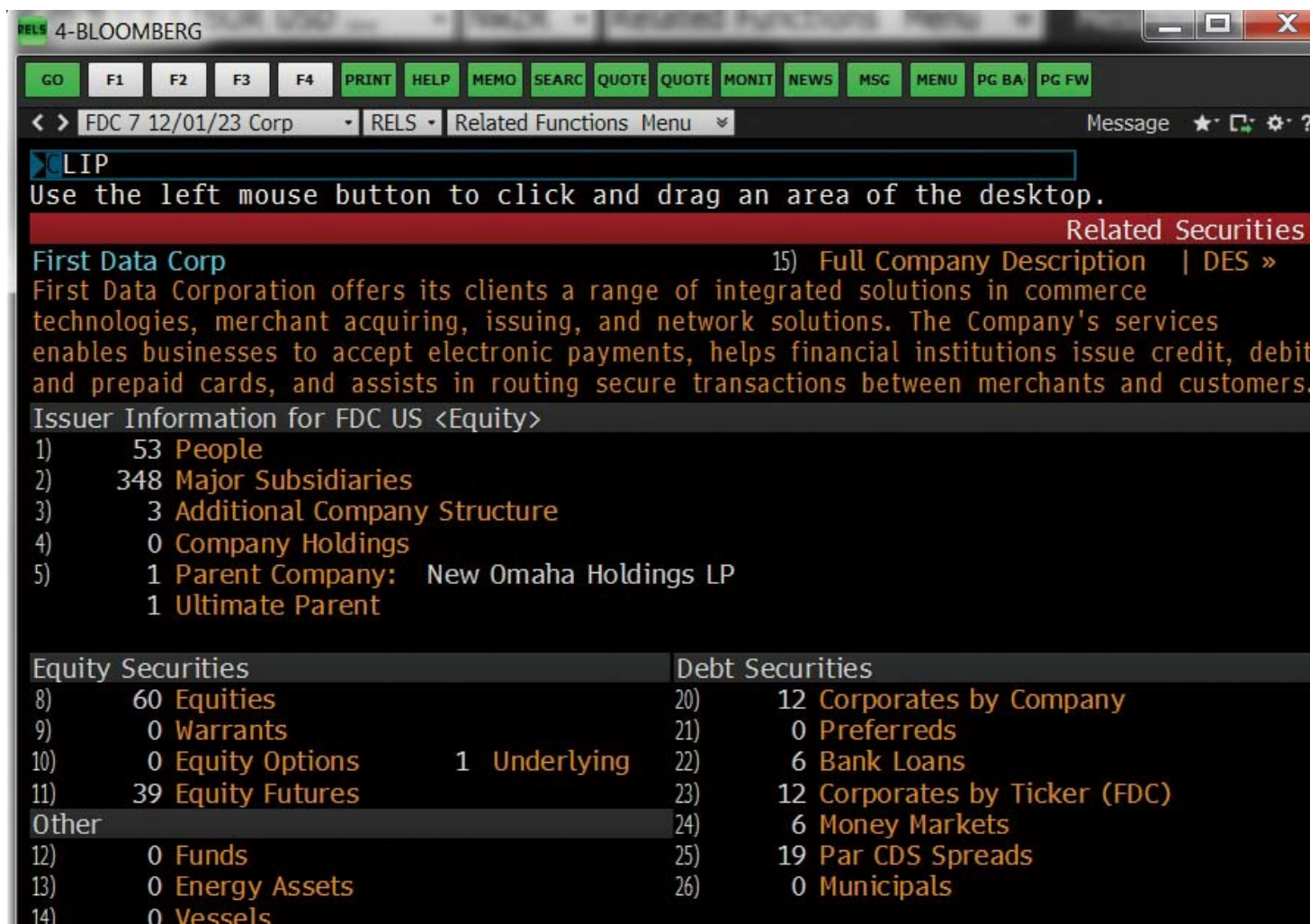


图3-9：First Data Corp公司的Related Securities屏幕

只要点击“Corporates by Company”，就会出现First Data Corp公司发行的有价值证券名单，根据有价值证券类型分成几个选项卡。注意，如图3-10所示，结果按照关联度排序（如第一列的Relevance Indicator图标所述）。基于彭博专有算法的Relevance Indicator能保证大多数相关有价值证券被包含在你的分析中。

SECF 4-BLOOMBERG

GO F1 F2 F3 F4 PRINT HELP MEMO SEARCH QUOTE QUOTE MONIT NEWS MSG MENU PG BA PG FW

< > FDC 7 12/01/23 Corp SECF Related Functions Menu Message

CLIP

Use the left mouse button to click and drag an area of the desktop.

BBID:100574 98) Export 12 results Security Finder

Category Fixed Income

Corp Govt Loan Pfd CDS CDS Idx Muni Futr Opt IRS IRS Vol Gen Govt

60) Exclusions: (55) Matured/Called 62) Advanced Search | SRCH » 61) Column Settings

R	Name	Ticker	Coupon	Maturity	Series	BB Rtg	Mty Type	Announce	Curr	Ask Px
1)	First Data Corp	FDC	7.000	12/01/2023	144A	B-	CALLABLE	10/29/2015	USD	106.000
2)	First Data Corp	FDC	5.000	01/15/2024	144A	BB-	CALLABLE	11/05/2015	USD	101.563
3)	First Data Corp	FDC	6.750	11/01/2020	144A	BB-	CALLABLE	08/02/2012	USD	104.000
4)	First Data Corp	FDC	5.375	08/15/2023	144A	BB-	CALLABLE	08/05/2015	USD	103.125
5)	First Data Corp	FDC	5.750	01/15/2024	144A	B-	CALLABLE	11/05/2015	USD	102.813
6)	First Data Corp	FDC	7.000	12/01/2023	REGS	B-	CALLABLE	10/29/2015	USD	105.182
7)	First Data Corp	FDC	5.000	01/15/2024	REGS	BB-	CALLABLE	11/05/2015	USD	102.375
8)	First Data Corp	FDC	5.000	01/15/2024	REGS	BB-	CALLABLE	03/09/2016	USD	102.562
9)	First Data Corp	FDC	6.750	11/01/2020	REGS	BB-	CALLABLE	08/02/2012	USD	104.625
10)	First Data Corp	FDC	5.375	08/15/2023	REGS	BB-	CALLABLE	08/05/2015	USD	103.250
11)	First Data Corp	FDC	5.750	01/15/2024	REGS	B-	CALLABLE	11/05/2015	USD	102.750
12)	First Data Corp	FDC	6.750	11/01/2020	SEPT	BB-	CALLABLE	08/02/2012	USD	103.875

图3-10：根据相关度检索的First Data Corp公司债券

3.4 途径1和2：Excel和Access

本节介绍创建一个新的工作簿的步骤，该工作簿中有公司、企业债券、贷款和指数工作表。通过把数据从彭博提取到这些工作表中，就将大量信息放到了你的指尖。此外，这些工作表也用于本书第二部分分析数据章节。如果你用途径2，本节创建的工作表与第5章中的Microsoft Access相关。

下面的表中选出的字段/列用于展示一些金融数据的基本类型。这些仅是彭博提供的数据中的沧海一粟。

3.4.1 企业债券、贷款和指数

首先，我们创建一个新的工作簿，命名为Bond。然后使用表3-3填入新工作表的前两行。表3-3包括四列：Excel列字母；存储在第一行的彭博字段（又名Mnemonic）；存储在第二行的人类可阅读的数据描述；引用的彭博字段描述。例如，单元格C1应当包含BOND_TO_EQY_TICKER，单元格C2应当包含CompanyID。同理，单元格D1应当包含SECURITY_DES，单元格D2应当包含“Security

Description”。注意彭博字段不区分大小写。填完Excel区域（A1至T2）之后，将仅包括第二行用到的单元格（A2至T2）的Excel区域转换为一个Excel表，命名为Bond。

表3-3：Bond作表列和彭博映射

列	第一行输入	第二行输入	描述
A	[Intentionally blank]	BondID	
B	[Intentionally blank]	BBID	
C	BOND_TO_EQY_TICKER	CompanyID	债券发行人的股票代码
D	SECURITY_DES	Security Description	有价证券的彭博描述
E	CPN_TYP	Coupon Type	利息类型（固定利息、浮动利息）
F	CPN	Fixed Coupon	当前利率
G	MATURITY	Maturity	债券到期日期
H	RTG_MOODY	Moody's Rating	穆迪评级
I	RTG_SP	S&P Rating	标准普尔评级
J	QUOTED_CRNCY	Currency	报价货币
K	PAYMENT_RANK	Rank	Payment 评级（特别不安全、安全等）
L	PX_LAST	Price	最终价格
M	CHG_PCT_YTD	YTD Px Chg	今年迄今价格变化（百分比）
N	CHG_PCT_3M	3M Px Chg	三个月价格变化（百分比）
O	YAS_YLD_SPREAD	YAS Spread	当前价格，债券收益和默认基准收益之间的基准点（bps）差。默认设置由YASD <GO> 屏幕控制
P	YAS_BOND_YLD	YAS Yield	在当前价格的债券收益。由YASD <GO> 屏幕控制默认设置
Q	CALLABLE	Callable?	债券能否被回购 / 赎回？
R	NXT_CALL_DT	Next Call Date	债券的下一个提前赎回日
S	NXT_CALL_PX	Next Call Price	下一个提前赎回日的债券赎回价
T	[Intentionally blank]	Bond Comments	

Bond表结果如图3-11所示。

为了把所有信息串起来，需要给每个债券、贷款、公司等设置一个唯一标识

符。毕竟，如果不能用公司所在行业就检索出债券，本书就没什么用了，因为该信息在另一张工作表上。不幸的是，为每个有价值证券设置一个唯一标识符很富于挑战性。例如，某个企业债券可能是在既符合Regulation S (Reg S) 又符合Rule 144a的条件下发行的，由此可以有两套标识符 (144a国际证券识别编码 (International Securities Identification Number, ISIN) 和美国统一证券辨认委员会 (Committee on Uniform Security Identification Procedures, CUSIP) 以及Reg S ISIN和CUSIP)。

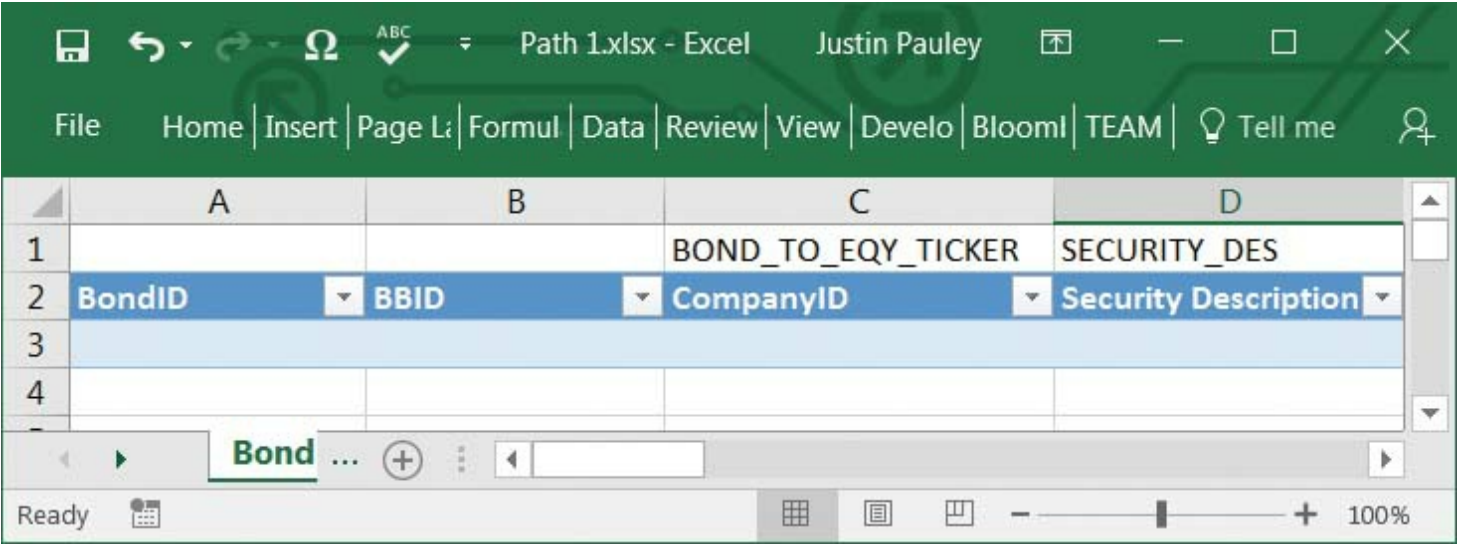


图3-11：初步排版之后的Bond工作表

还有更复杂麻烦的，尽管所有标识符引用同一个债券，不同的基金对于投资于144a系列和Reg S系列有不同的规则。由此，采用144a和Reg S规则的价格和流动性可能会有不同。此外，股票也存在类似的情况，因为未上市企业没有股票代码，上市企业在不同交易所可能有不同的股票代码。尽管没有完美的解决方案，只要保持一致就不会出乱子。在本书中的唯一标识符，对企业债券用144a ISIN，对公司用股票代码，对贷款用彭博标识符。注意，CUSIP是CUSIP Global Services的授权产品，在数据库里使用CUSIP可能需要获得授权。

用表3-3的排版，在Bond工作表A列的BondID标题下列出企业债券的144a ISIN。可以包含同一个公司发行的不同债券，但是记住不能用重复的ISIN。然后，添在第二列BBID上加一个公式，把ISIN转换为彭博能识别的有价值证券。如前所述，在债券的ISIN之后简单添加"Corp"，使其成为一个有效的彭博标识符，如下所示：

=[@BondID]&" Corp"

注意 “Corp” 之前的空格。这个空格很重要，结果看起来是这个样子的：“US004498AA90Corp”。在BBID列有一系列彭博标识符之后，就可以将彭博数据提取到工作表中了。在CompanyID列标题下的单元格C3中输入如下公式：

```
=BDP(Bond[@[BBID]:[BBID]], C$1, "Fill=-")
```

用Bond[@[BBID]:[BBID]]这样的公式来确定表和列，而非仅仅用[@BBID]，这样就锁定了单元格引用，可以被复制或者拖到工作表的其他部分。说明一下，没有单元格引用的公式如下所示：

```
=BDP("US004498AA90 Corp","BOND_TO_EQY_TICKER","Fill=-")
```

如前所述，公式从彭博中对每个债券提取BOND_TO_EQY_TICKER字段。BOND_TO_EQY_TICKER将返回债券发行人的股票代码（或者第一层母公司的股票代码及股票基础信息）。后面我们会用该数据为Company工作表提供一个公司表单。引用彭博字段，而不是直接把它硬编码成Excel BDP公式有三点好处。首先，列在每行上面可以迅速查找当前字段，而且只要改一个单元格就能换到另一个字段。其次，使用引用可以很方便地插入一行增加另一个字段，并在第一行放上彭博字段。最后，使用相同的公式减少犯错误的概率。

Bond工作表显示应该如图3-12所示（正在编辑单元格D3，以显示公式）。

	A	B	C	D	E
1	1		BOND_TO_EQY_TICKER	SECURITY_DES	CPN_TYP
2	BondID	BBID	CompanyID	Security Description	Coupon Type
3	US004498AA90	US004498AA90 Corp	ACIW US	[ID]], D\$1, "Fill=-")	FIXED
4	US007903AU15	US007903AU15 Corp	AMD US	AMD 7 3/4 08/01/20	FIXED
5	US007903BC08	US007903BC08 Corp	AMD US	AMD 7 07/01/24	FIXED
6	US007903AX53	US007903AX53 Corp	AMD US	AMD 7 1/2 08/15/22	FIXED

图3-12：带彭博数据的Bond工作表

你可以在Excel中选择A列，在重复的单元格上增加条件格式，然后在函数区Home键上，单击Conditional Formatting，指向Highlight Cell Rules，然后选择Duplicate Values。

Loan工作表的创建与Bond工作表基本一模一样，除了有一处不同：用Bloomberg ID Numbers（例如Zebra Technology公司定期贷款B用“BL2015081”），而非ISIN。如果你持续将BBID列的值设为一个有效的彭博标识符，你可以将一个不同的标识符作为Loan表的主关键字。用表3-4的前两列填入Loan工作表的前两列。将第二行转换为Excel表，命名为Loan。用与Bond表相同的公式填入其余的单元格。

表3-4：Loan工作表列和彭博映射

第一行输入	第二行输入	描述
[Intentionally blank]	LoanID	包含每笔贷款的唯一标识符
[Intentionally blank]	BBID	包含完整彭博标识符
ISSUER_PARENT_EQY_TICKER	CompanyID	贷款发行人的股票代码
SECURITY_DES	Security Description	有价证券的彭博描述
LN_CURRENT_MARGIN	Margin	票面上高于基准的利差或息差

表3-4：Loan工作表列和彭博映射（续）

第一行输入	第二行输入	描述
INDEX_FLOOR	Floor	指数下限（如适用）
RESET_IDX	Index	票面利息的基准指数
MATURITY	Maturity	到期日
RTG_MOODY	Moody's Rating	穆迪评级
RTG_SP	S&P Rating	标准普尔评级
PX_LAST	Price	最终价格
CHG_PCT_YTD	YTD Px Chg	今年迄今价格变化（百分比）
DISC_MRGN_ASK	DM	基于当前卖出价的贴现差额（DM）
YLD_YTM_ASK	Yield	基于卖出价的到期收益率
CALLABLE	Callable?	债券能否被回购 / 赎回？
NXT_CALL_DT	Next Call Date	发行人可以赎回债券的下一个日期
NXT_CALL_PX	Next Call Price	在下一个可赎回债券日期的债券价格
[Intentionally blank]	贷款备注	

创建一个Index工作表，命名为IDX（不是“Index”，因为该词在Microsoft Access中是保留词汇）。Index工作表与Loan工作表基本一样，除了一点：用“SPX Index”中的“Index”，而不用“BL2015081Corp”中BBID列的“Corp”。用表3-5的前两列填入IDX工作表的前两列。把第二行转换为Excel表，命名为“IDX”。用与Bond表中相同的公式填入其余的单元格。

表3-5：IDX工作表列和彭博映射

第一行输入	第二行输入	描述
[Intentionally blank]	IndexID	包含每笔贷款的唯一标识符
[Intentionally blank]	BBID	包含完整彭博标识符
NAME	Name	指数名称
PX_LAST	Price	最终价格
CHG_PCT_HIGH_52WEEK	52 Week High	过去 52 周内当前价格和最高价格的百分比差
CHG_PCT_LOW_52WEEK	52 Week Low	过去 52 周内当前价格和最低价格的百分比差
CHG_PCT_YTD	YTD Px Change	今年迄今价格百分比变化
CHG_PCT_3M	3M Px Change	过去三个月价格百分比变化
CURRENT_TRR_1YR	12M Total Return	一年总收益；利息进行再投资

3.4.2 公司工作表

Company（公司）工作表与Bond、Loan和IDX工作表有几处不同。首先，因为许多都是估计值，因此好几列中都需要货币重写（currency override）。其次，尽管大部分彭博列包含相同的公式，但是五年CDS费率需要一点修正，后面再讨论。最后，Company工作表包含一个需要手动填写（不是自动从彭博下载的）Category列和一个从其他彭博列衍生的Net Debt/EBITDA列。

创建一个新的工作表，命名为Company，在单元格A1写上Currency（货币），在单元格B1写上USD（美元）。后面，我们会在彭博公式中引用B1单元格，以确保所有美元金额的单位都是USD。

然后用表3-6的字段填入第二行和第三行的列。例如，单元格C2应包含“NAME”字样，单元格C3应包含“CompanyName”字样。同理，单元格E2应包含“GICS_SECTOR_NAME”字样，单元格E3应包含“Sector”字样。单元格A2和B2应为空。把第3行的单元格转换为名叫“Company”的Excel表。带星号（*）的列用货币重写，见本节后面的内容。

表3-6：Company工作表列和彭博映射

第二行输入	第三行输入	描述
[Intentionally blank]	CompanyID	包含每个公司的唯一标识符
[Intentionally blank]	BBID	包含完整的彭博标识符
NAME	CompanyName	公司名称
COMPANY_IS_PRIVATE	Private	表示公司是非上市企业
GICS_SECTOR_NAME	Sector	Global GICS 部门分类
GICS_INDUSTRY_NAME	Industry	Global GICS 行业分类
GICS_SUB_INDUSTRY_NAME	Sub-Industry	GICS 子行业分类
RTG_SP_LT_LC_ISSUER_CREDIT	S&P Rating	标准普尔长期 债务发行人评级
RTG_MDY_LT_CORP_FAMILY	Moody's Rating	穆迪长期企业 家族评级
CRNCY_ADJ_MKT_CAP*	Market Cap	经过货币调整的市值
SHORT_AND_LONG_TERM_DEBT*	Total Debt	短期和长期债务之和（百万美元计）
NET_DEBT*	Net Debt	公司的净债务
CRNCY_ADJ_CURR_EV*	Enterprise Value	经过货币调整的企业价值
TRAIL_12M_EBITDA*	TTM EBITDA	过去十二个月 EBITDA
PE_RATIO	PE Ratio	市盈率，即股票价格和公司每股收益比
EQY_DVD_YLD_IND	Dividend Gross Yield	最近宣布的年化毛息除以现价
CURRENT_TRR_1YR	12M Total Return	一年总收益。股息重新再投资
CHG_PCT_YTD	YTD Px Change	今年迄今百分比价格变化

表3-6：Company工作表列和彭博映射（续）

第二行输入	第三行输入	描述
CHG_PCT_3M	3M Px Change	过去三个月百分比价格变化
SALES_REV_TURN*	Total Revenue	公司的营业收入总额减去对销售总额的各种调整
TOT_DEBT_TO_EBITDA	Total Debt/EBITDA	总债务除以过去十二个月 EBITDA
INTEREST_COVERAGE_RATIO	Interest Coverage	息税前利润 (EBIT) 除以总利息
TRAIL_12M_FREE_CASH_FLOW*	FCF	过去十二个月净现金流
FCF_TO_TOTAL_DEBT	FCF/Total Debt	过去十二个月净现金流除以债务总额
CRNCY_ADJ_PX_LAST*	Price	经过货币调整的最终价格
CHG_PCT_HIGH_52WEEK	52 Week High Change	当前价格和过去 52 周中最高价格之间的百分比差
CHG_PCT_LOW_52WEEK	52 Week Low Change	当前价格和过去 52 周中最低价格之间的百分比差
CDS_SPREAD_TICKER_5Y	5yr CDS Spread Ticker	五年期信用违约互换票面价格息差的彭博名称缩写
TOT_BUY_REC	Buy Recommendations	提出买入建议的研究分析师总数
TOT_SELL_REC	Sell Recommendations	提出卖出建议的研究分析师总数
TOT_HOLD_REC	Hold Recommendations	提出持有建议的研究分析师总数
[Intentionally blank]	Net Debt/EBITDA	
[Intentionally blank]	5yr CDS Spread	
[Intentionally blank]	Category	
[Intentionally blank]	Company Comments	

你可能已经注意到，一些列（例如 Moody' sRating 和 Price）存在于多张表中，这与我之前说的在不同位置存储相同的信息是矛盾的。然而在这些情况下，Moody' sRating 列在 Company 表中表示 Moody' sCorporate Family Rating（穆迪企业家族评级），在 Bond 表中表示 Moody' sissue rating（穆迪发行评级）。类似地，Price 列在 Company 表中表示该公司的股票价格，而在 Loan 表中表示该公司的贷款价格。如果你觉得容易混淆，当然可以使用更精确的列名，例如 "Loan Price" 和 "Bond Price"。

然后，在A列的CompanyID列标题下输入你的公司股票名称缩写表单。该表单至少应当包括Bond和Loan工作表中CompanyID列的全部股票名称缩写。然后，在CompanyID列最后填上"Equity"，用如下公式在BBID列创建一个有效的彭博标识符：

```
=[@CompanyID]&" Equity"
```

需要再次注意“Equity”字样之前的空格，这样单元格应该看起来像“SVR US Equity”。然后，构建一个与Bond和Loan工作表使用的类似的BDP函数，除了如前所述添加货币重写。从第一行CompanyName标题下开始的每个单元格，除了四列（Net Debt/EBITDA、5yr CDS Spread、Category和Company Comments），应当包含：

```
=BDP(Company[@[BBID]:[BBID]], C$2,"EQY_FUND_CRNCY",$B$1,"Fill=-")
```

为了清晰起见，不含引用时该公式如下所示：

```
=BDP("SVR US Equity", "NAME","EQY_FUND_CRNCY","USD","Fill=-")
```

尽管彭博字段NAME没有也不需要货币重写，但是也不会导致什么问题，并且它还允许在大多数列用统一的公式。最后是Net Debt/EBITDA列，展示如何包含一个不从彭博提取数据的列。该列的公式是Net Debt列除以TTM EBITDA列（EBITDA列不是空白时）：

```
=IF([@[TTM EBITDA]]<>"-",[@[Net Debt]]/[@[TTM EBITDA]],"-")
```

信用违约互换（Credit Default Swap，CDS）是一种超出本书范围的、复杂的信用衍生工具。可以这么说，CDS是对债券（或其他有价证券）进行的保险，投资者可以买（例如，得到风险保护）或者卖（例如，为别人提供风险保护）。风险保护的成本（即息差）是市场对公司业绩预测的重要指标。在其他条件一样的情况下，CDS费率越大，潜在风险越大。用5yr CDS Spread Ticker列的彭博字段CDS_SPREAD_TICKER_5Y查询五年期CDS的名称缩写，如果存在，在5yr CDS Spread列输入如下公式获取CDS费率：

```
=IF([@[5yr CDS Spread Ticker]]="-"," ",BDP([@[5yr CDS Spread Ticker]]&" Corp","PX_LAST","Fill=-"))
```

与Bond和Loan工作表一样，主关键字（CompanyID）需要是唯一的。你可以按照前面Bond工作表中讲到的方法对重复之处突出显示。此外，在Category列填上每个公司的正确分类。

尽管大部分列来自于彭博，最重要的信息其实来自于你。例如，彭博会告诉你，根据全球行业分类标准（Global Industry Classification Standard，GICS），Apple是一家“技术硬件、存储和周边产品”（Technology Hardware，Storage，and Peripherals）公司。然而，彭博还会告诉你Western Digital是一家“技术硬件、存储和周边产品”公司，尽管两家公司差别很大。作为分析师，你自己必须拥有良好的判断，来确定进行业绩对比的不同公司究竟属于哪个类别。除了你自己进行分类之外，将数据与主观思考结合也将使你的分析更加强大。例如，彭博不能告诉你该公司的CFO有多坦诚，也不能告诉你是不是该相信该公司的预测。

3.4.3 引用和重写

下面的内容涉及从其他表引用相关信息，以及重写缺失的或错误的彭博信息。

引用

然后，我们修改工作表，从其他工作表导入列，例如包括Bond工作表中的公司全名（CompanyName）。尽管最常用的办法是Excel VLOOKUP函数，VLOOKUP还是有几个问题，见第2章。一个更好（但是有一点点复杂）的解决方案是结合Excel INDEX和MATCH函数。MATCH函数在一个区域搜索一个值，并返回其头寸。INDEX函数返回在某个头寸下一个单元格的值。在Bonds表增加一列，命名为“Company Name”，用如下公式：

```
=INDEX(Company[CompanyName],MATCH([@CompanyID],Company[CompanyID]
```

你可以在其他工作表随意增加多个引用，但是应当用不同颜色或者字体加粗等方式标明该数据来源于另一个工作表。还可以或者用数据验证（见第2章），或者在

Bond和Loan工作表包含一个标明CompanyID是否存在于Company工作表中的列。如果在Company工作表中找到该公司，则如下公式返回TRUE；否则返回FALSE：

```
=NOT(ISERROR(VLOOKUP([@CompanyID],Company[CompanyID],1,FALSE)))
```

重写

如果你发现缺失或者不正确的数据，直接更新工作表和替换彭博公式都将导致混乱。那么怎么处理呢？你可以创建重写工作表（Bond Override、Loan Override、Company Override），修改彭博返回的数据（或增加缺失数据）。这些重写工作表将包含你想修改的信息。

为Bond工作表创建一个重写，首先创建一个新的工作表，命名为Bond Override。然后在第一行的A和B列，分别增加列标题BondID和Override Date。然后，在第一行增加Bond表中的其他列标题，使它们可以被重写。然后，选择带列标题（包括BondID和Override Date）的单元格，并转换成Excel表（见第2章），命名为BondOverride。

要适应重写表，需要改变Bond工作表中的BDP函数公式。创建一个不依赖于列顺序的公式可能有点复杂。

对CompanyID的现有BDP公式

```
=BDP(Bond[@[BBID]:[BBID]], C$1, "Fill=-")
```

将变为：

```
=IF(IFERROR(VLOOKUP(Bond[@[BondID]:[BondID]],BondOverride,
MATCH(Bond[#Headers],[CompanyID],BondOverride[#Headers],0),
FALSE),"")="",BDP(Bond[@[BBID]:[BBID]], C$1, "Fill=-"),
VLOOKUP(Bond[@[BondID]:[BondID]],BondOverride,
MATCH(Bond[#Headers],[CompanyID],BondOverride[#Headers],0),
FALSE))
```

那可能看起来有点复杂，但分解之后其实挺简单的。如果BondID不存在于

BondOverride表中，或者如果BondOverride表中对应的CompanyID单元格（无论该单元格在哪一列）是空白的，公式将显示原始BDP函数的结果。

在第一部分中，公式在BondOverride表中搜索BondID（用VLOOKUP，因为我们始终知道列的位置），并返回与Bond表CompanyID列有相同列名的列的数据（用之前我们介绍过的MATCH函数）。如果导致错误（因为该列不存在，或者找不到BondID），将返回一个空白单元格。

然后，如果第一部分返回一个空白单元格，IF函数将返回原BDP函数的结果；否则，返回VLOOKUP的结果。调整后，BondOverride表上增加的行中的数据将出现在Bonds工作表上。对于贷款和公司增加重写工作表的过程是一样的。

3.5 途径3：彭博C#API

彭博提供一个功能强大的API，你可以用它在C#中获取之前在Excel中获取的信息。

3.5.1 设置Microsoft Access以用于C#

第一步，我们在Access中创建一个新的数据库。在这个数据库中创建四张表：Company、Bond、Loan和IDX。这些表将包含来自彭博的信息。第五张表用于将彭博字段的映射存储到数据库表和列中。

第二步，用表3-7中的模式创建Company表。将下表中带星号（*）的列设为主关键字。

在Microsoft Access中，当你将数据类型设为数字（Number），你需要在属性窗口将字段规模（Field Size）设为Double。

表3-7：Company表设计

字段名	数据类型	字段名	数据类型
CompanyID*	短文本	FCFToTotalDebt	数字
BBID	短文本	Price	数字

表3-7：Company表设计（续）

字段名	数据类型	字段名	数据类型
CompanyName	短文本	YrHi	数字
IsPrivate	短文本	YrLow	数字
Sector	短文本	CDS5YrTicker	短文本
Industry	短文本	NetDebtToEBITDA	数字
SubIndustry	短文本	CDSSpread5Yr	数字
SPRating	短文本	Category	短文本
MoodyRating	短文本	CompanyComments	长文本
MarketCap	数字	PERatio	数字
TotalDebt	数字	DVDYield	数字
NetDebt	数字	TotalReturn12M	数字
EV	数字	PxChgYTD	数字
EBITDA	数字	PxChg3M	数字
TotalRevenue	数字	RecBuy	数字
TotalDebtToEBITDA	数字	RecSell	数字
InterestCoverage	数字	RecHold	数字
FCF	数字		

第三步，用表3-8描述的模式创建Bond表。

表3-8：Bond表设计

字段名	数据类型	字段名	数据类型
BondID*	短文本	Rank	短文本
BBID	短文本	Price	数字
CompanyID	短文本	PxChgYTD	数字
SecurityDes	短文本	PxChg3M	数字
CpnType	短文本	YASSpread	数字
FixedCpn	数字	YASYield	数字
Maturity	日期 / 时间	IsCallable	短文本
MoodyRating	短文本	NextCallDate	日期 / 时间
SPRating	短文本	NextCallPrice	数字
Currency	短文本	BondComments	长文本

第四步，用表3-9中的模式创建Loan表。

表3-9：Loan表设计

字段名	数据类型	字段名	数据类型
LoanID*	短文本	Currency	短文本
BBID	短文本	Rank	短文本
CompanyID	短文本	Price	数字
SecurityDesc	短文本	PxChgYTD	数字
CpnType	短文本	PxChg3M	数字
Margin	数字	DM	数字
Floor	数字	Yield	数字
Index	短文本	IsCallable	短文本
Maturity	日期 / 时间	NextCallDate	日期 / 时间
MoodyRating	短文本	NextCallPrice	数字
SPRating	短文本	LoanComments	长文本

第五步，用表3-10中的模式创建Index表。

表3-10：Index表设计

字段名	数据类型	字段名	数据类型
IndexID*	短文本	YrLow	数字
BBID	短文本	PxChgYTD	数字
IndexName	短文本	PxChg3M	数字
Price	数字	TotalReturn12M	数字
YrHi	数字		

第六步，创建一个表，命名为Map，其中包含字段和我们的数据库表之间的映射（见表3-11）。DestTable和DestCol二者应共同被设为主关键字（即复合主关键字）。

表3-11：Map表设计

字段名	数据类型
DestTable*	短文本
DestCol*	短文本
BloombergFLD	短文本

最后，插入表3-12中的行。在本节后续部分，我们会讨论该映射将如何把彭博数据动态加载到你的Access数据库中。

表3-12：Map表数据

DestTable	DestCol	BloombergFLD
Company	CompanyName	NAME
Company	IsPrivate	COMPANY_IS_PRIVATE
Company	Sector	GICS_SECTOR_NAME
Company	Industry	GICS_INDUSTY_NAME
Company	SubIndustry	GICS_SUB_INDUSTY_NAME
Company	SPRating	RTG_SP_LT_LC_ISSUER_CREDIT
Company	MoodyRating	RTG_MDY_LT_CORP_FAMILY
Company	MarketCap	CRNCY_ADJ_MKT_CAP
Company	TotalDebt	SHORT_AND_LONG_TERM_DEBT
Company	NetDebt	NET_DEBT
Company	EV	CRNCY_ADJ_CURR_EV
Company	EBITDA	TRAIL_12M_EBITDA
Company	TotalRevenue	SALES_REV_TURN
Company	TotalDebtToEBITDA	TOT_DEBT_TO_EBITDA
Company	InterestCoverage	INTEREST_COVERAGE_RATIO
Company	FCF	TRAIL_12M_FREE_CASH_FLOW
Company	FCFToTotalDebt	FCF_TO_TOTAL_DEBT
Company	Price	CRNCY_ADJ_PX_LAST
Company	YrHi	CHG_PCT_HIGH_52WEEK
Company	YrLow	CHG_PCT_LOW_52WEEK
Company	CDS5YrTicker	CDS_SPREAD_TICKER_5Y
Company	PERatio	PE_RATIO
Company	DVDYield	EQY_DVD_YLD_IND
Company	TotalReturn12M	CURRENT_TRR_1YR
Company	PxChgYTD	CHG_PCT_YTD
Company	PxChg3M	CHG_PCT_3M
Company	RecBuy	TOT_BUY_REC
Company	RecSell	TOT_SELL_REC
Company	RecHold	TOT_HOLD_REC
Bond	CompanyID	BOND_TO_EQY_TICKER
Bond	SecurityDes	SECURITY_DES
Bond	CpnType	CPN_TYP
Bond	FixedCpn	CPN
Bond	Maturity	MATURITY

表3-12：Map表数据（续）

DestTable	DestCol	BloombergFLD
Bond	MoodyRating	RTG_MOODY
Bond	SPRating	RTG_SP
Bond	Currency	QUOTED_CRNCY
Bond	Rank	PAYMENT_RANK
Bond	Price	PX_LAST
Bond	PxChgYTD	CHG_PCT_YTD
Bond	PxChg3M	CHG_PCT_3M
Bond	YASSpread	YAS_YLD_SPREAD
Bond	YASYield	YAS_BOND_YLD
Bond	IsCallable	CALLABLE
Bond	NextCallDate	NXT_CALL_DT
Bond	NextCallPrice	NXT_CALL_PX
Loan	CompanyID	ISSUER_PARENT_EQY_TICKER
Loan	SecurityDesc	SECURITY_DES
Loan	CpnType	CPN_TYP
Loan	Margin	LN_CURRENT_MARGIN
Loan	Floor	INDEX_FLOOR
Loan	Index	RESET_IDX
Loan	Maturity	MATURITY
Loan	MoodyRating	RTG_MOODY
Loan	SPRating	RTG_SP
Loan	Currency	QUOTED_CRNCY
Loan	Rank	PAYMENT_RANK
Loan	Price	PX_LAST
Loan	PxChgYTD	CHG_PCT_YTD
Loan	PxChg3M	CHG_PCT_3M
Loan	DM	DISC_MRGN_ASK
Loan	Yield	YLD_YTM_ASK
Loan	IsCallable	CALLABLE
Loan	NextCallDate	NXT_CALL_DT
Loan	NextCallPrice	NXT_CALL_PX
Index	IndexName	NAME
Index	Price	PX_LAST
Index	YrHi	CHG_PCT_HIGH_52WEEK

表3-12：Map表数据（续）

DestTable	DestCol	BloombergFLD
Index	YrLow	CHG_PCT_LOW_52WEEK
Index	PxChgYTD	CHG_PCT_YTD
Index	PxChg3M	CHG_PCT_3M
Index	TotalReturn12M	CURRENT_TRR_1YR

3.5.2 彭博C#API

在本节中，我们讨论使用彭博的桌面API（C#语言编写）。本节介绍如何获取引用数据（类似于Excel中的BDP）和历史数据（类似于Excel中的BDH）。最后，展示

如何填入Access数据库。对于其他更高级的内容，彭博在WAPI<GO>屏幕上提供很多与其他API功能相关的文档。本节的大部分内容来自于彭博提供的示例和文档。

提醒一下，你会受到数据限制，因此建议不要对静态数据进行重复请求。

在写代码之前，访问彭博的WAPI<GO>屏幕，选择API Download Center，下载API库。点击桌面API旁边的Download按钮，保存并解压相关文件，一般存到C:\blp。

然后在Visual Studio中，右击你的项目，选择增加引用（Add Reference），创建一个新的C#控制台应用（console application）。浏览C:\blp\DAPI\APIv3\DotnetAPI\v3.10.1.2\lib（使用其他版本或本地blp文件夹时你可能需要调整），然后添加一个引用至BloombergIp.Blpapi.dll。

然后在Program.cs文件中，增加正确的using指令，以在代码中引用Bloomberg类：

```
using Event = BloombergIp.Blpapi.Event; using Element =
BloombergIp.Blpapi.Element; using Message =
BloombergIp.Blpapi.Message; using Name = BloombergIp.Blpapi.Name;
using Request = BloombergIp.Blpapi.Request; using Service =
BloombergIp.Blpapi.Service; using Session =
BloombergIp.Blpapi.Session; using SessionOptions =
BloombergIp.Blpapi.SessionOptions; using InvalidRequestException =
BloombergIp.Blpapi.InvalidRequestException; using Datetime =
BloombergIp.Blpapi.Datetime;
```

彭博推荐用GetElement和GetValue方法预先计算哈希值，也就是在Program类中加入如下代码：

```
class Program { private static readonly Name SECURITY_DATA =
new Name("securityData"); private static readonly Name SECURITY =
new Name("security"); private static readonly Name FIELD_DATA = new
Name("fieldData"); private static readonly Name RESPONSE_ERROR =
```

```
new Name("responseError"); private static readonly Name
SECURITY_ERROR = new Name("securityError"); private static readonly
Name FIELD_EXCEPTIONS = new Name("fieldExceptions"); private static
readonly Name FIELD_ID = new Name("fieldId"); private static
readonly Name ERROR_INFO = new Name("errorInfo"); private static
readonly Name CATEGORY = new Name("category"); private static
readonly Name MESSAGE = new Name("message");
```

最后，要在Program类的一个实例中，而非静态的Main函数中运行代码，就需要创建一个方法，在Program类中命名为BasicExample，从Main方法调用它：

```
static void Main(string[] args) { Program p = new Program();
p.BasicExample(); }
```

3.5.3 基本引用示例

BasicExample方法与彭博建立了关系，并发送数据请求到彭博的引用数据服务。来自彭博的引用数据服务的回复在下面介绍的ProcessResponse方法中进行处理。用API与彭博建立关系分成两个步骤，首先如下面的BasicExample方法代码示例所示，程序必须建立一个连接至你的彭博终端的会话控制（session），然后session代码用于获取彭博的一项服务。

所有的彭博数据都必须从其“服务”（service）之一获取。在引用数据时，使用其refdata服务。BasicExample方法尝试用OpenService方法打开refdata服务，如果成功就用GetService方法获取Service对象。

然后，用Service对象的CreateRequest方法（其中包含要获取的有价值证券和字段的表单），代码会创建一个Request对象。彭博推荐将多个有价值证券和字段请求打包发送，而非发送多个请求。

然后，把有价值证券贴到Request对象的字段元素上，使代码在请求中增加一个有价值证券表单。指定有价值证券可以有多重方式；下面的代码示例展示如何用CUSIP引用债券，如何用股票名称缩写引用公司。类似于“/cusip”和“/ticker”，“/isin”或“/bbgid”也可用于识别债券，分别

用的是ISIN或彭博全局标识符 (Bloomberg Global Identifier)。

然后，把字段贴到Request对象的field元素上，使请求增加字段。API用的字段名与Excel BDP函数一样。

如这个例子所示，要重写一个字段，例如把CRNCY_ADJ_MKT_CAP按欧元计价，把字段贴到Request对象的overrides元素上，并在发送请求前设置fieldId和value。

然后，代码发送请求至彭博，并捕获任何非法请求异常。

发送请求之后，代码用NextEvent方法轮询 (poll) Session对象，寻求反馈。对于较大的请求来说，API有可能返回部分反馈 (partial response)，在这种情况下应当继续轮调Session对象。然而，在API发送回一个非部分反馈 (或者会话状态反馈为结束) 之后，结束轮询并结束会话。在收到任何反馈之后，把事件传递给ProcessResponse方法 (后面会介绍)。

让我们看看代码：

```
private void BasicExample() { //Establish a Bloomberg Session,
//otherwise display error and exit SessionOptions sessionOptions =
new SessionOptions(); Session session = new Session(); bool
sessionStarted = session.Start(); if (!sessionStarted) {
System.Console.Error.WriteLine("Failed to start session."); return;
} //Open RefData Bloomberg Service if
(!session.OpenService("//blp/refdata")) {
System.Console.Error.WriteLine("Failed to open //blp/refdata");
return; } Service refDataService =
session.GetService("//blp/refdata"); // Create new request Request
request = refDataService.CreateRequest("ReferenceDataRequest");
//Add securities to request Element securities =
request.GetElement("securities");
securities.AppendValue("/ticker/AAPL US Equity");
securities.AppendValue("/cusip/319963BP8"); //Add Bloomberg Fields
```



```

to request Element fields = request.GetElement("fields");
fields.AppendValue("CRNCY_ADJ_MKT_CAP");
fields.AppendValue("PX_LAST"); //Adding override to request
//following 4 statements set EQY_FUND_CRNCY=EUR Element overrides =
request["overrides"]; Element override1 =
overrides.AppendElement(); override1.SetElement("fieldId",
"EQY_FUND_CRNCY"); override1.SetElement("value", "EUR"); //Send
Request try { session.SendRequest(request, null); } catch
(InvalidRequestException e) {
System.Console.WriteLine(e.ToString()); } //While we haven't
errored //or retrieved entire response bool done = false; while
(!done) { Event eventObj = session.NextEvent(); //A partial
response will be followed by // more partial responses or completed
response if (eventObj.Type == Event.EventType.PARTIAL_RESPONSE) {
ProcessResponse(eventObj); } //Most of the time, you just get a
completed response else if (eventObj.Type ==
Event.EventType.RESPONSE) { ProcessResponse(eventObj); done = true;
} else { foreach (Message msg in eventObj) {
System.Console.WriteLine(msg.AsElement); if (eventObj.Type ==
Event.EventType.SESSION_STATUS) { if
(msg.MessageType.Equals("SessionTerminated")) { done = true; } } }
} } session.Stop(); }

```

下一个方法是ProcessResponse方法。该方法处理前面提到的BasicExample方法中返回的彭博数据。在传递给ProcessResponse方法的Event对象中，有一系列Message对象。每个Message对象可以包含一个错误（表示请求失败），或者一个有价证券元素集合。每个有价证券元素包含一个错误（表示该有价证券有问题），或者一个字段集合和一个字段错误集合（可能有）。ProcessResponse函数应循环访问这些不同的集合，以获得返回值。

如果信息没有错误，代码将循环访问每个有价证券元素，提取返回字段的表单，并循环访问。尽管下面的代码将字段的值作为字符串检索，也有作为其他数据类型的方法（在本章后面讨论）。

最后，对每个有价证券元素，检查是否有一个字段异常的集合并显示。

代码如下：

```
private void ProcessResponse(Event eventObj) { //Responses can
contain multiple messages foreach (Message msg in eventObj) { // if
the message is an error, display and continue if
(msg.HasElement(RESPONSE_ERROR)) { Element error =
msg.GetElement(RESPONSE_ERROR); Console.WriteLine("Request failed:
" + error.GetElementAsString(CATEGORY) + " (" +
error.GetElementAsString(MESSAGE) + ")"); continue; } // For each
of the requested securities in the response Element securities =
msg.GetElement(SEcurity_DATA); for (int i = 0; i <
securities.NumValues; ++i) { //Create references for security
object and ticker Element security =
securities.GetValueAsElement(i); string ticker =
security.GetElementAsString(SEcurity); //If security has error,
display and continue if (security.HasElement("securityError")) {
Element error = security.GetElement(SEcurity_ERROR);
Console.WriteLine("Security Error: " +
error.GetElementAsString(CATEGORY) + " (" +
error.GetElementAsString(MESSAGE) + ")"); continue; } //For each
field in request // Display data from Bloomberg Element fields =
security.GetElement(FIELD_DATA); if (fields.NumElements > 0) { for
(int j = 0; j < fields.NumElements; ++j) { Element field =
fields.GetElement(j); System.Console.WriteLine(field.Name + "\t\t"
+ field.GetValueAsString()); } } //If there were exceptions with
particular fields //Display them Element fieldExceptions =
security.GetElement(FIELD_EXCEPTIONS); if
(fieldExceptions.NumValues > 0) { for (int k = 0; k <
fieldExceptions.NumValues; ++k) { Element fieldException =
fieldExceptions.GetValueAsElement(k); Element error =
fieldException.GetElement(ERROR_INFO); Console.WriteLine("Field
```

```
Exception: " + fieldException.GetElementAsString(FIELD_ID) + " " +
error.GetElementAsString(CATEGORY) + " (" +
error.GetElementAsString(MESSAGE) + ")"); } } } }
```

3.5.4 基本历史示例

为了仿真Excel BDH函数和提取彭博字段信息历史需要一些调整。首先，在Program类的主体中增加"date"的预先计算好的哈希值：

```
private static readonly Name DATE = new Name("date");
```

然后，对请求声明进行如下修改：

```
Request request =
refDataService.CreateRequest("HistoricalDataRequest");
```

然后，对历史请求设置几个额外的实参。在本章之前的部分中，我们讨论了BDH函数的几个实参。对等的API实参参见表3-13。

表3-13：历史请求的API实参

实参	BDH 对等	备注
startDate	—	起始日期 (YYYYMMDD 格式)
endDate	—	结束日期 (YYYYMMDD 格式)
periodicityAdjustment	—	ACTUAL、CALENDAR 或 FISCAL
periodicitySelection	—	DAILY、WEEKLY、MONTHLY、QUARTERLY、SEMI_ANNUALLY、YEARLY

表3-13：历史请求的API实参（续）

实参	BDH 对等	备注
currency	FX	三字节 ISO 代码；例如 USD、GBP
overrideOption	Quote	设为 OVERRIDE_OPTION_GPA 以便在报价计算中使用平均价格，而非收盘价格
pricingOption	QuoteType	PRICING_OPTION_PRICE（对于价格），PRICING_OPTION_YIELD（对于收益）
nonTradingDayFillOption	Days	NON_TRADING_WEEKDAYS、ALL_CALENDAR_DAYS、ACTIVE_DAYS_ONLY
nonTradingDayFillMethod	Fill	PREVIOUS_VALUE 或 NIL_VALUE（对于空白）
adjustmentNormal	CshAdjNormal	设为 true 或 false
adjustmentAbnormal	CshAdjAbnormal	设为 true 或 false
adjustmentSplit	CapChg	设为 true 或 false
adjustmentFollowDPDF	UseDPDF	设为 true 或 false
calendarCodeOverride	CDR	例如 “US” 或 “JN”

在发送请求之前，在BasicExample函数中，把Request对象的startDate和endDate字段设为 “YYYYMMDD” 日期格式：

```
request.Set("startDate", "20140601"); request.Set("endDate", "20140625");
```

和Excel一样，可以在Request对象中将periodicitySelection字段设为 DAILY、WEEKLY、MONTHLY、QUARTERLY或YEARLY：

```
request.Set("periodicitySelection", "DAILY");
```

整个方法代码如下：

```
private void HistoryExample() { SessionOptions sessionOptions =
new SessionOptions(); Session session = new Session(); bool
sessionStarted = session.Start(); if (!sessionStarted) {
System.Console.Error.WriteLine("Failed to start session."); return;
} if (!session.OpenService("//blp/refdata")) {
System.Console.Error.WriteLine("Failed to open //blp/refdata");
```

```

return; } Service refDataService =
session.GetService("//blp/refdata"); //Use HistoricalDataRequest
Request request =
refDataService.CreateRequest("HistoricalDataRequest"); Element
securities = request.GetElement("securities");
securities.AppendValue("/ticker/MSFT US Equity");
securities.AppendValue("/ticker/AAPL US Equity"); Element fields =
request.GetElement("fields"); fields.AppendValue("PX_LAST");
fields.AppendValue("PX_OPEN"); // Set Dates and period
request.Set("startDate", "20140601"); request.Set("endDate",
"20140612"); request.Set("periodicitySelection", "DAILY"); try {
session.SendRequest(request, null); } catch
(InvalidRequestException e) {
System.Console.WriteLine(e.ToString()); } bool done = false; while
(!done) { Event eventObj = session.NextEvent(); if (eventObj.Type
== Event.EventType.PARTIAL_RESPONSE) {
ProcessHistoryResponse(eventObj); } else if (eventObj.Type ==
Event.EventType.RESPONSE) { ProcessHistoryResponse(eventObj); done
= true; } else { foreach (Message msg in eventObj) {
System.Console.WriteLine(msg.AsElement); if (eventObj.Type ==
Event.EventType.SESSION_STATUS) { if
(msg.MessageType.Equals("SessionTerminated")) { done = true; } } }
} } session.Stop(); }

```

因为对历史数据请求的反馈不完全等同于对引用数据请求的反馈，因此你需要在while循环中将函数调用从ProcessResponse改为ProcessHistoryResponse。

ProcessHistoryResponse方法看起来很像ProcessResponse方法，但是两者有几处不同。一是与引用反馈不同，历史数据反馈中的SECURITY_DATA元素不是一个数组：它包括一个有价证券。二是FIELD_DATA包括一个值和一个日期。

代码将在FIELD_DATA集中循环，其中包含每天一个元素。在那些元素中的每一个都是每个请求字段的另一个Element集，包括日期本身。因为日期是元素集的一个

部分，它按照名称提取，并且在循环至其他字段的其他元素集时被忽略。

如下是完整代码：

```
private void ProcessHistoryResponse(Event eventObj) { foreach
(Message msg in eventObj) { if (msg.HasElement(RESPONSE_ERROR)) {
Element error = msg.GetElement(RESPONSE_ERROR);
Console.WriteLine("Request failed: " +
error.GetElementAsString(CATEGORY) + " (" +
error.GetElementAsString(MESSAGE) + ")"); continue; } Element
securityData = msg.GetElement(SEcurity_DATA); string security =
securityData.GetElement(SEcurity).GetValueAsString();
Console.WriteLine(security); Element fieldData =
securityData.GetElement(FIELD_DATA); if (fieldData.NumElements > 0)
{ for (int i = 0; i < fieldData.NumElements; i++) { Element element
= fieldData.GetValueAsElement(i); //Pull the date from the returned
field and display Datetime date =
element.GetElementAsDatetime(DATE);
Console.WriteLine(date.ToSystemDateTime().ToShortDateString());
//For the remaining fields (not DATE), display. for (int f = 0; f <
element.NumElements; f++) { Element field = element.GetElement(f);
if (!field.Name.Equals(DATE)) { Console.WriteLine(field.Name + " =
" + field.GetValueAsString()); } } } } }
```

3.5.5 填入Access数据库

在本节中，我们学习用彭博API填入数据库表。本节的代码在各自的表中循环访问债券、贷款和企业信息，并从彭博API中请求Map表中列出的彭博字段。Map表还包括对应列，用于存储来自彭博的信息结果。

开始之前

在开始之前，需要强调几点。首先，把C#连接到Microsoft Access，你需要安装OleDb驱动程序，版本要和你的Microsoft Access版本匹配。例如，用

Microsoft.ACE.OLEDB.12.0连接Microsoft Access 2016。

然后，在Visual Studio中为控制台应用创建一个新的C#解决方案，并在项目目录中输入你的Access数据库副本。然后用项目目录中的数据库文件，在Bond、Loan、Index和Company表中填入适当的标识符。在每张表中，填入主关键字列和BBID列，方法是或者手动填入标识符，或者从Excel中粘贴复制（提示：从Excel中复制几行，然后在Access的Home键上点击Paste，然后点击Paste Append）。把每个表中的其他列留成空白；它们都将用API填充数据。

创建类型数据集

有很多方法能够将C#代码连接到数据库，但是我喜欢用类型数据集，因为它简单，代码容易读，而且将数据库查询与代码进行了分隔。在Visual Studio中，右击你的项目，选择Add，选择New Item，然后选择DataSet。将新数据集命名为ADS.xsd。然后，在Server Explorer面板上，右击Data Connections并选择Add Connection。选择Microsoft Access Database File作为数据源，点击“Database file name”旁边的Browse按钮。然后，在项目目录上选择Access数据库文件，然后点击OK。接下来，在Server Explorer面板上，浏览并选择Tables文件夹中的表，并把它们拖动到你的新数据集中。你的数据集应该如图3-13所示。

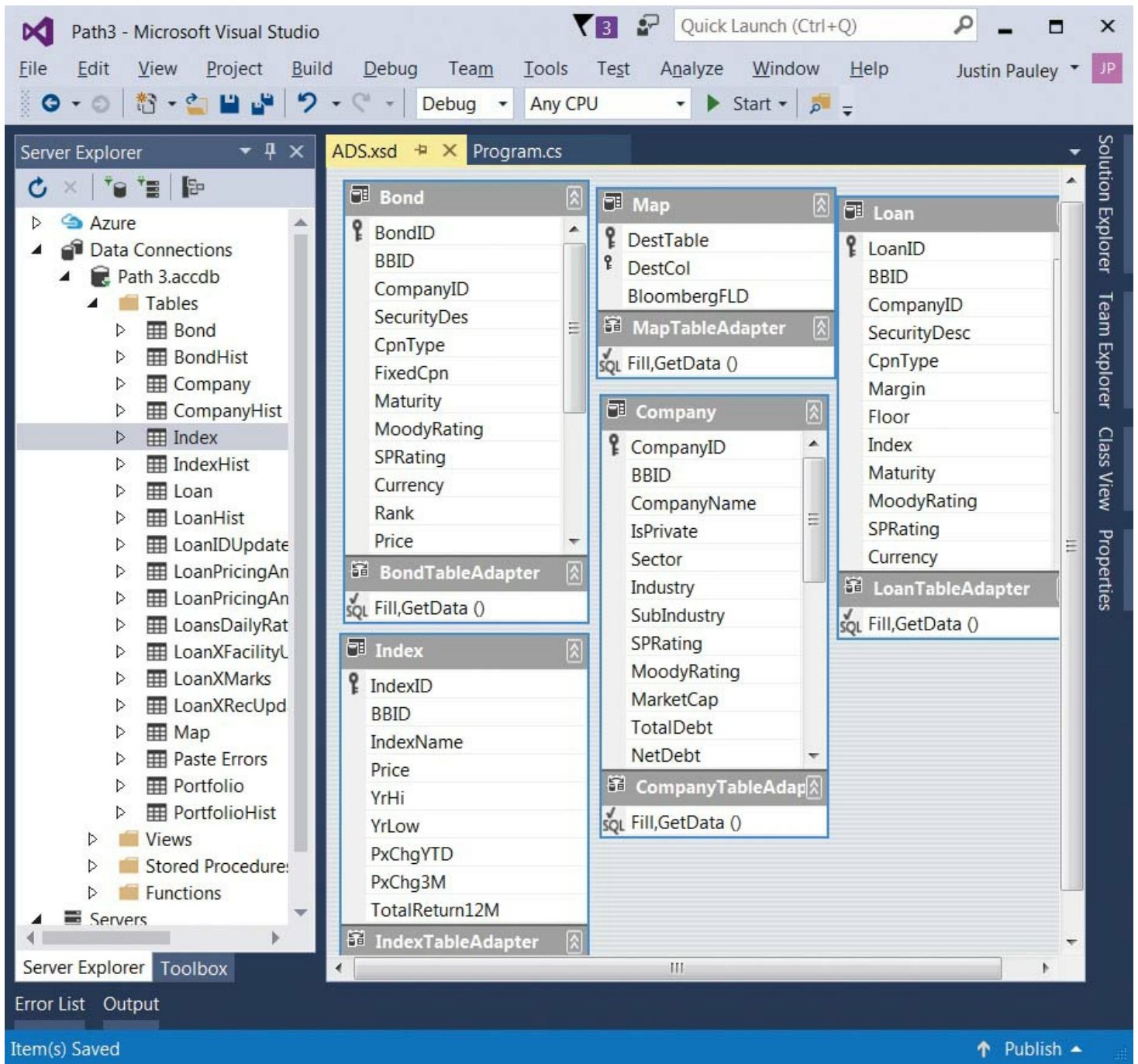


图3-13：ADS数据集

代码

该项目类似于我们用来获取引用数据的项目。首先增加与/b1p目录中的彭博.d11相同的引用，增加相同的using指令，增加相同的预先计算好的Name对象。在如下代码中，调用了Run方法，从Access填入数据集，提取债券、贷款和企业的数据，然后更新数据库：

```
using Event = BloombergIp.Blpapi.Event; using Element =
```

```

BloombergIp.Blpapi.Element; using Message =
BloombergIp.Blpapi.Message; using Name = BloombergIp.Blpapi.Name;
using Request = BloombergIp.Blpapi.Request; using Service =
BloombergIp.Blpapi.Service; using Session =
BloombergIp.Blpapi.Session; using DataType =
BloombergIp.Blpapi.Schema.Datatype; using SessionOptions =
BloombergIp.Blpapi.SessionOptions; using InvalidRequestException =
BloombergIp.Blpapi.InvalidRequestException; using System.Data;
namespace Path3_Load { class Program { private ADS DS = new ADS();
private static readonly Name SECURITY_DATA = new
Name("securityData"); private static readonly Name SECURITY = new
Name("security"); private static readonly Name FIELD_DATA = new
Name("fieldData"); private static readonly Name RESPONSE_ERROR =
new Name("responseError"); private static readonly Name
SECURITY_ERROR = new Name("securityError"); private static readonly
Name FIELD_EXCEPTIONS = new Name("fieldExceptions"); private static
readonly Name FIELD_ID = new Name("fieldId"); private static
readonly Name ERROR_INFO = new Name("errorInfo"); private static
readonly Name CATEGORY = new Name("category"); private static
readonly Name MESSAGE = new Name("message"); static void
Main(string[] args) { Program p = new Program(); p.Run(); } private
void Run() { FillDataSet(); RunBonds(); RunLoans(); RunCompanies();
RunIndex(); int rowc= UpdateDataSet(); }

```

FillDataSet方法比较简单，它用TableAdapterManager从Access填充数据集：

```

private void FillDataSet() { using
(ADSTableAdapters.TableAdapterManager tm = new
ADSTableAdapters.TableAdapterManager()) { tm.BondTableAdapter = new
ADSTableAdapters.BondTableAdapter(); tm.LoanTableAdapter = new
ADSTableAdapters.LoanTableAdapter(); tm.CompanyTableAdapter = new
ADSTableAdapters.CompanyTableAdapter(); tm.IndexTableAdapter = new

```

```

ADSTableAdapters.IndexTableAdapter(); tm.MapTableAdapter = new
ADSTableAdapters.MapTableAdapter();
tm.BondTableAdapter.Fill(DS.Bond);
tm.LoanTableAdapter.Fill(DS.Loan);
tm.CompanyTableAdapter.Fill(DS.Company);
tm.IndexTableAdapter.Fill(DS.Index);
tm.MapTableAdapter.Fill(DS.Map); tm.Connection.Close(); } }

```

在RunBonds方法中，循环访问数据库中的债券表单，将用于引用每个债券的有价证券标识符和对应的债券数据行填入一个Dictionary对象（secIds）。然后用Map表，将用于存储被请求信息的彭博字段和对应目标列填入另一个Dictionary对象（fields）。最后，将两个Dictionary对象传递给一个通用函数FetchData，由该类函数运行实际请求：

```

private void RunBonds() { Dictionary<string,DataRow> secIds =
new Dictionary<string, DataRow>(); Dictionary<string, string>
fields = new Dictionary<string, string>(); foreach (ADS.BondRow
bond in DS.Bond) { secIds.Add("/isin/" + bond.BondID,bond); }
foreach (ADS.MapRow map in DS.Map.Where(x => x.DestTable ==
"Bond")) { fields.Add(map.BloombergFLD, map.DestCol); }
FetchData(secIds, fields, "Bond"); }

```

FetchData方法创建一个彭博会话，得到引用数据服务的一个实例，从两个Dictionary对象（secIds和fields）向彭博请求增加有价证券标识符和字段表单，并发送请求。该方法还接受一个KeyValuePair的可选数组，以增加重写（后面用于获取公司数据）。最后，该方法轮询Session代码寻求反馈，在得到反馈或部分反馈时将Event代码以及Dictionary对象和表名传递至ProcessResponse方法：

```

private void FetchData( Dictionary<string, DataRow> secIds,
Dictionary<string, string> fields, string table, params
KeyValuePair<string, string>[] overrides) { // Open session and
service Session session = new Session(); bool sessionStarted =
session.Start(); if (!sessionStarted) {

```

```

System.Console.Error.WriteLine("Failed to start session."); return;
} if (!session.OpenService("//blp/refdata")) {
System.Console.Error.WriteLine("Failed to open //blp/refdata");
return; } Service refDataService =
session.GetService("//blp/refdata"); // Create request Request
request = refDataService.CreateRequest("ReferenceDataRequest"); //
add each security to the request Element securities =
request.GetElement("securities"); foreach (string id in
secIds.Keys) { securities.AppendValue(id); } // add each field to
the request Element requestedFields = request.GetElement("fields");
foreach (string field in fields.Keys) {
requestedFields.AppendValue(field); } // Optionally, if there are
overrides, add them to request if (overrides != null) { Element
overrideElement = request["overrides"]; foreach
(KeyValuePair<string, string> or in overrides) { Element o =
overrideElement.AppendElement(); o.SetElement("fieldId", or.Key);
o.SetElement("value", or.Value); } } // send the request try {
session.SendRequest(request, null); } catch
(InvalidRequestException e) {
System.Console.WriteLine(e.ToString()); } // Process the response
bool done = false; while (!done) { Event eventObj =
session.NextEvent(); if (eventObj.Type ==
Event.EventType.PARTIAL_RESPONSE) { ProcessResponse(eventObj,
fields, table, secIds); } else if (eventObj.Type ==
Event.EventType.RESPONSE) { ProcessResponse(eventObj, fields,
table, secIds); done = true; } else { foreach (Message msg in
eventObj) { System.Console.WriteLine(msg.AsElement); if
(eventObj.Type == Event.EventType.SESSION_STATUS) { if
(msg.MessageType.Equals("SessionTerminated")) { done = true; } } }
} } session.Stop(); }

```

ProcessResponse方法用来自彭博反馈的数据填入Dictionary对象secIds。

随着ProcessResponse循环至被请求数据，它用被请求的股票代码缩写查询在RunBonds方法中创建的secIds Dictionary的DataRow。

然后，代码提取返回字段的表单，用该字段的Dictionary获取对应的DataTable列，然后将DataRow对象的内容清空（即设为null）。然后，代码检查彭博返回的字段的“Datatype”属性，以确定获取返回值的恰当方法。例如，如果field.DataType被设置为DataType.FLOAT64，那么它就用field.GetValueAsFloat64（）方法；如果field.DataType被设置为DataType.DATETIME，那么它就用Field.GetValueAsDatetime（）方法。在将DataRow中的列设置成返回值之前，还要检查ADS数据集中的DataColumn是否会接受返回的数据类型值：

```
private void ProcessResponse( Event eventObj,
Dictionary<string, string> fields, string table, Dictionary<string,
DataRow> secIds ) { foreach (Message msg in eventObj) { if
(msg.HasElement(RESPONSE_ERROR)) { Element error =
msg.GetElement(RESPONSE_ERROR); Console.WriteLine("Request failed:
" + error.GetElementAsString(CATEGORY) + " (" +
error.GetElementAsString(MESSAGE) + ")"); continue; } Element
securities = msg.GetElement(SEcurity_DATA); for (int i = 0; i <
securities.NumValues; ++i) { Element security =
securities.GetValueAsElement(i); if
(security.HasElement("securityError")) { Element error =
security.GetElement(SEcurity_ERROR); Console.WriteLine("Security
Error: " + error.GetElementAsString(CATEGORY) + " (" +
error.GetElementAsString(MESSAGE) + ")"); continue; } Element
fieldExceptions = security.GetElement(FIELD_EXCEPTIONS); if
(fieldExceptions.NumValues > 0) { for (int k = 0; k <
fieldExceptions.NumValues; ++k) { Element fieldException =
fieldExceptions.GetValueAsElement(k); Element error =
fieldException.GetElement(ERROR_INFO); Console.WriteLine("Field
Exception: " + fieldException.GetElementAsString(FIELD_ID) + " " +
error.GetElementAsString(CATEGORY) + " (" +
```



```

error.GetElementAsString(MESSAGE) + ")"); } } // pulls the ticker
that we used in our request, like /ISIN/123123434 string ticker =
security.GetElementAsString(SEcurity); // the row from our dataset
with this id. DataRow row = secIds[ticker]; Element fieldElements =
security.GetElement(FIELD_DATA); if (fieldElements.NumElements > 0)
{ for (int j = 0; j < fieldElements.NumElements; ++j) { Element
field = fieldElements.GetElement(j); //Using the Map table to find
the destination column. DataColumn dc =
row.Table.Columns[fields[field.Name.ToString()]]; row[dc] =
DBNull.Value; switch (field.Datatype) { case DataType.BOOL: if
(dc.DataType == typeof(bool)) { row[dc] = field.GetValueAsBool(); }
break; case DataType.DATE: if (dc.DataType == typeof(DateTime)) {
row[dc] = field.GetValueAsDate().ToSystemDateTime(); } break; case
DataType.DATETIME: if (dc.DataType == typeof(DateTime)) { row[dc] =
field.GetValueAsDatetime().ToSystem DateTime(); } break; case
DataType.FLOAT32: if (dc.DataType == typeof(double)) { row[dc] =
Convert.ToDouble(field.GetValueAsFloat32()); } break; case
DataType.FLOAT64: if (dc.DataType == typeof(double)) { row[dc] =
field.GetValueAsFloat64(); } break; case DataType.INT32: if
(dc.DataType == typeof(int)) { row[dc] = field.GetValueAsInt32(); }
break; case DataType.INT64: if (dc.DataType == typeof(long)) {
row[dc] = field.GetValueAsInt64(); } break; case DataType.STRING:
if (dc.DataType == typeof(string)) { row[dc] =
field.GetValueAsString(); } break; case DataType.CHAR: if
(dc.DataType == typeof(char)) { row[dc] = field.GetValueAsChar(); }
else if (dc.DataType == typeof(string)) { row[dc] =
Convert.ToString(field.GetValueAsChar()); } else if (dc.DataType ==
typeof(bool)) { char c = field.GetValueAsChar(); if (c == 'Y')
row[dc] = true; else if (c == 'N') row[dc] = false; } break;
default: Console.WriteLine("Missing Column Type"); break; }
System.Console.WriteLine(field.Name + "\t\t" +
field.GetValueAsString()); } } } } }

```

RunLoans类似于RunBonds方法，除了它用股票名称缩写来识别贷款，而非用ISIN：

```
private void RunLoans() { Dictionary<string, DataRow> secIds =
new Dictionary<string, DataRow>(); Dictionary<string, string>
fields = new Dictionary<string, string>(); foreach (ADS.LoanRow
loan in DS.Loan) { secIds.Add("/ticker/" + loan.LoanID + " Corp",
loan); } foreach (ADS.MapRow map in DS.Map.Where(x => x.DestTable
== "Loan")) { fields.Add(map.BloombergFLD, map.DestCol); }
FetchData(secIds, fields, "Loan"); }
```

RunCompanies方法类似于RunLoans方法，除了将KeyValuePair传递给FetchData方法，以重写EQY_FUND_CRNCY。此外，还包括一个post-fetching代码部分，它用于设置数据库中的NetDebtToEBITDA列并用五年CDS股票名称缩写（如果存在）创建一个新的五年CDS费率请求。如下代码展示如何扩展其他函数，以增加不同的定制计算。

```
private void RunCompanies() { Dictionary<string, DataRow>
secIds = new Dictionary<string, DataRow>(); Dictionary<string,
string> fields = new Dictionary<string, string>(); foreach
(ADS.CompanyRow company in DS.Company) { secIds.Add("/ticker/" +
company.CompanyID + " Equity", company); } foreach (ADS.MapRow map
in DS.Map.Where(x => x.DestTable == "Company")) {
fields.Add(map.BloombergFLD, map.DestCol); } FetchData(secIds,
fields, "Company", new KeyValuePair<string, string>(
"EQY_FUND_CRNCY", "USD")); // Post-fetching tasks secIds.Clear();
fields.Clear(); foreach (ADS.CompanyRow company in DS.Company) {
//Set Net Debt/EBITDA if (company.IsNetDebtNull() == false &&
company.IsEBITDANull() == false && company.EBITDA > 0) {
company.NetDebtToEBITDA = company.NetDebt / company.EBITDA; } //
5yr cds if(company.IsCDS5YrTickerNull() == false) {
secIds.Add("/ticker/" + company.CDS5YrTicker + " Corp", company); }
} if(secIds.Count >0) { fields.Add("PX_LAST",
```

```
DS.Company.CDSSpread5YrColumn.ColumnName); FetchData(secIds, fields, "Company"); } }
```

RunIndex方法与RunLoans方法很相似，除了引用指数时在IndexID后面写上“Index”，而非“Corp”：

```
private void RunIndex() { Dictionary<string, DataRow> secIds = new Dictionary<string, DataRow>(); Dictionary<string, string> fields = new Dictionary<string, string>(); foreach (ADS.IndexRow index in DS.Index) { secIds.Add("/ticker/" + index.IndexID + "Index", index); } foreach (ADS.MapRow map in DS.Map.Where(x => x.DestTable == "Index")) { fields.Add(map.BloombergFLD, map.DestCol); } FetchData(secIds, fields, "Index"); }
```

最后一个方法是UpdateDataSet，它将更新的数据集保存回Access，并返回更新的行数：

```
private int UpdateDataSet() { int rowc = 0; using (ADSTableAdapters.TableAdapterManager tm = new ADSTableAdapters.TableAdapterManager()) { tm.BondTableAdapter = new ADSTableAdapters.BondTableAdapter(); tm.LoanTableAdapter = new ADSTableAdapters.LoanTableAdapter(); tm.CompanyTableAdapter = new ADSTableAdapters.CompanyTableAdapter(); tm.IndexTableAdapter = new ADSTableAdapters.IndexTableAdapter(); rowc = tm.UpdateAll(DS); tm.Connection.Close(); } return rowc; }
```

在Visual Studio内部运行代码时，它在你的项目的/bin/Debug文件夹创建数据库的本地副本，并将所有更新信息保存到该副本，而非更新你的项目目录中的版本。

3.6 本章小结

本章介绍了创建一个既能自动从彭博提取数据，又能方便存储的Excel工作簿或Access数据库的技术方法。尽管这对于金融分析已经很有用了，后面的章节还会介

绍数据分析和报告技术，以加强Excel工作簿或Access数据库。

第4章 IHS Markit：大企业数据

本章介绍如何从IHS Markit获取金融数据。IHS Markit是金融数据的最佳来源之一，其涉及领域包括企业债券、贷款以及其他一系列不同产品。与彭博不同，Markit为其客户提供几乎所有可交易债券和贷款的定价和引用相关数据的服务，没有什么限制。可以使用大规模的金融数据为进行多种有用的分析敞开了大门，例如基于息差、评级或其他属性，分析数千笔贷款的每日价格变动的趋势。许多银行、资产管理人和对冲基金都注册了Markit账户；然而，大部分数据都用于支持后台部门的内部系统运行，分析师的手上倒没有数据。

第8章讨论在大型数据集上进行不同种类的分析，而本章关注不同类型的数据、如何获取数据以及基于你采用的途径用Excel和Microsoft Access存储数据的技巧。

本章并未涉及IHS Markit提供的所有产品。想更了解Markit，请访问<https://www.markit.com>。在本章中用到的大部分字段描述来自于Markit文档。

4.1 企业贷款

企业贷款（也称为银行贷款、杠杆贷款或银团贷款）是企业发行的贷款，通常有非投资评级。Markit提供全球大部分可投资市场的参考数据（贷款信息、评级、标识符等）和业绩信息（定价、金融、分析等）。

4.1.1 数据请求

幸运的是，用Markit Loans Automated Data Exchange请求数据很方便。Markit提供的每种类型的数据被称为一个“渠道”（channel），你可以用自定义的Web请求，以逗号分隔值（Comma-Separated Value，CSV）或可扩展标记语言（Extensible Markup Language，XML）格式获取。Web请求就是一个包含参数（例如公司、用户名、密码和其他选项等）的URL（网站）。例如，获取CSV格式的

贷款价格（LoanXMarks渠道），只要在网页浏览器中访问如下URL就可以了：

```
https://loans.markit.com/loanx/LoanXMarks.csv?  
LEGALENTITY=firmname &USERNAME=user1&PASSWORD=password1
```

该URL会下载一个CSV文件，可以用Microsoft Excel打开。把该URL的.csv扩展名改为.xml，会把文件改为XML文件，这样既可以人类阅读，也可以机器阅读（设计为由应用阅读）。如果参数（例如你的用户名）包含特殊字符，必须编码避免出现问题；例如，你需要用“%2F”，而非“/”。在互联网上搜索“URL encoder”，会找到很多相关网站，帮助你编码。

 你可以从Markit的网站（<http://www.markit.com>）手动下载本节谈到的大部分信息。

4.1.2 贷款信息

Facility Update渠道将返回自上次请求以来被更新的每笔贷款的参考数据；联系support@markit.com获取贷款数据的初始文件。由此，如果你连着发出两个请求，中间没有间歇，则第二个请求返回的结果为空。用如下URL语法获取CSV格式的机构数据：

```
https://loans.markit.com/loanx/LoanXFacilityUpdates.csv?  
LEGALENTITY= mylegalentity&USERNAME=user1&PASSWORD=mypassword
```

返回的文件将包含表4-1中的列。因为Markit与标准普尔有合作关系，带星号（*）的列只能由也与标准普尔有协议的客户获取。此外，带双星号（**）的列只能通过特定请求从Markit（你可以联系support@markit.com）获得，才会出现在结果中。带三个星号（***）的列需要与标准普尔有协议，且向Markit发送特定请求。

表4-1：Facility Update列

列	说明
LoanX ID	每笔贷款的唯一标识符
PMD ID	与某特定发行人 / 分级组合相关的唯一标识符。可以是正数、也可以是负数
PMD Trans ID*	PMD/LCD 确认一笔交易或贷款计划的唯一标识符
Issuer Name	借款人或发行人的名称
Issuer ID*	与某特定发行人相关的唯一标识符
Deal Name	借款人的名称。通常与前面的单元格一样，但是可以包含交易日期和类型

表4-1：Facility Update列（续）

列	说明
Facility Type	特定贷款类型：TLB、过桥贷款等
LoanX Facility Type	Markit 将 PMD Facility Type 合并成当前 16 个标准值中的一个
Facility Status*	特定工具类型：例如过桥贷款，364 天贷款，次级贷款，定期贷款折旧等
LoanX Facility Type Code*	LoanX Facility Type + LoanX Facility Category 的代码标志
LoanX Facility Category*	Markit 将 PMD Facility Type 简化成如下之一：Institutional、RC、TLA、Other
Industry	基于 SIC 代码的行业划分
Initial Amount	机构金额（百万计）
Initial Spread	初始 LIBOR 息差
Maturity Date	到期日期
Ticker***	发行人的名称缩写
Currency***	贷款使用的货币
LoanX Currency Code*	标准货币缩写代码
SP Org ID***	标准普尔指定组织 ID
Commitment Fee*	承诺费
Sponsor*	贷款担保人
LoanX Sponsor Code*	贷款担保人数字代码
Launch Date*	贷款的发行日期
Close Date*	贷款的截止日期
State*	发行人的状态
Country*	发行人的国家
LoanX Country Code*	标准国家代码缩写
Pro Rata Assignment*	按比例分配最低额
Institutional Assignment*	按机构分配最低额
Pro Rata Fee*	按比例收费
Institutional Fee*	机构费用
Facility Fee*	为一笔贷款支付的年费总额
Consent*	代理、公司、二者
Security*	贷款担保资产
LoanX Security Code*	贷款担保资产标准代码缩写
Lead Agent*	牵头代理机构
LoanX Lead Agent Code*	与牵头代理机构相关的唯一标识符
Admin Agent*	行政代理机构

表4-1：Facility Update列（续）

列	说明
LoanX Admin Agent Code*	与行政代理机构相关的唯一标识符
Document Agent*	文件代理机构
LoanX Doc Agent Code*	与文件代理机构相关的唯一标识符
Syndicate Agent*	银团代理机构
LoanX Synd Agent Code*	与银团代理机构相关的唯一标识符
Initial SP Rating*	初始标准普尔评级
Industry Code*	行业代码
SIC Code*	SIC 代码
SIC Description*	SIC 说明
Industry Segment ID*	行业分类 ID
Industry Segment Description*	行业分类说明
Status Code*	内部状态代码
Status	内部状态说明
Cancelled*	表示交易已被取消的标志
Created Time	创建贷款记录的日期
Modified Time	修改贷款记录的日期
Term*	贷款期限（年）
RC Term*	RC 贷款期限（年）
TLA Term*	TLA 贷款期限（年）
TLB Term*	TLB 贷款期限（年）
TLD Term*	TLD 贷款期限（年）
OID*	受保险的贷款初始发行价格
Libor Floor*	低于特定地板价时支付的最低基准利率
Lien Type***	在借方资本结构下的留置权优先类型
Cov-Lite***	表示分级为不含借方保护条款的高风险贷款的标志

表4-1显示Facility Update数据包括大量有用信息，特别是对于标准普尔客户来说。你可以用很多方式使用该数据，例如，可以用它判断全球保险趋势（分评级或行业的息差和原始发行折扣[OID]）或者各个行业整体即将到来的到期日。LoanX ID列是主关键字，恰好对应一笔贷款。

Recommended Updates渠道提供关于再融资的信息，以及可能导致一笔贷款变

得不处于支付阶段的原因。与Facility Update渠道类似，Recommended Updates 渠道只会返回你上次请求以来新的记录。你可以用如下URL语法获取Recommended Updates：

```
https://loans.markit.com/loanx/LoanXRecUpdates.csv?  
LEGALENTITY=myfirm&USERNAME= user1&PASSWORD=mypassword
```

表4-2描述了Recommended Updates请求返回的列。

表4-2：推荐更新列

列	描述
LoanX ID	每笔贷款的唯一标识符
LCD ID	S&P LCD 数据的标识符
Issuer Name	借款人或发行人的名称
Dealname	借款人的名称。通常与前面的单元格一样，但是可能包括日期和交易类型
Facility Type	特定贷款类型；TLB，过桥贷款等
Industry	基于 SIC 代码的行业分类
Initial Amount	机构金额（百万计）
Final Maturity	最终到期日日期
Initial Spread	初始 LIBOR 息差
Facility Status	状态处于支付阶段 / 不处于支付阶段（A 或 I）
Inactive Date	状态改变日期，总是代表过去
Inactive Reason	状态改变的原因
Replacement LoanX ID	重置贷款的 LoanX ID
Replacement PMD ID	与一个特定发行人 / 分级组合相关的重置唯一标识符。可以是正数，也可以是负数
Replacement Issuer Name	重置借款人或发行人的名称
Replacement Deal Name	重置借款人的名称；通常与上一个单元格相同，但是可以包括日期和交易类型
Replacement Facility Type	重置贷款特定类型；TLB，过桥贷款等
Replacement Industry	基于 SIC 代码的重置行业分类
Replacement Initial Amount	重置贷款机构金额（百万计）
Replacement Final Maturity	重置贷款最后到期日期
Replacement Initial Spread	重置贷款最初 LIBOR 息差
Replacement Status	重置贷款机构状态

Recommended Update数据集中前面几列（LoanX ID至Facility Status）包括原始贷款在变得不处于支付阶段之前的具体信息。Inactive Date和Inactive Reason表示原始贷款变得不处于支付阶段的日期和原因。其余列（前面带有“Replacement”字样）是替代了原始贷款的新贷款，即重置贷款（如有）的详细信息。例如，如果一笔贷款的LoanX ID是LX123456，它进行了再融资，新贷款的LoanX ID是LX98765，就会有一行内容，前面的LoanX ID列是LX123456，后面的Replacement LoanX ID列是LX98765。如果替代列是空白的，就说明没有对应的新贷款。原始LoanX ID和Replacement LoanX ID都对应于Facility Update数据

中的行。重置贷款还有可能随后被再融资，作为LoanX ID出现在Recommend Update数据的另一行中。

尽管该表的目的是在贷款再融资或者结束时更新投资组合的最新信息，该数据还有许多其他用途。例如，你可以用该数据追踪再融资趋势，例如过去三个月中评级为B+的贷款所占的百分比和平均息差收紧情况。你还可以用它判断一笔典型贷款在最终到期日之前进行再融资的平均月数。

Daily Rating渠道针对每笔处于支付阶段的贷款提供穆迪和标准普尔的最新评级信息。使用如下URL语法请求最新的Daily Rating数据（仅CSV）：

```
https://loans.markit.com/loanx/LoansDailyRatings.csv?  
LEGALENTITY=firmname &USERNAME=user1&PASSWORD=pw239876
```

或者提供一个可选的DATE实参，用MM-DD-YY格式标明发送评级的日期。可以获得最近10个工作日和最近三个月月末的历史评级数据。例如，要获得2016年5月14日以来的每日评级，用如下：

```
https://loans.markit.com/loanx/LoansDailyRatings.csv?  
LEGALENTITY=firmname &USERNAME=user1&PASSWORD=pw239876&DATE=05-14-  
16
```

表4-3包括Daily Rating请求返回的列。

表4-3：Daily Rating级列

列	描述
As of Date	创建文件的日期
LoanX ID	每笔贷款的唯一标识符
Price Date	买卖报价日期
Moody's Rating	穆迪评级
Moody's Rating Date	穆迪评级上次更新评级的日期
Moody's Watch	来自穆迪评级的监视名单描述
Moody's Watch Date	上次更新监视名单的日期
Moody's Outlook	穆迪评级展望
Moody's Outlook Date	穆迪评级展望最后一次更新日期
S&P Rating	标准普尔评级
S&P Rating Date	标准普尔评级最后一次更新日期
S&P Watch	来自标准普尔评级的监视名单描述
S&P Watch Date	上次更新监视名单的日期
S&P Outlook	标准普尔评级展望
S&P Outlook Date	标准普尔评级展望最后一次更新日期

因为Markit提供评级更新的日期，所以你可以用该数据建构各笔贷款的历史，或者从更大处着眼，追踪一个行业、一系列公司的评级变化，或者确定贷款价格下降和评级下降之间的相关性。

LoanID Updates渠道提供从Markit的LoanX ID到CUSIP或其他标识符的映射。类似于Facility Update渠道，LoanID Updates渠道只返回从上次请求以来的新记录。用如下URL语法请求LoanID Updates：

```
https://loans.markit.com/loanx/LoanIDUpdates.csv?
LEGALENTITY=mylegalentity &USERNAME=user1&PASSWORD=mypassword
```

表4-4包含从LoanID Updates请求返回的列。

表4-4：LoanID Updates列

列	描述
Identifier	一个与某特定发行人 / 分级组合相关的行业标准唯一标识符（通常存储 CUSIP）
Identifier Type	标明标识符来源（例如 “CUSIP”）
LoanX ID	每笔贷款的唯一标识符
Valid From	映射标识符到 LoanX ID 的日期
Valid To	取消标识符到 LoanX ID 的映射关系的日期
Modified Time	修改标识符映射的日期

Identifier Type列包含 “CUSIP” 字样时，返回的数据集的Identifier列将包含一个CUSIP。LoanX ID和CUSIP之间的映射很重要，因为LoanX ID只存在于Markit中，而多套系统（包括彭博）使用CUSIP。提醒一下，CUSIP是CUSIP Global Services公司的注册产品，在数据库中存储和使用CUSIP可能需要预先得到授权。

4.1.3 贷款定价、金融和分析

用Marks渠道可以获取数以千计的美国和欧洲贷款的贷款价格。下面是基本的URL语法：

```
https://loans.markit.com/loanx/LoanXMarks.xml?
LEGALENTITY=firmname &USERNAME=user1&PASSWORD=password
```

该URL可以处理两个额外的参数。一是RELATIVEVERSION参数，用于请求过去最多10个工作日内数据。例如，RELATIVEVERSION值为-1将返回今天之前两个工作日的数值（“0”是默认值，返回前一个工作日的数值）：

```
https://loans.markit.com/loanx/LoanXMarks.xml?
LEGALENTITY=firmname
&USERNAME=user1&PASSWORD=password&RELATIVEVERSION=-1
```

北美买家可以将EOD参数设为Y，在美国东部时间下午4点以后查询当天的收盘价格：

<https://loans.markit.com/loanx/LoanXMarks.xml?>

LEGALENTITY=firmname &USERNAME=user1&PASSWORD=password&EOD=Y

欧洲客户要查询欧洲有价证券的收盘价可以在格林威治时间下午4点以后用另一个URL：

<https://loans.markit.com/loanx/LoanXMarksUK.csv?>

LEGALENTITY=firmname &USERNAME=eurouser&PASSWORD=password&EOD=Y

表4-5包含Marks渠道返回的列的描述。

表4-5：Marks列描述

列	描述
LoanXID	每笔贷款的唯一标识符
Mark Date	标注日期
Evaluated Price	收盘卖价 / 收盘买价之间的中间点
Bid	在标注日期对机构的平均卖价。根据交易日不同而变化
Offer	在标注日期对机构的平均买价。根据交易日不同而变化
Depth	Depth 一般表示参与的交易者的数量
Close Bid	东部时间下午 4 点时的收盘卖价。不会发生改变
Close Offer	东部时间下午 4 点时的收盘买价。不会发生改变
Close Date	收盘卖价和买价的日期
Contributed	如果公司参与平均记分，则返回 “Yes”

华尔街的许多公司每夜都用Markit贷款价格标记，不过该数据还可以用于许多地方。你可以用它回答问题，例如：

·现在有多少笔贷款的交易价格高于90美元，而曾经低于70美元？

·我们售出的贷款有没有卖出后价格下跌超过10%的？

·有哪些行业或者评级组的表现好于/弱于市场？

·今天、本周、本月、本季度、今年哪些贷款的波动最大？

·评级下降和价格变化之间是否强相关？

·哪些贷款按照票面价格交易，且当前可赎回？

更进一步，Markit的Loan Performance渠道对每笔贷款提供每日分析。用如下URL语法查询贷款业绩（仅CSV）：

```
https://loans.markit.com/loanx/LoanPricingAndAnalytics.csv?  
LEGALENTITY=firmname &USERNAME=user1&PASSWORD=password
```

返回的贷款业绩数据集超过200列，包含了其他渠道（facility、pricing、ratings）大部分有用的列，以及如下：

·每日、月度、年度收益。

·息差；久期；修正久期；PV01；剩余加权平均期限；到期收益率；一年、二年、三年、四年、五年赎回收益率。

·当前收益。

·30天、60天、90天平均出价。

·每日、月度、年度价格变化（按百分比和按小数表示）。

最后，Markit还可用Financial Statement渠道为Capital IQ客户获取标准普尔Capital IQ金融数据。用如下URL语法查询2007年1月以来的财务报表数据（仅CSV）：

```
https://loans.markit.com/loanx/FinancialStatement.csv?  
LEGALENTITY=firmname &USERNAME=user&PASSWORD=password
```

URL语法还有一个可选参数TIMEFRAME，你可以把它设置成LatestAvailable，以仅仅返回可获得的最新财务报表数据：

```
https://loans.markit.com/loanx/FinancialStatement.csv?  
LEGALENTITY=firmname
```

表4-6包含Financial Statement数据的列描述。

表4-6：Financial Statement列

列	描述
SP_COMPANY_ID	标准普尔 Capital IQ 标识符
Currency	财务数据货币
Year	财务报表年份
Quarter	财务报表季度
Is_Annual	表示为年度数据
Is_Latest	表示为最新可获得信息
Total_Sr_Secured_EBITDA	有担保债务总额 /EBITDA
Sr_Debt_EBITDA	优先债务 /EBITDA
Sr_Sub_Debt_EBITDA	高级次级债务 /EBITDA
Jr_Sub_Debt_EBITDA	初级次级债务 /EBITDA
Sub_Debt_EBITDA	次级债务 /EBITDA
Total_Debt_EBITDA	总债务 /EBITDA
Net_Debt_EBITDA	(总债务 – 现金和短期投资) /EBITDA
Total_Assets	总资产

表4-6：Financial Statement列（续）

列	描述
Revenue	收益
EBITDA	税息折旧及摊销前利润
Retained_Earnings	留存收益
EBITDA_INT	EBITDA/ 利息花费
Quick_Ratio	(总现金和短期投资 + 应收账款) / 当前资产总额
Current_Ratio	当前资产总额 / 当前负债总额
Total_Debt_Capital	债务总额 / 资本总额
Total_Debt_Equity	债务总额 / 权益总额

尽管第3章讨论了如何从彭博提取大量信息，Markit的这个功能却能提取大量

企业的完整历史。不幸的是，要将该信息连接到一个LoanX ID，你需要用如下语法对Financial Statement Map渠道单独进行请求（仅CSV）

```
https://loans.markit.com/loanx/FinancialStatementMap.csv?  
LEGALENTITY=firname &USERNAME=user&PASSWORD=xxxxx
```

该URL请求返回数据包含一个LXID（LoanX ID）列以及一个SP_COMPANY_ID列，你可以用来连接其他Markit数据集和S&P Capital IQ财务报表数据。

4.2 企业和主权债券

除了提供企业贷款定价信息之外，Markit还为企业和主权债券提供每日价格和分析CSV文件。与贷款定价不同，债券价格通过SSH文件传输协议（SSH File Transfer Protocol，SFTP）传输。SFTP服务器和凭证都是由Markit提供的。要为几千种债券标价，Markit“用有效数据，辅以多种可观测数据确证，建立发行人曲线”。没有可确定数据来源的债券的价格是通过对可观测数据用演算法或者外推法确定的。

Markit的债权定价数据超过90列，包括：

- 静态债券参考数据（CUSIP、Maturity、Currency等）。
- 债券定价的市场数据来源。
- 买入价、卖出价和中间价的有效数据（clean）和“脏数据”（dirty）。
- 用买入价、卖出价和中间价算出的最低赎回利率（YTW）。
- 用买入价、卖出价和中间价算出的资产互换息差。
- 基于买入价、卖出价和中间价的Z-息差。
- 基于买入价、卖出价和中间价的G-息差。
- PV01。

- 基于买入价、卖出价和中间价的有效久期、修正久期和麦考利久期。
- 基于买入价、卖出价和中间价的凸性。
- 基于买入价、卖出价和中间价的期权调整息差（OAS）。
- 深度和流动性统计。

4.3 途径1：在Excel中存储Markit信息

幸运的是，Markit的所有内容都在CSV文件中，Excel都能打开。不幸的是，Excel难以处理大型数据集，尽管新版本已经有所改进。此外，Excel需要一些额外步骤删除重复的行。尽管如此，本节介绍将来自于CSV文件的Markit数据存入Excel的技巧。尽管你可以把Markit数据和彭博数据保存在同一个工作簿中，还是推荐把Markit数据存储在单独的工作簿中，因为在Excel处理大量数据很慢。

首先为每个想处理的数据集（渠道）创建一个新的工作表。对于Facility Update和LoanID Updates工作表，联系support@markit.com，并请求完整的CSV文件，其中包括每笔贷款的全部机构信息和从LoanX ID到CUSIP的整个映射。还可以从其他渠道请求历史信息。对于其他表，用在本章前面讲过的URL语法为每个渠道下载一个CSV文件。

然后，将CSV文件的内容复制粘贴到它们对应的工作表中（例如，Marks CSV数据复制粘贴到Marks工作表）。在第一行最后面增加一列，命名为“Duplicate Check”。如第2章中所述，选择所有单元格（包括“Duplicate Check”标题），把每个工作表中的每个区域转换为Excel表，然后在函数区上，在Home键上，点击“Format as Table”按钮（或按Ctrl+T键），并选择你喜欢的表样式。弹出后选择“My table has headers”。为每个Excel表选择一个名字，例如“LoanFacilities” “LoanUpdates” 和 “LoanPrices”。

一些表（Facility Update、Recommended Update和LoanID Updates）可以把LoanX ID作为主关键字，因为只有最新的数据才重要，保存历史表没有什么意义。如果表中新增加的行有相同的LoanX ID，就删除现有的行。为此，在Duplicate Check列增加该公式，计算有相同的LoanX ID（或者对于LoanID

Updates用标识符) 和较新的修改时间Modified Time (或者对于RecUpdates用Inactive Date) 的行数 :

```
=SUMPRODUCT(N([LoanX ID]=[@[LoanX ID]]),N([Modified Time]>
[[@[Modified Time]]]))
```

本公式展示如何结合使用Excel SUMPRODUCT和N函数,以返回基于多个列的计数。当条件语句实参为真 (LoanX ID列中有一个LoanX ID与当前行的LoanX ID匹配,且同一行的Modified Time大于当前行的Modified Time), N函数返回1,否则返回0。SUMPRODUCT返回N函数结果之和。如果两个N语句都为真,则公式会增加1到结果 (1乘以1等于1), 但如果为假,它会增加0到结果 (1乘以0等于0)。你可以安全地删除任何公式结果大于0的行,因为有另一行的LoanX ID相同,但是修改时间 (Modified Time) 离现在更近。

下面,在用URL从Markit请求一个更新的CSV文件之后,在这些表上增加数据,从CSV文件中 (除了标题) 复制所有的行和列,在目标工作表上,右击行标题 (最左边包含行号那个) 并选择 “Insert Copied Cell”, 以从CSV文件向工作表插入该行。然后保证Duplicate Check列中的公式被复制到这些新的行。删除任何重复的输入。

对于可能保存了历史的表 (例如Marks、Daily Ratings和Financial Statements), 方法一是仅保存最新数据,即每次从Markit获取数据时清除整个表格 (或者用与其他表相同的公式,以保证只存在一个LoanX ID), 方法二是在Duplicate Check列使用如下公式,以返回有相同LoanX ID和Mark Date (或者As of Date) 的行数 :

```
=SUMPRODUCT(N([@[LoanX ID]]=[LoanX ID]),N([@[Mark Date]]=[Mark
Date]))
```

4.4 途径2 : 将Markit数据导入Microsoft Access

本节介绍如何将Markit CSV文件导入Access,并维护数据。首先联系support@markit.com,要求得到包含每笔贷款的全部机构信息的完整Facility Update和LoanID CSV文件,以及从LoanX ID到CUSIP的整个映射。然后进行下面

的步骤：

- 1.在Access中，在函数区点击External Data，然后选择Text File。
- 2.点击browse按钮，选择从Markit获取的某一个CSV文件，例如LoanXFacilityUpdates.csv。
- 3.选择“Import the source data into a new table in the current database”。这样就用CSV文件中的数据创建了一个新表。
- 4.点击Okay。选择Delimited选项，然后点击Next。
- 5.在下一个屏幕上，选择“Comma”分隔符，在Text Qualifier下拉框选择双引号（"），然后勾选，命名为First Row Including Field Names。
- 6.在下一个屏幕上，点击Next。这样表示今后再改变数据类型。
- 7.在下一个屏幕上，选择“No primary key”。这样表示今后再设置主关键字。
- 8.点击Finish按钮，结束导入。
- 9.重复第1至8步，为每个渠道导入所有的CSV文件。
- 10.右击一个表，然后选择Design View。
- 11.选择表4-7中标明的列，然后点击Primary Key按钮。该步将适当的列设置为表的主关键字。
- 12.将包含日期的列的数据类型调整为Date/Time，然后将数字数据类型设为Double。

想了解关于每个表的数据类型的完整信息，请阅读Markit文档。

表4-7：Markit表的主关键字

表名	主关键字
LoanIDUpdates	Identifier, Modified Time
LoanPricingAndAnalytics	PricingAsOf, LoanX ID
LoansDailyRatings	Date, LoanX ID
LoanXFacilityUpdates	LoanX ID, Modified Time
LoanXMarks	LoanX ID, Mark Date
LoanXRecUpdates	LoanX ID, Inactivation Date
FinancialStatement	SP_COMPANY_ID, Year, Quarter
FinancialStatementMap	LXID

表4-7的主关键字表包含日期列，以防止Access从未来的导入中排除更新行。要删除有相同LoanX ID但是更早的Modified Date的行，用如下查询（根据需要调整表和列名）：

```
Delete FROM LoanXFacilityUpdates r where r.[Modified Time] <
(select max(r2.[Modified Time]) from LoanXFacilityUpdates r2 where
r2.[LoanX ID]=r.[LoanX ID])
```

随后，用新的CSV文件更新这些表，用函数区上External Data选项卡上相同的Text File按钮，但是选择“Append a copy of the records to the table”选项，并为你选择的文件选择适当的表。用第一次导入时的方式再做一遍。在最后一个屏幕上，选择“Save import steps”，以创建一个重复这些步骤的简单方式。要重新运行导入，在External Data菜单上选择Saved Imports之后，选择被保存的导入名，然后点击Run。

4.5 途径3：用C#导入Markit数据

本节介绍如何用C#将Markit中的数据导入Microsoft Access。首先，按照途径2中的步骤创建初始表，通过从CSV文件导入这些表和设置适当的主关键字及数据类型。然后在Visual Studio中，创建一个新的控制台应用项目。右击References并选择Add Reference复选框（在Assemblies/Framework下靠近Mirosoft.VisualBasic），以向Micorosft.VisualBasic添加一个引用。

在Program.cs中，首先增加如下using指令：

```
using System.Data; using System.Data.OleDb; using System.Net;
using Microsoft.VisualBasic.FileIO;
```

在Program类中，添加如下属性，修改数据库地址和Markit证书：

```
private string ConnStr =
"Provider=Microsoft.ACE.OLEDB.12.0;Data Source=..\..\Path
3.accdb"; private string FIRM = "MyCompanyName"; private string
USERNAME = "MyMarkitUserName"; private string PASS =
"MyMarkitPassword";
```

如前所述，对Microsoft Access要安装和使用匹配的OleDb驱动程序。然后，在Main方法中，创建一个Program类实例，并调用后面将创建的Run方法：

```
static void Main(string[] args) { Program p = new Program();
p.Run(); }
```

在Run方法中，用与Markit请求、Access数据库中的表名以及应被搜索和更新（而非附加）的表的主关键字名相关的URL语法，调用ProcessURL方法（将在后面创建）。对于创建历史的表，对主关键字实参传递null值：

```
public void Run() { ProcessURL(
"https://loans.markit.com/loanx/LoanXFacilityUpdates.csv?
LEGALENTITY=" + FIRM + "&USERNAME=" + USERNAME + "&PASSWORD=" +
PASS, "LoanXFacilityUpdates", "LoanX ID"); ProcessURL(
"https://loans.markit.com/loanx/LoanXRecUpdates.csv?LEGALENTITY=" +
FIRM + "&USERNAME=" + USERNAME + "&PASSWORD=" + PASS,
"LoanXRecUpdates", "LoanX ID"); ProcessURL(
"https://loans.markit.com/loanx/LoanIDUpdates.csv?LEGALENTITY=" +
FIRM + "&USERNAME=" + USERNAME + "&PASSWORD=" + PASS,
"LoanIDUpdates", "Identifier"); ProcessURL(
"https://loans.markit.com/loanx/LoanXMarks.csv?LEGALENTITY=" + FIRM
```

```
+ "&USERNAME=" + USERNAME + "&PASSWORD=" + PASS, "LoanXMarks",  
null); ProcessURL(  
"https://loans.markit.com/loanx/LoansDailyRatings.csv?LEGALENTITY=" +  
FIRM + "&USERNAME=" + USERNAME + "&PASSWORD=" + PASS,  
"LoansDailyRatings", null); ProcessURL(  
"https://loans.markit.com/loanx/LoanPricingAndAnalytics.csv?  
LEGALENTITY=" + FIRM + "&USERNAME=" + USERNAME + "&PASSWORD=" +  
PASS, "LoanPricingAndAnalytics", null); }
```

ProcessURL方法用url参数使用WebClient从Markit下载CSV文件。然后随着它在CSV文件中循环，它会检查它是否应基于主关键字更新或者插入一个新的行。在列名匹配CSV文件中的列标题时，更新或者插入数据。

ProcessURL方法开始时用数据库的当前行填充一个数据集（用后面介绍的FillDataSet方法）。然后，它用WebClient类从Markit下载CSV文件，存到一个临时文件地址。

然后，该方法实例化一个TextFieldParser类，对CSV文件语法分析设置属性，并启动一个while循环直到到达文件最后。在第一行，它用语法分析的列数组填入标题List实例，其中将包含来自于CSV文件的列名。在后面的行，它用语法分析的列数组填入col字符串数组；这将包含一行数据以填入数据库。

该方法随后检查primaryKey实参是否通过；如果通过，找到数据库中的对应行。如果primaryKey实参没通过，或者数据库中不存在对应行，创建一个新的DataRow。

然后，如果CSV文件包括一个有相同名的列，且数据可以被转换成该列的数据类型，就填入数据库表的列。

如果DataRow实例是创建的，而非被发现的，就需要把它添加到DataTable然后更新DataSet（用后面定义的UpdateDataSet方法）。

代码如下：

```
private void ProcessURL(string url, string table, string
```

```

primaryKey) { DataSet ds = FillDataSet(table); WebClient client =
new WebClient(); string tmpfile =
System.Environment.GetFolderPath(Environment.SpecialFolder.Internet
+ "\\\" + Guid.NewGuid().ToString() + ".csv";
client.DownloadFile(url, tmpfile); using (TextFieldParser parser =
new TextFieldParser(tmpfile)) { parser.Delimiters = new string[] {
"," }; parser.TextFieldType = FieldType.Delimited;
parser.SetDelimiters(","); parser.TrimWhiteSpace = true;
List<string> header = null; while (!parser.EndOfData) { if (header
== null) { header = new List<string>(parser.ReadFields());
continue; } string[] col = parser.ReadFields(); DataRow row = null;
if (primaryKey != null) { var rows = ds.Tables[table].Select("[ " +
primaryKey + "]='" + col[header.IndexOf(primaryKey)] + "'"); if
(rows.Count() > 0) { row = rows[0]; } } if (row == null) { row =
ds.Tables[table].NewRow(); } foreach (DataColumn dc in
ds.Tables[table].Columns) { if (header.Contains(dc.ColumnName)) {
string val = col[header.IndexOf(dc.ColumnName)]; if (dc.DataType ==
typeof(string)) { row[dc] = val; } else if (dc.DataType ==
typeof(DateTime)) { DateTime foo; if (DateTime.TryParse(val, out
foo)) row[dc] = foo; } else if (dc.DataType == typeof(double)) {
double foo; if (Double.TryParse(val, out foo)) row[dc] = foo; }
else if (dc.DataType == typeof(Int32)) { int foo; if
(Int32.TryParse(val, out foo)) row[dc] = foo; } else
Console.WriteLine("Unhandled Column Type"); } } if (row.RowState ==
DataRowState.Detached) ds.Tables[table].Rows.Add(row); }
parser.Close(); } int rowc = UpdateDataSet(ds, table);
Console.WriteLine("Updated " + rowc + " in table " + table); }

```

最后剩下的两种方法是用Access数据库文件的内容填入DataSet实例的FillDataSet方法，以及更新Access数据库文件的UpdateDataSet方法：

```

private DataSet FillDataSet(string table) { DataSet ds = new
DataSet(); using (OleDbConnection conn = new

```



```
OleDbConnection(ConnStr)) { string cmdStr = "SELECT * FROM " +  
table; OleDbCommand cmd = new OleDbCommand(cmdStr, conn);  
OleDbDataAdapter da = new OleDbDataAdapter(cmd); conn.Open();  
da.Fill(ds, table); conn.Close(); } return ds; } private int  
UpdateDataSet(DataSet ds, string table) { int rowc = 0; using  
(OleDbConnection conn = new OleDbConnection(ConnStr)) { string  
cmdStr = "SELECT * FROM " + table; OleDbCommand cmd = new  
OleDbCommand(cmdStr, conn); OleDbDataAdapter da = new  
OleDbDataAdapter(); da.SelectCommand = cmd; OleDbCommandBuilder cb  
= new OleDbCommandBuilder(da); cb.QuotePrefix = "["; cb.QuoteSuffix  
= "]"; conn.Open(); da.UpdateCommand = cb.GetUpdateCommand();  
da.InsertCommand = cb.GetInsertCommand(); rowc = da.Update(ds,  
table); conn.Close(); } return rowc; }
```

4.6 本章小结

几乎所有公司都是以作为投资者或者交易者的方式参与市场活动的，而无论是作为投资者或者交易者都需要获取报价服务，例如Markit提供的报价服务。然而，这些数据通常都是由后台人员在使用，用于会计用途或者风险防控，而前台那些能赚钱的分析师却几乎没使用过。尽管Markit只是众多数据提供商之一，不过在本章中展示的它简单却能够获取大量信息的API，对于技术资源比较匮乏的分析师来说却是理想的工具。诚然，获取信息是有用的，但是对大量金融数据进行多种分析才是更有价值的。第8章将介绍对大量金融数据进行分析的方法。

第5章 金融数据分析

在有数据前就进行理论化，是个极大的错误。

——夏洛克·福尔摩斯

前几章中收集的数据可以帮助确定有价值证券的内在价值或者绝对价值，但是却不能讲述完整的故事，因为我们需要把数据放在一定的背景下进行分析。例如，知

道一个公司的股票过去六个月上涨了5%，并不能说明它比市场整体表现好还是表现差。金融分析的目标，也是本部分的目标，就是把金融数据放在一定的背景下进行分析，以支撑投资决策。

下面几章把金融分析分成三类：

相对价值分析

第6章主要探讨将一个有价值证券与其他同等债券和更大范围的市场进行比较的技巧。相对价值不是简单地将两个有价值证券的过往业绩进行比较，而是要确定一个有价值证券比另一个有价值证券便宜还是贵。说起来容易，做起来难，因为量化一个投资的好坏是很主观的，而且几乎是决定一个分析师成败的关键。例如，假设两个债券都是按照低于票面价格的一个折扣价格交易，风险差不多，但是其中一个比另一个的到期日早一年，那么到期时间较长的那个债券需要收益多多少，才能让二者成为等价的投资呢？答案取决于很多不同因素，包括市场展望。第6章介绍如何衡量这些不同的因素和用自定义的“分数”对有价值证券进行排序。

风险分析

第7章介绍从有价值证券和投资组合两个层面确定潜在风险的技巧。对于一个有价值证券来说，指标（例如价格、信用违约互换（CDS）息差和评级）可用于发现异常值。例如，有风险的异常值可能是现价低于70美元的有价值证券或者上个月价格下跌达到10%的有价值证券。对于一个投资组合来说，有价值证券指标（例如到期日、评级、行业、国家等）可用于预测风险，例如行业集中度过高等。

市场分析

第8章关注于对大量市场数据进行整合、总结及分析。例如，把从Markit获取的数千条每日价格按照行业、评级或者其他性质进行划分，能够帮助研究判断趋势。这些趋势可用于对一个有价值证券进行假设。例如，如果交易价格高于票面价格、点差超过450bps的医疗贷款中有85%在过去两个月中进行了再融资，那么你的投资组合中有类似性质的贷款就很有可能进行再融资。市场分析可以帮助你理解故事，并且掌握经验证据也比仅听道听途说的证据好。

在我们开始金融分析之前，需要先讲讲一些概念和任务。本章介绍如何保证你

在后面章节分析的数据既准确又充分。此外，本章还介绍如何创建你在后面的章节中将用到的Portfolio工作表或数据库表。最后，因为可能不时要追踪你的投资组合和其他数据，我们还要介绍留存记录的技巧。

5.1 数据真实性

任何金融分析的第一步，也是最关键的步骤之一，就是保证数据的正确性。然而，还是存在使用的数据不完美的情况。那时，你必须理解用不太完美的信息将如何影响你的结果。

5.1.1 检查数据

大部分数据源（例如Markit或彭博提供的金融信息）是正确的，但是可以肯定其中可能有错误。在作出投资决策之前，要预先审阅和证实你所使用的数据，这是很重要的。此外，即使来自这些源头的信息是正确的，它也可能误导人或者被错误使用。如下是要检查的一些内容：

- 在对金融数据进行数学运算（加减乘除）时，确保所有的金额都用了相同的货币单位。

- 有些金融数据可能用不同的单位（千、百万、整数等）存储，确保所有的单位一致。

- 对于每个实体，金融数据报告并不总是用同一个频率。因此在比较不同金融信息时，要考虑“截至”（as of）日期。

5.1.2 样本规模

分析中用到的有价证券数量是很重要的。包含的有价证券太少使分析不可信，而太多可能导致误导结果。我们不过多讨论数学知识，要确定最小样本规模，用如下公式。

$$n = \frac{N \times Z^2 \times p \times (1 - p)}{(N - 1) \times e^2 + Z^2 \times p \times (1 - p)}$$

图5-1：样本规模公式

如下是变量含义：

n

样本最小规模。

N

总体规模。

Z

基于置信水平的 z 值：90%=1.65，95%=1.96，99%=2.58。例如，假设结果正确率为95%，将把 z 值设为1.96。其他 z 值可以在网上查询到。

p

如果你想要特定的比例，可以用它缩小所需的样本规模。如果不知道比例，用正态分布（设 $p=50\%$ ）。

e

残差。例如，如果你同意将结果准确度降低5%，以缩小所需的样本规模，设 $e=5\%$ 。

一个例子是Markit数据库中的4226笔贷款，置信水平95%，误差边际5%，分析应包括352笔贷款。

5.1.3 异常值

检验一个值和中位数之间的标准差，是检验异常值是否会导致数据问题的好办法。一个值距离中位数的标准差越大，它就越可能是异常值，因此产生潜在数据问题。用一个值和整个数据序列的中位数（中位数）之差的绝对值，除以该序列的标准差。在Microsoft Excel中，用MEDIAN和STDEV函数。那么，计算“3M Px Chg”值与中位数的标准差的公式就如下：

`=ABS([@[3M Px Chg]]-MEDIAN([3M Px Chg]))/STDEV([3M Px Chg])`

要基于另一个列中的值计算一行的中位数，在MEDIAN函数中用IF函数。例如，用如下数组公式计算标准普尔评级为BB+的Market Cap中位数（需要按Ctrl+Shift+Enter组合键）：

`=MEDIAN(IF(Company[S&P Rating]="BB+",Company[Market Cap]))`

Excel中有求中位数和标准差的函数。而在C#中，你需要编写如下函数：

```
public static double GetMedian(List<double> list) { double
median = 0; if (list.Count != 0) { // create new instance of list
// so source list isnt modified // List should be ordered.
List<double> sortedList = list.OrderBy(x => x).ToList(); int size =
sortedList.Count; int mid = size / 2; if (size % 2 != 0) median =
sortedList[mid]; else median = (sortedList[mid] + sortedList[mid -
1]) / 2; } return median; } public static double
GetStandardDev(List<double> list) { double stdev = 0; if
(list.Count != 0) { double average = list.Average(); stdev =
Math.Sqrt((list.Sum(x => Math.Pow(x - average, 2))) / (list.Count()
- 1)); } return stdev; }
```

用这种方法比较不类似的有价值证券数据（例如总债务或企业价值）是没有意义的。

此外，在计算大型数据集的平均值时，用中位数而非平均值可能有助于避免出现大的异常值的情况。例如，如果一个债券明天到期，那么对一个债券投资组合的到期日期列取简单平均值就可能产生误导。

5.2 投资组合

现在已经用彭博数据或者手动填好了Bond、Loan、IDX和Company工作表（或数据库表）。本节介绍创建一个Portfolio工作表（按途径1）和投资组合数据库表（按途径2和3）。尽管Portfolio表可以用于一个在某个日期买入的、有特定规模

和价格的`Portfolio`集合，但还是可以把它用于任意`Portfolio`子集合。我们用后面章节的`Portfolio`工作表/表来与其他`Portfolio`集合和整个市场进行比较。

`Portfolio`工作表/表中的每一行中都包含一个`PositionID`，用于确定唯一的头寸以及其他属性：

`PositionID`

这是`Portfolio`表的唯一标识符（主关键字）。可以用递增数字作为标识符（如1、2、3等）。

`Type`

指投资类型（债券、贷款、股权等）。

`SecurityID`

`SecurityID`是一个外关键字，它指向投资的主关键字。例如，根据该投资是股票、贷款或债券，它可以是一个`CompanyID`、`LoanID`或`BondID`。这是少数几个外关键字和主关键字不同的情况之一。

`Size`

规模可以是股份数，也可以是有价证券的名义金额。记住永远不能将股份数和名义金额进行加和。

`PurchasePx`

购买有价证券花费的美元。本书假设投资都用了同一种货币，但是我们可以扩展该表以处理多种货币。

`PurchaseDate`

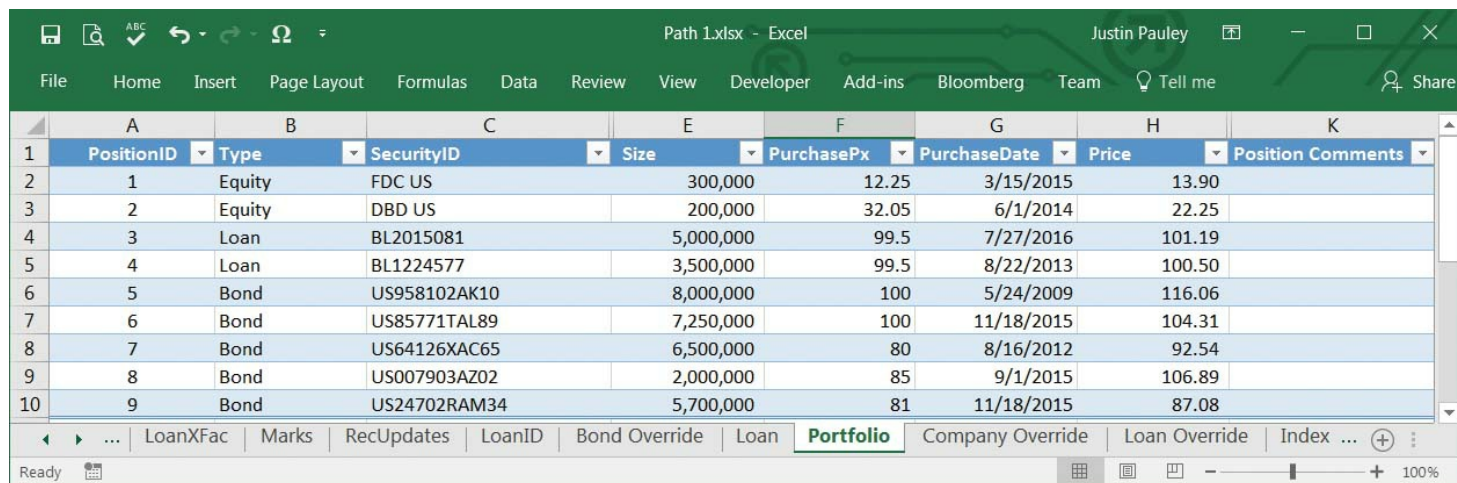
购买有价证券的日期。

`Position Comments`

用该字段存储你对头寸的备注，例如你的投资观点心得。

5.2.1 投资组合工作表

如果你用Excel创建一个表，其中包括前面描述的列，并填入不同的头寸。在Portfolio工作表中输入数据之后，如图5-2所示（我们在本节后面讨论Price）。



PositionID	Type	SecurityID	Size	PurchasePx	PurchaseDate	Price	Position Comments
1	Equity	FDC US	300,000	12.25	3/15/2015	13.90	
2	Equity	DBD US	200,000	32.05	6/1/2014	22.25	
3	Loan	BL2015081	5,000,000	99.5	7/27/2016	101.19	
4	Loan	BL1224577	3,500,000	99.5	8/22/2013	100.50	
5	Bond	US958102AK10	8,000,000	100	5/24/2009	116.06	
6	Bond	US85771TAL89	7,250,000	100	11/18/2015	104.31	
7	Bond	US64126XAC65	6,500,000	80	8/16/2012	92.54	
8	Bond	US007903AZ02	2,000,000	85	9/1/2015	106.89	
9	Bond	US24702RAM34	5,700,000	81	11/18/2015	87.08	

图5-2：Portfolio工作表示例

用MATCH和INDEX函数，从其他工作表提取信息，增强Portfolio工作表。这将是基于Type和匹配SecurityID提取列。例如，基于Type和SecurityID列提取Company Name或Security Description，用如下公式：

```
=IF([@Type]="Equity",INDEX(Company[CompanyName],MATCH([@SecurityID],Company[SecurityID],0)),IF([@Type]="Loan",INDEX(Loan[SecurityDescription],MATCH([@SecurityID],Loan[LoanID],0)),IF([@Type]="Bond",INDEX(Bond[SecurityDescription],MATCH([@SecurityID],Bond[BondID],0)))))
```

该公式首先检查类型是否是Equity；如果是，则从Company工作表的CompanyName列提取CompanyID与Portfolio工作表的SecurityID列匹配的单元格。如果不是，它会检查类型是否为Bond（之后检查类型是否为Loan），并提取适当的Security Description列。你可以用同样的技术提取当前价格（存入一个新的列，命名为Price）：

```
=IF([@Type]="Equity",INDEX(Company[Price],MATCH([@SecurityID],  
Company[CompanyID],0)),IF([@Type]="Loan",INDEX(Loan[Price],MATCH([@  
Loan[LoanID],0)),IF([@Type]="Bond",INDEX(Bond[Price],  
MATCH([@SecurityID],Bond[BondID],0))))))
```

增加那两列之后，就可以计算每个头寸的市值。创建一个新的列，命名为“MarketValue”，用如下公式：

```
=IF([@Type]="equity",[@Price]*[@Size],[@Size]*[@Price]/100)
```

该公式将债券和贷款价格除以100，因为那些价格代表百分比，而股票价格不除以100。

增加另一列，命名为“BBID”，其中包含每个有价证券的彭博ID。在该列用如下公式：

```
=IF([@Type]="Equity",INDEX(Company[BBID],MATCH([@SecurityID],  
Company[CompanyID],0)),IF([@Type]="Loan",INDEX(Loan[BBID],MATCH([@S  
Loan[LoanID],0)),IF([@Type]="Bond",INDEX(Bond[BBID],  
MATCH([@SecurityID],Bond[BondID],0))))))
```

最后，添加一个“%of Portfolio”列，其中包含每个头寸的市值在投资组合中的比重。

在新的列中用该公式：

```
=[@MarketValue]/SUM([MarketValue])
```

右击表，添加一个Total行（在表底部计算每列的加总值），然后，在打开的快捷菜单上点击表，然后选择Totals Row。

在Total行里可以选择一个聚合函数（例如求和、求最大值、求最小值、求标准差），用于每列的值。点击MarketValue列Total行交叉的那个单元格，然后从下拉菜单选择Sum，对MarketValue列求和。对于“%of Portfolio”列，在Total行下拉菜单选择Max，显示投资组合中市值比重最大的头寸。

5.2.2 投资组合数据库表

用表5-1中的模式创建一个将PositionID作为主关键字的Portfolio表。

表5-1：Portfolio表设计

字段名	数据类型
PositionID*	数字
Type	短文本
SecurityID	短文本
Size	数字
PurchasePx	数字
PurchaseDate	日期 / 时间
Position Comments	短文本

手动向数据表输入数据之后，用如下查询从其他表提取额外的列，以增强Portfolio视图：

```
SELECT p.*, a.Price, a.SecurityDes,  
iif(p.type='equity',Price,Price/100) * p.size as MarketValue,  
iif(p.type='equity',Price,Price/100) * p.size/( select  
sum(iif(p2.type='equity',a2.Price,a2.Price/100) * p2.size) from  
portfolio p2 inner join ( select BondID as  
SecurityID,Price,SecurityDes from Bond UNION select LoanID as  
SecurityID,Price,SecurityDesc from Loan UNION select CompanyID as  
SecurityID,Price,CompanyName as SecurityDes from Company ) a2 on  
a2.SecurityID=p2.SecurityID; ) as PctOfPortfolio FROM Portfolio p  
inner join ( select BondID as SecurityID,Price,SecurityDes from  
Bond UNION select LoanID as SecurityID,Price,SecurityDesc from Loan  
UNION select CompanyID as SecurityID,Price,CompanyName as  
SecurityDes from Company ) a on a.SecurityID=p.SecurityID;
```

该查询使Portfolio表与一个查询结果为Bond、Loan和Company表的相关列的内查询相连接。内连接还会在子查询中复制，该子查询当前行的市值除以投资组合

的总市值。如果价格列是债券或贷款价格，IIF函数就会将其除以100，因为这些是百分比。

5.3 连接Excel工作表和Microsoft Access

如果使用途径2，必须把第3章创建的Excel工作表连接到Access。第一步，对于在第一行有彭博字段的每个工作表（Loan、Bond、IDX和Company），你需要创建一个命名区域，以正确地将数据连接到Access。步骤如下：

1.在Excel工作簿的函数区上，在Formulas按钮上，Defined Names群中，点击Name Manager。

2.在打开的对话框上部，点击New按钮，然后在Name文本框中输入BondTable，将Scope列表框设为Workbook，然后在“Refers to”文本框中，输入=Bond!\$A\$2:\$AZ\$9999。点击OK。

该步创建一个命名区域，名为BondTable，其中包括Bond工作表中Bloomberg Fields行下面的所有单元格。

3.重复第1~2步，创建LoanTable、IDXTTable和CompanyTable，分别添加=Loan!\$A\$2:\$AZ\$9999、=IDX!\$A\$2:\$AZ\$9999和='ompany?\$A\$3:\$AZ\$9999。CompanyTable始自第三行，因为货币字段有额外一行。

如果你所用到的列范围没有达到“AZ”，你可以指定一个更小的区域。因为Portfolio工作表始自第一行，所有不需要一个命名区域。

4.保存Excel工作簿，转向Access，然后在函数区External Data选项卡上，点击Import Excel Spreadsheet。

5.在打开的对话框的第一页，用Browse按钮选择你的Excel工作簿，选择“Link to the data source by creating a linked table”，然后点击OK。

6.在下一个屏幕上，点击Show Named Ranges，选择BondTable，然后点击Next。

7.在下一个屏幕上，选择First Row Contains Column Headings，然后点击Next。

8.在下一个屏幕上，把Linked Table Name改为Bond，然后点击Finish。

你能打开Bond表，看到来自Excel的所有数据。

9.重复该步骤以连接到Loan、IDX和Company工作表。

10.用相同的步骤连接Portfolio表，只不过在第二个屏幕不选择Show Named Ranges，而是从表单中选择Portfolio。

5.4 留存记录

从整个过程看，留存记录（例如你的投资组合或者分析结果）是很有用的。如果没有留存记录，很可能难以或者无法重建记录。尽管在开头看起来，留存记录的重要性并不显而易见，但是等到需要的时候，你就会庆幸自己留存了记录。本节针对不同途径，介绍维护数据历史的多种技巧。

根据经验，（硬盘上的）数据存储很便宜，并且存储过多数据也比存储过少数数据要好。

5.4.1 途径1：Excel

不幸的是，Excel不是存储记录的最好工具，因为它难以处理大量数据。本节介绍如何通过在一个单一的便于维护的工作表中保存特定值的历史数据，而在其他工作簿中用简单的批处理文件保存全部历史数据，来解决这个问题。

历史工作表

首先在第3章创建的工作簿中，确定一个你想跟踪的单元格，并用左上角的Name Box给它命名。例如，要跟踪你的投资组合的市值，选择Portfolio工作表Total行和MarketValue列交叉的那个单元格，然后在Name Box中，输入PortfolioMV，如图5-3所示。

Path 1.xlsx - Excel Justin Pauley

File Home Insert Page Layout Formulas Data Review View Developer Add-ins Bloomberg Team Design Tell me

PortfolioMV $\times$ $\checkmark$ f_x =SUBTOTAL(109,[MarketValue])

	F	G	H	I	J	K
	PurchasePx	PurchaseDate	Price	MarketValue	% Of Portfolio	Position Comments
1						
2	12.25	3/15/2015	14.60	4,380,000	9.2%	
3	32.05	6/1/2014	24.00	4,800,000	10.1%	
4	99.5	7/27/2016	100.63	5,031,250	10.6%	
5	99.5	8/22/2013	100.44	3,515,313	7.4%	
6	100	5/24/2009	115.25	9,219,944	19.5%	
7	100	11/18/2015	103.73	7,520,063	15.9%	
8	80	8/16/2012	91.17	5,926,148	12.5%	
9	85	9/1/2015	106.93	2,138,598	4.5%	
10	81	11/18/2015	85.28	4,860,762	10.3%	
11				47,392,076	19.5%	

RecUpdates LoanID Bond Override Loan **Portfolio** Hist Company Override ...

Ready 100%

图5-3：在Excel中命名单元格

对包含投资组合中最大市值头寸的单元格重复该步骤（“%of Portfolio”列的单元格）；将单元格命名为MaxPortfolioPct。

然后，创建一个新的工作表，命名为“Hist”，用它来跟踪这些单元格。该工作表从第一步开始就将包含一系列与历史数据的日期相对应，一系列与每个被命名的单元格相对应。完成该工作表后，你可以通过增加一个列标题并复制公式跟踪新增的已命名单元格。然而，因为这些单元格与其他工作表中的现值相连接，所以每天你需要从Hist工作表中手动复制粘贴现值。该步在本节后面有更详细的解释。

在Hist工作表中，首先在第一个单元格（A1）中输入Date，然后在B列和C列输入两个单元格名，这样第一行分别是Date、PortfolioMV和MaxPortfolioPct。

在Date列（从单元格A2开始）输入以下公式：

=IF(ISFORMULA(A1),"-",TODAY())

拖动时该公式会检查它上面的单元格是否包含公式；如果上面的单元格不包含公式，就显示今天的日期。因为在后面的步骤中，每个历史行将包含值而非公式，该公式会在最后的历史值（或者在标题后显示第一天）后面显示今天的日期，并在

它下面的每一行显示一个破折号（“-”）。

在PortfolioMV列（单元格B2），添加如下公式：

```
=IF(ISFORMULA(B1), "-", INDIRECT(B$1))
```

和Date列公式一样，该公式用INDIRECT函数，如果上面的单元格是一个历史值（若第一次出现，则为列标题单元格）；否则，显示破折号。INDIRECT函数显示一个已命名单元格的值。换句话说，公式将返回没有历史的最后一行中的现值；否则，显示破折号。把该公式复制到MaxPortfolioPct列（单元格C1），如下所示：

```
=IF(ISFORMULA(C1), "-", INDIRECT(C$1))
```

然后把第一行的公式向下拖几行。第一行应包括今天的日期、PortfolioMV的现值和MaxPortfolioPct的现值。其他每行应该有一系列破折号。

要增加额外列，只要简单命名该单元格，把单元格名增为一个列标题，并从其他已命名单元格列之一复制公式。

最后，因为这些公式连接着其他单元格，如果PortfolioMV或MaxPortfolioPct的值改变，我们必须把带有今天日期的行的公式转换成值。要转换公式，右击包含今天日期的单元格旁边的行号，然后在快捷菜单上，选择Copy。然后，再次右击包含今天日期的单元格旁边的行号，并在粘贴选项上选择“Values（V）”。结果是该行的公式转换为值，由此，被转换的行下面那一行将显示今天的日期和当前信息。

如果你每天打开该工作表，那么破折号之前的最后一行现在应包含那个日期，如果你改变了PortfolioMV或MaxPortfolioPct的值，那些值只会在最后一行改变。每天你想要记录，就必须复制粘贴最后一行的值。工作表结果如图5-4所示。

	A	B	C
1	Date	PortfolioMV	MaxPortfolioPct
2	11/12/2016	47,392,076	19.5%
3	10/20/2016	47,268,922	18.7%
4	10/25/2016	47,145,768	18.7%
5	10/30/2016	47,022,614	19.1%
6	11/4/2016	46,899,460	19.1%
7	11/9/2016	48,097,762	19.1%
8	11/11/2016	47,392,076	19.5%
9	11/12/2016	=IF(ISFORMULA(B8),'-',INDIRECT(B\$1))	19.5%
10	-	-	-
11	-	-	-

图5-4：显示公式的Hist工作表

Backup.bat

本节介绍如何创建一个每天自动备份你的工作簿的脚本。首先运行Notepad应用。在新的文本文件中粘贴如下（再次输入"C:\drive\Path 1.xlsx"，在两处都输入完整的文件名，注意用"%date%"语法）：

```
copy "C:\drive\Path 1.xlsx" "C:\drive\Path
1_%date:~10,4%%date:~4,2%%date:~7,2%.xlsx"
```

该命令创建你的Excel文件（在此情况下为Path 1.xlsx）的副本，并把今天的日期（yyyyMMdd）加到文件名上（Path 1_20161112.xlsx）。

保存文件，命名为backup.bat。后缀为 ".bat" 的文件是批处理文件，意味着启动文件之后，Microsoft Windows会在其中执行命令。

然后，运行Task Scheduler应用和选择创建Basic Task，这样能每天自动运

行该批处理文件。在对话框打开的一个屏幕上，输入名称Excel Analysis Backup，然后点击Next。在下一个屏幕上，选择Daily（或者你想要的频率），然后点击Next。在下一个屏幕上，选择你想创建备份文件的日期，然后点击Next。在下一个屏幕上，选择“Start a program，”然后点击Next。在下一个屏幕上，点击Browse按钮，选择你刚创建的backup.bat文件，然后点击Next。点击Finish，结束工作安排。

 为避免打开历史工作表时从彭博更新数据或者重新计算其他单元格，首先打开一个空白工作簿，然后，在函数区的Formulas按钮上，选择Calculation Options，选择Manual，然后打开历史工作簿。

或者，为了从历史Excel文件中提取到一张工作表中，要创建一个新的工作簿文件，并按照这样的方法在文件中引用单元格（其中PortfolioMVVal是11/12/2016创建的文件中的一个已命名单元格）：

```
= 'C:\drive\Path 1_20161112.xlsx'!PortfolioMVVal
```

你不能直接引用Excel表中的单元格，除非那个工作簿是打开的。要引用PortfolioMV单元格，必须在Excel表以外创建一个单元格（命名为PortfolioMVVal），它用如下公式在表内引用PortfolioMV单元格：

```
=PortfolioMV
```

你也不能用INDIRECT函数引用外部单元格：每个引用必须在公式栏写明。

5.4.2 途径2：Microsoft Access

为了在Access中维护历史数据，首先复制一张现有的表，创建一张表存储历史信息。步骤如下：

1. 要维护Portfolio表的历史，在Access中右击该表，然后在快捷菜单上，选择Copy，再次右击，然后选择Paste。

2. 把新表命名为PortfolioHist，然后在Paste Options下，选择Structure Only（Local Table）。

3.重复第1步和第2步，为另一张表创建历史表。

4.右击PortfolioHist表，选择Design View。

5.在PortfolioHist表的Design View中，在表模式的上部添加一行，并把新字段命名为HistDate，把数据类型设为Date/Time，然后点击Save。

6.对其他表重复第4步和第5步。

7.创建如下查询，该查询从关联表向新的历史表插入HistDate列中写有今天日期的行：

```
BondHistQuery INSERT INTO BondHist SELECT now() AS HistDate, *  
FROM Bond WHERE BondID is not null; PortfolioHistQuery INSERT INTO  
PortfolioHist SELECT now() AS HistDate, * FROM Portfolio WHERE  
SecurityID is not null; CompanyHistQuery INSERT INTO CompanyHist  
SELECT now() AS HistDate, * FROM Company WHERE CompanyID is not  
null; LoanHistQuery INSERT INTO LoanHist SELECT now() AS HistDate,  
* vFROM Loan WHERE LoanID is not null; IndexHistQuery INSERT INTO  
IndexHist SELECT now() AS HistDate, * FROM Index WHERE IndexID is  
not null;
```

如果基础表（Index、Loan、Company、Portfolio、Bond）模式有变化，而历史表（IndexHist、LoanHist等）模式没有进行相应修改，那么这些查询就不能运行。

8.在Create选项卡上，点击Macro。

9.在宏上添加一个SetWarnings操作，并将Warnings On设为No。

10.对于你创建的每个查询（PortfolioHistQuery、CompanyHistQuery、LoanHistQuery、BondHistQuery和IndexHistQuery），在宏上增加一个OpenQuery操作，并选择适当的查询。

11.把新的宏保存为HistMacro。

新的宏将执行所有的查询，并把每个表中的所有数据复制到适当的历史数据库中。

自动运行这个宏。

12. 在HistMacro最后添加一个QuitAccess操作。

这指示Access在宏结束的时候关闭。

13. 运行Task Scheduler应用，然后选择创建Basic Task。

14. 给该任务命名，然后点击Next。

15. 选择你喜欢的频率，然后点击Next。

16. 指定宏应当运行的时间，然后点击Next。

17. 选择 "Start a program" ，然后点击Next。浏览你的msaccess.exe。

一般是C:\Program Files (x86)\Microsoft Office\root\Office16\MSACCESS.EXE。

18. 增加可选实参："c:\path\to\access.accdb"/x HistMacro，然后点击Next和Finish。

/x实参指示Access在启动时开始宏。

5.4.3 途径3：C#

 用写代码的方式填入历史表是可选的，用途径2中展示的宏也可以。然而，如果喜欢全部用代码，就用如下办法。

与途径2一样，首先在Access中创建一个表，从一个现有的表中复制结构存储历史信息。例如，要维护Portfolio表的历史，在Access中右击该表，选择Copy，然后再次右击，选择Paste。把新表格命名为PortfolioHist，然后在Paste Options下选择Structure。在PortfolioHist表中添加一

个 "HistDate" Date/Time列，并在主关键字中包含它（即，PortfolioHist表的主关键字是PortfolioID和HistDate的组合）。在其他表重复该步。

然后，在Visual Studio中创建一个新的C#控制台应用。在Program.cs文件中，包含如下using指令：

```
using System.Data; using System.Data.OleDb;
```

然后添加如下连接字符串属性（把路径调整至Access数据库文件）：

```
private string ConnStr =  
"Provider=Microsoft.ACE.OLEDB.12.0;Data Source=..\..\Path  
3.accdb";
```

和本书前面的代码一样，实例化一个Program对象，调用我们马上要创建的Run方法：

```
static void Main(string[] args) { Program p = new Program();  
p.Run(); }
```

Run方法实例化一个带有初始数据表和对应的历史数据表的Dictionary对象。对于Dictionary的每次输入，Run方法从Access数据库（用FillDataSet方法）填入数据，将数据复制到历史数据表（用CopyData方法），最后在Access数据库填入新的历史行（用UpdateDataSet方法）：

```
public void Run() { DataSet ds = null; Dictionary<string,  
string> histTables = new Dictionary<string, string>() {  
{"Bond","BondHist"}, {"Loan","LoanHist"},  
{"Company","CompanyHist"}, {"Portfolio","PortfolioHist"},  
{"Index","IndexHist"} }; int rowsUpdated = 0; foreach(string  
origtable in histTables.Keys) { ds =  
FillDataSet(origtable,histTables[origtable]); CopyData(ds,  
origtable, histTables[origtable]); rowsUpdated+= UpdateDataSet(ds,  
histTables[origtable]); } Console.WriteLine(rowsUpdated + " rows  
updated"); }
```


FillDataSet方法在初始表中填入数据，然后在历史表中获取模式（schema）。因为我们只在历史表中插入数据，因此仅需要模式。该模式用在CopyTable方法中，以保证历史表包含的列与初始表中是一样的：

```
private DataSet FillDataSet(string table, string histTable) {
    DataSet ds = new DataSet(); using (OleDbConnection conn = new
    OleDbConnection(ConnStr)) { string cmdStr = "SELECT * FROM [" +
    table+"]"; OleDbCommand cmd = new OleDbCommand(cmdStr, conn);
    OleDbDataAdapter da = new OleDbDataAdapter(cmd); conn.Open();
    da.Fill(ds, table); cmdStr = "SELECT * FROM " + histTable; cmd =
    new OleDbCommand(cmdStr, conn); da = new OleDbDataAdapter(cmd);
    da.FillSchema(ds, SchemaType.Source, histTable); conn.Close(); }
    return ds; }
```

CopyData方法循环访问初始表中的所有行，在历史表中创建一个新的对应行，将HistDate列设为今天的日期，并复制到所有对应列：

```
private void CopyData(DataSet ds, string origtable, string
histTable) { foreach(DataRow row in ds.Tables[origtable].Rows) {
    DataRow newrow = ds.Tables[histTable].NewRow(); newrow["HistDate"]
    = DateTime.Now; foreach(DataColumn dc in
    ds.Tables[origtable].Columns) { if(row.IsNull(dc) == false &&
    ds.Tables[histTable].Columns.Contains(dc.ColumnName)) {
    newrow[dc.ColumnName] = row[dc]; } }
    ds.Tables[histTable].Rows.Add(newrow); } }
```

最后，UpdateDataSet方法更新数据库中的历史表，并返回增加的行数：

```
private int UpdateDataSet(DataSet ds, string table) { int rowc
= 0; using (OleDbConnection conn = new OleDbConnection(ConnStr)) {
    string cmdStr = "SELECT * FROM " + table; OleDbCommand cmd = new
    OleDbCommand(cmdStr, conn); OleDbDataAdapter da = new
    OleDbDataAdapter(); da.SelectCommand = cmd; OleDbCommandBuilder cb
    = new OleDbCommandBuilder(da); cb.QuotePrefix = "["; cb.QuoteSuffix
```

```
= "]]"; conn.Open(); da.UpdateCommand = cb.GetUpdateCommand();  
da.InsertCommand = cb.GetInsertCommand(); rowc = da.Update(ds,  
table); conn.Close(); } return rowc; }
```

5.5 本章小结

本章介绍你在学习后面的章节时会用到的一些基本概念。此外，我们填入了Portfolio工作表/表，然后展示了如何将一个中心工作表/表中的信息连接到你在第3章中创建的其他工作表/表上去。最后，本章介绍了维护数据和分析历史的技巧。下面三章会深入讲解买入价、卖出价和中间价，风险，以及市场金融数据分析。

第6章 相对价值分析

研究一项投资的价值有两种主要方法。一是确定投资的内在价值。内在价值是你在不考虑其他因素的情况下，认为一项投资价值多少，它可能与其市值有偏差，例如尽管一个债券的市场价格可能是40美元，但是在你的计算假设中，其净现值（NPV）高达50美元。量化风险和设置不同计算假设（场景）及概率需要对投资基本面有全面了解，这也是金融分析的核心。另一方面，买入价、卖出价和中间价将一项投资的风险调整后的收益与其他潜在投资相比较，以确定它是贵了、便宜了、还是价格公道。换句话说，在其他因素都相同的情况下，一项回报率15%、但是比较有风险的投资比另一项回报率15%、但是风险没有那么高的投资要贵。类似地，尽管一个债券可能看起来便宜，因为它的市值为40美元，而内在价值为50美元，但是可能还有更便宜的债券（同等风险，市值为40美元，而内在价值为60美元）。

确定买入价、卖出价和中间价的关键是区分和量化各种投资之间的区别。例如，比较同一家公司发行的、两个几乎相同的债券（票息和到期日不同），实质上就是量化额外期限的价值。此外，量化区别可能是非常主观的。例如，如果两个别墅位于同一社区，但是一家带游泳池，有些人会认为游泳池是加分项，而有些人会认为游泳池是减分项。不幸的是，投资之间的差异太多了，实在难以量化。本书不能教你如何进行基础研究或者确定一项投资的内在价值，不过我们会讲到通过区分和量化各种相似投资之间的区别以确定买入价、卖出价和中间价的几个技巧。

确定一个有价值证券的市值时常会用到买入价、卖出价和中间价。在很多情况下，对一个有价值证券进行估值的唯一办法是将其与近期市场已经进行定价（或交易）的有价值证券进行比较。过程是先找到时间离得最近的相关公开市场交易，用它作为未定价有价值证券的基准。例如，如果近期发行的一个有价值证券收益率为9%，一个类似的有价值证券即将进入市场，如果该有价值证券的风险更大则收益率应比9%高，如果该有价值证券的风险更小则收益率应比9%低。

本章介绍用Microsoft Excel、Access或C#，将债券、贷款和股票与其对等组和市场进行比较的办法。与前面的章节一样，本章分为三条途径。推荐你读懂途径1（Excel）的解释，以更好地理解Access和C#。

6.1 途径1：Excel

下面介绍Excel中的相对价值技巧。

6.1.1 Excel中的相关性与回归

比较一个有价值证券与指数的关系或者两个有价值证券之间的关系，理解它们之间的相关性很有帮助。相关性告诉我们两项投资之间的关系（正相关、负相关或者不相关）以及关系的强度。正相关性表示两个有价值证券的价格同向变化。例如，生产汽车的企业和生产汽车使用的原材的企业很可能是正相关的。负相关性表示两个有价值证券的价格反向变化。例如，随着石油价格变化，石油公司和大量消费石油的企业（例如航空公司和卡车公司）很可能是负相关的。这种关系的强度（用相关系数计算）告诉我们两项投资在同向而行（正相关性）和逆向而行（负相关性）时有多紧密。

确定投资之间的相关性是重要的，原因如下。第一，如果你的投资组合中包含的投资都是高度相关的，就失去多元化投资的优势了。第二，测量投资之间的相关性，用于找到确定负相关的投资，以进行适当的防范。第三，也是本章的核心——帮你找到合适的基准有价值证券。例如，对比Microsoft的股票业绩和Russell 3000 Restaurant Index就不太恰当。

回归类似于相关性，除了回归会给你一个方程式，用于在已知一个投资价值时

预测另一个投资的价值。例如，如果Russell 3000Technology指数下跌10%，回归方程式能预测Microsoft股价的变化。显然，相关性并不完美，因此预测也不完美。尽管如此，有一些办法测量回归方程式对历史数据的解释程度有多好。

本节带你学习在Excel中确定两项投资之间的相关性和回归方程式的办法。此外还展示如何建立相关性矩阵，类似于图3-9所示的Bloomberg Peer Correlation 屏幕的输出。

1. 添加一个新的Excel工作表，命名为“Regression”（回归）。

（图6-2中为该表。）

在该工作表中，列A至C包含Microsoft（MSFT）的每日价格和价格变化，列E至G包含Russell 3000Technology Index（RGUST）的每日价格和价格变化。列J包含标签（Start Date、Correlation等），列K包含对应那些标签的值。

2. 在单元格A1，输入一个指数或股票的名字。

对该示例，用“MSFT Equity”代表Microsoft公司。

3. 在第一行E列（单元格E1），输入另一个指数或股票的名字。

对该示例，用“RGUST Index”代表Russell 3000Technology Index。

4. 跨过几列，在单元格J1，用如下公式标记输出表（读作“Regression of MSFT Equity to RGUST Index”）：

```
= "Regression of"&A1&"to"&E1
```

5. 在单元格J2，输入“Start Date”，在旁边的单元格（K2），输入如下公式得到今天之前12个月的那天：

```
=EDATE（TODAY（），-12）
```

6. 在单元格A2，输入如下公式：

```
=BDH（A1，“PX_LAST”，$K$2，“”，"sort=D"）
```

该公式提取MSFT Equity从初始日期到今天的全部历史价格，按从最近到最久远排序。

7.类似地，在单元格E2，输入如下公式提取RGUST Index的历史价格：

=BDH (E1 , "PX_LAST" , \$K\$2 , "" , "sort=D")

8.在单元格C2，输入如下公式：

= (B2-B3) /B3

该公式用于计算MSFT Equity价格的每日百分比变化。

9.从第8步向下复制公式（除了最后一行，因为公式要求现在单元格下面有一个赋值的单元格）。

10.类似地，对RGUST Index，在单元格G2用如下公式计算其每日百分比变化，并向下复制：

= (F2-F3) /F3

下面的公式填入列J和K的输出表。每个公式的标签都放在J列，每个公式本身都放在K列。

11.把单元格J3命名为“Correlation”。

12.在单元格K3，输入如下公式（对包含值的行进行调整）：

=CORREL (C2 : C253 , G2 : G253)

该公式用CORREL函数，计算MSFT Equity和RGUST Index每日价格变动之间的相关系数。

CORREL函数返回相关系数，该系数介于-1和1之间，表示MSFT Equity和RGUST Index之间的线性关系。1表示两个有价证券完全正相关，而-1表示两个有价证券完全负相关。尽管多大的相关系数值表示强、弱或者一般相关并没有划一条绝对的线，不过一般说来0.7和1之间的值或者-0.7和-1之间的值表示强相关。在本例中，

相关系数是0.83（根据日期不同可能有变化）。

知道两个有价值证券之间的相关性是有价值的，线性回归对两个有价值证券之间的关系建立模型。已知一个有价值证券的变化，我们可以用它预测另一个的变化。要将关系模型可视化，在工作表中插入一个散点图，X赋值为G列的值，Y赋值为C列的值。然后，在图表中右击series数据，然后在打开的快捷菜单上，选择AddTrendline。在显示的Format Trendline对话框中，选择Linear，并勾选“Display Equation on chart”和“Display R-Squared value on chart”。结果如图6-1所示。

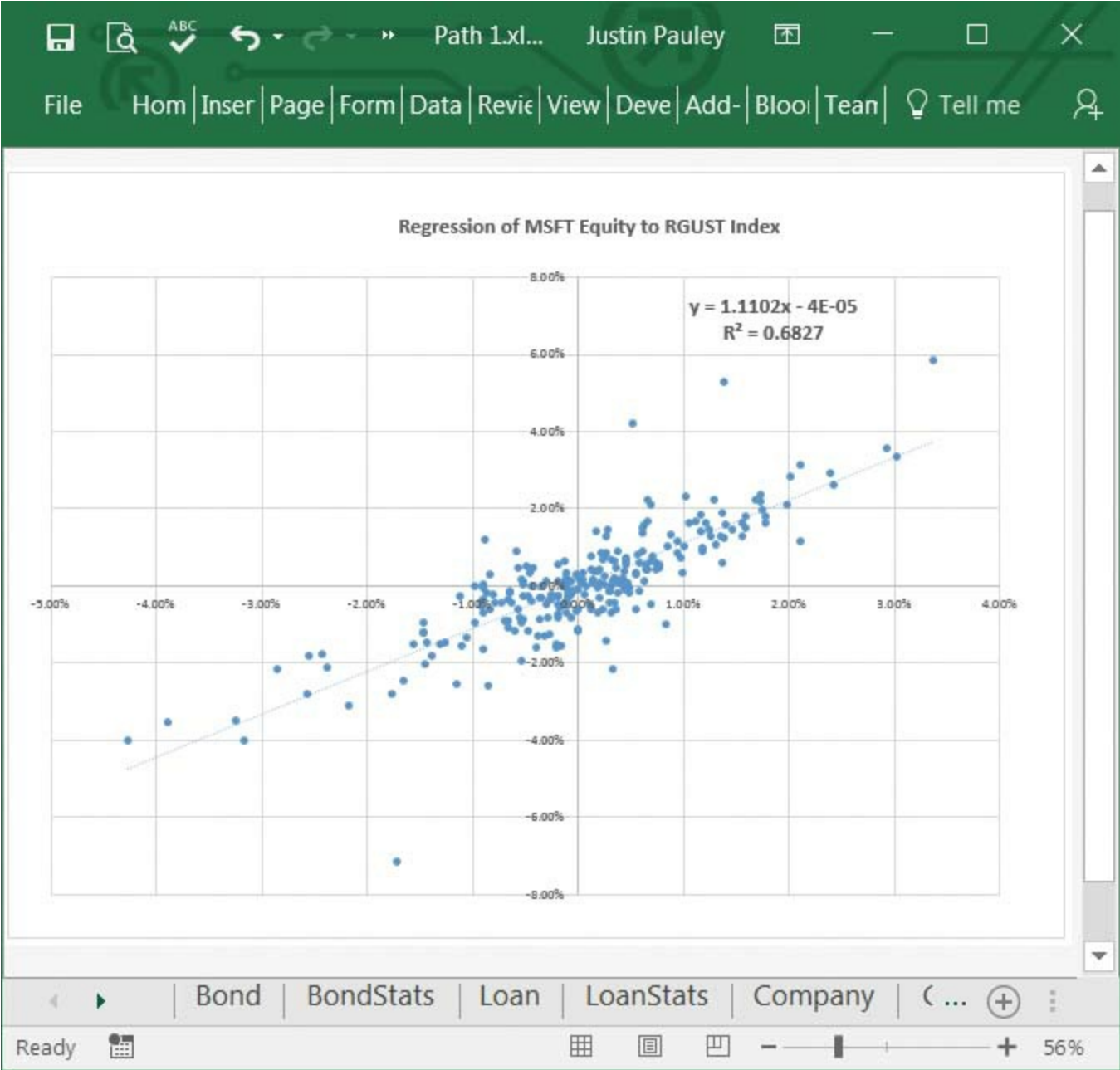


图6-1：MSFT和RGUST的回归

回归公式 “ $y=1.1102x-4E-05$ ” , 表示x (Russell 3000Technology Index) 每波动1% , y (Microsoft Equity) 大约会波动1.11% ($4E-05$ 是 4×10^{-5} 的科学计数法, 大约等于零)。1.11x是回归线的斜率, 也是该有价值证券相对于指数的Beta。将有价值证券和指数的每日变化作为实参, 则可以通过Excel中的SLOPE函数计算Beta。

下一个 (标签在单元格J4, 公式在单元格K4) 是R平方值, 介于零和代表回归方程式与历史数据拟合度之间的数值。R平方值表示的是回归方程式与数据拟合度有多高。用如下公式在单元格K4计算R平方值 (对有价值的单元格进行调整) :

=RSQ (C2 : C253 , G2 : G253)

RSQ公式的结果应该与图表中的数字相匹配, 在本例中是0.6827。R平方值的数值最理想的情况下是接近于1, 但是如你在图6-1中看到的, 回归线不能完美地匹配数据。

下一个公式 (单元格J5和K5) 是F统计值, 代表的是回归方程式与数据拟合的几率。例如, F统计值为15%表示回归方程式完全拟合的几率是15%。F统计值超过10%意味着你或许不应用该回归方程式。在此情况下, F统计值是 3.36×10^{-64} , 基本接近于0。计算F统计值的公式如下:

=FDIST (INDEX (LINEST (C2 : C253 , G2 : G253 , 1 , 1) , 4 , 1) , 1 ,
INDEX (LINEST (C2 : C253 , G2 : G253 , 1 , 1) , 4 , 2))

类似地, 我们需要计算P值, 代表的是Russell 3000Technology Index的值是否将对Microsoft股票产生实质影响。与F统计值一样, P值越小越好, 应小于5%。在此情况下, P值是 3.4×10^{-64} , 基本接近于0。不幸的是, 在Excel中没有简单的计算方法。所以分成三步计算。第一步计算见单元格J6和K6, 返回的就是相关性和回归分析中的观测数值:

=COUNT (C2 : C253)

第二步计算t统计, 见单元格J7和K7:

= (K3 * SQRT (K6 - 2)) / (SQRT (1 - K3^2))

最后是第三步计算，在单元格J8和K8中返回P值：

=TDIST (K7 , K6-2 , 2)

你可以用返回的工作表（见图6-2）显示两个投资之间的相关性和回归。例如，将RGUST Index改为S5INFT Index（S&P 500Information Technology Sector Index GICS Level 1），显示更高的相关性（0.84）、更高的R平方值（0.71）和接近于零的F统计值和P值。

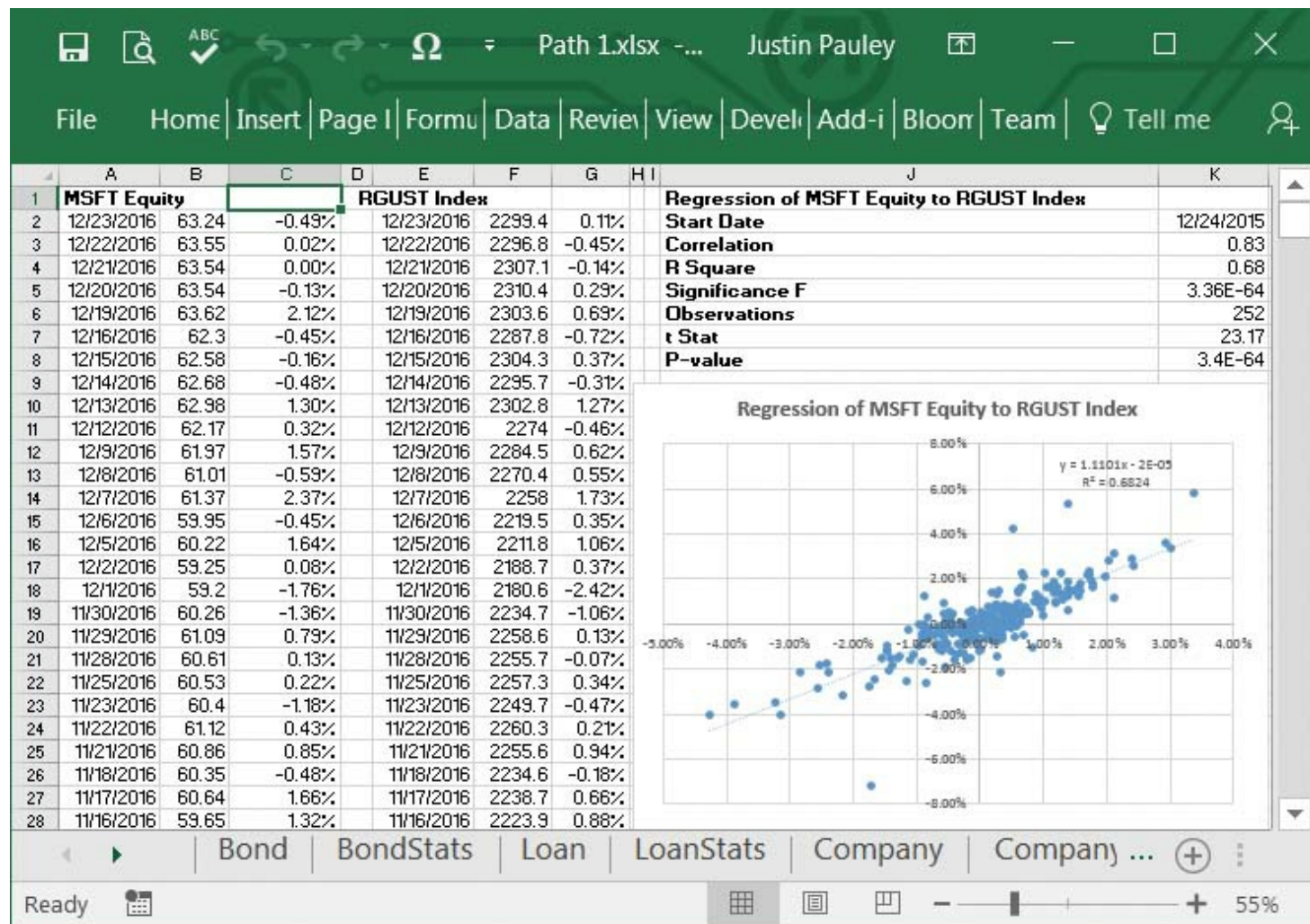


图6-2：回归工作表

我们可以创建一个矩阵同时比较多个有价值证券，这样就不用反复改变有价值证券来进行比较了。从单元格M3开始，逐行列出几个有价值证券。然后类似地，从单元格N2开始，逐列列出几个有价值证券或指数。这样，同一个有价值证券可能既出现在行里，又出现在列里。在单元格M2，输入历史数据的起始日期，或用之前的公式提取今天之前一年那一天：

=EDATE (TODAY () , -12)

最后，用如下数组公式在单元格N3计算与交叉单元格（N2和M3）中的有价证券之间的相关性；由于这是一个数组公式，因此在输入公式之后需要按Ctrl+Shift+Enter组合键：

```
=CORREL(
BDH($M3,"chg_pct_1d",$M$2,"","dates=hide","array=true","PER=CD","FI
"CDR=US"),
BDH(N$2,"chg_pct_1d",$M$2,"","dates=hide","array=true","PER=CD","FI
"CDR=US"))
```

前面的数组公式用彭博BDH函数提取两个有价证券的历史一日价格变化值数组，并把它作为CORREL函数的参数。PER、FILL和CDR实参（详见第3章）保证按日期排列。将该公式复制到矩阵中的所有单元格里。最后，选择带CORREL公式的单元格，然后，在函数区Home键上，点击Conditional Formatting，选择Color Scales下的第一个选项，让矩阵产生热图效应，其中强相关性用绿色标示，弱相关性用红色标示。在图6-3显示的相关性矩阵中，交叉单元格的行标题与列标题是相同的（在单元格N3中为MSFT Equity和MSFT Equity）。

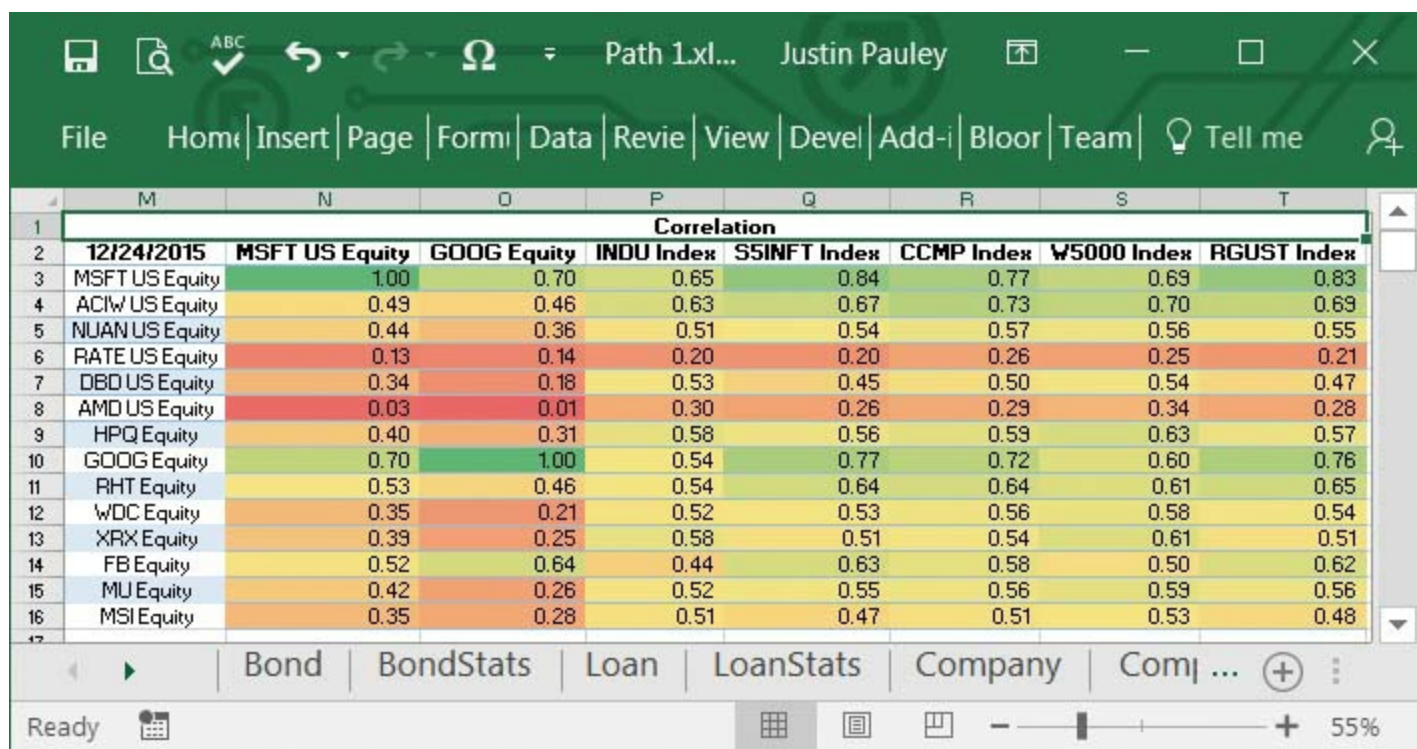


图6-3：相关性矩阵

你可以用相同的方法为R平方值创建一个矩阵，除非你用RSQ公式而不用CORREL公式。

6.1.2 对等组

比较投资优劣的下一步是将Loan、Bond和Company工作表中的有价值证券根据相似程度分成几个对等组。为有价值证券确定适当的组是一件主观的工作，难度不小。例如，你可以用债券的行业、评级、到期月份或者几种属性的组合等要素对债券进行分组。目标是分成一组的有价值证券有相似的风险和收益性质，可以在同组之间进行比较。是不是要进行更细致的分组，例如分成“金融软件开发企业”“5年或更久以后到期的高收益债券”，还是分成更宽泛的“短期推荐投资级企业债券”，可以由你自己决定。这些分组在后面都可以再进行优化。

首先检查本书中前面所述的Company工作表中的Category列。该列应包含对你最有意义的分类。花点时间读读每个公司的介绍，理解一下它为何存在是重要的一步，这能帮助你找出比彭博直接提供的行业列更加恰当的分组。确保用3.4.3节的技术，从Company工作表向Bond和Loan工作表的Category列添加一个引用。

然后，在Bond工作表中添加一列，命名为“Months Until Maturity”，用它来帮助判断投资是长期还是短期的。在列中填入如下公式，返回距离到期日还有几个月：

```
=DATEDIF ( TODAY ( ) , [@Maturity] , "m" )
```

然后还是在Bond工作表中添加一列，命名为“Peer Groups”，其中对每个债券包含一个对等组。我曾经提到过，确定适当的对等组是主观的，但还是建议先看新的Months Until Maturity列，并对什么是“短期”债券画一条清晰的界限。因为较短期的债券倾向于有类似的业绩性质，所以我把所有在四年（48个月）以内要到期的债券分组为“Short CCC”（如果评级为“CCC+”或更低）或“Short HY”（其他评级）。此外，我把其他所有“CCC+”或低评级债券都归入“CCC Corp”组，再用公司分类进行备注，例如用“FinTech”代表金融科技企业。我推荐你使用自己的分类，并用公司的类别和债券收益率作为对类似债券进行分类的指标。对Loan工作表重复该步。

6.1.3 评级

你可以很容易地对数字列（例如Spread、Yield、Coupon等）进行加总和比较，但是却不能对评级求平均值。此外，评级也不能转化为线性数字（例如AAA=1，AA+=2等），因为它们代表的默认概率是非线性的。幸运的是，穆迪提供了一个解决方案，名为评级因子（Rating Factor，RF）。Moody's RF用近似数字代表穆迪对于各企业10年默认评级的理想化概率累积值。用表6-1显示的评级因子数值除以10000，得到的就是10年默认评级的理想化概率累积值。例如，Baa1（RF值为260）的10年默认评级的理想化概率累积值为2.6%（260除以10000）。

表6-1：穆迪评级因子（来源：穆迪投资服务公司）

评级	评级因子	评级	评级因子
Aaa	1	Ba2	1 350
Aa1	10	Ba3	1 766
Aa2	20	B1	2 220
Aa3	40	B2	2 720
A1	70	B3	3 490
A2	120	Caa1	4 770
A3	180	Caa2	6 500
Baa1	260	Caa3	8 070
Baa2	360	Ca	10 000
Baa3	610	C	10 000
Ba1	940		

要把它填入工作簿，创建一个新的工作表，命名为“RF”，并在列A和B增加表6-1中的内容。然后，在Company、Loan和Bond工作表中添加一列，命名为“RF”，填入如下公式：

```
=IFERROR(VLOOKUP(IF(ISERR(FIND(" ",[@[Moody's Rating]])),  
[@[Moody's Rating]], LEFT([@[Moody's Rating]],FIND(" ",  
[@[Moody's Rating]])-1)), RF!A:B,2,FALSE),"")
```

该公式检查评级列中是否有空格（彭博可能将评级检测信息和评级一起返回，

我们想避免该情况发生)。如果有空格,就抓取空格之前的内容;否则抓取整个单元格,并用它从RF工作表VLOOKUP评级因子。如果在RF工作表上不存在评级因子(因为评级被撤销(WR)或者没有填入),公式就会返回一个空的字符串。

6.1.4 统计工作表

在本节中,我们要创建一个新的工作表,其中包含按照对等组划分的几个属性的中位数和标准差。该工作表将有助于将数据放入具体环境,分析趋势和潜在问题。首先创建一个新的工作表,命名为“BondStats”。在BondStats工作表中,从单元格A1开创建一个Excel表,命名为“MedianBondStats”,设置如下列标题:

- Peer Group
- Count
- YTD Px Chg
- 3M Px Chg
- YAS Spread
- YAS Yield
- Fixed Coupon
- Months Until Maturity
- RF

在Peer Group(对等组)列中,列出Bond工作表中Peer Group列中列出的所有不同的组。(提示:你可以整行粘贴对等组,并用Data选项卡上的Remove Duplicates按钮。)在Count列中,用如下公式计算每组的债券数量:

=COUNTIF(Bond[Peer Group],[@[Peer Group]])

在YTD Px Chg列，用如下数组公式计算每组今年迄今为止的价格变化。因为它是一个数组公式，因此在输入公式之后需要同时按Ctrl+Shift+Enter组合键。

```
=MEDIAN ( IF ( Bond[Peer Group]=[@[Peer Group]] , Bond[YTD Px Chg] ) )
```

对其他列重复该步骤。例如，3M Px Chg列应该用如下数组公式：

```
=MEDIAN ( IF ( Bond[Peer Group]=[@[Peer Group]] , Bond[3M Px Chg] ) )
```

完成后，你的BondStats工作表如图6-4所示。

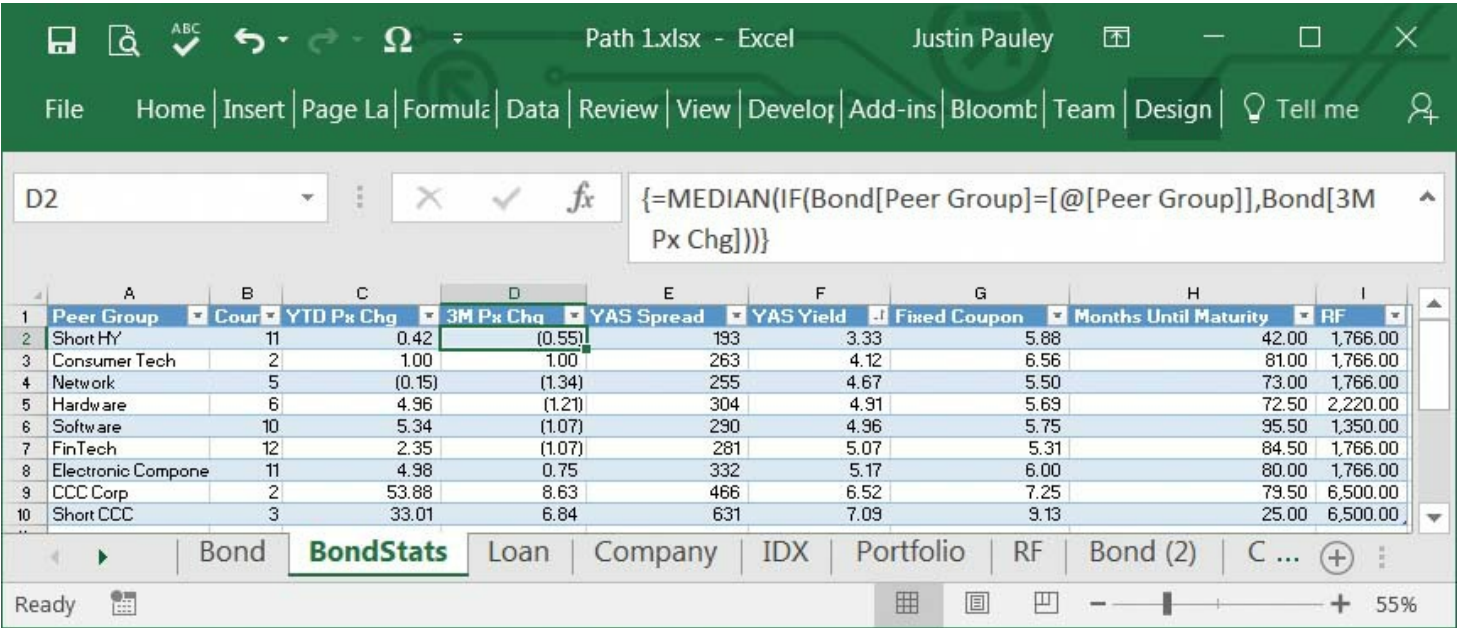


图6-4：BondStats工作表

继续向下之前，花点时间想一想这个数据表示什么，它是不是确实有意义。例如，和预计的一样，Short HY组的收益率最低，因为在其他条件都一样的情况下，债券期限越短，则风险越小，收益越低。同理，CCC组应该风险最大，收益率最高（评级因子也最高）。CCC Corp组今年迄今的价格变化达到53%，看起来有点高的离谱，但是只要简单看看该组里的两个债券，就知道两个从年初至今差不多都翻倍了。你的数据中包含的债券越多，该工作表越有意义。

然后，在MedianBondStats Excel表右边，创建另一个表，命名为“StdDevBondStats，”该表显示的是数据的离差。该表与MedianBondStats表

一样，除了不用MEDIAN函数，而用STDEV数组函数：

```
=STDEV ( IF ( Bond[Peer Group]=[@[Peer Group]] , Bond[YAS  
Spread] ) )
```

然后，在StdDevBondStats Excel表右边，创建另一个同样的表，命名为“AvgBondStats”。该表计算平均值，而非中位数或标准差。该表应该与前两个表一样，除了它使用AVERAGE数组函数，而不用MEDIAN或STDEV函数：

```
=AVERAGE ( IF ( Bond[Peer Group]=[@[Peer Group]] , Bond[YAS  
Spread] ) )
```

然后，重复上述步骤，创建LoanStats工作表，但是使用如下列标题：

·Peer Group

·Count

·YTD Px Chg

·3M Px Chg

·DM

·Yield

·Margin

·Months Until Maturity

·RF

最后，重复上述步骤，创建“CompanyStats”工作表，但是使用如下列标题。注意在CompanyStats工作表中用Category，而不用Peer Group。

·Category

- Count
- Market Cap
- Total Debt
- Total Debt/EBITDA
- Net Debt/EBITDA
- FCF/Total Debt
- 52Week High Chg
- 52Week Low Chg
- YTD Px Chg
- 3M Px Chg
- 12M Total Return

现在需要回头看一看，理解一下这些数据到底在说什么。试着按每列排序整理，看看有什么引人注目的。是否有某些组业绩较好/较差？今年迄今统计结果和过去三个月统计结果之间的差别是否能显示一定趋势？这些中位数是交叉分析数据的第一有用方法。

6.1.5 并排比较

下一步，我们把在BondStats、LoanStats和CompanyStats工作表中计算出的median stats代回入Bond、Loan和Company工作表。这样我们可以将单个有价证券与其所在组中的中位数进行并排比较。用MEDIAN函数，而非AVERAGE，以避免出现大的异常值的问题。

首先在Bond表RF列右侧插入一列：右击RF列右侧的列，然后，在Insert选项卡上选择“Table Column to the Left”。将该列命名为“Median RF”，填入如

下公式：

=INDEX (MedianBondStats[RF] , MATCH ([@[Peer Group]] ,
MedianBondStats[Peer Group] , 0))

该公式从Company工作表提取列的公式一样，除了在当前行的Peer Group单元
格与MedianBondStats表的Peer Group匹配的情况下，它从MedianBondStats表
提取RF列。推荐突出标示该列，以表明其包含的是对等组的中位数信息，而非关于
当前债券的数据。结果应该如图6-5所示。

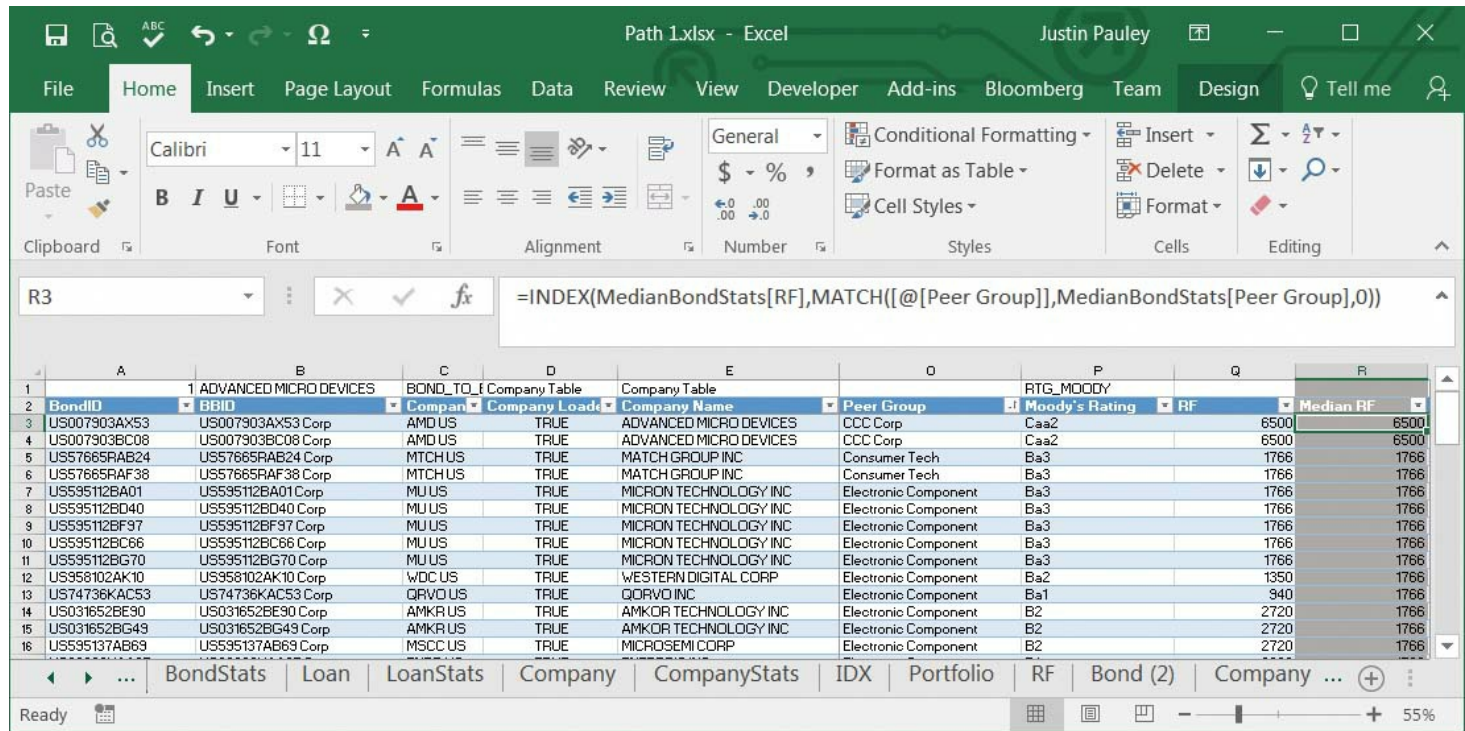


图6-5：带Median RF的Bond工作表

重复该步，从MedianBondStats表向Bond表增加其他列。例如，如下是增加
Median Fixed Coupon列的公式：

=INDEX(MedianBondStats[Fixed Coupon],MATCH([@[Peer Group]],
MedianBondStats[Peer Group],0))

在从MedianBondStats之后增加每列之后，花点时间审视一下Bond表。随着整
理筛查数据，你可以用中位数数据对有价值证券在上下文中进行具体解释。例如，你
可能注意到了某个债券的YTD价格变化高于平均值，但现在你可能注意到与其对等债

券是一致的。或者如果它的业绩高于其对等债券，你可以对数值进行量化。

重复该步骤，用如下公式从MedianLoanStats向Loan表增加列：

```
=INDEX(MedianLoanStats[Margin],MATCH([@[Peer Group]],  
MedianLoanStats[Peer Group],0))
```

然后，重复该步骤，用如下公式从MedianCompanyStats向Company表增加列，公式里用到的是Category，而非Peer Group：

```
=INDEX(MedianCompanyStats[12M Total Return],MATCH([@Category],  
MedianCompanyStats[Category],0))
```

完成该步骤之后，对Peer Group（或者Company表中的Category）进行整理排序，并将每个有价值证券与其同组进行比较。使这些业绩指标在上下文中有意义是确定买入价、卖出价和中间价的重要环节。你可能注意到了，例如某有价值证券基本面不错，技术面也在改善，但是交易价格较低，因为它被某机构评为CCC级。或者某有价值证券滞后于同组的其他有价值证券。在进行比较的时候，你可以有大量有价值的发现。随着潜在数据或对等组变化，median stats会自动进行更新。

6.1.6 指数

除了将有价值证券与对等组进行比较外，还要将其与指数（也就是更完备的参照基准）进行比较。每个有价值证券都可以与一个或多个指数进行比较。在本节中，我们介绍如何将Company工作表中的股票与一个指数进行比较。你可以将该技术应用到将有价值证券与多个指数比较中去，或者扩展到Loan和Bond工作表。

首先，在Company工作表中增加一列，命名为“IndexID”。在该列，输入你想用作每个股票对比指数那个IDX工作表的IndexID。然后，再增加三列：“Index YTD Px Change” “Index 3M Px Change” 和 “Index 12M Total Return”。然后在Index YTD Px Change列，用INDEX函数从IDX工作表对给定IndexID提取YTD Px Change列：

```
=INDEX(IDX[YTD Px Change],MATCH([@IndexID],IDX[IndexID],0))
```

重复该步骤从IDX工作表向Company工作表提取3M Px Change和12M Total Return列。你可以选择添加一个列“Index Correlation”，使用上个日历年度的数据，用如下数组公式（按Ctrl+Shift+Enter组合键）返回公司和指数之间的相关性：

```
=CORREL(BDH([@BBID],"chg_pct_1d","-1CY","", "dates=hide", "array=true"),  
BDH(INDEX(IDX[BBID],MATCH([@IndexID],IDX[IndexID],0)),  
"chg_pct_1d","-1CY","", "dates=hide", "array=true"))
```

Company工作表上有指数信息之后，你可以把Index列放到公司列旁边，并列对比业绩。

6.1.7 加权Z值

由于指标太多，投资之间的差异也千差万别，因此很难对有价值证券从“最好”到“最差”进行排序。例如，某债券可能票面利息高，但是评级差。此外，一些值用百分数表示，而另一些值则使用不同的单位。简化这些指标成为一个“分数（score）”的一种技巧是用Z值（Z-score，也称为标准分）。Z值测量一个数据点和平均值之间的标准差数值。换句话说，为了使各种指标标准化，Z值显示该指标与平均值相比更好或更差的程度。Z值的算法是用数据点和平均值之差，除以标准差，公式如下：

$$Z\text{值} = (\text{值} - \text{所有值的平均值}) / \text{所有值的标准差}$$

例如，如果某债券票面利息为8%，而所有债券的票面利息平均值为5.63%，所有债券的票面利息标准差为1.13%，则债券票面利息的Z值为2.09——
 $Z = (8\% - 5.63\%) / 1.13\%$ 。该Z值表示，它的票面利息比平均值高两个标准差值。Z值也可能是负值；Z值等于-1代表该值比平均值低一个标准差值。

尽管确定一个值与平均值之间的标准差数是有用的，不过Z值还可以将多个属性值简化成一个单一分数，方法是对这些Z值取一个加权平均值，其中基于一个指标相对于其他指标的重要程度来设置权重。例如，如果我们计算了某债券的票面利息、评级和距离到期日月份数的Z值，我们就可以对这些Z值取一个加权平均值，并对票面利息Z值设置一个较大的权重，以显示更喜欢票面利息较高的债券。

不幸的是，在Bond、Loan和Company工作表中增加z值需要好几个步骤：

1.在BondStats工作表AvgBondStats表右边，增加四个新的列标题，命名为Field、Mean、StdDev和Weight。

2.在Field列标题下，列出所有要比较的字段，例如YTD Px Chg、3M Px Chg、YAS Spread、YAS Yield、Fixed Coupon、Months Until Maturity和RF。

3.在Mean列标题下，用AVERAGE函数计算Bond表中每一列的平均值。例如，挨着“YTD Px Chg”的列用如下公式：

```
=AVERAGE ( Bond[YTD Px Chg] )
```

4.同理，在StdDev列，用STDEV函数为每列计算标准差。例如，在StdDev列中挨着YAS Yield，用如下公式：

```
=STDEV ( Bond[YAS Yield] )
```

5.在Weight列下，输入一个百分比权重值，所有百分比权重值加和等于百分之百。

你可以在后面修改权重值，并且在开始阶段对每个字段设置相等的字段比较容易。

结果表如图6-6所示。

	AE	AF	AG	AH	AI
1	Field	Mean	StdDev	Weight	
2	YTD Px Chg	0.30	0.53	14.3%	
3	3M Px Chg	0.48	3.26	14.3%	
4	YAS Spread	276.53	167.78	14.3%	
5	YAS Yield	4.30	1.93	14.3%	
6	Fixed Coupon	6.14	1.37	14.3%	
7	Months Until Maturity	78.92	50.50	14.3%	
8	RF	2,236.81	1,481.12	14.3%	
9				100.0%	
10					

图6-6：Z值权重表

6.用左上角的Name框，为Mean、StdDev和Weight列中的每个单元格命名。

每个定义名的开头都应为“Bond”，随后是字段名，最后是列标题。例如，将包含YTD Px Chg列平均值的单元格命名为“BondYTD Px ChgMean”，并将包含Fixed Coupon列权重值的单元格命名为“BondFixedCouponWeight”。

尽管为表中每个单元格命名有点麻烦，但是这使在Bond工作表中构建复杂的Z值计算变得简单了许多。

7.在新表格中的每个单元格都被命名之后，在Bond工作表中添加一个新的列，命名为“Z值”。

8.在该列，对于权重值越高越好的列，将Z值分别与其权重值相乘之后再相加，再减去权重值越低越好的列Z值分别与其权重值相乘结果之和。

例如，公式开头是计算YTD Px Change的Z值，然后与其权重值相乘：

= (([@[YTD Px Chg]] - BondYTD Px ChgMean) / BondYTD Px ChgStddev * BondYTD Px ChgWeight)

9.增加下一个字段3M Px Chg，使公式变为：

$$=(([@[YTD Px Chg]] - \text{BondYTD Px Chg Mean}) / \text{BondYTD Px Chg Stdev} * \text{BondYTD Px Chg Weight}) + (([@[3M Px Chg]] - \text{Bond3M Px Chg Mean}) / \text{Bond3M Px Chg Stdev} * \text{Bond3M Px Chg Weight})$$

在增加其他列之后，最终公式应该看起来如下所示：

$$\begin{aligned} &=(([@[YTD Px Chg]] - \text{BondYTD Px Chg Mean}) / \text{BondYTD Px Chg Stdev} * \text{BondYTD Px Chg Weight}) + (([@[3M Px Chg]] - \text{Bond3M Px Chg Mean}) / \text{Bond3M Px Chg Stdev} * \text{Bond3M Px Chg Weight}) + \\ &(([@[YAS Spread]] - \text{BondYAS Spread Mean}) / \text{BondYAS Spread Stdev} * \text{BondYAS Spread Weight}) + \\ &(([@[YAS Yield]] - \text{BondYAS Yield Mean}) / \text{BondYAS Yield Stdev} * \text{BondYAS Yield Weight}) + (([@[Fixed Coupon]] - \text{BondFixed Coupon Mean}) / \\ &\text{BondFixed Coupon Stdev} * \text{BondFixed Coupon Weight}) - (([@[Months Until Maturity]] - \text{BondMonths Until Mat Mean}) / \\ &\text{BondMonths Until Mat Stdev} * \text{BondMonths Until Mat Weight}) - (([@RF] - \text{BondRF Mean}) / \text{BondRF Stdev} * \text{BondRF Weight}) \end{aligned}$$

注意对于Months Until Maturity和RF的加权z值用减法，而不是用加法，因为这些值越小越好。

现在，你可以对z值列从高到低（从“好”到“坏”）进行排序了。花点时间调整一下权重值，观察一下对z值列的影响。例如，如果除了Months Until Maturity之外的所有权重都变为零，Months Until Maturity的权重设为100%，则z值最大的就是最短期的债券。把Fixed Coupon和RF的权重值各设为50%，其他的权重值设为零，再次对z值列进行排序，则表单越上面的债券票面利息越高，而评级越低越排在表单下面。你可以对Loan和Company工作表重复z值分析。

与本书中的其他分析一样，投资决策不能仅靠z值。它仅是聚焦投资观点，分析投资数据的一种工具而已。

6.2 途径2：Access

因为Access数据库中的表连接到Excel工作簿上，所以用前一节（途径1）的步骤，更新Excel工作簿和Access数据库表。在Excel中计算相关性，回归和中位数等内容就不在Access部分赘述了，但是理解如何计算也是有帮助的。本节介绍Excel（途径1）中的内容如何用Microsoft Access技巧实现。

正如前面的章节在多个工作表（Bond、Loan、Company）中增加了列，连接的Access表也将包含这些新的列。结果是，用于维护历史（BondHistQuery、CompanyHistQuery等）的查询会失败，除非相应的历史表（BondHist、CompanyHist等）也进行了修改，以匹配相同的列（模式）。

6.2.1 Access中的相关性和回归

为了实现本节的目标，我们创建一个简化的表，命名为“Comp”，其中包含两个有价值证券的历史每日价格变动。按照表6-2的模式创建Comp表（在Microsoft Access中的“Create”菜单下选择“Table”）。该表只在本节使用，后面不会再用到了。

表6-2：Comp表模式

字段名	数据类型
HistDate	日期 / 时间（设为主关键字）
X	数字（字段规模设为 Double）
Y	数字（字段规模设为 Double）

打开新创建的Comp表，填入值。在X和Y列填入不同有价值证券的每日价格百分比变化值；或者从Regression工作表复制粘贴（在Access中用Paste Append）列A（HistDate列）、G（X列）和C（Y列）以匹配Excel结果。

要计算X和Y列之间的相关性，用如下查询：

```
SELECT (Avg(X * Y) - Avg(X) * Avg(Y)) / (StDevP(X) * StDevP(Y))  
AS Correlation FROM Comp;
```

要计算回归方程式，先用如下查询计算回归系数：

```
select (Avg(X * Y) - Avg(X) * Avg(Y)) / VarP(X) AS RegressCoeff
from Comp;
```

然后用如下公式计算截距 (intercept)：

```
select Avg(Y) - ((Avg(X * Y) - Avg(X) * Avg(Y)) / VarP(X)) *
Avg(X) AS Intercept from Comp;
```

用两个结果创建回归方程式： $Y = \text{RegressCoeff} * X + \text{Intercept}$ 。用Regression工作表中的数据作为一个例子，知道Russell 3000Technology Index的值，要预测Microsoft的股价，就用Russell 3000Technology Index的值乘以回归系数，再加上截距。

要确定回归方程式与历史数据的匹配程度，用下面的查询计算R平方值：

```
select (Avg(X * Y) - Avg(X) * Avg(Y)) ^ 2 / (VarP(X) * VarP(Y))
as RSquare from Comp;
```

6.2.2 Access中的中位数

不幸的是，中位数并不是Access包含的标准的聚合函数，因此需要一些Visual Basic Application (VBA) 代码进行计算。幸运的是，Microsoft提供了所需代码 ([https://msdn.microsoft.com/en-us/library/dd789431\(v=office.12\).aspx](https://msdn.microsoft.com/en-us/library/dd789431(v=office.12).aspx))。

首先，在Access中，在Createtab上点击Module。然后在代码窗口插入如下Microsoft代码函数，然后关闭Visual Basic窗口：

```
Public Function DMedian( _ ByVal strField As String, ByVal
strDomain As String, _ Optional ByVal strCriteria As String) As
Variant ' Purpose: ' To calculate the median value ' for a field in
a table or query. ' In: ' strField: the field. ' strDomain: the
table or query. ' strCriteria: an optional WHERE clause to ' apply
```

```

to the table or query. ' Out: ' Return value: the median, if
successful; ' Otherwise, an Error value. Dim db As DAO.Database Dim
rstDomain As DAO.Recordset Dim strSQL As String Dim varMedian As
Variant Dim intFieldType As Integer Dim intRecords As Integer Const
errAppTypeError = 3169 On Error GoTo HandleErr Set db = CurrentDb()
' Initialize return value. varMedian = Null ' Build SQL string for
recordset. strSQL = "SELECT " & strField & " FROM " & strDomain '
Only use a WHERE clause if one is passed in. If Len(strCriteria) >
0 Then strSQL = strSQL & " WHERE " & strCriteria End If strSQL =
strSQL & " ORDER BY " & strField Set rstDomain =
db.OpenRecordset(strSQL, dbOpenSnapshot) ' Check the data type of
the median field. intFieldType = rstDomain.Fields(strField).Type
Select Case intFieldType Case dbByte, dbInteger, dbLong, _
dbCurrency, dbSingle, dbDouble, dbDate ' Numeric field. If Not
rstDomain.EOF Then rstDomain.MoveLast intRecords =
rstDomain.RecordCount ' Start from the first record.
rstDomain.MoveFirst If (intRecords Mod 2) = 0 Then ' Even number of
records. ' No middle record, so move to the ' record right before
the middle. rstDomain.Move ((intRecords \ 2) - 1) varMedian =
rstDomain.Fields(strField) ' Now move to the next record, the ' one
right after the middle. rstDomain.MoveNext ' And average the two
values. varMedian = _ (varMedian + rstDomain.Fields(strField)) / 2
' Make sure you return a date, even when ' averaging two dates. If
intFieldType = dbDate And Not IsNull(varMedian) Then varMedian =
CDate(varMedian) End If Else ' Odd number or records. ' Move to the
middle record and return its value. rstDomain.Move ((intRecords \
2)) varMedian = rstDomain.Fields(strField) End If Else ' No
records; return Null. varMedian = Null End If Case Else ' Non-
numeric field; so raise an app error. Err.Raise errAppTypeError End
Select DMedian = varMedian ExitHere: On Error Resume Next
rstDomain.Close Set rstDomain = Nothing Exit Function HandleErr: '
Return an error value. DMedian = CVErr(Err.Number) Resume ExitHere

```

End Function

DMedian函数有三个实参：列名、表格名和where子句标准（可选）。注意每个参数都是“被引用的（quoted）”，列名应该在方括号中（[and]）。例如，计算Bond表中YAS Spread超过250bps的RF中位数，用如下查询：

```
select dMedian("rf","Bond","[YAS Spread] > 250")
```

6.3 途径3：C#

本节介绍6.1节中的内容如何用C#实现。如果你还没有读过6.1节，我推荐你还是读一读，以便更深入地理解这些计算及其目的。

6.3.1 相关性和回归

本节创建一个应用，用于从彭博提取两个有价证券的历史每日价格变动，并计算其相关性、回归方程式和R平方值。该代码需要用到一个很流行的数学库MathNet。开始一个新的控制台应用，并在新项目中右击References，选择Manage NuGet Packages，以添加一个引用到MathNet。搜索MathNet.Numerics，然后点击Install。此外，添加一个引用到第3章用过的彭博库。

在Program.cs中，首先增加如下using指令：

```
using Element = Bloomberglp.Blpapi.Element; using Message =  
Bloomberglp.Blpapi.Message; using Name = Bloomberglp.Blpapi.Name;  
using Request = Bloomberglp.Blpapi.Request; using Service =  
Bloomberglp.Blpapi.Service; using Session =  
Bloomberglp.Blpapi.Session; using DataType =  
Bloomberglp.Blpapi.Schema.Datatype; using SessionOptions =  
Bloomberglp.Blpapi.SessionOptions; using InvalidRequestException =  
Bloomberglp.Blpapi.InvalidRequestException; using Datetime =  
Bloomberglp.Blpapi.Datetime; using System.Collections; using  
MathNet.Numerics.Statistics; using MathNet.Numerics;
```

然后在Program类，增加在前面的例子中用过的典型彭博语句：

```
private static readonly Name SECURITY_DATA = new
Name("securityData"); private static readonly Name SECURITY = new
Name("security"); private static readonly Name FIELD_DATA = new
Name("fieldData"); private static readonly Name RESPONSE_ERROR =
new Name("responseError"); private static readonly Name
SECURITY_ERROR = new Name("securityError"); private static readonly
Name FIELD_EXCEPTIONS = new Name("fieldExceptions"); private static
readonly Name FIELD_ID = new Name("fieldId"); private static
readonly Name ERROR_INFO = new Name("errorInfo"); private static
readonly Name CATEGORY = new Name("category"); private static
readonly Name MESSAGE = new Name("message"); private static
readonly Name DATE = new Name("date");
```

增加Main函数的主体部分，该函数实例化Program类并调用Run方法。

```
static void Main(string[] args) { Program p = new Program();
p.Run(); }
```

Run方法从彭博获取历史数据值，然后用MathNet计算相关性、截距、斜率和R平方值。Run方法从GetHistory方法（在后面定义）得到历史彭博数据，GetHistory方法将对特定证券、字段和起始日期返回一个包含日期-值（date-value）对的Dictionary。结束日期总是今天。GetHistory返回数据值之后，用MathNet库计算必要的统计数据：

```
public void Run() { //Declare the securities to compare X,Y
string secY = "/ticker/MSFT US Equity"; string secX =
"/ticker/S5INFT Index"; //Declare the field to use to compare
string field = "chg_pct_1d"; // One day price change //Declare the
starting date, end date is today. DateTime startDate = new
DateTime(2015, 12, 24); //Retrieve the historical points from
Bloomberg Dictionary<DateTime, double> resultY = GetHistory(secY,
field, startDate); Dictionary<DateTime, double> resultX =
```



```

GetHistory(secX, field, startDate); //Use MathNet to calculate the
correlation double correlation =
Correlation.Pearson(resultY.Values, resultX.Values); //Use MathNet
to calculate the intercept and slope Tuple<double, double> p =
Fit.Line(resultX.Values.ToArray(), resultY.Values.ToArray());
double intercept = p.Item1; double slope = p.Item2; //Use MathNet
to calculate the R Square double rsqr =
GoodnessOfFit.RSquared(resultX.Values.Select(x => intercept + slope
* x), resultY.Values); Console.WriteLine("Correlation is " +
correlation.ToString()); Console.WriteLine("Regression equation is
y=" + slope.ToString() + "x+" + intercept.ToString());
Console.WriteLine("R Square is " + rsqr.ToString());
Console.ReadLine(); }

```

GetHistory方法和前面的彭博示例很像，除了它将一个Dictionary对象传递给ProcessHistoryResponse，以存储请求结果：

```

private Dictionary<DateTime, double> GetHistory (string
security, string field, DateTime startDate) { Dictionary<DateTime,
double> date2value = new Dictionary<DateTime, double>();
SessionOptions sessionOptions = new SessionOptions(); Session
session = new Session(); bool sessionStarted = session.Start(); if
(!sessionStarted) { System.Console.Error.WriteLine("Failed to start
session."); return null; } if
(!session.OpenService("//blp/refdata")) {
System.Console.Error.WriteLine("Failed to open //blp/refdata");
return null; } Service refDataService =
session.GetService("//blp/refdata"); Request request =
refDataService.CreateRequest("HistoricalDataRequest"); Element
securities = request.GetElement("securities");
securities.AppendValue(security); Element fields =
request.GetElement("fields"); fields.AppendValue(field);
request.Set("startDate", startDate.ToString("yyyyMMdd")); try {

```

```

session.SendRequest(request, null); } catch
(InvalidRequestException e) {
System.Console.WriteLine(e.ToString()); } bool done = false; while
(!done) { Event eventObj = session.NextEvent(); if (eventObj.Type
== Event.EventType.PARTIAL_RESPONSE) {
ProcessHistoryResponse(eventObj, date2value); } else if
(eventObj.Type == Event.EventType.RESPONSE) {
ProcessHistoryResponse(eventObj, date2value); done = true; } else {
foreach (Message msg in eventObj) {
System.Console.WriteLine(msg.AsElement); if (eventObj.Type ==
Event.EventType.SESSION_STATUS) { if
(msg.MessageType.Equals("SessionTerminated")) { done = true; } } }
} } session.Stop(); return date2value; }

```

类似地，ProcessHistoryResponse方法与在第3章中的一样，除了它用从彭博请求中得到的日期和值填入Dictionary对象参数：

```

private void ProcessHistoryResponse(Event eventObj,
Dictionary<DateTime, double> date2value) { foreach (Message msg in
eventObj) { if (msg.HasElement(RESPONSE_ERROR)) { Element error =
msg.GetElement(RESPONSE_ERROR); Console.WriteLine("Request failed:
" + error.GetElementAsString(CATEGORY) + " (" +
error.GetElementAsString(MESSAGE) + ")"); continue; } Element
securityData = msg.GetElement(SEcurity_DATA); string security =
securityData.GetElement(SEcurity).GetValueAsString();
Console.WriteLine(security); Element fieldData =
securityData.GetElement(FIELD_DATA); for (int i = 0; i <
fieldData.NumValues; i++) { Element element =
fieldData.GetValueAsElement(i); DateTime date =
element.GetElementAsDatetime(DATE).ToSystemDateTime(); double?
value = null; for (int f = 0; f < element.NumElements; f++) {
Element field = element.GetElement(f); if
(!field.Name.Equals(DATE)) { if (field.Datatype ==

```

```
DataType.FLOAT32) value = Convert.ToDouble(field.  
GetValueAsFloat32()); else if (field.Datatype == DataType.FLOAT64)  
value = field.GetValueAsFloat64(); } } if (value != null)  
date2value.Add(date, value.Value); } } }
```

6.3.2 对等组

和途径1一样，本节介绍如何向Bond和Loan表添加一个“Peer Group”列，创建并填入新表，以存放Bond、Loan和Company表的摘要统计信息，并并排查询摘要统计及其分别对应的有价证券。

第一步是向Access中的Bond和Loan两个表都添加一个“PeerGroup”列，类型为“短文本 (Short Text)”。然后按照途径1所述，在新的PeerGroup列为每个有价证券填入适当的对等组。

6.3.3 评级

然后，我们需要像在本章之前讨论的那样合并穆迪评级因子。首先创建一个新的表，命名为“RF”，用表6-3描述的模式。之后，用表6-1的内容填入RF表。

表6-3：RF表模式

字段名	数据类型
Rating (主关键字)	短文本
Factor	数字

你可以用如下查询把RF表连接到Bond或Loan表：

```
Select SecurityDes,MoodyRating,Factor from Bond b left outer  
join RF on RF.Rating= iif(instr(b.MoodyRating,"  
)>0,left(b.MoodyRating,instr(b.MoodyRating," ")-1), b.MoodyRating)
```

该查询选择Bond表中的每个债券的描述、穆迪评级和穆迪评级因子。在连接实参中，用iif函数从MoodyRating列删除任何评级警告信息。

6.3.4 统计表

本节介绍如何创建表和用每个对等组的整体统计数据填入表。首先，创建一个表，存储不同的整体统计数据，例如中位数、平均值和标准差。把新创建的表命名为“MedianBondStats”，用表6-4的模式。你可以在该表中增加更多的列，但是它们的名字和大小写应该与在Bond表中的一致。

表6-4：MedianBondStats表模式

字段名	数据类型
PeerGroup (主关键字)	短文本
Count	数字
PxChgYTD	数字 (字段规模设为 Double)
PxChg3M	数字 (字段规模设为 Double)
YASSpread	数字 (字段规模设为 Double)
YASYield	数字 (字段规模设为 Double)
FixedCpn	数字 (字段规模设为 Double)
MonthsUntilMaturity	数字 (字段规模设为 Double)
RF	数字

然后，复制粘贴该表两次，这样就创建了两个一模一样的表，分别命名为“MeanBondStats”和“StdevBondStats”。

重复这些步骤，用表6-5的模式创建MedianLoanStats、MeanLoanStats和Stdev-LoanStats表。

表6-5：MedianLoanStats表模式

字段名	数据类型
PeerGroup (主关键字)	短文本
Count	数字
PxChgYTD	数字 (字段规模为 Double)
PxChg3M	数字 (字段规模为 Double)
DM	数字 (字段规模为 Double)
Yield	数字 (字段规模为 Double)
Margin	数字 (字段规模为 Double)
MonthsUntilMaturity	数字 (字段规模为 Double)
RF	数字

再次重复这些步骤，用表6-6的模式创建MedianCompanyStats、MeanCompanyStats和StdevCompanyStats表。

表6-6：MedianCompanyStats表模式

字段名	数据类型
Category (主关键字)	短文本
Count	数字
MarketCap	数字 (字段规模为 Double)
TotalDebt	数字 (字段规模为 Double)
TotalDebtToEBITDA	数字 (字段规模为 Double)
NetDebtToEBITDA	数字 (字段规模为 Double)
FCFToTotalDebt	数字 (字段规模为 Double)
YrHi	数字 (字段规模为 Double)
YrLow	数字 (字段规模为 Double)
PxChgYTD	数字 (字段规模为 Double)
PxChg3M	数字 (字段规模为 Double)
TotalReturn12M	数字 (字段规模为 Double)

然后，创建一个新的C#控制台应用以填入这些表。代码在Bond、Loan和Company表中循环，将数据按照对等组（或者Company表中的Category）和列名分成几个表单。然后它按照中位数、平均值和标准差对各表单进行合计。最后，它用

合计统计数据插入或更新中位数、平均值和标准差表。

和之前连接到Access数据库的代码组一样，程序开始时先声明一个连接字符串，并在Main函数中调用Program类的Run方法：

```
private string ConnStr =  
"Provider=Microsoft.ACE.OLEDB.12.0;Data Source=..\..\Path  
3.accdb"; static void Main(string[] args) { Program p = new  
Program(); p.Run(); }
```

然后，定义Run方法，并调用ProcessTable方法，把表名和要分组的列作为实参。ProcessTable方法返回中位数、平均值和标准差表中更新的行数。

```
public void Run() { int rowc = 0; rowc += ProcessTable("Bond",  
"PeerGroup"); rowc += ProcessTable("Loan", "PeerGroup"); rowc +=  
ProcessTable("Company", "Category"); Console.WriteLine(rowc + "  
rows updated"); }
```

ProcessTable方法将全部数据从一个表（表参数）中提取出来，按照PeerGroup列（或者Company表中的Category）对数据进行分组，计算中位数、平均数和标准差，并存储在MedianStats、MeanStats和StdevStats表中（例如MedianBondStats）。

ProcessTable方法先定义那些合计表的名称，并调用FillDataSet从数据库填入内容。后面定义的FillDataSet方法把每个表的内容作为实参填入数据集代码。此外，在适当地使用SQL的情况下，它会排除缺失Peer Group或Category的行，并增加RF和MonthsUntilMaturity列。

然后，ProcessTable方法将数据集、表名和要分组的列传递给CategorizeData方法（后面定义），返回一个多维数组，把表中的数据按PeerGroup（或者Company表中的Category）和列名进行分解。

变量peergroup2column2values的类型声明很复杂，所以我们分解一下。该对象包括一个对等组表单，对于每个对等组都包括一个列名单表，对于每个对等组和列名组合，对象都包括一个数据库中的值的表单。换句话说，它允许你循环访问

peerGroup2column2values["Short CCC"]["YASSpread"], 以便对 "Short CCC" 对等组引用所有的YASSpread double值。

然后, ProcessTable方法循环访问对等组表单, 对该对等组引用中位数、平均数和标准差表中已存在的行。如果行不存在, 就创建行。

然后, 该方法循环访问与对等组相关的列的表单, 计算中位数、平均数和标准差, 并对Dictionary对象中的值进行计数。最后, 该方法用UpdateDataSet方法在适当的行设定对应值, 并更新数据集(后面定义)。

让我们看看相关代码:

```
public int ProcessTable(string table, string groupBy) { string
mediantable = "Median" + table + "Stats"; string meantable = "Mean"
+ table + "Stats"; string stdevtable = "Stdev" + table + "Stats";
//Get data for table, median, mean, and stddev tables // from the
Access database DataSet ds = FillDataSet(table, mediantable,
meantable, stdevtable); // This Data structure will hold a
Dictionary // of Peer Groups -> Column -> Values // For instance,
to get all of the Price values for the Tech peer group // you would
iterate through the // Double List peerGroup2column2values["Tech"]
["Price"] Dictionary<string, Dictionary<string, List<double>>>
peerGroup2- column2values = CategorizeData(ds, table, mediantable,
groupBy); foreach (string peerGroup in
peerGroup2column2values.Keys) { DataRow medianRow = null; DataRow
meanRow = null; DataRow stdDevRow = null; //Find the rows for this
Peer Group in the median, mean, and stdDev tables medianRow =
ds.Tables[mediantable].AsEnumerable(). SingleOrDefault(x =>
x.Field<string>(groupBy) == peerGroup); meanRow =
ds.Tables[meantable].AsEnumerable(). SingleOrDefault(x =>
x.Field<string>(groupBy) == peerGroup); stdDevRow =
ds.Tables[stdevtable].AsEnumerable(). SingleOrDefault(x =>
x.Field<string>(groupBy) == peerGroup); //If they couldn't be
```



```

found, add them. if (medianRow == null) { medianRow =
ds.Tables[mediantable].NewRow(); medianRow[groupBy] = peergroup;
ds.Tables[mediantable].Rows.Add(medianRow); } if (meanRow == null)
{ meanRow = ds.Tables[meantable].NewRow(); meanRow[groupBy] =
peergroup; ds.Tables[meantable].Rows.Add(meanRow); } if (stdDevRow
== null) { stdDevRow = ds.Tables[stdevtable].NewRow();
stdDevRow[groupBy] = peergroup;
ds.Tables[stdevtable].Rows.Add(stdDevRow); } foreach (string column
in peergroup2column2values[peergroup].Keys) { // For each column
in each peer group, calculate the // median, stddev, mean, and
count. double median = GetMedian(peergroup2column2values[peergr-
oup][column]); double stddev =
GetStandardDev(peergroup2column2values[peergroup][column]); double
mean = peergroup2column2values[peergroup][column].Average(); int
count = peergroup2column2values[peergroup][column].Count; // put
the statistics in their correct tables meanRow[column] = mean;
medianRow[column] = median; if (Double.IsNaN(stddev))
stdDevRow[column] = DBNull.Value; else stdDevRow[column] = stddev;
meanRow["Count"] = count; medianRow["Count"] = count;
stdDevRow["Count"] = count; } } int rowc = UpdateDataSet(ds,
mediantable, meantable, stdevtable); return rowc; }

```

CategorizeData方法按照对等组和列名对表中的数据分组，并将数据返回到ProcessTable方法。它只包括在对应MedianStats表中也存在的列。

CategorizeData方法还向Dictionary对象增加一个“Total”对等组，其中是每个对等组所有值的合计。这样，就能看到所有对等组的所有值的中位数、平均数和标准差：

```

private Dictionary<string, Dictionary<string, List<double>>>
CategorizeData( DataSet ds, string table, string mediantable,
string groupBy) { //Create an instance of the data structure
Dictionary<string, Dictionary<string, List<double>>>

```

```

peergroup2column2values = new Dictionary<string, Dictionary<string,
List<double>>>(); // Add a Total peer group
peergroup2column2values.Add("Total", new Dictionary<string,
List<double>>()); foreach (DataRow row in ds.Tables[table].Rows) {
//Get the PeerGroup value from each row // (or category for Company
table) string peergroup = (string)row[groupBy]; foreach (DataColumn
dc in ds.Tables[table].Columns) { // Loop through each column // If
the column also exists in the MedianTable // add it to the
peergroup2column2values dictionary string columnName =
dc.ColumnName; if (
ds.Tables[mediantable].Columns.Contains(columnName) &&
row.IsNull(dc) == false && columnName != groupBy ) { if
(peergroup2column2values.ContainsKey(peergroup) == false)
peergroup2column2values.Add(peergroup, new Dictionary<string,
List<double>>()); if (peergroup2column2values[peergroup].
ContainsKey(columnName) == false)
peergroup2column2values[peergroup]. Add(columnName, new
List<double>()); if (peergroup2column2values["Total"].
ContainsKey(columnName) == false) peergroup2column2values["Total"].
Add(columnName, new List<double>());
peergroup2column2values[peergroup][columnName].
Add(Convert.ToDouble(row[dc])); peergroup2column2values["Total"]
[columnName]. Add(Convert.ToDouble(row[dc])); } } } return
peergroup2column2values; }

```

下一个方法是FillDataSet，用来自Access数据库的内容填入DataSet对象。你用什么查询取决于要填入的表。查询Loan和Bond表是相同的；选择Bond或Loan表中的每一列，包括RF表的穆迪评级因子列，并用DATEDIFF函数包含距到期日月数列。查询Company表很类似，除了没有到期日。其他表（中位数、平均数和标准差表）则选择每一列：

```

private DataSet FillDataSet(params string[] tables) { DataSet
ds = new DataSet(); using (OleDbConnection conn = new

```

```
OleDbConnection(ConnStr)) { conn.Open(); foreach (string table in
tables) { string cmdStr = null; switch (table.ToUpper()) { case
"LOAN": case "BOND": cmdStr= "Select b.*,Factor as RF,
DATEDIFF(\"m\",now(),b.Maturity) as MonthsUntilMaturity"; cmdStr +=
" from " + table+ " b"; cmdStr += " left outer join RF on RF.Rating
= iif(instr(b.MoodyRating, \" \") > 0, left(b.MoodyRating,
instr(b.MoodyRating, \" \") - 1), b.MoodyRating)"; cmdStr += "
where b.PeerGroup is not null"; break; case "COMPANY": cmdStr =
"Select b.*,Factor as RF"; cmdStr += " from " + table + " b";
cmdStr += " left outer join RF on RF.Rating =
iif(instr(b.MoodyRating, \" \") > 0, left(b.MoodyRating,
instr(b.MoodyRating, \" \") - 1), b.MoodyRating)"; cmdStr += "
where b.Category is not null"; break; default: cmdStr = "SELECT *
FROM [" + table + "]"; break; } OleDbCommand cmd = new
OleDbCommand(cmdStr, conn); OleDbDataAdapter da = new
OleDbDataAdapter(cmd); da.Fill(ds, table); } conn.Close(); } return
ds; }
```

最后三个方法与前面的代码示例一样：

```
private int UpdateDataSet(DataSet ds,params string[] tables) {
int rowc = 0; using (OleDbConnection conn = new
OleDbConnection(ConnStr)) { conn.Open(); foreach (string table in
tables) { string cmdStr = "SELECT * FROM " + table; OleDbCommand
cmd = new OleDbCommand(cmdStr, conn); OleDbDataAdapter da = new
OleDbDataAdapter(); da.SelectCommand = cmd; OleDbCommandBuilder cb
= new OleDbCommandBuilder(da); cb.QuotePrefix = "["; cb.QuoteSuffix
= "]"; da.UpdateCommand = cb.GetUpdateCommand(); da.InsertCommand =
cb.GetInsertCommand(); da.DeleteCommand = cb.GetDeleteCommand();
rowc += da.Update(ds, table); } conn.Close(); } return rowc; }
public static double GetMedian(List<double> list) { double median =
0; if (list.Count != 0) { // create new instance of list // so
source list isnt modified // List should be ordered. List<double>
```

```
sortedList = list.OrderBy(x => x).ToList(); int size =
sortedList.Count; int mid = size / 2; if (size % 2 != 0) median =
sortedList[mid]; else median = (sortedList[mid] + sortedList[mid -
1]) / 2; } return median; } public static double
GetStandardDev(List<double> list) { double stdev = 0; if
(list.Count != 0) { double average = list.Average(); stdev =
Math.Sqrt((list.Sum(x => Math.Pow(x - average, 2))) / (list.Count()
- 1)); } return stdev; }
```

图6-7显示返回的MedianBondStats表。和预计的一样，CCC组的RF中位数为6500，息差、收益率和今年迄今价格变化等信息显示这些评级为CCC的债券在2016年表现超过市场。

PeerGroup	Count	PxChgYTD	PxChg3M	YASSpread	YASYield	FixedCpn	Monthsl	RF
CCC Corp	2	0.81	6.32	380.51	5.73	7.25	78.5	6,500
Consumer Tech	2	0.41	-0.93	235.81	3.74	6.56	80.0	1,766
Electronic Component	11	0.34	1.73	316.61	4.47	6.00	79.0	1,766
FinTech	12	0.41	-0.40	274.19	4.53	5.31	83.5	1,766
Hardware	6	0.23	-0.11	245.12	3.36	5.69	71.0	2,220
Network	5	0.04	-0.82	249.16	4.60	5.50	72.0	1,766
Short CCC	3	0.13	0.92	163.48	2.84	9.13	24.0	6,500
Short HY	11	0.00	-0.23	169.15	2.58	5.88	41.0	1,766
Software	10	0.26	-1.05	261.42	4.75	5.75	94.5	1,350
Total	62	0.17	0.03	260.14	4.35	5.88	74.0	1,766

图6-7：MedianBondStats表

6.3.5 并排比较

现在在我们的数据库中已经有了合计统计数据，我们可以将各个有价值证券与其对等有价值证券进行比较，再进行排序。如下查询将Bond表连接到MedianBondStats表，其中Bond表和MedianBondStats表的列交替排列。按照PeerGroup排序，会使

得在同一个组中看到不同的有价证券变得更容易：

```
SELECT Bond.BondID, Bond.SecurityDes, Bond.PeerGroup,
Bond.PxChgYTD, MedianBondStats.PxChgYTD, Bond.PxChg3M,
MedianBondStats.PxChg3M, MedianBondStats.YASSpread, Bond.YASYield,
MedianBondStats.YASYield FROM Bond INNER JOIN MedianBondStats ON
Bond.PeerGroup = MedianBondStats.PeerGroup ORDER BY Bond.PeerGroup;
```

类似地，在Company表中增加一列，命名为“IndexID”，填入Index表中对应于该公司的相关指数IndexID。如下查询返回每个公司的信息，其中公司业绩指标及其对应的指数的业绩指标交替排列：

```
Company.Price, Index.Price, Company.YrHi, Index.YrHi,
Company.YrLow, Index.YrLow, Company.PxChgYTD, Index.PxChgYTD,
Company.PxChg3M, Index.PxChg3M, Company.TotalReturn12M,
Index.TotalReturn12M FROM Company INNER JOIN [Index] ON
Company.IndexID = Index.IndexID;
```

6.3.6 加权Z值

要在你的分析中添加一个加权Z值，首先创建一个表来存储不同的权重。例如，表6-7包括BondZWeights的模式，其中包含Bond表的Z值权重值。该表只包括一行，而且不需要主关键字。

表6-7：BondZWeights模式

字段名	数据类型
PxChgYTD	数字 (字段规模为 Double)
PxChg3M	数字 (字段规模为 Double)
YASSpread	数字 (字段规模为 Double)
YASYield	数字 (字段规模为 Double)
FixedCpn	数字 (字段规模为 Double)
MonthsUntilMaturity	数字 (字段规模为 Double)
RF	数字 (字段规模为 Double)

在表中填入百分比，加总为100%。值越大，该字段对Z值的影响越大。数据库中填入权重之后，你手里拥有的数字就足够计算一个查询的Z值了：

```
SELECT b.BBID, b.SecurityDes, b.PxChgYTD, b.PxChg3M,
b.YASSpread, b.YASYield, b.FixedCpn, (select Factor from RF where
RF.Rating= iif(instr(b.MoodyRating, " ") > 0, left(b.MoodyRating,
instr(b.MoodyRating, " ") - 1), b.MoodyRating)) as RatingFactor,
DATEDIFF("m",now(),b.Maturity) as MonthsTilMaturity, ((b.PxChgYTD-
mean.PxChgYTD)/sd.PxChgYTD*z.PxChgYTD) +((b.PxChg3M-
mean.PxChg3M)/sd.PxChg3M*z.PxChg3M) +((b.YASSpread-
mean.YASSpread)/sd.YASSpread*z.YASSpread) +((b.YASYield-
mean.YASYield)/sd.YASYield*z.YASYield) +((b.FixedCpn-
mean.FixedCpn)/sd.FixedCpn*z.FixedCpn) -(((select Factor from RF
where RF.Rating= iif(instr(b.MoodyRating, " ") > 0,
left(b.MoodyRating, instr(b.MoodyRating, " ") - 1),
b.MoodyRating))-mean.RF)/sd.RF*z.RF) -
((DATEDIFF("m",now(),b.Maturity)-mean.MonthsUntilMaturity)/
sd.MonthsUntilMaturity*z.MonthsUntilMaturity) AS WeightedZScore
FROM Bond AS b, MeanBondStats AS mean, StdevBondStats AS sd,
BondZWeights AS z WHERE mean.PeerGroup='Total' and
sd.PeerGroup='Total';
```

该查询返回各个债券、其Z值中包含的指标以及用MeanBondStats和StdevBondStats表中的“Total”对等组与BondZWeights表中的权重加权相乘所得到的Z值。

6.4 本章小结

本章介绍了将一个有价值证券的金融数据与其他同类或者更大范围的市场进行比较，以确定有价值证券的相对价值所需的方法和工具。确定适当的对等组和Z值权重，可以使你的分析从罗列数据变成发现新投资和评价现有投资的有用工具。此外，考虑有价值证券之间的相关性对于构建投资组合和防范风险也是有益的。第7章将介绍风险分析以及如何在一个投资组合内比较有价值证券（而不是与其他潜在投资进行比

较)。

第7章 组合风险分析

每个人在脸上挨了一拳之前都很自信。

——拳王泰森

本章介绍测量投资组合的风险和收益的三种不同技巧。第一种测量风险的技术是计算一个投资组合的方差和标准差。尽管该计算可能有点复杂，并需要对相关性系数进行矩阵乘法，但是它是测量风险和多元化优势的一种很有用的方法。此外，你可以用投资组合标准差及预期收益计算投资组合的夏普比率 (Sharpe Ratio)。夏普比率是测量一个投资组合进行风险调整后的收益的一种传统方法。

第二种测量风险的技术简单一点，是把各个头寸分解为不同的“桶” (bucket)。该技术能关注并不显而易见的趋势。例如，按到期日对头寸分组，你可以迅速发现投资组合中即将到到期日的有价值证券所占的百分比。一些特征 (例如评级) 可能不是你的投资决策的重点，但是却可能影响流动性和价格波动性。

最后一种技术为容易监视的不同指标出现特殊情况建立阈值。例如，该方法展示如何关注价格剧烈变动或者处于52周末期的头寸。它还能关注某贷款接近赎回保障末期、某债券即将到到期日或者某头寸的市场价格可能出现潜在违约等情况。

第6章中讨论了买入价、卖出价和中间价，有关技术和目的将在7.1节详细介绍，而7.2节和7.3节仅包含操作细节。推荐阅读7.1节关于Excel的内容。

7.1 途径1：Excel

本节介绍Excel操作，如果你用途径2，则所有的内容应按照本节展示的在Excel中操作，然后在Access的连接表中获取。

7.1.1 方差、波动性和标准差

在开始复杂的计算之前，需要明白几个术语。在金融世界里，方差和标准差都是用于测量波动性和风险的。标准差和方差是相关的术语；实际上，标准差就是方差的平方根。一个投资组合的方差是单个头寸的规模相对于投资组合的规模（或“比重”）、该头寸的历史收益的标准差（波动性）和投资组合中各个头寸的相关性（或协方差）的函数。了解头寸的相关性很重要，因为它能测量多元化的优势大小。我们不对数学进行过多探讨，只是通过计算标准差测量投资组合的风险。

 一个投资组合的标准差一般指历史收益的标准差。计算投资组合历史收益的标准差能测量那些历史收益的波动性。本章的重点是，通过计算投资组合当前的标准差，理解当前的风险。有点复杂的是用历史标准差为每个相关头寸计算当前标准差。

不幸的是，计算一个投资组合的标准差并不简单，需要好几步。最初几步创建一张总结表，其中是彭博返回的数据：

1. 创建一个新的工作表，命名为PortfolioStats。

该工作表将用彭博BDH函数提取每个头寸的月度历史价格，计算年化收益和标准差，并生成一张总结表，其中包括投资组合的预期收益、标准差和夏普比率。

2. 在PortfolioStats工作表单元格A1和A2中，输入Start Date和End Date。在单元格B1和B2中，输入价格历史的起始日期和结束日期。例如，B1为1/31/1990，B2为1/16/2017。

3. 在第六行，从单元格A6开始到单元格H6结束，依次输入PositionID、BBID、Historical Return、StdDev、Weight、WtdStd、Forecasted Return和Return。

4. 从单元格A7开始，在前一步创建的PositionID标题下，逐行输入数字，从1开始，到Positions工作表中最大的PositionID值结束。例如，单元格A7应当包含1，单元格A8应当包含2，以此类推。

5. 在单元格B7（在BBID列标题下），输入如下公式，并向下复制到第4步使用的最后一行：

=IFERROR (INDEX (Portfolio[BBID] , MATCH (A7 ,
Portfolio[PositionID] , 0)) , "")

该公式从Portfolio工作表提取Bloomberg ID (BBID) , 其PositionID (列A) 与Position Excel表中的PositionID相匹配。

图7-5展示第1至5步的结果。下面的步骤是从彭博提取必要的信息，以填入上面三步创建的总结表。

6.在单元格J2，输入如下公式：

=VLOOKUP (ROUNDDOWN (COLUMN () /6 , 0) , \$A : \$B , 2 , FALSE)

该公式用列数从列B提取BBID。例如，列J是第10列，10除以6向下舍入等于1，那么就提取第一个头寸。同理，第二个头寸历史在列P，即第16列（16除以6向下舍入等于2）。

7.在单元格J3，输入如下公式：

=BDH (J2 , "px_last" , \$B\$1 , \$B\$2 , "PER=CM")

该公式用第2步的起始日期和结束日期在第6步中获得BBID月度历史价格，然后填入列J和K。

8.在单元格L4，输入如下公式，并向下复制到L500：

=IF (K4>0 , (K4-K3) /K3 , "")

该公式计算价格的月度变化。如果历史价格不在列K中，就显示一个空白单元格。很重要的是向下复制500行或更多（如果你用了一个更早的起始日期），即使该有价证券的历史没有那么长，因为该公式会被复制和使用到有更长历史的有价证券中去。

9.从单元格M4开始，到单元格M10结束，逐行输入如下标签：Starting Price、Ending Price、Day Count、Return、Annualized Return、Standard Deviation、Annualized Standard Deviation。

10.在单元格N4，输入如下公式：

=VLOOKUP (MIN (J : J) , J : K , 2 , FALSE)

该公式用VLOOKUP中的最小日期返回初始价格。列J包括历史日期。

11.在单元格N5，输入如下公式：

=VLOOKUP (MAX (J : J) , J : K , 2 , FALSE)

类似地，用VLOOKUP中的最大日期返回结束价格。

12.在单元格N6，输入如下公式：

=MAX (J : J) -MIN (J : J)

该公式计算起始日期和结束日期之间的天数（可能与第2步中输入的起始日期和结束日期不同，因为每个有价证券都始自1990年或者你使用的起始日期）。

13.在单元格N7，输入如下公式：

= (N5 -N4) /N4

该公式计算起始日期和结束日期之间的收益。

14.在单元格N8，输入如下公式：

= (1+N7) ^ (365/N6) -1

该公式年化一个收益数值。

15.在单元格N9，输入如下公式：

=STDEV (L : L)

该公式返回月度价格变化的标准差。

16.在单元格N10，输入如下公式：

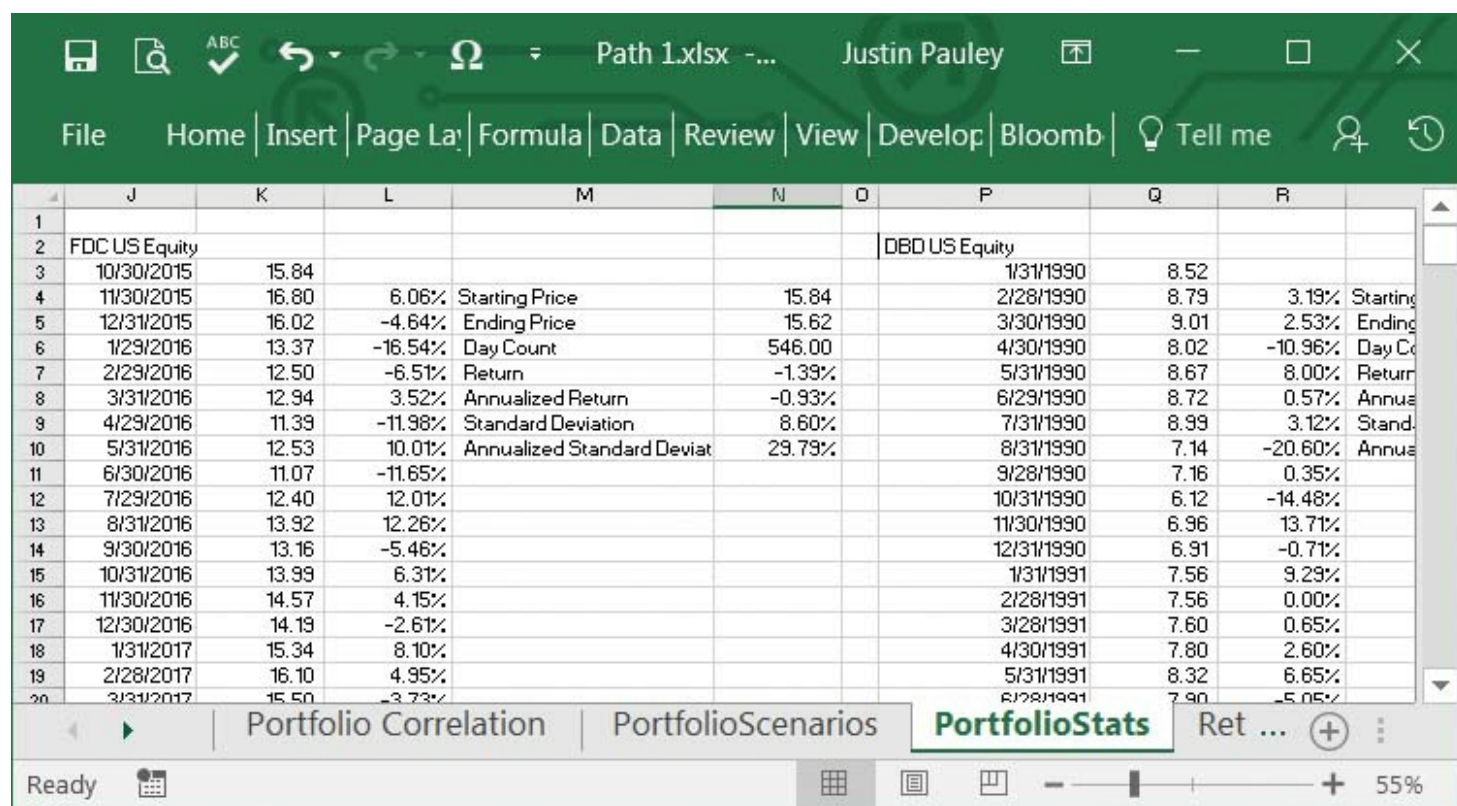
=N9*SQRT (12)

该对第15步的标准差进行年化。

17.复制列J至N，粘贴到单元格P1。重复该步，复制整个五列，跳过一列靠右侧粘贴。这五列（J至N）应粘贴到列P、V、AB、AH等，直到显示每个头寸。

因为单元格J2中的公式用列号返回对应的BBID，你可以复制粘贴该五列，它们应自动显示下一个对应的BBID。

结束第6至17步之后，你的工作表应该如图7-1所示。



	J	K	L	M	N	O	P	Q	R
1									
2	FDC US Equity						DBD US Equity		
3	10/30/2015	15.84					1/31/1990	8.52	
4	11/30/2015	16.80	6.06%	Starting Price	15.84		2/28/1990	8.79	3.19%
5	12/31/2015	16.02	-4.64%	Ending Price	15.62		3/30/1990	9.01	2.53%
6	1/29/2016	13.37	-16.54%	Day Count	546.00		4/30/1990	8.02	-10.96%
7	2/29/2016	12.50	-6.51%	Return	-1.39%		5/31/1990	8.67	8.00%
8	3/31/2016	12.94	3.52%	Annualized Return	-0.93%		6/29/1990	8.72	0.57%
9	4/29/2016	11.39	-11.98%	Standard Deviation	8.60%		7/31/1990	8.99	3.12%
10	5/31/2016	12.53	10.01%	Annualized Standard Deviat	29.79%		8/31/1990	7.14	-20.60%
11	6/30/2016	11.07	-11.65%				9/28/1990	7.16	0.35%
12	7/29/2016	12.40	12.01%				10/31/1990	6.12	-14.48%
13	8/31/2016	13.92	12.26%				11/30/1990	6.96	13.71%
14	9/30/2016	13.16	-5.46%				12/31/1990	6.91	-0.71%
15	10/31/2016	13.99	6.31%				1/31/1991	7.56	9.29%
16	11/30/2016	14.57	4.15%				2/28/1991	7.56	0.00%
17	12/30/2016	14.19	-2.61%				3/28/1991	7.60	0.65%
18	1/31/2017	15.34	8.10%				4/30/1991	7.80	2.60%
19	2/28/2017	16.10	4.95%				5/31/1991	8.32	6.65%
20	3/31/2017	15.50	-3.73%				6/28/1991	7.90	-5.05%

图7-1：PortfolioStats工作表

注意：从彭博获取了历史数据，计算了每个头寸的年化收益和标准差，用下面的步骤把它们放入总结表。

18.在单元格C7，输入如下公式，并向下复制到列A中的最后一行：

=IF (B7="", "", OFFSET (\$N\$8 , 0 , (A7-1) *6))

与J2中的公式一样，该公式用Position ID确定每个头寸的年化收益数值位置。OFFSET函数在第28步中详细定义。

19.在单元格D7，输入如下公式，并向下复制到列A中的最后一行：

=IF (B7="", "", OFFSET (\$N\$10, 0, (A7-1) *6))

类似地，该公式用每个头寸的Position ID引用合适的列，以获取年化标准差。例如，列A的Position ID等于2（在单元格A8），（A8-1）*6就等于6，OFFSET函数处理的是列N向右数六列，即第二个头寸计算结果的位置。

20.在单元格E7，输入如下公式，并向下复制到列A中的最后一行：

=IFERROR (INDEX (Portfolio[%Of Portfolio], MATCH (A7, Portfolio[PositionID], 0)), "")

和用于获取BBID的公式一样，该公式返回该头寸在Excel Positions表的整个投资组合中所占的权重（或占整个市值的百分比）。

21.在单元格F7，输入如下公式，并向下复制到列A中的最后一行：

=IF (E7<>"", E7*D7, "")

该公式计算头寸市值标准差。

现在，我们的表中（从单元格C6开始）包含投资组合中每个头寸的历史收益标准差，那么需要计算每个头寸的相关性。如果我们在另一个工作表中创建相关性矩阵，就能看起来更简洁一些。

22.创建一个工作表，命名为Portfolio Correlation（投资组合相关性）。

23.在列A，从第三行开始（单元格A3），逐行输入数字，从1开始，到Positions工作表中最大的PositionID结束。例如，单元格A4应包含数字2，单元格A5应包含数字3，以此类推。

24.和第23步一样，在各列输入PositionID，在单元格C1输入数字1，在单元

格D1输入数字2，以此类推。

25.在单元格B3，输入如下公式，并向下复制到带PositionID的最后一行：

=IFERROR (VLOOKUP (A3 , PortfolioStats ! A : B , 2 , FALSE) , "")

该公式从PortfolioStats工作表中提取列A中PositionID对应的BBID。

26.在单元格C2，输入如下公式，并逐列复制到有PositionID的最后一列：

=IFERROR (VLOOKUP (C1 , PortfolioStats ! \$A : \$B , 2 , FALSE) , "")

与第25步一样，该公式从PortfolioStats工作表中提取第一行的PositionID对应的BBID。第25～26步的结果应该是一个投资组合头寸矩阵，上边那行和左边那列都是BBID。

27.在单元格B2，输入一个起始日期，以计算在矩阵中每个单元格交叉的头寸之间的相关性。

矩阵中的每个单元格将包含在该单元格相交叉的那两个有价值证券（BBID）的相关性。例如，单元格D4的内容是在单元格D2中的有价值证券和在B4中的有价值证券之间的相关性。第28步的相关性计算将用到这两个有价值证券的历史收益数据（来自PortfolioStats工作表）。因此，投资组合中的每个有价值证券的历史收益都有单元格B2的日期那天的值是很重要的。

28.在单元格C3，输入如下公式，复制到各行各列，这样在第二行和列B中每个单元格里都有一个对应的BBID：

=IF(AND(\$B3<>"",C\$2<>""), CORREL(OFFSET(PortfolioStats!\$A:\$A, MATCH(\$B\$2,OFFSET(PortfolioStats!\$A:\$A,0,(6*C\$1)+3),0)-1,(6*C\$1)+5,10,1), OFFSET(PortfolioStats!\$A:\$A,MATCH(\$B\$2, OFFSET(PortfolioStats!\$A:\$A,0,(6*\$A3)+3),0)-1,(6*\$A3)+5,10,1)), "")

这是个复杂的公式，让我们分解看一下。

1) 如果行标题和列标题中没有BBID，就显示一个空白单元格。

2) CORREL是一个相关性函数，其中有两个历史价格变化数组。该函数用OFFSET和MATCH函数找到PositionIDs对应的正确数组。

3) CORREL函数的第一个实参是：

```
OFFSET(PortfolioStats!$A:$A,  
MATCH($B$2,OFFSET(PortfolioStats!$A:$A,0,(6*C$1)+3),0)-1,  
(6*C$1)+5,10,1)
```

4) OFFSET函数有三个参数：第一个实参是起始点（在此情况下是PortfolioStats工作表的第一列整列）；第二个实参是要向下移动的行数；最后一个实参是要移动的列数。计算最后一个实参的方法是，和PortfolioStats基于PositionID确定一个固定点的对应值的方法一样的。

5) 要计算需要向下移动的行数（OFFSET函数的第二个实参），用MATCH函数：

```
MATCH($B$2,OFFSET(PortfolioStats!$A:$A,0,(6*C$1)+3),0)
```

6) MATCH函数的功能是告诉外OFFSET函数需要向下挪动的行数，以找到起始相关性日期（第27步在单元格B2中输入。MATCH函数有三个参数：查询日期（单元格B2）、要查询的数组和一个类型（此处为零）。

7) 要计算出找到日期的数组，公式用另一个OFFSET函数返回给定PositionID的日期列。

8) CORREL函数的第二个函数与第一个相同，除了内函数对于给定行，而非给定列引用PositionID。

包含各个有价证券之间的相关性的矩阵结果应该如图7-2所示。

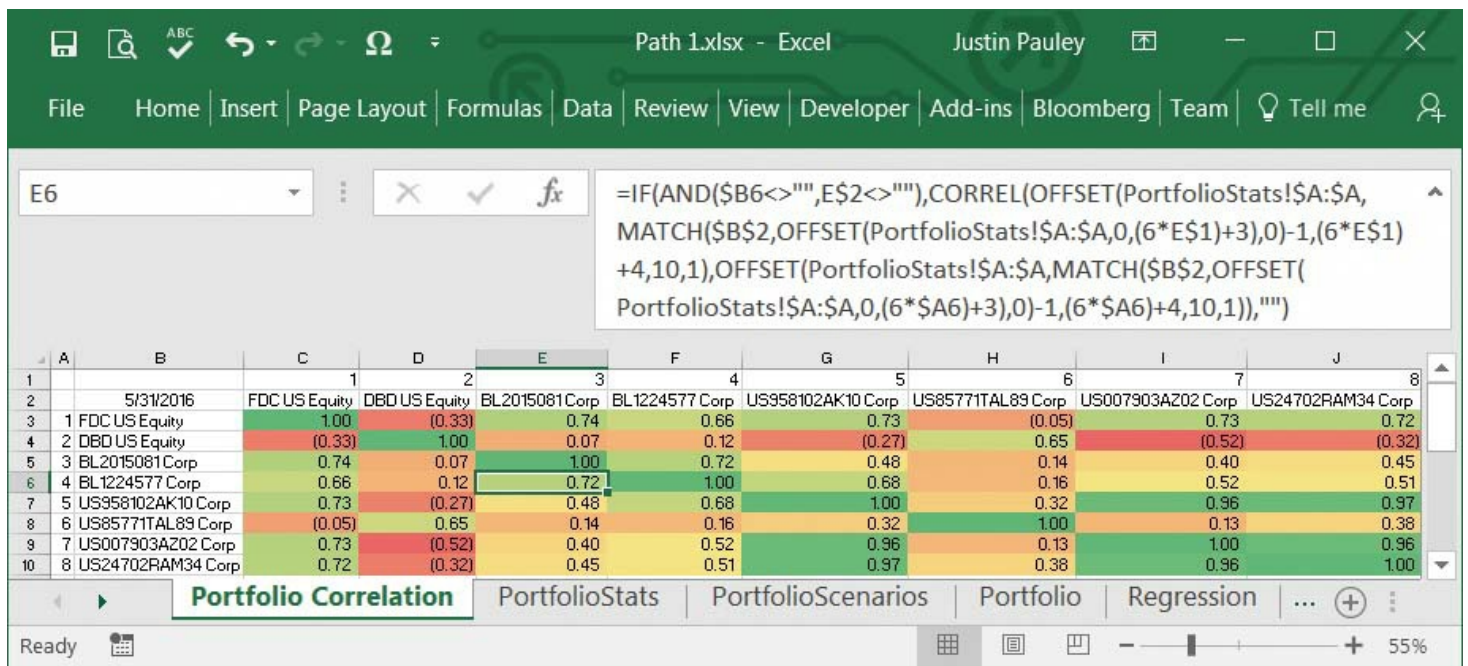


图7-2：投资组合相关性矩阵

完成投资组合相关性矩阵后，下一步是计算投资组合的标准差。记住，投资组合的标准差是方差的平方根，是关于每个头寸的权重、历史收益的标准差和对其他每个头寸的相关性的函数。方差公式见图7-3。我们不过多地研究数学问题，就用MMULT函数做矩阵乘法，由PortfolioStats工作表中的表的WtdStd列与PortfolioCorrelation工作表中的相关性矩阵相乘。

29.在PortfolioStats工作表中，在单元格D1，增加标签StdDev。

$$\text{方差} = [w_1\sigma_1 \quad \dots \quad w_n\sigma_n] \times \begin{bmatrix} 1 & \rho_{12} & \dots & \rho_{1n} \\ \rho_{21} & 1 & \dots & \rho_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \rho_{n1} & \dots & \dots & 1 \end{bmatrix} \times \begin{bmatrix} w_1\sigma_1 \\ \vdots \\ w_n\sigma_n \end{bmatrix}$$

$w_n\sigma_n$ = 头寸权重 x 的标准差
 ρ_{12} = 头寸 1 和头寸 2 的相关性

图7-3：方差公式

30.在单元格E1，输入如下公式：

=SQRT(MMULT(MMULT(TRANPOSE(F7:F14),'PortfolioCorrelation'!C3:J10), PortfolioStats!F7:F14))

该公式用矩阵乘法来计算方差，即用WtdStd列的加权标准差（转置）乘以相关性矩阵，然后再次乘以加权标准差（非转置）。该公式返回标准差（即方差结果的平方根）。

得出的标准差可用于增加或减少一个头寸将如何增加或减少你的投资组合的风险（标准差越大表示风险越高），以及将一个投资组合的风险与其他投资组合的风险进行比较。

7.1.2 带历史或预期收益的夏普比率

夏普比率（Sharpe Ratio）是诺贝尔奖得主William F.Sharpe提出的，用于测量一个投资组合的风险调整后的收益。夏普比率主要告诉我们，一个投资组合的收益是来自于额外的风险，还是来自于更好的投资。夏普比率越高，意味着投资组合收益高，而且风险越小。尽管计算标准差（见上一节）对于理解一个投资组合的波动性和风险是有用的，但你可以用夏普比率测量相对于预期收益的波动性。该公式（见图7-4）就是将平均投资组合收益和无风险利率之差除以投资组合的标准差。然而，还有几个小问题值得讨论一下。

$$\text{夏普比率} = \frac{\bar{R}_p - R_f}{\sigma_p}$$

$\bar{R}_p$ = 平均投资组合收益
 R_f = 无风险利率
 σ_p = 投资组合标准差

图7-4：计算夏普比率的公式

和计算标准差一样，计算夏普比率更常见、更简便的方法是用投资组合历史收益（和历史收益的标准差），而非投资组合预期收益和当前标准差。因为关注的是当前风险调整后的收益，我们需要用当前投资组合的收益和标准差。幸运的是，我们在前面已经计算了标准差。无风险利率一般指与投资组合期限匹配的美国国债利率（债券投资组合应考虑最长的国债期）。在实践中，许多参与者采用更短期的利率，例如三个月伦敦银行同业拆借利率（LIBOR）。

不幸的是，确定未来收益比使用历史组合收益数据难得多。很少有方法能计算

在夏普比率公式中使用的投资组合收益。首先，特别是对于股票，历史年化收益（见上一节PortfolioStats工作表列C）可能是计算未来收益的唯一媒介（尽管可能不太确定）。其次，对于固定收益，也可能使用几种收益率之一进行计算，例如到期收益率或最差收益率。最后，也是最理想的，设置不同的假设情景和不同的概率，以概率计算加权收益。

下面的步骤创建一个工作表，用历史或概率加权收益来计算概率加权收益和夏普比率。

31. 创建一个新的工作表，命名为PortfolioScenarios。

32. 把单元格A1命名为PositionID，在列A中逐行输入PositionID（如单元格A2应包含1，单元格A3应包含2，等等）。

33. 把Excel区域转换成Excel表，命名为PortfolioScenario。

34. 把单元格B1命名为Description，并在单元格B2输入如下公式，并向下复制到列A的最后一个PositionID：

```
=IFERROR ( INDEX ( Portfolio[Description] , MATCH ( A2 ,  
Portfolio[PositionID] , 0 ) ) , "" )
```

该公式通过将Portfolio表中从列A到PositionID列的PositionID进行匹配，从Portfolio工作表提取Description列。

35. 把单元格C1命名为SecurityID，然后在单元格C2中输入如下公式并向下复制到列A中的最后一个PositionID：

```
=IFERROR ( INDEX ( Portfolio[SecurityID] , MATCH ( A2 ,  
Portfolio[PositionID] , 0 ) ) , "" )
```

该公式从Portfolio工作表中提取SecurityID。

36. 把单元格D1命名为Type，在该列填入如下公式：

```
=IFERROR ( INDEX ( Portfolio[Type] , MATCH ( A2 ,
```

Portfolio[PositionID],0)),"")

37.把单元格E1命名为Coupon/Margin，在该列填入如下公式：

=IF([@Type]="Loan",INDEX(Loan[Margin],MATCH([@SecurityID],Loan[LoanID],0)),IF([@Type]="Bond",INDEX(Bond[Fixed Coupon],MATCH([@SecurityID],Bond[BondID],0)),""))

该公式从Loan工作表提取Margin列，如果有价证券是一笔贷款或债券，就提取Coupon列。如果有价证券不是一笔贷款或债券，就返回一个空的单元格。

38.把单元格F1命名为Yield，在该列填入如下公式：

=IF([@Type]="Loan",INDEX(Loan[Yield],MATCH([@SecurityID],Loan[LoanID],0)),IF([@Type]="Bond",INDEX(Bond[YAS Yield],MATCH([@SecurityID],Bond[BondID],0)),""))

与Coupon/Margin列一样，该公式将提取债券或贷款的Yield列。

39.把单元格G1命名为12M Equity Total Return，在该列填入如下公式：

=IF([@Type]="Equity",INDEX(Company[12M Total Return],MATCH([@SecurityID],Company[CompanyID],0)),"")

该公式提取股票的12M Total Return列。

40.把单元格H1、I1和J1分别命名为：Best Return、Average Return和Worst Return。

41.用列E至G的值作为指南，在列H、I和J手动填入你估计的每个头寸最佳、平均和最差收益的发生概率。

例如，在最佳情景下，某个证券头寸可能获得高达30%的收益，而没有回购保障、与票面价值相当的贷款可能收益很少。

42.把单元格K、L和M分别命名为Best Probability、Average Probability

和Worst Probability。

43.在列K、L和M手动填入你估计的最佳、平均和最差情况的发生概率。列K、L和M之和应等于100%。

44.把单元格N1命名为Forecasted Return，在该列填入如下公式：

$$=[@[Best\ Return]]*[@[Best\ Probability]]+ [@[Average\ Return]]* [@[Average\ Probability]]+ [@[Worst\ Return]]*[@[Worst\ Probability]]$$

该公式基于列H ~ M的潜在结果和概率，返回一个概率加权收益。

包含预期收益的PortfolioScenarios工作表结果应该如图7-5所示。

	B	C	D	E	F	G	H	I	J	K	L	M	N
	Description	SecurityID	Type	Coupon/Margin	Yield	12M Equity Total Return	Best Return	Average Return	Worst Return	Best Probability	Average Probability	Worst Probability	Forecasted Return
1	FIRST DATA COR FDC US		Equity			10.42	15%	9%	-14%	30%	50%	20%	6.2%
2	DIEBOLD NIXDO DBD US		Equity			3.41	10%	6%	-15%	20%	50%	30%	0.5%
3	ZBRA TL B 1L US	BL2015081	Loan	325	3.98		7%	4%	1%	30%	50%	20%	4.3%
4	FDC TL B 1L USD	BL1224577	Loan	400	4.71		10%	4%	-5%	20%	50%	30%	2.5%
5	WDC 10 1/2 04, US958102AK10		Bond	10.5	5.27		10%	10%	-12%	20%	50%	30%	3.4%
6	STATSP 8 1/2 11 US85771TAL89		Bond	8.5	6.65		10%	8.50%	-2%	30%	50%	20%	6.9%
7	AMD 6 3/4 03/C US007903AZ02		Bond	6.75	2.92		10%	7%	-2%	30%	50%	20%	6.1%
8	DELL 5.4 09/10/ US24702RAM34		Bond	5.4	6.51		10%	5%	-2%	30%	50%	20%	5.1%

图7-5：PortfolioScenarios工作表

现在，我们已经计算了预测收益，我们可以把它们代回PortfolioStats工作表并计算夏普比率。

45.在PortfolioStats工作表中，在单元格A3输入标签Risk Free Rate，在单元格B3输入如下公式：

$$=BDP("CT10\ Govt", "ASK_YIELD")/100$$

该公式得到当前10年期美国国债的当前收益率。你可以把任何数值作为你的投资组合的无风险利率，再次输入该公式。其久期应与你的头寸的久期接近。

46.把单元格A4命名为Use Forecasted ,然后在单元格B4输入TRUE。

该单元格将用于在夏普比率中切换使用历史收益或预测收益。

47.在PortfolioStats工作表中的单元格G7下,在Forecasted Return标题下输入如下公式并向下复制:

```
=IFERROR(INDEX(PortfolioScenario[Forecasted Return],  
MATCH(A7,PortfolioScenario[PositionID],0)), "")
```

该公式用PositionID从PortfolioScenarios工作表获取预测收益。

48.在Return标题下,在单元格H7中输入如下公式并向下复制:

```
=IF($B$4=TRUE,G7,C7)
```

该列将包含Historical Return列或Forecasted Return列的值(取决于单元格B4的值)。

49.把单元格D2命名为Return,然后在单元格E2中输入如下公式:

```
=SUMPRODUCT(H7:H45,E7:E45) (如超过45行,需调整公式)。
```

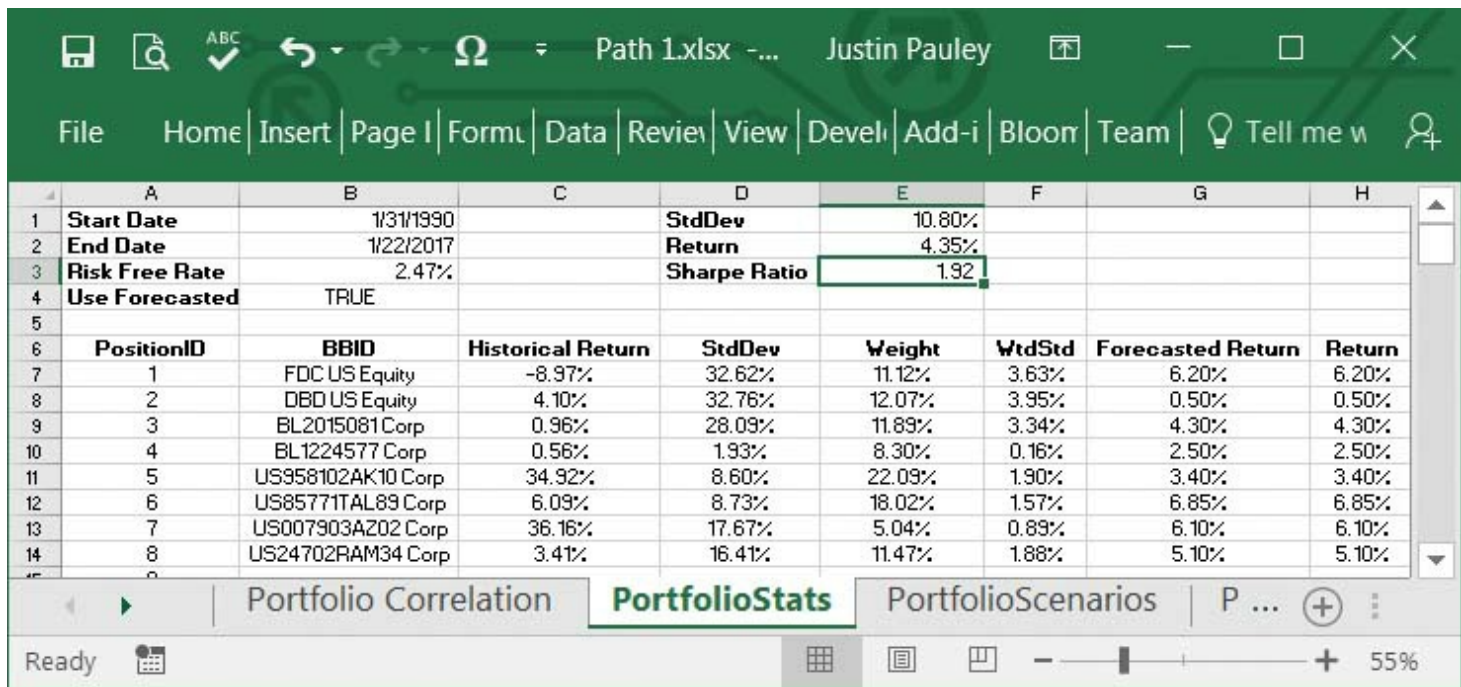
该公式用历史收益或者根据单元格B4的预测收益返回投资组合的加权平均值。

50.把单元格D3命名为Sharpe Ratio,然后在单元格E3输入如下公式,以计算夏普比率:

```
=(E2-B3)/E1
```

根据经验,夏普比率为1时是好的,为2时是很好,为3时是非常好的。然而,夏普比率在进行比较时更有价值,无论是比较不同的投资组合,还是在一个投资组合内比较改变头寸后的情形。由此,对于检测假设性交易来说,它是一个有力的工具。夏普比率不能作为判定一项投资是否优于另一项的唯一判断方法;它仅是一个完整的投资分析中的一个数据点。此外,夏普比率在收益正态分布时效果最佳,并会受到计算中不同输入变量的影响,包括潜在头寸收益使用的期限。图7-6显示

PortfolioStats工作表上返回的表。



	A	B	C	D	E	F	G	H
1	Start Date	1/31/1990		StdDev	10.80%			
2	End Date	1/22/2017		Return	4.35%			
3	Risk Free Rate	2.47%		Sharpe Ratio	1.92			
4	Use Forecasted	TRUE						
5								
6	PositionID	BBID	Historical Return	StdDev	Weight	WtdStd	Forecasted Return	Return
7	1	FDC US Equity	-8.97%	32.62%	11.12%	3.63%	6.20%	6.20%
8	2	DBD US Equity	4.10%	32.76%	12.07%	3.95%	0.50%	0.50%
9	3	BL2015081 Corp	0.96%	28.09%	11.89%	3.34%	4.30%	4.30%
10	4	BL1224577 Corp	0.56%	1.93%	8.30%	0.16%	2.50%	2.50%
11	5	US958102AK10 Corp	34.92%	8.60%	22.09%	1.90%	3.40%	3.40%
12	6	US85771TAL89 Corp	6.09%	8.73%	18.02%	1.57%	6.85%	6.85%
13	7	US007903AZ02 Corp	36.16%	17.67%	5.04%	0.89%	6.10%	6.10%
14	8	US24702RAM34 Corp	3.41%	16.41%	11.47%	1.88%	5.10%	5.10%

图7-6：PortfolioStats工作表

7.1.3 投资组合分解

本节介绍将一个投资组合中的头寸分解成多个分类或者“装入不同的桶里”的技巧。对投资组合进行解构可以关注到如果仅看每个头寸难以看到的风险。例如，如果不整合投资组合，你可能就难以注意到某个行业所占的比重过大、投资组合中丧失未来六个月提前偿还保护的有价值证券所占的百分比、评级为CCC的头寸数量等问题。本节还探讨如何对金融数据进行分组，例如根据价格、市值或距到期日时间划分。尽管本节提供了很多示例，不过金融分析的重要内容是对金融信息（特别是风险指标）拥有你自己的观点。

下面的步骤从相关的Bond、Loan和Company工作表提取更多数据，加强Portfolio工作表。从一张表上总结数据比从多个工作表提取数据容易得多。注意方法是：先将每个Bond、Loan和Company工作表中所有关于有价值证券层次的信息提取出来的，在各个工作表中进行所有计算，然后将结果列提取到工作表。之后再从Portfolio工作表总结数据。

前面一些列主要是固定收益（债券和贷款）有价值证券；后面才是债券列。

1.在Portfolio工作表上增加一列，命名为“Months Until Maturity”，在该列填入如下公式：

```
=IF([@Type]="Loan",INDEX(Loan[Months Until Maturity],  
MATCH([@SecurityID],Loan[LoanID],0)),  
IF([@Type]="Bond",INDEX(Bond[Months Until Maturity],  
MATCH([@SecurityID],Bond[BondID],0)), ""))
```

该公式根据Portfolio工作表中的Type列，从Loan或Bond工作表的Months Until Maturity列提取数据。股票（Equity）没有到期日，因此显示一个空白单元格。

2.新增一列Coupon Type，然后输入如下公式：

```
=IF([@Type]="Loan",INDEX(Loan[Coupon Type],  
MATCH([@SecurityID],Loan[LoanID],0)),IF([@Type]="Bond",  
INDEX(Bond[Coupon Type],MATCH([@SecurityID],Bond[BondID],0)), ""))
```

该公式从Bond和Loan表提取Coupon Type（固定或者浮动）。股票（Equity）没有票面利息，因此显示一个空白单元格。

3.新增一列Coupon/Margin，然后输入如下公式：

```
=IF([@Type]="Loan",INDEX(Loan[Margin],MATCH([@SecurityID],  
Loan[LoanID],0)),IF([@Type]="Bond",INDEX(Bond[Fixed Coupon],  
MATCH([@SecurityID],Bond[BondID],0)), ""))
```

该公式或者从Loan工作表提取Margin列，或者从Bond工作表提取Coupon。

4.新增一列Yield，然后输入如下公式：

```
=IF([@Type]="Loan",INDEX(Loan[Yield],MATCH([@SecurityID],Loan[LoanID],0)),  
IF([@Type]="Bond",INDEX(Bond[YAS Yield],MATCH([@SecurityID],  
Bond[BondID],0)), ""))
```

该公式对贷款提取Yield列，对债券提取YAS Yield列，对股票为空白单元格。

5.新增一列Callable Ever，然后输入如下公式：

```
=IF([@Type]="Loan",INDEX(Loan[Callable?],MATCH([@SecurityID],  
Loan[LoanID],0)),  
IF([@Type]="Bond",INDEX(Bond[Callable?],MATCH([@SecurityID],  
Bond[BondID],0)), ""))
```

该公式返回Bond和Loan表“Callable？”列是“Y”还是“N”，代表该有价证券可否在到期日之前提前回购。

6.新增一列Callable Now，然后输入如下公式：

```
=IF([@Type]="Loan",IFERROR(INDEX(Loan[Next Call  
Date],MATCH([@SecurityID], Loan[LoanID],0))<=  
TODAY(),FALSE),IF([@Type]="Bond", IFERROR(INDEX(Bond[Next Call  
Date],MATCH([@SecurityID], Bond[BondID],0))<= TODAY(),FALSE),""))
```

该公式有点复杂，它检查Bond和Loan工作表中“Next Call Date”的值是否是今天的日期还是尚未到期（由Excel TODAY（）函数返回）。如果当前可以选择回购该有价证券，则该公式的结果是TRUE。

7.新增一列Callable Next 6Months，然后输入如下公式：

```
=IF([@Type]="Loan",IFERROR(EDATE(INDEX(Loan[Next Call Date],  
MATCH([@SecurityID],Loan[LoanID],0)),-6)<= TODAY(),FALSE),  
IF([@Type]="Bond",IFERROR(EDATE(INDEX(Bond[Next Call Date],  
MATCH([@SecurityID],Bond[BondID],0)),-6)<= TODAY(),FALSE),""))
```

和第6步一样，该公式用Excel EDATE函数将Next Call Date减去六个月，并将结果日期与今天的日期进行比较。如果有价证券从今天起六个月内可赎回，则结果为TRUE。

8.新增一列Facility Rating，然后输入如下公式：

```
=IF([@Type]="Loan",INDEX(Loan[Moody]'s
```

```
Rating],MATCH([@SecurityID],
Loan[LoanID],0)),IF([@Type]="Bond",INDEX(Bond[Moody's Rating],
MATCH([@SecurityID],Bond[BondID],0)), ""))
```

该公式从Bond或Loan工作表中提取穆迪评级。尽管公司都有企业家族评级，但是该评级是贷款机构或债券所特有的。因为在处理破产时，贷款比其他债务拥有更高优先级，因此其评级比企业更高。

9.新增一列Facility RF，然后输入如下公式：

```
=IF([@Type]="Loan",INDEX(Loan[RF],MATCH([@SecurityID],Loan[LoanID],0)),
IF([@Type]="Bond",INDEX(Bond[RF],MATCH([@SecurityID],
Bond[BondID],0)), ""))
```

该公式返回可能对于投资组合进行评级分组有用的穆迪评级因子。穆迪评级因子参见第6章。

后面的列从Company工作表提取数据；对于债券和贷款，这些列引用有价证券的发行人。例如，Company Rating返回AMD的企业评级，可能与AMD的债券或贷款评级不同。

10.新增一列CompanyID，然后输入如下公式：

```
=IF([@Type]="Loan",
INDEX(Loan[CompanyID],MATCH([@SecurityID],Loan[LoanID],0)),
IF([@Type]="Bond",INDEX(Bond[CompanyID],MATCH([@SecurityID],
Bond[BondID],0)),[@SecurityID]))
```

对于债券和贷款，该公式返回Bond和Loan工作表的CompanyID列。股票显示SecurityID列，与CompanyID一样。下面的步骤从Company工作表提取字段，用该字段找到正确的公司。

11.新增一列Category，然后输入如下公式：

```
=INDEX(Company[Category],MATCH([@CompanyID],Company[CompanyID],0))
```

该公式根据CompanyID（见第10步），从Company工作表提取Category列。

12.新增一列Company Rating，然后输入如下公式：

```
=INDEX(Company[Moody's  
Rating],MATCH([@CompanyID],Company[CompanyID],0))
```

该公式返回该公司的穆迪评级（企业家族评级）。

13.添加一列，命名为Company RF，然后输入如下公式：

```
=INDEX(Company[RF],MATCH([@CompanyID],Company[CompanyID],0))
```

该公式返回企业家族评级的穆迪评级因子（见第12步）。

14.新增一列Company Market Cap，然后输入如下公式：

```
=INDEX(Company[Market  
Cap],MATCH([@CompanyID],Company[CompanyID],0))
```

该公式返回每个公司的Market Cap值。

15.新增一列Total Debt/EBITDA，然后输入如下公式：

```
=INDEX(Company[Total  
Debt/EBITDA],MATCH([@CompanyID],Company[CompanyID],0))
```

该公式返回每个公司的Total Debt/EBITDA。

现在Portfolio工作表有很丰富的内容了，我们可以分解数据，以关注潜在风险。下面的步骤创建一个单独的工作表，命名为PortfolioBreakdown，其中包含几个Excel表可以随意处理Portfolio表中的信息。在每个示例中，头寸市值（MV）都用于测量头寸的真实风险。这些表位于另一个工作表上，以避免Portfolio表的行或列增加时不得不挪来挪去。

16.创建一个新的工作表，命名为Portfolio Breakdown。

17.从单元格A1开始，创建一个Excel表。列标题（单元格A1、B1和C1）为：Type、MV和%of Portfolio；行标题（单元格A2、A3和A4）为：Bond、Loan和Equity。

该表根据头寸类型对头寸进行分组，加和各组市值，并计算该组在整个投资组合中所占的百分比。你要用Excel功能区上的Insert选项卡里的Table选项，将这个区域转换成一个Excel表；或者也可以选择该区域，并按Ctrl+T键，将这个区域转换成一个Excel表。

18.在MV列，输入如下公式：

```
=SUMIF(Portfolio[Type],[@Type],Portfolio[MarketValue])
```

该公式将Portfolio工作表中的市值列进行加和，条件是Portfoliobreakdown工作表的单元格A2、A3和A4中的类型与Portfolio工作表中的类型相匹配。

19.在Design选项卡上，点击Table Style Options，然后选择Total Row，向Excel表添加一个总计行。在总计行和MV列交叉的那个单元格，点击打开下拉式菜单，然后选择Sum，显示MV列之和。

该步向MV列添加了一个总计行，且结果必须与Portfolio工作表中所有头寸的市值总和匹配。这些数字匹配很重要，以保证你准确地包含所有头寸。

20.在“%of Portfolio”列，用每行的MV单元格除以总计行的总数（见第19步图7-7中的单元格B5）。

该步显示每种有价值证券类型在整个投资组合中所占的比重。“%of Portfolio”加和应为100%。

前面四个步骤（第17~20步）是重要步骤，在本节将重复好几次。你的Excel表应该如图7-7所示。为简略起见，后面将把这四个步骤整合成一个步骤，即“创建一个带如下列标题和行标题的Excel表”。

	A	B	C
1	Type	MV	% of Portfolio
2	Bond	24,240,188	56.64%
3	Loan	8,571,875	20.03%
4	Equity	9,982,000	23.33%
5	Total	42,794,063	100.00%

图7-7：投资组合按有价证券类型划分

下面的步骤根据票面利息类型（固定或浮动），而非有价证券类型，将投资组合分解为债券和贷款。此外，它包含一列，展示有价证券的票面利息或利润加权平均值。

21. 在PortfolioBreakdown工作表中增加一个Excel表，输入如下列标题（逐列）：Coupon Type、Market Value、%of Fixed Income和Weighted Avg Cpn/Margin。输入行标题（逐行）：Fixed和Floating。

22. 在Market Value列，输入如下公式：

`=SUMPRODUCT(N(Portfolio[Type]<>"Equity"), N(Portfolio[Coupon Type]=[@[Coupon Type]]), Portfolio[MarketValue])`

该公式根据票面利息类型不同对非债券头寸的市值进行加和。

Market Value列之和（见总计行）应当与前面的步骤创建的Excel表中的债券和贷款市场之和相等。

23. “%of Fixed Income” 列应该用Fixed和Float各自的Market Value列除以总计行中的总和值。

因为该表中的头寸只有固定收益（债券和贷款），因此百分比并不能反映完整的投资组合。

24. Weighted Avg Cpn/Margin列应包含如下公式：

=SUMPRODUCT(N(Portfolio[Type]<>"Equity"), N(Portfolio[Coupon Type]=[@[Coupon Type]]), Portfolio[Coupon/Margin], Portfolio[MarketValue])/[@[Market Value]]

该公式将每个头寸的市值与Portfolio工作表中按照票面利息类型划分的Coupon/Margin进行加权相乘。在总计行中对该列进行加和没有意义。

下面的步骤总结固定收益头寸的回购保障情况。此外，我们计算每个分类的平均价格。一般说来，可赎回的有价值证券可以由发行人按照票面价格（即价格100%）回购，因此，可以按照高于票面价格的价格进行交易的溢价有价值证券存在被回购的风险。

在Market Value列中的公式不会是全部一样的，以避免重复计算有价值证券。第26步显示的是Market Value和不同的回购保障类型相交的那个单元格中需要填入的不同公式。

25. 增加一个Excel表，输入如下列标题：Call Protection、Market Value、%of Fixed Income和Average Price。输入如下行标题：Callable Now、Callable in 6MO、Callable Later和Not Callable。

26. 在Market Value列（应对固定收益头寸进行加和），对不同的回购保障类型用如下公式：

·Callable Now应对Callable Ever和Now都为真的项进行加和：

=SUMPRODUCT(N(Portfolio[Type]<>"Equity"), N(Portfolio[Callable Ever]="Y"), N(Portfolio[Callable Now]=TRUE), Portfolio[MarketValue])

·因为我们不想重复计算现在能马上赎回的有价值证券，Callable in 6MO应当只包括六个月内可以赎回、但是现在不能马上赎回的有价值证券：

=SUMPRODUCT(N(Portfolio[Type]<>"Equity"), N(Portfolio[Callable Ever]="Y"), N(Portfolio[Callable Now]=FALSE), N(Portfolio[Callable

Next 6 Months]=TRUE),Portfolio[MarketValue])

·Callable Later是可赎回的有价证券，但是不能现在或者在六个月内赎回：

=SUMPRODUCT(N(Portfolio[Type]<>"Equity"), N(Portfolio[Callable Ever]="Y"),N(Portfolio[Callable Now]=FALSE), N(Portfolio[Callable Next 6 Months]=FALSE),Portfolio[MarketValue])

·Not Callable是Callable Ever项为false的有价证券：

=SUMPRODUCT(N(Portfolio[Type]<>"Equity"), N(Portfolio[Callable Ever]="N"),Portfolio[MarketValue])

27. "%of Fixed Income" 列应当用Market Value列中每个单元格除以总计行中的总市值。

28.Average Price列模仿Market Value列的SUMPRODUCT公式，除了包括Price列，并除以Market Value列，以生成一个加权平均价格。例如，对于Callable in 6MO，用如下公式：

=SUMPRODUCT(N(Portfolio[Type]<>"Equity"),N(Portfolio[Callable Ever]="Y"), N(Portfolio[Callable Now]=FALSE), N(Portfolio[Callable Next 6 Months]=TRUE),Portfolio[MarketValue], Portfolio[Price])/[@[Market Value]]

下一个Excel表按照Category对投资组合进行划分，按照如何分解投资组合的多个字段对有价证券类型进行划分。

29.增加一个Excel表，列标题如下：Category、Bond、Loan、Equity、Total和%of Portfolio。行标题应包括Portfolio工作表Category列值的不同表单。

30.在Bond列，输入如下公式：

=SUMPRODUCT(N(Portfolio[Type]="Bond"),N(Portfolio[Category]=[@Category]), Portfolio[MarketValue])

该公式加和与交叉分类相匹配那些债券头寸的市值。

31.在Loan列，输入如下公式：

```
=SUMPRODUCT(N(Portfolio[Type]="Loan"),N(Portfolio[Category]=  
[@Category]), Portfolio[MarketValue])
```

这与第30步相同，但是加和的是贷款头寸市值。

32.在Equity列，输入如下公式：

```
=SUMPRODUCT(N(Portfolio[Type]="Equity"), N(Portfolio[Category]=  
[@Category]),Portfolio[MarketValue])
```

33.在Total列，增加Bond、Loan和Equity列。

本列之和应当等于投资组合的市值总和。

34.在“%of Portfolio”列中，用Total列的值除以Total列的和（在总计行）。

三个Excel表的结果如图7-8所示。注意前两个表只涉及投资组合的固定收益（债券和贷款）部分，而最后一个表分解整个投资组合。因此，前两个表的余额总数应当匹配，且匹配于单元格B25和C25之和。

Path 1.xlsx - Excel Justin Pauley						
File Home Insert Page Layout Formulas Data Review View Developer Add-ins Bloomberg Team Tell me						
	A	B	C	D	E	F
7	Coupon Type	Market Value	% of Fixed Income	Weighted Avg Cpn/Margin		
8	Fixed	24,240,188	74%	8.50		
9	Floating	8,571,875	26%	356		
10	Total	32,812,063	100%			
11						
12	Call Protection	Market Value	% of Fixed Income	Average Price		
13	Callable Now	3,521,875	11%	100.63		
14	Callable in 6MO	9,486,536	29%	118.58		
15	Callable Later	7,687,719	23%	106.0		
16	Not Callable	12,115,934	37%	96.17		
17	Total	32,812,063	100%			
18						
19						
20	Category	Bond	Loan	Equity	Total	% of Portfolio
21	Financial Services/Software	-	3,521,875	9,982,000	13,503,875	31.56%
22	Electronic Component	19,326,625	-	-	19,326,625	45.16%
23	Network Hardware	-	5,050,000	-	5,050,000	11.80%
24	Enterprise Software	4,913,564	-	-	4,913,564	11.48%
25	Total	24,240,188	8,571,875	9,982,000	42,794,063	100.00%

图7-8：额外投资组合分解表

下面三个Excel表用穆迪评级分解投资组合。在下面的例子中，评级因子用于将几个评级整合成更显著的大类（如AAA-A、CCC and Lower等）。还要说明一下，尽管我们引用了穆迪评级，不过我们用的评级分类与其他评级机构（标准普尔、惠誉等）更相关，因为它们更常用。创建第一个表的步骤（即分解固定收益头寸的贷款评级）会在后面重复，用于分解债券的企业家族发行人评级和所有头寸的企业家族评级。

35.增加一个Excel表，设置如下列标题：Fixed Income Facility、Max RF、Bonds、Loans、Total和%of Fixed Income。设置行标题：AAA-A、BBB、B、CCC and Lower和Not Rated。

36.Max RF列依次填入如下评级因子行：180、610、1766、3490、10000。Not Rated行应当为空。

将投资组合按照评级分解的最简单方法分成显著的大类，将各个评级归入这些

显著的大类最简单的方法是使用其评级因子。Max RF列包含组中最大的评级因子。例如，表6-1显示AAA-A的最大评级因子是180，从180到610都是BBB评级（从Baa1到Baa3）。但对于第二行再次输入I1及I2，对于第三行再次输入I1及I3，等等。

37.在Bonds列，输入如下公式：

```
=SUMPRODUCT(N(Portfolio[Type]="Bond"), N(Portfolio[Facility RF]
<=[@[Max RF]]), N(Portfolio[Facility
RF]>I1),Portfolio[MarketValue])
```

该公式加和头寸评级因子介于当前Max RF和当前行上一行的Max RF之间的债券头寸市值。例如，BBB行是评级因子小于等于610但大于180的债券的市值之和。请注意，必须把公式中的I1改成列标题Max RF的位置，这样向下拖动公式时就会引用当前行上一行的Max RF。对于第一行（AAA-A），公式引用行标题是可以的，因为没有比AAA更高的评级。

38.重复第37步填入Loans列，除了用"Loan"而非"Bond"，例如：

```
=SUMPRODUCT(N(Portfolio[Type]="Loan"), N(Portfolio[Facility RF]
<=[@[Max RF]]), N(Portfolio[Facility
RF]>I1),Portfolio[MarketValue])
```

与Bond列一样，该列对所有评级队伍的贷款头寸市值进行加和。

39.在Total列，对Bonds和Loans列的市值求和：

```
=[@Loans]+[@Bonds]
```

该合计数应当匹配于Loans和Bonds的市值之和。

40.在"%of Fixed Income"列，用Total列的值除以Total列的和（在总计行）。应相加等于100%。

下面的步骤创建一个表，总结债券头寸的企业家族评级。

41.与前面的表一样，创建一个Excel表，设置如下列标题：Equity Corp

Rating、Max RF、Market Value和%of Equity。设置如下行标题：AAA-A、BBB、B、CCC and Lower和Not Rated。

42.在Max RF列的各行依次填入如下评级因子：180、610、1766、3490、10000。Not Rated行应当为空。

43.Market Value列应包含如下公式：

=SUMPRODUCT(N(Portfolio[Type]="Equity"), N(Portfolio[Company RF]<=[@[Max RF]]), N(Portfolio[Company RF]>I10),Portfolio[MarketValue])

这与前面Excel表中用的公式一样，除了它过滤股票头寸并引用Company RF，而非Facility RF。

44. "%of Equity" 列应该用Market Value列除以Market Value列之和（在总计行）。加和总数为百分之百。

最后一个以评级为基础的表分解所有头寸的企业家族评级。尽管一个债券或贷款的机构评级可能与其发行人的评级不一样，但是发行人的评级（企业家族评级）对于确定有价值证券的风险和潜在流动性还是很重要的。

45.与前面的表一样，创建一个Excel表，设置如下列标题：Portfolio Corp Ratio、Max RF、Market Value和%of portfolio。设置如下行标题：AAA-A、BBB、B、CCC and Lower和Not Rated。

46.在Max RF列，按顺序填入如下评级因子：180、610、1766、3490、10000。Not Rated行应为空。

47.与第43步一样，在Market Value列填入：

=SUMPRODUCT(N(Portfolio[Company RF]<=[@[Max RF]]), N(Portfolio[Company RF]>I19),Portfolio[MarketValue])

48.重复第44步，填入 "%of Portfolio" 。

评级分解表应该如图7-9所示。

	H	I	J	K	L	M
1	Fixed Income Facility	Max RF	Bonds	Loans	Total	% of Fixed Income
2	AAA-A	180	-	-	-	0.0%
3	BBB	610	4,913,564	-	4,913,564	15.0%
4	BB	1766	9,486,536	5,050,000	14,536,536	44.3%
5	B	3490	7,687,719	-	7,687,719	23.4%
6	CCC and Lower	10000	2,152,370	-	2,152,370	6.6%
7	Not Rated		-	3,521,875	3,521,875	10.7%
8	Total		24,240,188	8,571,875	32,812,063	100.0%
9						
10	Equity Corp Rating	Max RF	Market Value	% of Equity		
11	AAA-A	180	-	0.0%		
12	BBB	610	-	0.0%		
13	BB	1766	5,290,000	53.0%		
14	B	3490	4,692,000	47.0%		
15	CCC and Lower	10000	-	0.0%		
16	Not Rated		-	0.0%		
17	Total		9,982,000	100.0%		
18						
19	Portfolio Corp Rating	Max RF	Market Value	% of Portfolio		
20	AAA-A	180	-	0.0%		
21	BBB	610	4,913,564	11.5%		
22	BB	1766	19,826,536	46.3%		
23	B	3490	8,213,875	19.2%		
24	CCC and Lower	10000	2,152,370	5.0%		
25	Not Rated		7,687,719	18.0%		
26	Total		42,794,063	100.0%		

图7-9：评级分解表

在后面几个例子中，我们按照价格分档、到期年限、总债务等不同因素分解投资组合。因为这些步骤对于每个表来说非常相似，只提供一个完整的示例，但你可以很容易地重复。

下面的步骤按照债券头寸价格对投资组合市值进行分组。

49. 创建一个Excel表，设置如下列标题：Equity Price、End Price、Market Value和% of Equity。设置如下行标题：0、15、25、40。

50.在End Price列，按顺序填入如下价格：15、25、40、150。

Equity Price和End Price列的价格区间为0~15、15~25、25~40、40~150。最终使用的价格（在该例中为150）应当能够覆盖任何债券头寸的最高价格。

51.在Market Value列，输入如下公式：

```
=SUMPRODUCT(N(Portfolio[Type]="Equity"), N(Portfolio[Price]<=[@End Price]), N(Portfolio[Price]>[@Equity Price]),Portfolio[MarketValue])
```

该公式计算价格小于或等于End Price但是大于Equity Price的债券头寸的总市值。

52.在“%of Equity”字段，用Market Value列除以Market Value列之和。加和应为百分之百。

你可以用最后四步总结不同评级的任何指标。用Months Until Maturity字段（而非Price字段）重复上面最后四个步骤，会关注到接近到期日的有价证券和远期有价证券的头寸。图7-10展示几种不同的以评级为基础的Excel表的例子。

O	P	Q	R	S	T	U	V	W
1	Market Cap	End Size	Market Value	% of Portfolio	Equity Price	End Price	Market Value	% of Equity
2	-	5,000	22,941,282	54.0%	0	15	-	0.0%
3	5,000	10,000	-	0.0%	15	25	4,692,000	47.0%
4	10,000	15,000	10,093,875	23.7%	25	40	5,290,000	53.0%
5	15,000	25,000	9,486,536	22.3%	40	150	-	0.0%
6		Total	42,521,693	100.0%		Total	9,982,000	100.0%
7								
8	Total Debt/EBITDA	End Amt	Market Value	% of Portfolio	Months Until Maturity	End Month	Market Value	% of Fixed Income
9	-	5	7,687,719	18.1%	0	24	1,880,000	5.8%
10	5	10	29,543,975	69.5%	24	48	7,687,719	23.6%
11	10	15	-	0.0%	48	56	8,571,875	26.3%
12	15	100	5,290,000	12.4%	56	500	14,400,100	44.3%
13		Total	42,521,693	100.0%		Total	32,539,693	100.0%
14								
15	Fixed Income Price	End price	Market Value	% of Fixed Income				
16	0	90	4,913,564	15.1%				
17	90	100	1,880,000	5.8%				
18	100	103	8,571,875	26.3%				
19	103	150	17,174,255	52.8%				
20		Total	32,539,693	100.0%				

7.1.4 警告标志

在本章中讨论的第三个也是最后一个方法是在你的组合中建立警告标志（warning sign），以关注特定趋势或者可能需要重视的变化。例如，三个月中价格下降超过5%，评级为CCC，低于票面价格的有价证券即将到期，等等。本节介绍如何在Portfolio工作表增加warning列，并视情况用颜色突出显示，以生成警告标志。本节只有几个例子，但是你可以根据自身需要设置多个warning列。

第一个warning列关注未来24个月以内到期且价格低于票面价格的头寸。在接近到期日，打折的固定收益有价证券价格接近票面价格；由此，应关注打折销售且接近到期日的有价证券。如果你觉得这个公式过于敏感，你可以调整到一个较低的价格底线。

1. 在Portfolio表中增加一列，命名为Maturity Warning。

2. 在该列填入如下公式：

```
=AND([@[Months Until Maturity]]<=24,[@Price]<100)
```

该公式检查24个月或更短时间内将到期且价格低于票面价格的固定收益头寸的Months Until Maturity列。如果头寸打折且到期日较近，则返回TRUE。

3. 选择列，然后在函数区上点击Home键。在Styles组点击Conditional Formatting按钮，把鼠标移动至Highlight Cell Rules，然后选择Equals To。在左边的文本框中，输入TRUE，然后选择一个颜色突出显示选项，例如“Light Red Fill with Dark Red Text”。

第二个warning列检查在未来六个月可回购且当前价格高于回购价格的固定收益有价证券。

4. 在Portfolio表添加一列，命名为Next Call Price，在该列填入如下公式：

```
=IF([@[Callable Ever]]="Y",IF([@Type]="Loan",INDEX(Loan[Next
```

```
Call Price],  
MATCH([@SecurityID],Loan[LoanID],0)),IF([@Type]="Bond",  
INDEX(Bond[Next Call Price],MATCH([@SecurityID],  
Bond[BondID],0)),"")), "")
```

该公式从贷款或Bond工作表返回Next Call Price；否则单元格为空。

5.添加一列，命名为Call Warning，在该列填入如下公式：

```
=AND([@[Callable Next 6 Months]]=TRUE,[@Price]>[@Next Call  
Price]))
```

如果该有价证券在未来六个月可赎回，且交易价格高于Next Call Price，该公式返回TRUE。

6.按照第3步一样的方式进行条件格式化。

第三个warning列仅关注穆迪贷款或企业家族评级为CCC或更低的头寸。

7.添加一列，命名为CCC Warning，在该列填入如下公式：

```
=OR(LEFT([@[Facility Rating]],1)="C",LEFT([@[Company  
Rating]],1)="C")
```

该公式检查Facility Rating和Company Rating的第一个字母，如果二者之一开头字母是“C”，则返回TRUE。应用适当的条件格式化。

最后一个warning列关注价格在过去三个月下跌5%或以上的头寸。

8.添加一列，命名为3M Px Change，在该列填入如下公式：

```
=IF ([@Type]="Equity", INDEX ( Company[3M Px Change] ,  
MATCH ( [@SecurityID] , Company[CompanyID] , 0 ) ) ,  
IF ( [@Type]="Loan" , INDEX ( Loan[3M Px Chg] , MATCH ( [@SecurityID] ,  
Loan[LoanID] , 0 ) ) , IF ( [@Type]="Bond" , INDEX ( Bond[3M Px Chg] ,  
MATCH ( [@SecurityID] , Bond[BondID] , 0 ) ) ) ) )
```

该公式从适当的Bond、Loan或Company工作表中提取3M Px Change列。

9.添加一列，命名为Price Warning，在该列填入如下公式：

```
=[@[3M Px Change]]<-5
```

如果3M Px Change下跌大于5%（例如变化小于-5），则该公式返回TRUE。记住，对于该字段彭博返回的数值是整数，不过表示的是百分比。可进行适当条件格式化。

或者，你可以基于相对于基准价格（针对股票）的三个月价格变化或者第6章所述的有价证券对等组的三个月价格变化中位数等设置Price Warning。如果整个市场已经卖出超过5%，且你只想关注奇怪的事情，那么这就是有用的。为此，用如下公式增加该公司股票相对于基准价格的三个月价格变化：

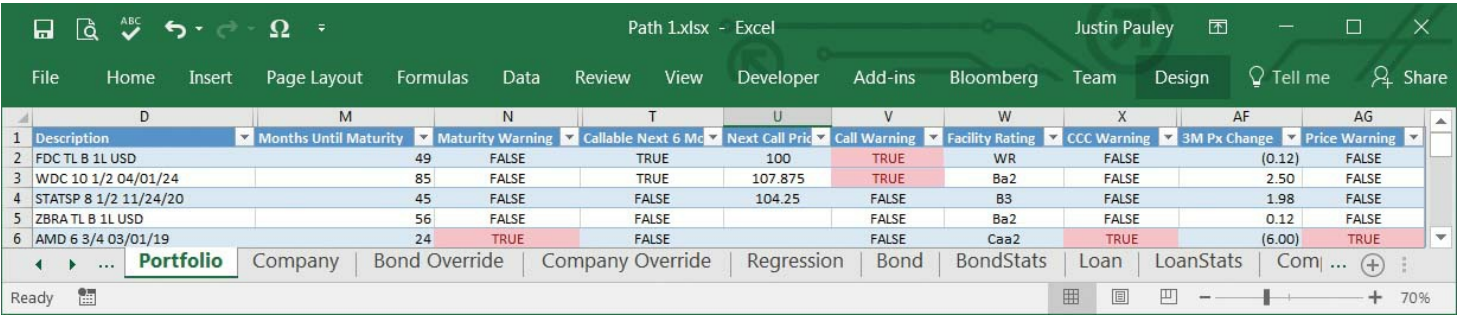
```
=IF([Type]="Equity",INDEX(Company[Index 3M Px Change],  
MATCH([SecurityID],Company[CompanyID],0)),")
```

然后，用下列公式增加有价证券的对等组三个月价格变化中位数：

```
=IF([@Type]="Equity",INDEX(Company[Median 3M Px Chg],  
MATCH([@SecurityID],Company[CompanyID],0)),IF([@Type]="Loan",  
INDEX(Loan[Median 3M Px Chg],MATCH([@SecurityID],Loan[LoanID],0)),  
IF([@Type]="Bond",INDEX(Bond[Median 3M Px Chg],  
MATCH([@SecurityID],Bond[BondID],0))))))
```

最后，根据个人意愿调整Price Warning列。例如，仅仅在该有价证券的三个月价格变化和对等组三个月价格变化中位数之差小于-5%时，才将其设为TRUE。

结果应该如图7-11所示（数字进行了调整，以作展示之用）。



	D	M	N	T	U	V	W	X	AF	AG
	Description	Months Until Maturity	Maturity Warning	Callable Next 6 Mo	Next Call Price	Call Warning	Facility Rating	CCC Warning	3M Px Change	Price Warning
2	FDC TL B 1L USD	49	FALSE	TRUE	100	TRUE	WR	FALSE	(0.12)	FALSE
3	WDC 10 1/2 04/01/24	85	FALSE	TRUE	107.875	TRUE	Ba2	FALSE	2.50	FALSE
4	STATSP 8 1/2 11/24/20	45	FALSE	FALSE	104.25	FALSE	B3	FALSE	1.98	FALSE
5	ZBRA TL B 1L USD	56	FALSE	FALSE		FALSE	Ba2	FALSE	0.12	FALSE
6	AMD 6 3/4 03/01/19	24	TRUE	FALSE		FALSE	Caa2	TRUE	(6.00)	TRUE

图7-11：投资组合warning列

7.2 途径2：Access

本节扩展7.1节的内容。特别是本节展示如何用查询来观察投资组合分解表，以及检查带有警告标志的头寸。

7.2.1 投资组合分解

Microsoft Access用简单的查询代替了复杂的Excel公式，使分析和分组数据变得更容易。例如，在Access中，你可以用如下查询重新创建多步骤线程，其中包含按照资产类型对组合头寸进行分解的复杂公式：

```
SELECT Type, Format(Sum(MarketValue),"#,##0") AS MV,
FormatPercent(sum([% Of Portfolio]),2) as PctofPortfolio FROM
Portfolio WHERE Type Is Not Null GROUP BY Type;
```

该查询按照Portfolio表中的Type列对各行进行分组，并对MarketValue和“%of Portfolio”列求和。用Format和FormatPercent函数对图7-12中的结果调整格式。

Type	MV	PctofPortfolio
Bond	24,049,461	56.11%
Equity	10,226,000	23.86%
Loan	8,582,500	20.03%

图7-12：在Access中按有价证券类型分解投资组合

类似地，你可以用如下查询，按票面利息类型分解固定收益有价证券：

```
SELECT [Coupon Type], Format(sum(MarketValue),"#,##0") as MV,
FormatPercent(Sum(MarketValue)/(select sum(marketvalue) from
Portfolio where type = 'Bond' or type='loan'),0) as PctOfFI,
format(Sum(MarketValue * [Coupon/Margin])/Sum(MarketValue),
iif([Coupon Type]="Floating", "#,##0", "0.00")) as WtdAvg from
Portfolio where (type='bond' or type='loan') and [Coupon Type] is
not null Group by [Coupon Type]
```

在该查询中显示固定收益资产百分比的方法是用MarketValue列之和除以一个子查询，该子查询返回全部固定收益资产的总市值。此外，该查询用MarketValue和Coupon/Margin列的乘积除以MarketValue之和，计算Coupon/Margin加权平均值。我们用IIF函数展示不带小数点之后数字的浮动利率，因为已经在基准点中了。

重新创建Excel表并展示按回购保护进行分类的查询有点复杂。因为结果表中每一行的计算都有细微差别，我们用UNION结合几个有细微差别的查询：

```
SELECT 'Callable Now' as CallProtection,
Format(sum(MarketValue),"#,##0") as MV,
FormatPercent(Sum(MarketValue)/(select sum(marketvalue) from
Portfolio where type = 'Bond' or type='loan'),0) as PctOfFI,
```

```

format(Sum(MarketValue * [Price])/Sum(MarketValue),"0.00") as
WtdAvgPx from Portfolio where Type <> 'Equity' and [callable
ever]='Y' and [callable now]=True UNION SELECT 'Callable In 6MO' as
CallProtection, Format(sum(MarketValue),"#,##0") as MV,
FormatPercent(Sum(MarketValue)/(select sum(marketvalue) from
Portfolio where type ='Bond' or type='loan'),0) as PctOfFI,
format(Sum(MarketValue * [Price])/Sum(MarketValue),"0.00") as
WtdAvgPx from Portfolio where Type <> 'Equity' and [callable
ever]='Y' and [callable now]=False and [callable next 6
months]=true UNION SELECT 'Callable Later' as CallProtection,
Format(sum(MarketValue),"#,##0") as MV,
FormatPercent(Sum(MarketValue)/(select sum(marketvalue) from
Portfolio where type ='Bond' or type='loan'),0) as PctOfFI,
format(Sum(MarketValue * [Price])/Sum(MarketValue),"0.00") as
WtdAvgPx from Portfolio where Type <> 'Equity' and [callable
ever]='Y' and [callable now]=False and [callable next 6
months]=false UNION SELECT 'Not Callable' as CallProtection,
Format(sum(MarketValue),"#,##0") as MV,
FormatPercent(Sum(MarketValue)/(select sum(marketvalue) from
Portfolio where type ='Bond' or type='loan'),0) as PctOfFI,
format(Sum(MarketValue * [Price])/Sum(MarketValue),"0.00") as
WtdAvgPx from Portfolio where Type <> 'Equity' and [callable
ever]='N'

```

该查询用与Excel工作簿相同的逻辑，并用前面讨论过的聚合函数，来展示固定收益资产百分比和加权平均值价格。

尽管前面的查询用UNION来分隔表的每行中的逻辑，不过你还可以用IIF函数分隔每行中的逻辑。如下查询按Category和资产类别分解投资组合，用IIF函数过滤每列中包含的头寸：

```

SELECT Category,
Format(sum(iif(Type='Bond',MarketValue,0)),"#,##0") as Bond,

```

```
Format(sum(iif(Type='Loan',MarketValue,0)), "#,##0") as Loan,  
Format(sum(iif(Type='Equity',MarketValue,0)), "#,##0") as Equity,  
Format(sum(MarketValue), "#,##0") as Total, FormatPercent(sum([% Of  
Portfolio]),2) as PctofPortfolio from Portfolio where Category is  
not null Group by Category
```

下一个查询用评级因子按照Facility Rating分解投资组合，它其实结合了前两项技巧，既用UNION连接了按照评级因子过滤的不同查询，又用IIF函数按照有价证券类型过滤了列：

```
SELECT 'AAA-A' as Rating, 1 as Ord,  
Format(sum(iif(Type='Bond',MarketValue,0)), "#,##0") as Bond,  
Format(sum(iif(Type='Loan',MarketValue,0)), "#,##0") as Loan,  
Format(sum(MarketValue), "#,##0") as Total,  
FormatPercent(Sum(MarketValue)/(select sum(marketvalue) from  
Portfolio where type ='Bond' or type='loan'),0) as PctOfFI from  
Portfolio where [Facility RF] <= 180 and Type <> 'Equity' UNION  
SELECT 'BBB' as Rating, 2 as Ord,  
Format(sum(iif(Type='Bond',MarketValue,0)), "#,##0") as Bond,  
Format(sum(iif(Type='Loan',MarketValue,0)), "#,##0") as Loan,  
Format(sum(MarketValue), "#,##0") as Total,  
FormatPercent(Sum(MarketValue)/(select sum(marketvalue) from  
Portfolio where type ='Bond' or type='loan'),0) as PctOfFI from  
Portfolio where [Facility RF] >180 and [Facility RF] <= 610 and  
Type <> 'Equity' UNION SELECT 'BB' as Rating, 3 as Ord,  
Format(sum(iif(Type='Bond',MarketValue,0)), "#,##0") as Bond,  
Format(sum(iif(Type='Loan',MarketValue,0)), "#,##0") as Loan,  
Format(sum(MarketValue), "#,##0") as Total,  
FormatPercent(Sum(MarketValue)/(select sum(marketvalue) from  
Portfolio where type ='Bond' or type='loan'),0) as PctOfFI from  
Portfolio where [Facility RF] >610 and [Facility RF] <= 1766 and  
Type <> 'Equity' UNION SELECT 'B' as Rating, 4 as Ord,  
Format(sum(iif(Type='Bond',MarketValue,0)), "#,##0") as Bond,
```



```

Format(sum(iif(Type='Loan',MarketValue,0)), "#,##0") as Loan,
Format(sum(MarketValue), "#,##0") as Total,
FormatPercent(Sum(MarketValue)/(select sum(marketvalue) from
Portfolio where type = 'Bond' or type='loan'),0) as PctOfFI from
Portfolio where [Facility RF] >1766 and [Facility RF] <= 3490 and
Type <> 'Equity' UNION SELECT 'CCC and Lower' as Rating, 5 as Ord,
Format(sum(iif(Type='Bond',MarketValue,0)), "#,##0") as Bond,
Format(sum(iif(Type='Loan',MarketValue,0)), "#,##0") as Loan,
Format(sum(MarketValue), "#,##0") as Total,
FormatPercent(Sum(MarketValue)/(select sum(marketvalue) from
Portfolio where type = 'Bond' or type='loan'),0) as PctOfFI from
Portfolio where [Facility RF] >3490 and [Facility RF] <= 10000 and
Type <> 'Equity' UNION SELECT 'Not Rated' as Rating, 6 as Ord,
Format(sum(iif(Type='Bond',MarketValue,0)), "#,##0") as Bond,
Format(sum(iif(Type='Loan',MarketValue,0)), "#,##0") as Loan,
Format(sum(MarketValue), "#,##0") as Total,
FormatPercent(Sum(MarketValue)/(select sum(marketvalue) from
Portfolio where type = 'Bond' or type='loan'),0) as PctOfFI from
Portfolio where [Facility RF] is null and Type <> 'Equity' order by
Ord

```

该查询合并了一个顺序列，命名为Ord，以控制展示不同行的顺序。如图7-13所示，结果表可以用Ord列排序，以便按照正确的顺序展示评级分类。

Rating	Ord	Bond	Loan	Total	PctOfFI
AAA-A	1				
BBB	2	4,891,429	0	4,891,429	15%
BB	3	9,492,112	5,056,250	14,548,362	45%
B	4	7,785,920	0	7,785,920	24%
CCC and Lower	5	1,880,000	0	1,880,000	6%
Not Rated	6	0	3,526,250	3,526,250	11%

图7-13：按照Ord排序的评级分解

本节的最后一个查询用UNION将投资组合分解成不同的市值区间。你可以用该技术根据其他列的内容分解投资组合。它还用Ord列控制展示各行的顺序：

```
SELECT '0 - 5,000' as MarketCap, 1 as Ord,
Format(Sum(MarketValue),"#,##0") AS MV, FormatPercent(sum([% Of
Portfolio]),2) as PctofPortfolio from Portfolio where [Company
Market Cap] between 0 and 5000 UNION SELECT '5,000-10,000' as
MarketCap, 2 as Ord, Format(Sum(MarketValue),"#,##0") AS MV,
FormatPercent(sum([% Of Portfolio]),2) as PctofPortfolio from
Portfolio where [Company Market Cap] between 5000 and 10000 UNION
SELECT '10,000-15,000' as MarketCap, 3 as Ord,
Format(Sum(MarketValue),"#,##0") AS MV, FormatPercent(sum([% Of
Portfolio]),2) as PctofPortfolio from Portfolio where [Company
Market Cap] between 10000and 15000 UNION SELECT '15,000-25,000' as
MarketCap, 4 as Ord, Format(Sum(MarketValue),"#,##0") AS MV,
FormatPercent(sum([% Of Portfolio]),2) as PctofPortfolio from
Portfolio where [Company Market Cap] between 15000 and 25000 order
by ord
```

7.2.2 警告标志

本节介绍如何将一个简单的查询变成一个基础报告，其中对Excel Portfolio 工作表中的警告行突出显示。如下查询展示warning列，后面的其他列是Excel投资组合关联表中warning为真的那些列：

```
SELECT [Maturity Warning], [Call Warning], [CCC Warning],
[Price Warning], * from Portfolio where [Maturity Warning]=true or
[Call Warning]=true or [CCC Warning]=true or [Price Warning]=true
```

在执行该查询后，在函数区Create选项卡上，点击Report，用查询结果创建一个报告。然后在Design视图中，为了给报告增加条件格式化，选择Warning列。然后在Format选项卡上，点击Control Formatting按钮，然后选择Conditional Formatting。添加一个规定，使得单元格值等于 “Yes” 时就改变格式。报告结果应该如图7-14所示。

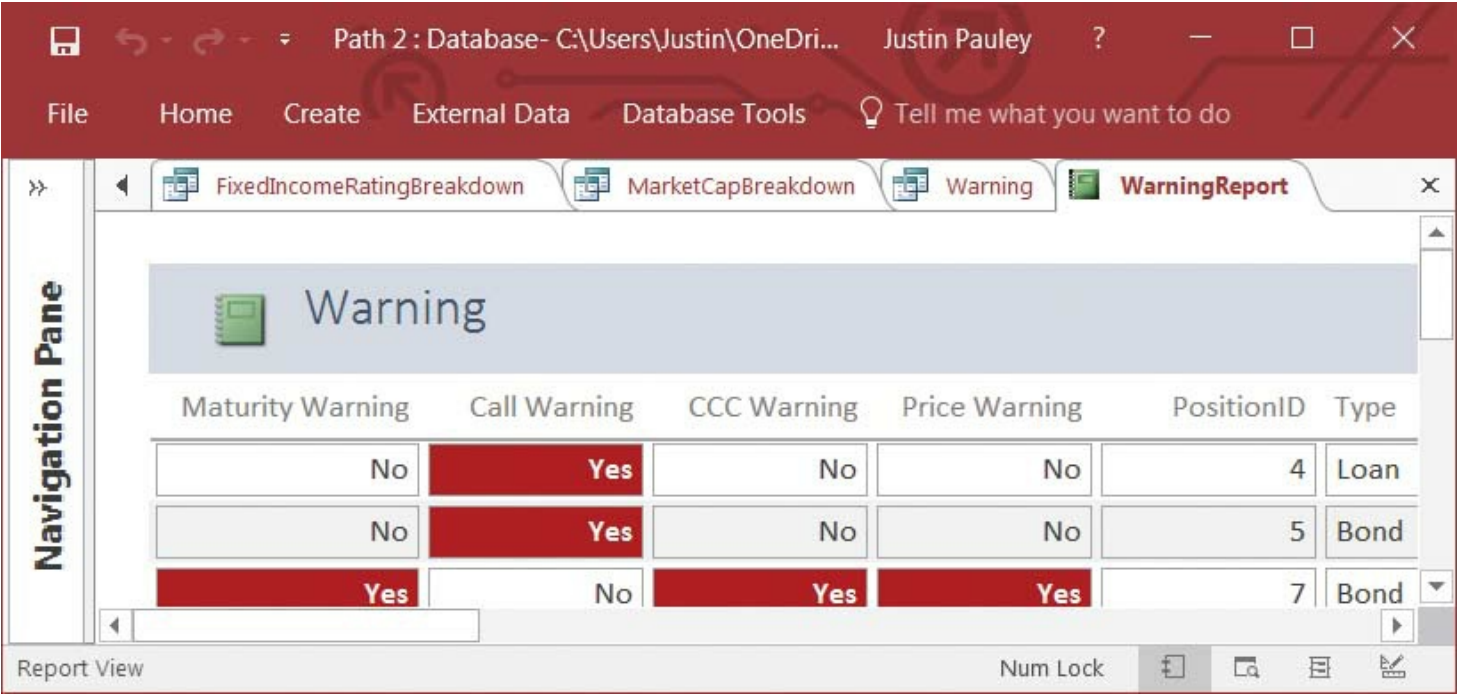


图7-14：警告报告

根据你的偏好，你还可以创建不同的宏，用Create选项卡上的Macros选项，打印或用email发送该报告。

7.3 途径3：C#

本节介绍7.1节的内容如何同C#实现。本节还很好地展示了如何将Excel中一个复杂的任务转换成简便的应用。

7.3.1 历史或预测收益的夏普比率

本节带你学习创建一个C#控制台应用，使用历史或预测收益、投资组合的标准差和投资组合的夏普比率来计算投资组合加权平均年化收益。步骤比较简单：

- 1.在数据库中查询头寸信息。
- 2.用彭博计算每个头寸的历史收益和标准差。
- 3.创建一个相关性矩阵，以计算投资组合标准差。
- 4.计算夏普比率。

在开始编C#代码之前，用表7-1中的模式创建一个数据库表，命名为“Portfolio-Scenarios”，其中包括Excel途径（见图7-5）中PortfolioScenarios工作表那样的情景。将每个头寸填入表中。

表7-1：投资组合情景模式

字段名	数据类型
PositionID（主关键字）	数字
BestReturn	数字（double）
AverageReturn	数字（double）
WorstReturn	数字（double）
BestProbability	数字（double）
AverageProbability	数字（double）
WorstProbability	数字（double）

创建一个新的C#控制台应用，和前面的例子一样，增加适当的引用至彭博API以及MathNet。添加一个数据集项（Dataset Item），命名为ADS。在ADS数据集上增加两个TableAdapter，连接到你的Access数据库。

第一个TableAdapter命名为Portfolio , 查询代码如下 :

```
SELECT p.PositionID, p.Type, p.SecurityID, p.[Size],  
a.BBID,a.Price FROM Portfolio p inner join ( select BondID as  
SecurityID,Price,BBID from Bond UNION select LoanID as  
SecurityID,Price,BBID from Loan UNION select CompanyID as  
SecurityID,Price,BBID from Company ) a on a.SecurityID=p.SecurityID
```

第二个TableAdapter命名为PortfolioScenarios , 查询代码如下 :

```
SELECT PositionID, BestReturn, AverageReturn, WorstReturn,  
BestProbability, AverageProbability, WorstProbability, BestReturn *  
BestProbability + AverageReturn * AverageProbability + WorstReturn  
* WorstProbability AS ForecastedReturn FROM PortfolioScenarios
```

添加如下using指令至Program.cs :

```
using Event = BloombergIp.Blpapi.Event; using Element =  
BloombergIp.Blpapi.Element; using Message =  
BloombergIp.Blpapi.Message; using Name = BloombergIp.Blpapi.Name;  
using Request = BloombergIp.Blpapi.Request; using Service =  
BloombergIp.Blpapi.Service; using Session =  
BloombergIp.Blpapi.Session; using DataType =  
BloombergIp.Blpapi.Schema.Datatype; using SessionOptions =  
BloombergIp.Blpapi.SessionOptions; using InvalidRequestException =  
BloombergIp.Blpapi.InvalidRequestException; using System.Data;  
using MathNet.Numerics.LinearAlgebra; using  
MathNet.Numerics.Statistics;
```

在Program类中声明如下变量 :

```
private ADS DS = new ADS(); private static readonly Name  
SECURITY_DATA = new Name("securityData"); private static readonly  
Name SECURITY = new Name("security"); private static readonly Name
```

```

FIELD_DATA = new Name("fieldData"); private static readonly Name
RESPONSE_ERROR = new Name("responseError"); private static readonly
Name SECURITY_ERROR = new Name("securityError"); private static
readonly Name FIELD_EXCEPTIONS = new Name("fieldExceptions");
private static readonly Name FIELD_ID = new Name("fieldId");
private static readonly Name ERROR_INFO = new Name("errorInfo");
private static readonly Name CATEGORY = new Name("category");
private static readonly Name MESSAGE = new Name("message"); private
static readonly Name DATE = new Name("date"); //Date range for
historical price changes private DateTime StartDate = new
DateTime(1990, 1, 31); private DateTime EndDate = DateTime.Now;
//Start Date for correlation matrix private DateTime
CorrelationDate = new DateTime(2016, 6, 30); //UseForecasted=true -
> use PortfolioScenarios Table //UseForecasted=false -> use
historical annualized returns private bool UseForecasted = true;

```

声明一个类，命名为PositionStat，其中存储用于计算相关性的彭博数据和历史价格变化的计算结果。

```

private class PositionStat { public int PositionID = 0; public
double StartingPrice = 0; public double EndingPrice = 0; public
double DayCount = 0; public double HistoricalReturn = 0; public
double AnnualizedReturn = 0; public double StdDev = 0; public
double AnnualizedStdDev = 0; public double MarketValue = 0; public
double Weight = 0; public double ForecastedReturn = 0; public
double Return = 0; public Dictionary<DateTime, double> Hist = new
Dictionary<DateTime, double>(); }

```

声明一个List集合，其中包含一个PositionStat表单：

```
List<PositionStat> Stats = new List<PositionStat>();
```

在Main方法中实例化一个Program实例，并调用控制程序基本流动的Go方法：

```

static void Main(string[] args) { new Program().Go(); } private
void Go() { // Populate DS with data from Access Database
FillDataSet(); //Populate Stats collection using data from
Bloomberg PopulatePositionStats(); //Use Stats collection to
calculate portfolio Standard Deviation double StdDev =
CalcStdDev(); // Calculate weighted average return of portfolio
double Return = Stats.Sum(x => x.Weight * x.Return); //Fetch the
10-year US treasury yield for risk free rate Dictionary<DateTime,
double> riskfree = GetHistory("CT10 Govt", "ASK_YIELD",
DateTime.Now.AddDays(-1),false); //Calculate Sharpe Ratio double
Sharpe = (Return - riskfree.ElementAt(0).Value/100) / StdDev;
//Output Console.WriteLine("Std Dev: "+StdDev.ToString("p2"));
Console.WriteLine("Return: " + Return.ToString("p2"));
Console.WriteLine("Sharpe Ratio: " + Sharpe.ToString("p2")); }

```

FillDataSet方法用Access数据库的数据填入DS变量：

```

private void FillDataSet() { using
(ADSTableAdapters.PortfolioScenariosTableAdapter ta = new
ADSTableAdapters.PortfolioScenariosTableAdapter()) {
ta.Fill(DS.PortfolioScenarios); ta.Connection.Close(); } using
(ADSTableAdapters.PortfolioTableAdapter ta = new
ADSTableAdapters.PortfolioTableAdapter()) { ta.Fill(DS.Portfolio);
ta.Connection.Close(); } }

```

PopulatePositionStats函数循环访问DS数据集中用FillDataSet函数填入的每个头寸，从彭博请求历史价格，并用计算完的数据填入Stats集合：

```

private void PopulatePositionStats() { //Iterate through the
positions foreach (ADS.PortfolioRow position in
DS.Portfolio.OrderBy(x => x.PositionID)) { // Populates prices
variable with historical dates // and prices from Bloomberg //
GetHistory(Bloomberg ID, Bloomberg Field, Historical Start Date, //

```



```

Fetch Monthly Data?) Dictionary<DateTime, double> prices =
GetHistory(position.BBID, "PX_LAST", StartDate, true); // If less
than two historical dates are returned // there is not much that
can be done. // Error handling should be added in the event there
// are less than 2 positions if (prices.Count > 2) { PositionStat
stat = new PositionStat(); stat.PositionID = position.PositionID;
// price on the earliest date returned stat.StartingPrice =
prices.OrderBy(x => x.Key).First().Value; //price on the latest
date returned stat.EndingPrice = prices.OrderByDescending(x =>
x.Key).First().Value; // number of days returned stat.DayCount =
(prices.OrderByDescending(x => x.Key).First().Key -
prices.OrderBy(x => x.Key).First().Key).TotalDays - 1;
stat.HistoricalReturn = (stat.EndingPrice - stat.StartingPrice) /
stat.StartingPrice; //annualize historical returns
stat.AnualizedReturn = Math.Pow(1 + stat.HistoricalReturn, 365 /
stat.DayCount) - 1; // Loop through returned prices // populate
Hist property with // monthly price change for (int i = 1; i <
prices.Count; i++) { DateTime currDate = prices.OrderBy(x =>
x.Key).ElementAt(i).Key; DateTime prevDate = prices.OrderBy(x =>
x.Key).ElementAt(i - 1).Key; double currPx = prices[currDate];
double prevPx = prices[prevDate]; double chgPx = (currPx - prevPx)
/ prevPx; stat.Hist.Add(currDate, chgPx); } //Calculate standard
deviation of monthly price changes stat.StdDev =
Statistics.StandardDeviation(stat.Hist.Values); //annualize
standard deviation stat.AnualizedStdDev = stat.StdDev *
Math.Sqrt(12); //Calculate Market Value of the position double
price = 0; if (position.Type.ToUpper() == "EQUITY") price =
position.Price; else price = position.Price / 100; stat.MarketValue
= price * (double)position.Size; // find the position in the
PortfolioScenarios table // Populate the ForecastedReturn property
of stat with // data from the Access database
ADS.PortfolioScenariosRow scenario =

```

```

DS.PortfolioScenarios.FindByPositionID(stat.PositionID);
stat.ForecastedReturn = scenario.ForecastedReturn; //set
stat.Return depending on if we are // looking at forecasted returns
// or annualized historical returns if (UseForecasted) stat.Return
= stat.ForecastedReturn; else stat.Return = stat.AnnualizedReturn;
Stats.Add(stat); } } // Once market value is calculated for each
position // loop through and set the Weight property to // the
market value percentage of the portfolio double totalMV =
Stats.Sum(x => x.MarketValue); foreach (PositionStat stat in Stats)
stat.Weight = stat.MarketValue / totalMV; }

```

GetHistory和ProcessHistoryResponse方法与6.3.1节中的程序代码类似，除了GetHistory接受一个请求月度价格（而非每日价格）的布尔实参：

```

private Dictionary<DateTime, double> GetHistory ( string
security, string field, DateTime startDate, bool monthly) {
Dictionary<DateTime, double> date2value = new Dictionary<DateTime,
double>(); SessionOptions sessionOptions = new SessionOptions();
Session session = new Session(); bool sessionStarted =
session.Start(); if (!sessionStarted) {
System.Console.Error.WriteLine("Failed to start session."); return
null; } if (!session.OpenService("//blp/refdata")) {
System.Console.Error.WriteLine("Failed to open //blp/refdata");
return null; } Service refDataService =
session.GetService("//blp/refdata"); Request request =
refDataService.CreateRequest("HistoricalDataRequest"); Element
securities = request.GetElement("securities");
securities.AppendValue(security); Element fields =
request.GetElement("fields"); fields.AppendValue(field);
request.Set("startDate", startDate.ToString("yyyyMMdd")); //if
Monthly argument is set //to true, set periodicity to Calendar
Month if (monthly) { request.Set("periodicitySelection",
"MONTHLY"); request.Set("periodicityAdjustment", "CALENDAR"); } try

```

```

{ session.SendRequest(request, null); } catch
(InvalidRequestException e) {
System.Console.WriteLine(e.ToString()); } bool done = false; while
(!done) { Event eventObj = session.NextEvent(); if (eventObj.Type
== Event.EventType.PARTIAL_RESPONSE) {
ProcessHistoryResponse(eventObj, date2value); } else if
(eventObj.Type == Event.EventType.RESPONSE) {
ProcessHistoryResponse(eventObj, date2value); done = true; } else {
foreach (Message msg in eventObj) {
System.Console.WriteLine(msg.AsElement); if (eventObj.Type ==
Event.EventType.SESSION_STATUS) { if
(msg.MessageType.Equals("SessionTerminated")) { done = true; } } }
} } session.Stop(); return date2value; } private void
ProcessHistoryResponse(Event eventObj, Dictionary<DateTime, double>
date2value) { foreach (Message msg in eventObj) { if
(msg.HasElement(RESPONSE_ERROR)) { Element error =
msg.GetElement(RESPONSE_ERROR); Console.WriteLine("Request failed:
" + error.GetElementAsString(CATEGORY) + " (" +
error.GetElementAsString (MESSAGE) + ")"); continue; } Element
securityData = msg.GetElement(SEcurity_DATA); string security =
securityData.GetElement(SEcurity).GetValueAsString();
Console.WriteLine(security); Element fieldData =
securityData.GetElement(FIELD_DATA); for (int i = 0; i <
fieldData.NumValues; i++) { Element element =
fieldData.GetValueAsElement(i); DateTime date =
element.GetElementAsDatetime(DATE).ToSystemDateTime(); double?
value = null; for (int f = 0; f < element.NumElements; f++) {
Element field = element.GetElement(f); if
(!field.Name.Equals(DATE)) { if (field.Datatype ==
DataType.FLOAT32) value = Convert.ToDouble(field.GetValueAs-
Float32()); else if (field.Datatype == DataType.FLOAT64) value =
field.GetValueAsFloat64(); } } if (value != null)

```

```
date2value.Add(date, value.Value); } } }
```

CalcStdDev方法是在矩阵中填入每个头寸组合的相关性。然后将相关性矩阵与每个头寸的加权标准差向量相乘：

```
private double CalcStdDev() { int count = Stats.Count; //Matrix
to store the correlation between each position Matrix<double>
corrMatrix = Matrix<double>.Build.Dense(count, count); //Vector to
store the weighted standard deviation of each position // where the
weight is the percentage the position makes up // of the portfolio
and the standard deviation // is the annualized standard deviation
of historical returns. Vector<double> weightVector =
Vector<double>.Build.Dense(count); for (int i = 0; i < count; i++)
{ weightVector[i] = Stats[i].Weight * Stats[i].AnnualizedStdDev;
for (int j = 0; j < count; j++) { corrMatrix[i, j] =
GetCorrelation(Stats[i], Stats[j]); } } //Matrix multiplication
Vector<double> variance = weightVector * corrMatrix *
weightVector.ToColumnMatrix(); return Math.Sqrt(variance[0]); }
```

GetCorrelation方法用从应用开始时宣布的CorrelationDate之后的历史月度价格变化，计算两个头寸之间的相关性：

```
private double GetCorrelation(PositionStat stat1, PositionStat
stat2) { double corr = 0; //Get the list of monthly price changes
from the first position // where the date of the price change is
greater than or equal to // the CorrelationDate variable set at the
//start of the program List<double> set1 = (from stat in stat1.Hist
where stat.Key >= CorrelationDate select stat.Value).ToList();
//Like set1, this gets the monthly price changes // for the second
position List<double> set2 = (from stat in stat2.Hist where
stat.Key >= CorrelationDate select stat.Value).ToList(); //MathNet
to calculate correlation corr = Correlation.Pearson(set1, set2);
return corr; }
```

7.3.2 投资组合分解和警告标志

对于绝大部分内容来说，7.2节的查询可以很方便地修改，然后用于C#应用的表结构。推荐你将第2章中计算每个头寸市值的投资组合查询保存为“PortfolioQuery”，并在请求市值的查询中使用它。例如，如下查询用一个保存好的投资组合查询对头寸按照类型进行分类：

```
SELECT Type, Format(Sum(MarketValue), "#,##0") AS MV,  
FormatPercent(sum(PctofPortfolio),2) as PortfolioPct FROM  
PortfolioQuery WHERE Type Is Not Null GROUP BY Type;
```

7.4 本章小结

第6章关注各个有价值证券的相对价值，而本章介绍了测量一个投资组合的风险和收益的不同方法。你应当把两章的技术相结合，做出更好的投资决策。例如，某个有价值证券可能从相对价值角度看起来不错，但是却可能让你的投资组合过于集中于某一个行业。第8章还将继续扩大范围，涵盖整个贷款市场。

第8章 市场分析

本章讨论把更广义的市场趋势融入你的分析的重要性。例如，零售企业的股票看起来比科技企业的股票更好的原因之一是市场情绪普遍对零售行业施加了压力。尽管我们可以比较容易地在证券市场上测量市场情绪，但是要在透明度较低的市场（例如高回报债券和企业贷款市场）追踪市场情绪就比较难了。本章展示如何利用从Markit收集的数据，判断企业贷款市场的整体趋势。把在本章中学到的内容稍加改动，就可以分析数据集比较类似的其他市场了。

本章将Markit贷款数据分解为三个部分，并在每部分展示不同种类的分析。

第一部分用贷款更新（facility update）信息研判新发放贷款的趋势。可以用该信息确定每月进入市场的贷款数量及其平均票面利息、规模和期限（距到期日年限）。此外，该信息提示某行业相对于市场整体定价宽松程度，揭示其风险。

第二部分用推荐更新 (recommended update) 信息研判贷款再融资的趋势。聚合 (aggregated) 贷款再融资数据展示了重要信息，例如到期后延期的平均年数以及息票的平均变化。该信息可以分解到行业，能够帮助我们了解市场的整体趋势。

最后一部分将贷款市场信息用于多重目的。我们可以用历史价格分行业考察价格变动趋势、辨识出表现特别好或表现不佳的贷款、找到符合特定条件的贷款（例如“现在价格超过90美元，但是曾经低于70美元”）。

不幸的是，Microsoft Excel在分析大型数据集合时有一定局限。例如，行数最多为20万行，天数最多为125天。由此，本章对于Excel只提供基础案例，而对Microsoft Access进行更深入的分析。

8.1 途径1：Excel

Markit提供大量贷款市场信息，但不幸的是，Excel不能很好地处理大量数据。推荐你勤勉地维护好每个工作表，且只保存必要的历史数据。此外，如果多达几千行，就不要在Markit数据上直接添加公式。

8.1.1 新发放贷款分析

Markit的贷款更新数据包含每笔企业贷款的静态信息，包括决算日、规模、期限和息差。因为大部分贷款的利率都是浮动的，其息差就是该笔贷款在一个指数（一般用伦敦同业拆借利率 (London Interbank Offered Rate , LIBOR) ）之上多支付的部分。例如，如果一笔贷款的票面是三个月期LIBOR加上300bps，其息差为300bps。由于大部分新发放贷款价格与票面价格差不多，其息差大约等于其贴现差额 (DM) ，也是风险的最佳指标。在其他条件相同的情况下，一个息差为300bps的新发放贷款的风险小于一个息差为500bps的新发放贷款的风险。

然而，市场的风险偏好会随时间而变化，两个月前价格为300bps息差的贷款，可能今天的价格高达500bps息差。可以用下面的步骤根据时间的不同平均息差水平，确定数据中可观察的息差，以追踪市场情绪：

1. 创建一个新的工作表，命名为New Issue。

2.在第一行各列，添加如下标签：Date、Count、Total Size、Avg Size、Term、All Spread、Healthcare、Retail和Oil&Gas。

3.在A列的Date列标题下方，用每月第一天添加一个月度日期区域。例如，在单元格A2，输入2/1/2017；在单元格A3，输入1/1/2017；等等。

4.把Excel区域转换为Excel表。

5.在Count列（B列），在每一行都输入如下公式：

```
=SUMPRODUCT(N(LoanFacilities[Close Date]>=[@Date]),  
N(LoanFacilities[Close Date]<=EDATE([@Date],1)-1),  
N(LoanFacilities[Currency]="US Dollar"), N(LoanFacilities[LoanX  
Facility Category]="Institutional"),  
N(LoanFacilities[Status]="Active"),N(LoanFacilities[Initial  
Spread]>0))
```

该公式假设你把包含Markit贷款数据的Excel表命名为“LoanFacilities”。它用SUMPRODUCT函数和N函数相结合，只计算LoanFacilities表中符合条件的行。第一个N函数过滤掉交割日期在列A显示的日期之前的那些行。下一个N函数过滤掉交割日期在当月最后一天（EDATE函数在列A的日期上加上一个月，然后从该日期减去一天，以得到当月最后一天的日期）之后的那些行。下一组N函数将结果过滤限制为贷款类型为机构贷款（除了循环贷款（revolvers）和按比例子贷款（pro-rata tranches）），货币按美元计算，状态为活跃，且已填入息差（除了一些错误的行）。

6.在Total Size列，输入如下公式：

```
=SUMPRODUCT(N(LoanFacilities[Close Date]>=[@Date]),  
N(LoanFacilities[Close Date]<=EDATE([@Date],1)-1),  
N(LoanFacilities[Currency]="US Dollar"), N(LoanFacilities[LoanX  
Facility Category]="Institutional"),  
N(LoanFacilities[Status]="Active"), N(LoanFacilities[Initial  
Spread]>0),LoanFacilities[Initial Amount])
```


该公式与第5步的公式相同，除了增加了最后一个参数

LoanFacilities[Initial Amount]，结果是SUMPRODUCT返回了符合标准的各行对应的Initial Amount列之和。

7.在Avg Size列，输入如下公式：

=[@[Total Size (\$mm)]]/[@Count]

该公式就是Total Size列除以Count，以计算在该阶段发行的贷款的平均规模。

8.在Term列，输入如下公式：

=SUMPRODUCT(N(LoanFacilities[Close Date]>=[@Date]),
N(LoanFacilities[Close Date]<=EDATE([@Date],1)-1),
N(LoanFacilities[Currency]="US Dollar"), N(LoanFacilities[LoanX
Facility Category]="Institutional"),
N(LoanFacilities[Status]="Active"), LoanFacilities[Initial Amount],
N(LoanFacilities[Initial Spread]>0), LoanFacilities[Term])/[@[Total
Size]]

该公式与第6步相同，除了它在SUMPRODUCT的实参列表中增加了Term列除以Total Size列，结果是期限的加权平均值。看到不同项相对于其初始量所占的比重（而非进行简单平均）是很重要的，因为有些小型贷款可能歪曲结果。

9.在All Spread列，输入如下公式：

=SUMPRODUCT(N(LoanFacilities[Close Date]>=[@Date]),
N(LoanFacilities[Close Date]<=EDATE([@Date],1)-1),
N(LoanFacilities[Currency]="US Dollar"), N(LoanFacilities[LoanX
Facility Category]="Institutional"),
N(LoanFacilities[Status]="Active"),LoanFacilities[Initial Amount],
N(LoanFacilities[Initial Spread]>0), LoanFacilities[Initial
Spread])/[@[Total Size]]

与第8步类似，该步计算Initial Spread的加权平均值。

10.在Healthcare列，输入如下公式：

```
=SUMPRODUCT(N(LoanFacilities[Close Date]>=[@Date]),  
N(LoanFacilities[Close Date]<=EDATE([@Date],1)-1),  
N(LoanFacilities[Currency]="US Dollar"), N(LoanFacilities[LoanX  
Facility Category]="Institutional"),  
N(LoanFacilities[Status]="Active"),LoanFacilities[Initial Amount],  
N(LoanFacilities[Initial Spread]>0),LoanFacilities[Initial Spread],  
N(LoanFacilities[Industry]="Healthcare"))/  
SUMPRODUCT(N(LoanFacilities[Close Date]>=$A2),  
N(LoanFacilities[Close Date]<=EDATE([@Date],1)-1),  
N(LoanFacilities[Currency]="US Dollar"), N(LoanFacilities[LoanX  
Facility Category]="Institutional"),  
N(LoanFacilities[Status]="Active"),LoanFacilities[Initial Amount],  
N(LoanFacilities[Initial Spread]>0),  
N(LoanFacilities[Industry]="Healthcare"))
```

该公式与前面步骤里的过滤公式相同，但是增加了一个行业类型限制为医疗健康（healthcare）。此外，我们用医疗健康贷款的初始总值（而非总值）作为分母。计算所有医疗健康贷款的总值使用的公式与分子中的公式相同，但是不包括初始息差（Initial Spread）的数值。结果是医疗健康贷款的加权平均息差。如果该时段没有发行医疗贷款，该公式可能导致被除数为零的错误。

11.对Retail和Oil&Gas列用与第10步相同的公式，但是把Healthcare过滤公式修改为Retailing和Oil&Gas过滤公式。

返回Retail和Oil&Gas息差列的加权平均值。

图8-1是结果表，其中显示相对于整个市场来说，医疗、零售和油气行业的息差较大。它还显示油气行业贷款在前几个月有普遍改善。

	A	B	C	D	E	F	G	H	I
1	Date	Count	Total Size	Avg Size	Term	All Spread	Healthcare	Retail	Oil & Gas
2	2/1/2017	128	79,125	618	5.82	343	424	#DIV/0!	403
3	1/1/2017	110	80,961	736	5.38	336	339	310	366
4	12/1/2016	153	77,915	509	5.69	361	367	505	986
5	11/1/2016	121	62,441	516	6.27	401	456	929	564
6	10/1/2016	135	75,955	563	5.34	382	440	342	1,145
7	9/1/2016	128	52,255	408	5.80	363	553	358	358
8	8/1/2016	115	57,317	498	5.88	432	467	390	875

图8-1：新发放贷款分析

在本书曾多次说过，尽管该信息对于判断趋势有用，但是分析师总是应该审视贷款的潜在信息，以确保不歪曲数字。

8.1.2 再融资

与抵押贷款类似，企业可以对贷款进行再融资（在非行权期之后），变成一个息差更低、到期日可能更晚的新贷款。尽管某个公司再融资变成一个息差更低的贷款，一般说明该公司业绩良好（出借人愿意以更低的收益借款给该公司），不过还可能是因为该公司要应对马上要到来的到期日。再融资增多一般说明市场情绪好转。

与贷款更新信息一样，Markit的推荐更新数据可用作帮助判断再融资趋势的价格点。每次一笔贷款进行重组、再融资或者付清，Markit会指定一个新的LoanX ID，并注意推荐更新数据输入的变化。该输入包含许多字段，包括息差和到期日的初始和新信息。该数据有助于判断各行业以及整个市场的息差。

下面的步骤创建一个新的工作表，其中整合了再融资贷款的信息：

12. 创建一个新的工作表，命名为Refinancings。

13.在第一行各列分别添加如下标签：Date、Count、Size、Years Extension、Original Spread、New Spread、Spread Change、Healthcare和Retail。

14.与第3步一样，在Date列标题下（A列），用每月第一天的日期输入月度日期。

15.在Count列，输入如下公式：

```
=SUMPRODUCT(N(LoanUpdates[Inactivation Date]>=[@Date]),  
N(LoanUpdates[Inactivation Date] < EDATE([@Date],1)-1),  
N(LoanUpdates[InactivationReason]="Refinanced"))
```

该公式假设推荐更新Excel表被命名为LoanUpdates。与上一节的步骤一样，该公式用于过滤在LoanUpdates期限内进行了再融资的贷款。

16.在Years Extension列，输入如下公式：

```
=SUMPRODUCT(N(LoanUpdates[Inactivation Date]>=[@Date]),  
N(LoanUpdates[Inactivation Date] <= EDATE([@Date],1)-1),  
N(LoanUpdates[InactivationReason]="Refinanced"), N(LoanUpdates[Repl  
Maturity Date] >= LoanUpdates[Maturity Date]), LoanUpdates[Initial  
Amount], LoanUpdates[Repl Maturity Date]-LoanUpdates[Maturity  
Date])/ SUMPRODUCT(N(LoanUpdates[Inactivation Date]>=[@Date]),  
N(LoanUpdates[Inactivation Date] <= EDATE([@Date],1)-1),  
N(LoanUpdates[InactivationReason]="Refinanced"), N(LoanUpdates[Repl  
Maturity Date] >= LoanUpdates[Maturity Date]), LoanUpdates[Initial  
Amount])/365
```

该公式增加了一些过滤公式，包括新的到期日期晚于初始到期日期的行，标示进行延期的贷款。它还增加了一个过滤公式，计算新的到期日和前一个到期日之间的差，以计算贷款延期天数的加权平均值。该数值除以365，即得到按年计算的值。

17.在Original Spread列，输入如下公式：

```
=SUMPRODUCT(N(LoanUpdates[Inactivation Date]>=[@Date]),
N(LoanUpdates[Inactivation Date] <= EDATE([@Date],1)-1),
N(LoanUpdates[InactivationReason]="Refinanced"), N(LoanUpdates[Repl
Initial Spread]>0), N(LoanUpdates[Initial Spread]>0),
LoanUpdates[Initial Amount],LoanUpdates[Initial Spread])/
SUMPRODUCT(N(LoanUpdates[Inactivation Date]>=[@Date]),
N(LoanUpdates[Inactivation Date] <= EDATE([@Date],1)-1),
N(LoanUpdates[InactivationReason]="Refinanced"), N(LoanUpdates[Repl
Initial Spread]>0), N(LoanUpdates[Initial
Spread]>0),LoanUpdates[Initial Amount])
```

该公式返回平均初始息差（在初始和当前息差都存在的情况下）。

18.在New Spread列，输入如下公式：

```
=SUMPRODUCT(N(LoanUpdates[Inactivation Date]>=[@Date]),
N(LoanUpdates[Inactivation Date] <= EDATE([@Date],1)-1),
N(LoanUpdates[InactivationReason]="Refinanced"), N(LoanUpdates[Repl
Initial Spread]>0), N(LoanUpdates[Initial
Spread]>0),LoanUpdates[Initial Amount], LoanUpdates[Repl Initial
Spread])/ SUMPRODUCT(N(LoanUpdates[Inactivation Date]>=[@Date]),
N(LoanUpdates[Inactivation Date] <= EDATE([@Date],1)-1),
N(LoanUpdates[InactivationReason]="Refinanced"), N(LoanUpdates[Repl
Initial Spread]>0), N(LoanUpdates[Initial
Spread]>0),LoanUpdates[Initial Amount])
```

类似地，该公式返回加权平均重置息差。

19.在Spread Change列，输入如下公式：

```
=SUMPRODUCT(N(LoanUpdates[Inactivation Date]>=[@Date]),
N(LoanUpdates[Inactivation Date] <= EDATE([@Date],1)-1),
N(LoanUpdates[InactivationReason]="Refinanced"), N(LoanUpdates[Repl
Initial Spread]>0), N(LoanUpdates[Initial
```

```
Spread]>0),LoanUpdates[Initial Amount], LoanUpdates[Repl Initial
Spread]-LoanUpdates[Initial Spread])/
SUMPRODUCT(N(LoanUpdates[Inactivation Date]>=[@Date]),
N(LoanUpdates[Inactivation Date] <= EDATE([@Date],1)-1),
N(LoanUpdates[InactivationReason]="Refinanced"), N(LoanUpdates[Repl
Initial Spread]>0), N(LoanUpdates[Initial
Spread]>0),LoanUpdates[Initial Amount])
```

该公式返回息差的加权平均值变化。

20.在Healthcare列，输入如下公式：

```
=SUMPRODUCT(N(LoanUpdates[Inactivation Date]>=[@Date]),
N(LoanUpdates[Inactivation Date] <= EDATE([@Date],1)-1),
N(LoanUpdates[InactivationReason]="Refinanced"), N(LoanUpdates[Repl
Initial Spread]>0), N(LoanUpdates[Initial
Spread]>0),LoanUpdates[Initial Amount], LoanUpdates[Repl Initial
Spread]-LoanUpdates[Initial Spread],
N(LoanUpdates[Industry]="Healthcare"))/
SUMPRODUCT(N(LoanUpdates[Inactivation Date]>=[@Date]),
N(LoanUpdates[Inactivation Date] <= EDATE([@Date],1)-1),
N(LoanUpdates[InactivationReason]="Refinanced"), N(LoanUpdates[Repl
Initial Spread]>0), N(LoanUpdates[Initial
Spread]>0),LoanUpdates[Initial Amount],
N(LoanUpdates[Industry]="Healthcare"))
```

该公式返回医疗行业贷款息差的加权平均值变化。当然，如果那段时间没有医疗行业再融资，可能会出现除以零这种错误。

21.在Retail列输入与第20步相同的公式，但是把医疗改为零售（Retailing）行业。

这些步骤的结果如图8-2所示。用结果表判断跨月份和行业息差缩减的趋势。

	A	B	C	D	E	F	G	H	I
1	Date	Count	Size	Years Extension	Original Spread	New Spread	Spread Change	Healthcare	Retail
2	2/1/2017	1	3,323	-	325.00	300.00	(25.00)	#DIV/0!	#DIV/0!
3	1/1/2017	2	500	1.65	325.00	250.00	(75.00)	#DIV/0!	#DIV/0!
4	12/1/2016	40	21,966	1.86	343.76	322.45	(21.31)	(21.30)	#DIV/0!
5	11/1/2016	50	44,146	2.15	279.04	256.39	(22.64)	-	#DIV/0!
6	10/1/2016	77	40,683	2.19	393.29	355.76	(37.54)	(87.83)	(58.29)
7	9/1/2016	35	21,493	1.91	324.53	289.74	(34.79)	(26.09)	(25.00)
8	8/1/2016	37	19,932	2.20	372.92	330.76	(42.16)	-	18.22
9	7/1/2016	40	16,771	2.35	367.11	340.40	(26.71)	213.61	(150.00)
10	6/1/2016	44	28,758	2.76	407.63	366.41	(41.22)	34.52	(75.00)

图8-2：贷款再融资分析

你可以用该数据判断公司节省了多少利息（息差缩小）以及它们延长贷款到期日的能力。与贷款机构信息一样，分析师应当回顾推荐更新数据中可能引起结果变化的异常值。注意息差用基点。

8.1.3 价格历史

Markit从买方代理人那里收集大部分企业贷款信息，并每日发布。除了提供买方出价和卖方出价之外，Markit还提供一个Depth列，表明价格贡献者的数量。Depth可以被视为贷款流动性的媒介，数值越大一般表示流动性越强。公司常用该数据标注每日头寸，但是每日贷款价格其实有更多用途，包括如下：

- 整体市场信息，包括所有贷款的每日平均价格变化，或者分行业或者分评级的贷款每日平均价格变化。
- 一笔贷款投资组合的平均流动性。
- 一笔贷款的价格区间，揭示其在不同信誉循环中的业绩情况。
- 业绩低于或超过市场的公司。

然而，大数据也有大问题。因为Markit按照“第三范式”提供数据，因此贷款定价数据中包含的关于贷款本身的信息仅有其标识符（LoanX ID），用它可以连接到LoanFacilities表。而且你会发现，每天连接超过6000笔贷款信息会导致Excel

变慢，甚至崩溃。解决该问题的一种办法是用VLOOKUP或INDEX/MATCH函数从LoanFacilities表向LoanPrices表增加列，并将结果从一个公式转换成值（删除连接）。然而，该解决方案也有几个缺点：

- 每次从Markit获得每日贷款价格，你都必须重复上述步骤从LoanFacilities表填入facility信息。

- 用增加另一个表中已经存在的数据，以删除“第三范式”的方法，可能导致数据问题。例如，如果LoanPrices表从LoanFacilities表复制了某笔贷款的行业信息，然后行业信息得到了更新，那么LoanPrices表中的信息就是错误的，因为数据没有被链接。

- 用VLOOKUP链接数据也不能保证Excel肯定不会崩溃。

因此，推荐用Microsoft Access对大数据集进行分析，而非Excel。尽管如此，对于那些想“玩火”的人，下面的步骤展示如何从LoanFacilities表向LoanPrices表链接行业和规模列。

22.在Marks工作表中，对LoanPrices表增加一列，命名为Industry，然后输入如下公式：

```
=INDEX(LoanFacilities[Industry],MATCH([@[LoanX ID]],  
LoanFacilities[LoanX ID],0))
```

该公式从LoanFacilities表提取特定LoanX ID的Industry信息。

23.选择Industry列下面的所有单元格，右击，然后在弹出的快捷菜单上选择Copy。然后，再次右击，在同一列选择Paste Values。

该步骤将INDEX公式用Industry值进行了重写。去掉两张表之间的链接将使Excel运行得更好。

24.在LoanPrices表增加另一个列，命名为Initial Amount，然后输入如下公式：

```
=INDEX(LoanFacilities[Initial Amount],MATCH([@[LoanX ID]],
```

LoanFacilities[LoanX ID],0))

与第22步一样，该公式从LoanFacilities表提取Initial Amount列。

25.选择Initial Amount列标题下的每个单元格，右击并选择Copy，然后再次右击，并选择Paste Values。

下面的步骤展示如何创建整个市场和分行业的贷款价格变化情况摘要。

26.创建一个新的工作表，命名为Price Changes。

27.在第一行，为A列至G列增加如下列标题：Date、Count、All、%Change、Healthcare、Retail、Oil&Gas。

28.在单元格A2，在Date列标题下，增加如下公式：

=MAX(LoanPrices[Mark Date])

返回LoanPrices表的最新日期。

29.在单元格A3，即包含最新日期的单元格下，输入公式并向下拖拽想要的天数：

=MAXIFS(LoanPrices[Mark Date],LoanPrices[Mark Date],"< "&A2)

该公式返回最新的定价日期。该公式用于获得连续定价日期，因为周末和节日可能使定价日期不连续。

30.将Excel区域（至G列）转换成Excel表。

31.在B列，在Count列标题下，输入如下公式：

=COUNTIF(LoanPrices[Mark Date],[@Date])

该函数查询在每个指定日期的贷款价格数量。

32.在All列标题下，用如下公式：

`=SUMPRODUCT(N(LoanPrices[Mark Date]=[@Date]),LoanPrices[Initial Amount], LoanPrices[Evaluated Price])/SUMPRODUCT(N(LoanPrices[Mark Date]=[@Date]), LoanPrices[Initial Amount])`

该公式返回在某个指定日期根据每笔贷款的初始数量计算的加权平均价格。

33.在单元格C2，即在“%Change”列用如下公式，然后向下拖拽：

`=([@A11]-C3)/C3`

该公式将A11列的值与前一天的值进行比较，计算每日百分比价格变化。注意最后一行将返回错误，因为没有前一天的价格可进行比较。

34.在Healthcare下，输入如下公式：

`=SUMPRODUCT(N(LoanPrices[Mark Date]=[@Date]),
N(LoanPrices[Industry]="Healthcare"),LoanPrices[Initial Amount],
LoanPrices[Evaluated Price])/ SUMPRODUCT(N(LoanPrices[Mark Date]=
[@Date]), N(LoanPrices[Industry]="Healthcare"),LoanPrices[Initial
Amount])`

该公式与第32步一样，计算每日贷款价格的加权平均值，但该公式只包括医疗贷款。

35.在Retail列，输入如下公式：

`=SUMPRODUCT(N(LoanPrices[Mark Date]=[@Date]),
N(LoanPrices[Industry]="Retailing"),LoanPrices[Initial Amount],
LoanPrices[Evaluated Price])/ SUMPRODUCT(N(LoanPrices[Mark Date]=
[@Date]), N(LoanPrices[Industry]="Retailing"),LoanPrices[Initial
Amount])`

该公式与第34步一样，计算每日贷款价格的加权平均值，但该公式只包括零售贷款。

36.在Oil&Gas列，输入如下公式：

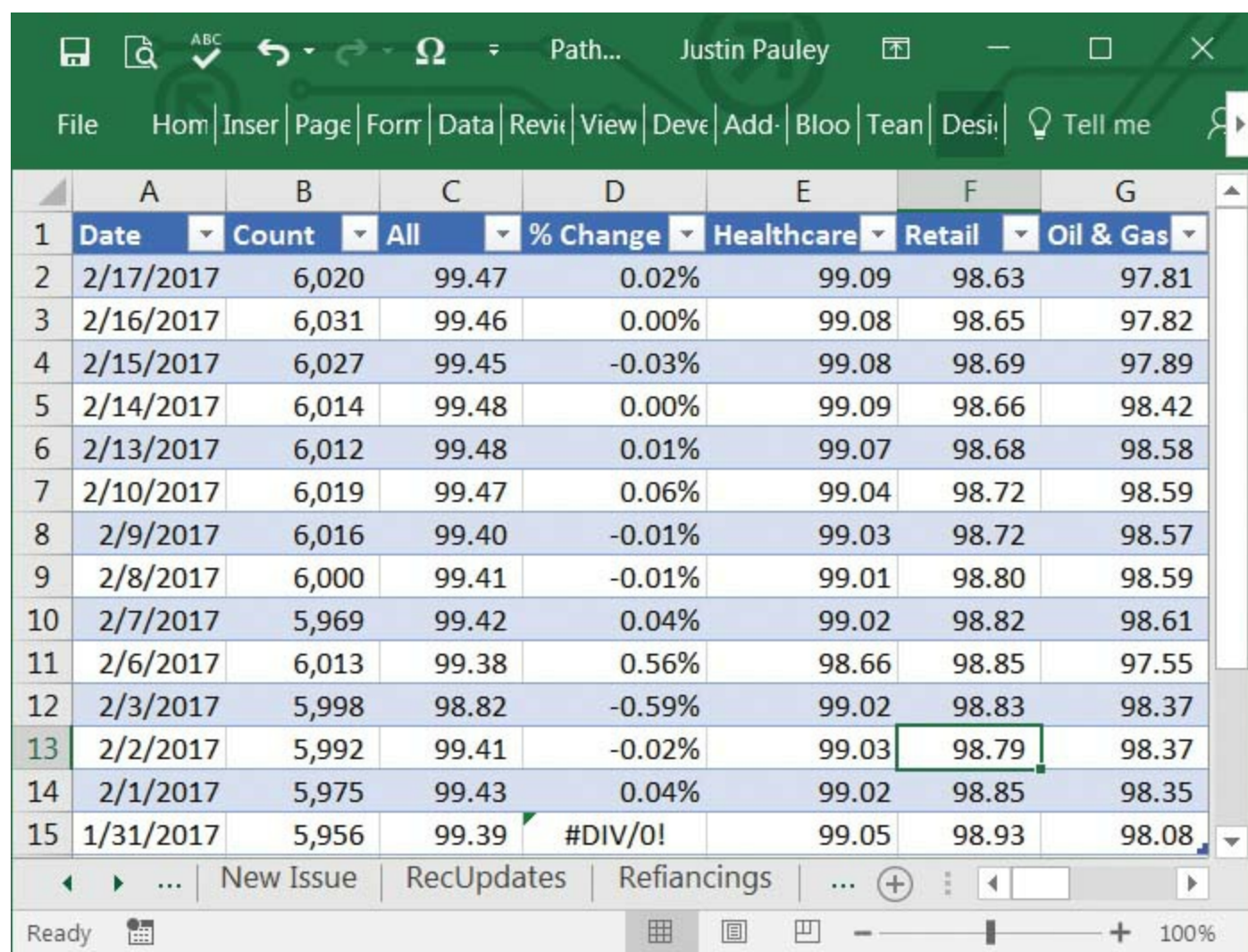
=SUMPRODUCT(N(LoanPrices[Mark Date]=[@Date]),
N(LoanPrices[Industry]="Oil & Gas"),LoanPrices[Initial Amount],
LoanPrices[Evaluated Price])/ SUMPRODUCT(N(LoanPrices[Mark Date]=
[@Date]), N(LoanPrices[Industry]="Oil & Gas"),LoanPrices[Initial
Amount])

该公式与上两列一样，但只包括油气行业的贷款。

结果应该如图8-3所示。

你可以用分行业每日贷款价格分解和价格随时间变动情况，判断哪部分市场承压或有特殊需求。

你可以追踪整个市场的每日价格变动，在总结每个贷款的情况时比用BDH函数更快更省力。下面的步骤展示如何对特定贷款总结历史价格：



	A	B	C	D	E	F	G
1	Date	Count	All	% Change	Healthcare	Retail	Oil & Gas
2	2/17/2017	6,020	99.47	0.02%	99.09	98.63	97.81
3	2/16/2017	6,031	99.46	0.00%	99.08	98.65	97.82
4	2/15/2017	6,027	99.45	-0.03%	99.08	98.69	97.89
5	2/14/2017	6,014	99.48	0.00%	99.09	98.66	98.42
6	2/13/2017	6,012	99.48	0.01%	99.07	98.68	98.58
7	2/10/2017	6,019	99.47	0.06%	99.04	98.72	98.59
8	2/9/2017	6,016	99.40	-0.01%	99.03	98.72	98.57
9	2/8/2017	6,000	99.41	-0.01%	99.01	98.80	98.59
10	2/7/2017	5,969	99.42	0.04%	99.02	98.82	98.61
11	2/6/2017	6,013	99.38	0.56%	98.66	98.85	97.55
12	2/3/2017	5,998	98.82	-0.59%	99.02	98.83	98.37
13	2/2/2017	5,992	99.41	-0.02%	99.03	98.79	98.37
14	2/1/2017	5,975	99.43	0.04%	99.02	98.85	98.35
15	1/31/2017	5,956	99.39	#DIV/0!	99.05	98.93	98.08

图8-3：每日贷款价格汇总

37.在第1行，为I列至Q列，增加如下列标题：LoanX ID、Deal Name、Current Price、Max Price、Max Date、Min Price、Min Date、Count、StdDev。

38.在LoanX ID列对贷款输入LoanX标识符，以总结LoanXFacility工作表。在LoanX ID列标题下，每行输入一个标识符。

39.把Excel区域转换为Excel表。

40.在Deal Name列标题下，输入如下公式：

```
=INDEX(LoanFacilities[Deal Name],MATCH([@[LoanX ID]],  
LoanFacilities[LoanX ID],0))
```

该公式用INDEX和MATCH函数，从LoanFacilities表对给定的LoanX ID提取Deal Name列。

41.在Current Price列，输入如下公式：

```
=SUMPRODUCT(N(LoanPrices[Mark Date]=MAX(LoanPrices[Mark  
Date])), N(LoanPrices[LoanX ID]=[@[LoanX ID]]),LoanPrices[Evaluated  
Price])
```

该公式通过在LoanPrices表中过滤Mark Date，返回给定LoanX ID的最新贷款价格。

42.在Max Price列，输入如下公式：

```
=MAXIFS(LoanPrices[Evaluated Price],LoanPrices[LoanX ID],  
[@[LoanX ID]])
```

该公式对一个给定LoanX ID返回LoanPrices中的最高价格。

43.在Max Date列，输入如下公式：

```
=MAXIFS(LoanPrices[Mark Date],LoanPrices[LoanX ID],[@[LoanX ID]], LoanPrices[Evaluated Price],[@[Max Px]])
```

该公式用MAXIFS函数返回贷款价格最高的最近一个日期。

44.在Min Price列，输入如下公式：

```
=MINIFS(LoanPrices[Evaluated Price],LoanPrices[LoanX ID],[@[LoanX ID]])
```

类似于第42步，该公式对一个给定LoanX ID返回LoanPrices中的最低价格。

45.在Min Date列，输入如下公式：

```
=MAXIFS(LoanPrices[Mark Date],LoanPrices[LoanX ID],[@[LoanX ID]],LoanPrices[Evaluated Price],[@[Min Px]])
```

类似于第43步，该公式返回贷款价格最低的最近一个日期。

46.在Count列标题下，输入如下公式：

```
=COUNTIF(LoanPrices[LoanX ID],[@[LoanX ID]])
```

该公式返回某指定贷款处于特定价格的天数。

47.在StdDev列，输入如下数组公式：

```
=STDEV(IF(LoanPrices[LoanX ID]=[@[LoanX ID]],LoanPrices[Evaluated Price]))
```

该公式返回一个特定贷款的贷款价格标准差。如上一章所述，标准差测量波动性和风险。注意因为该公式是数组公式，输入之后必须按Ctrl+Shift+Enter组合键。

结果应该如图8-4所示。做投资决策时了解标准差和高低价格是很有益的，尽管过往业绩不能保证未来业绩。了解高低价格出现的日期也是很有益的，可以让你知道贷款价格变动受整体市场情况影响多，还是受该贷款自身影响多。这些日期还能

告诉你，贷款现在或者近期是否处于最高或最低价格。

	I	J	K	L	M	N	O	P	Q
1	LoanX ID	Deal Name	Current Px	Max Px	Max Date	Min Px	Min Date	Count	StdDev
2	LX143540	Community Health 3/1	99.36	99.55	10/14/2016	98.76	1/4/2017	34	16%
3	LX148423	Nelson Education 10/	57.75	57.83	2/9/2017	52.67	10/14/2016	34	89%
4	LX157886	EW Scripps 12/16 Cov	100.75	100.75	2/17/2017	100.56	1/3/2017	33	6%

图8-4：贷款价格总结

8.2 途径2和3：Access和C#

因为本章中使用的数据已在数据库中，因此不需要写C#代码。本节介绍如何用Microsoft Access分析Markit贷款数据，无论数据是从Access中用CSV导入的，还是用C#导入的。与前面的章节一样，本节关注8.1节的实现理念。因为Access更适用于查询数据，因此本节的实现要容易得多。

8.2.1 新发放贷款分析

如下查询结果和图8-1中的Excel表一样：

```
SELECT DateSerial(Year([close date]),Month([close date]),1) AS
MonthlyDate, Count(*) AS [Count], format(Sum(x.[Initial
Amount]),"#,##0") AS TotalSize, format(Sum(x.[Initial
Amount])/Count(*),"#,##0") AS AvgSize, format(Sum(term*[initial
amount])/Sum([initial amount]),"0.00") AS WtdTerm,
format(Sum([initial spread]*[initial amount])/Sum([initial
amount]),"0.00") AS WtdSprd, format(
Sum(iif([Industry]='Healthcare',[Initial Spread]*[initial
amount],0))/ Sum(iif([Industry]='Healthcare',[initial amount],0))
,"#,##0") AS WtdHealthcareSpread, format(
Sum(iif([Industry]='Retailing',[Initial Spread]*[initial
amount],0))/ Sum(iif([Industry]='Retailing',[initial amount],0))
```



```
, "#,##0") AS WtdRetailSpread, format( Sum(iif([Industry]='Oil & Gas', [Initial Spread]*[initial amount], 0))/ Sum(iif([Industry]='Oil & Gas', [initial amount], 0)) , "#,##0") AS WtdOilGasSpread FROM LoanXFacilityUpdates AS x WHERE (((x.Currency)='us dollar') And ((x.[LoanX Facility Category])='Institutional') And ((x.Status)='Active') And ((x.[Initial Spread])>0)) GROUP BY DateSerial(Year([close date]), Month([close date]), 1);
```

MonthlyDate列使用DATESERIAL函数，用数据库中存储的日期的年和月，将每个日期都转换成当月第一天。同一个函数还用在GROUP BY命令中，以按月聚合所有的值。

Count、TotalSize和Size列使用标准COUNT和SUM函数，以便按照MonthlyDate列聚合行。WtdTerm列是Term列和Initial Amount列的加权，以返回每月贷款期限的加权平均值。类似地，WtdSprd列返回每月贷款息差的加权平均值。

WtdHealthcareSpread列与WtdSprd列的计算一样，除了它用IIF函数以便只包含医疗行业的贷款。同理，WtdRetailSpread和WtdOilGasSpread分别只包含零售和油气行业的贷款。

8.2.2 再融资

如下查询结果和图8-2中的Excel表一样：

```
SELECT DateSerial(Year([Inactivation Date]), Month([Inactivation Date]), 1) AS MonthlyDate, Count(*) AS [Count], format(Sum(x.[Initial Amount]), "#,##0") AS TotalSize, format( (Sum( iif([Repl Maturity Date] >= [Maturity Date], (datediff("d", [Maturity Date], [Repl Maturity Date])) * [initial amount], 0)) / Sum(iif([Repl Maturity Date] >= [Maturity Date], [initial amount], 0))) / 365 , "0.00") AS YrsExt, format(Sum([initial spread]*[initial amount])/Sum([initial amount]), "0.00") AS OrigSprd, format(Sum([Repl initial spread]*[initial amount])/Sum([initial
```

```
amount]),"0.00") AS NewSprd, format(Sum(([Repl initial spread]-
[initial spread])*[initial amount])/ Sum([initial amount]),"0.00")
AS SprdDiff, format( Sum(iif([Industry]='Healthcare', ([Repl
initial spread]-[initial spread])*[initial amount],0))
/Sum(iif([Industry]='Healthcare', [initial amount],0)) ,"0.00") AS
WtdHealthcareSpreadChg, format( Sum(iif([Industry]='Retailing',
([Repl initial spread]- [initial spread])*[initial amount],0))
/Sum(iif([Industry]='Retailing',[initial amount],0)) ,"0.00") AS
WtdRetailSpreadChg FROM LoanXRecUpdates AS x WHERE
[inactivationreason]='Refinanced' and [Repl Initial Spread] > 0 and
[Initial Spread] > 0 GROUP BY DateSerial(Year([Inactivation
Date]),Month([Inactivation Date]),1);
```

与前面的查询一样，MonthlyDate列用的也是再融资日期（包含在Inactivation Date中），而且也是在GROUP BY命令中使用，以便按照月份聚合各行。Count和TotalSize列分别返回行数和每月再融资总额。

YrsExt列更复杂一点，它返回从某到期日之后延期的年份加权平均值。该命令用旧的到期日和新的到期日之间的加权差（包含在Repl Maturity Date中）除以365，得到年数。IIF函数用于限制正常到期或延期的贷款的分子和分母。DATEDIFF函数用于返回当前融资和再融资到期日之间的天数。

OrigSprd和NewSprd列返回初始融资和再融资息差的加权平均值。SprdDiff返回初始融资和再融资息差之间的加权平均值差。类似地，WtdHealthcareSpreadChg和WtdRetailSpreadChg与SprdDiff列使用相同的逻辑，除了分别只包括医疗和零售贷款。

8.2.3 价格历史

如下查询结果和图8-3中的Excel表一样：

```
SELECT [Mark Date], Count(*) AS [Count], Format(sum([initial
amount] * [evaluated price])/sum([initial amount]),"0.00") AS
WtdPx, Format( sum([initial amount] * [evaluated
```

```
price])/sum([initial amount]) / (SELECT top 1 sum(x2.[initial amount] * m2.[evaluated price])/sum(x2.[initial amount]) FROM LoanXMarks AS m2 INNER JOIN LoanXFacilityUpdates AS x2 ON x2.[loanx id]=m2.[loanx id] where m2.[mark date] < m.[mark date] group by m2.[mark date] order by m2.[mark date] desc )-1 , "0.00%") AS PctChg, format( Sum(iif([Industry]='Healthcare',[evaluated price]*[initial amount],0))/ Sum(iif([Industry]='Healthcare',[initial amount],0)) , "0.00") AS WtdHealthcarePx, format( Sum(iif([Industry]='Retailing',[evaluated price]*[initial amount],0)) / Sum(iif([Industry]='Retailing',[initial amount],0)) , "0.00") AS WtdRetailPx, format( Sum(iif([Industry]='Oil & Gas', [evaluated price]*[initial amount],0)) /Sum(iif([Industry]='Oil & Gas',[initial amount],0)) , "0.00") AS WtdOilGasPx FROM LoanXMarks AS m INNER JOIN LoanXFacilityUpdates AS x ON x.[loanx id]=m.[loanx id] GROUP BY [Mark Date] ORDER BY [mark date] DESC;
```

该结果表显示每天的数据，Mark Date被包含在GROUP BY命令中。Count和WtdPx列分别返回行数和规模加权的價格。

展示每日波动百分比的PctChg列是一个复杂的公式，因为不能像在Excel中那样简单引用下面的行。相反，该列用一个子查询返回前一天的波动百分比。尽管在本书中的大部分例子里，大部分百分比都是用现值和原值之差除以原值，而另一个百分比变化公式则是用现值除以原值再减去一。PctChg的第一部分使用与WtdPx列相同的逻辑，但是之后会除以前一天的WtdPx，再减去一。该子查询从LoanXMarks中选择日期小于当前行日期的第一行，以返回前一行的WtdPx。子查询会使查询变慢，所以应当删除，以提高查询效率。

WtdHealthcarePx、WtdRetailPx和WtdOilGasPx列使用与WtdPx列相同的逻辑，但是使用了IIF命令，分别只返回医疗、零售和油气行业的值。

该查询还内连接LoanXFacilityUpdates表，因为Initial Amount和Industry列并不是LoanXMarks表的一部分。

如下查询结果和图8-4中的Excel表一样：

```

SELECT [LoanX ID], [Deal Name], format( (select top 1
[evaluated price] from LoanXMarks m where m.[loanx id]=x.[loanx id]
order by [mark date] desc ) ,"0.00") AS CrrPx, format( (select top
1 max([evaluated price]) from LoanXMarks m where m.[loanx id]=x.
[loanx id] group by [evaluated price] order by [evaluated price]
desc ), "0.00") AS MaxPx, (select top 1 max([mark date]) from
LoanXMarks m where m.[loanx id]=x.[loanx id] group by [evaluated
price] order by [evaluated price] desc) AS MaxDt, format( (select
top 1 max([evaluated price]) from LoanXMarks m where m.[loanx
id]=x.[loanx id] group by [evaluated price] order by [evaluated
price] ) ,"0.00") AS MinPx, (select top 1 max([mark date]) from
LoanXMarks m where m.[loanx id]=x.[loanx id] group by [evaluated
price] order by [evaluated price] ) AS MinDt, (select count(*) from
LoanXMarks m where m.[loanx id]=x.[loanx id] ) AS [Count], format(
(select stdev([evaluated price]) from LoanXMarks m where m.[loanx
id]=x.[loanx id]) ,"0%") AS StdDev FROM LoanXFacilityUpdates AS x
WHERE x.[loanx id] in ('LX143540','LX148423','LX157886');

```

该查询对于一系列不同的LoanX ID从LoanXFacilityUpdates选择行，而且用了几个子查询（而非内连接），因为它需要用不同的方法聚合LoanXMarks数据。第一个子查询对于特定LoanX ID从LoanXMarks选择按照最新价格排序的第一个价格，这样就返回了现价（CrrPx）。

下一个子查询（MaxPX）返回某指定LoanX ID的最高价格，方法是将LoanXMarks价格从高到低排序，然后选择第一个价格。Access与Microsoft SQL Server的区别是它需要将MAX函数与GROUP BY结合使用，以确保该子查询只返回一行，尽管包含了TOP语句。

MaxDt子查询与MaxPX子查询一样，除了MaxDt返回价格最高那行的日期。复制MaxPX和MaxDt的逻辑以查询MinPX和MinDt列，除了子查询中的行按照价格从最低到最高排列。

Count和StdDev子查询用Access COUNT和STDEV聚合函数，分别返回标记数及

其标准差。该查询在WHERE子句中的IN语句中，以确定要包括在其中的LoanX ID表单。

8.2.4 更进一步

本节包括途径1未涉及的其他查询，以展示Access中更复杂的技术。

现在价格高于90美元、曾经价格低于80美元的贷款

运用大型数据集（例如Markit贷款数据）可以进行一个更有趣的分析，即发现异常情况。这类分析之一是看曾经价格低于80美元，但是由于业绩转好，现在价格高于90美元的贷款：

```
SELECT m.[LoanX ID], x.[Deal Name], m.[Evaluated Price],  
n.MinPx, (select max(m3.[mark date]) from LoanXMarks m3 where m3.  
[loanx id]=m.[loanx id] and m3.[evaluated price]=n.minPx ) AS  
DateOfMinPrice FROM (LoanXMarks AS m INNER JOIN  
LoanXFacilityUpdates AS x ON x.[loanx id]=m.[loanx id]) INNER JOIN  
( SELECT m2.[LoanX ID], Min(m2.[Evaluated Price]) AS MinPx FROM  
LoanXMarks AS m2 WHERE m2.[evaluated price] < 80 and m2.[mark date]  
> dateadd("d",-90,now()) GROUP BY m2.[LoanX ID]) AS n ON n.[LoanX  
ID]=m.[LoanX ID] WHERE m.[mark date] = (select max([mark date])  
from LoanXMarks) and m.[evaluated price] > 90;
```

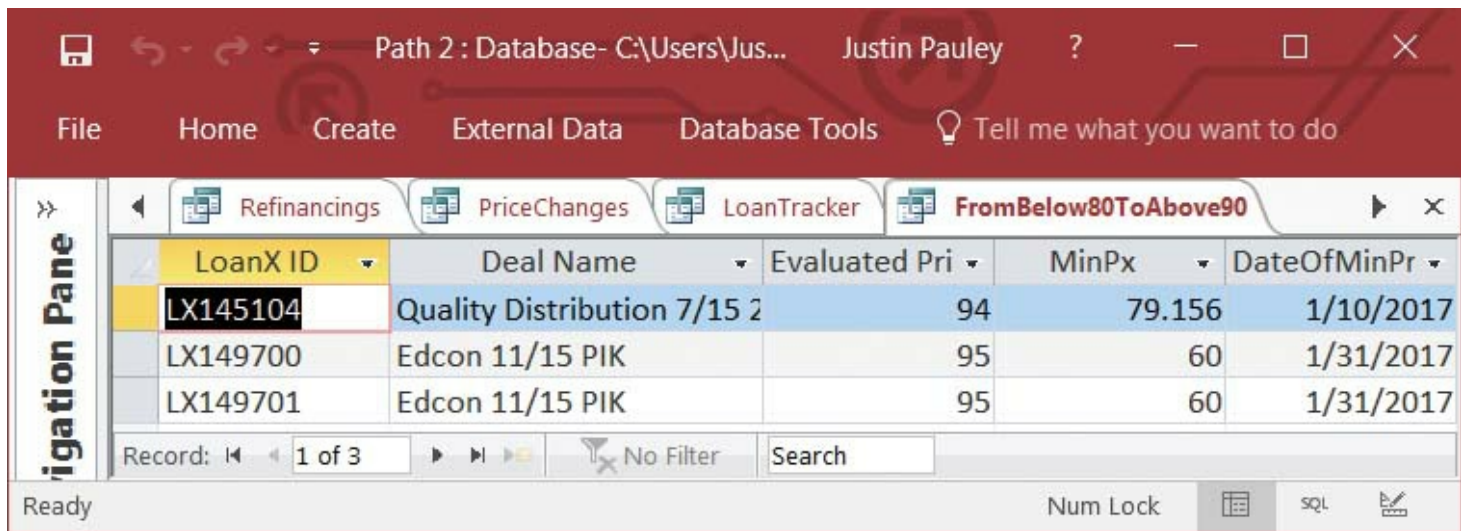
在前一个查询中，从LoanXMarks表选择了LoanX ID和Evaluated Price列，而从进行了内连接的LoanXFacilityUpdates表选择了Deal Name列。

MinPx列也来自于进行了内连接的子查询。该子查询从LoanXMarks中选择每个最低价格低于80美元且处于最后90天的LoanX ID所对应的LoanX ID和最低价格。Access与SQL Server的另一区别在于它只允许你内连接一张表，除非你用额外的括号进行分组。

DateOfMinPrice列使用一个子查询返回贷款最低价格低于80美元的最近日期。

WHERE子句用一个子查询过滤LoanXMarks表中日期并非最新的标记。此外，它

过滤了所有现在价格没有超过90美元的贷款。查询结果应该如图8-5所示。



LoanX ID	Deal Name	Evaluated Pri	MinPx	DateOfMinPr
LX145104	Quality Distribution 7/15 2	94	79.156	1/10/2017
LX149700	Edcon 11/15 PIK	95	60	1/31/2017
LX149701	Edcon 11/15 PIK	95	60	1/31/2017

图8-5：查询结果贷款集合

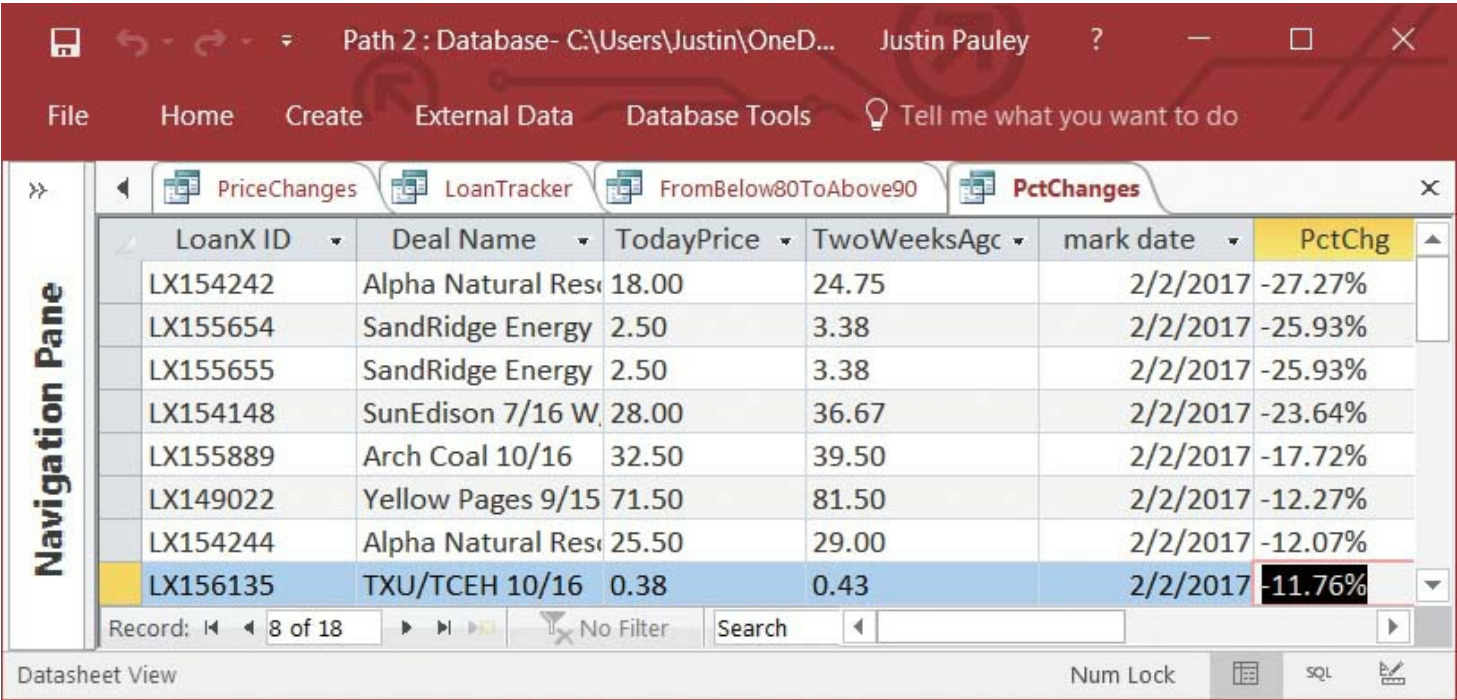
另一个有用的查询返回过去两周中价格变化超过10%的所有贷款。你可以很方便地修改如下查询，使其时间范围或价格限制有所不同：

```
SELECT m.[LoanX ID], [Deal Name], format(m.[Evaluated
Price],"0.00") AS TodayPrice, format(twoweeksago.[Evaluated
Price],"0.00") AS TwoWeeksAgo, twoweeksago.[mark date], format( (m.
[Evaluated Price]-twoweeksago.[Evaluated Price])/ twoweeksago.
[Evaluated Price],"0.00%") AS PctChg FROM (LoanXMarks AS m INNER
JOIN LoanXFacilityUpdates AS x ON x.[loanx id]=m.[loanx id]) INNER
JOIN ( SELECT m2.[LoanX ID], m2.[Mark Date], m2.[Evaluated Price]
FROM LoanXMarks AS M2 WHERE m2.[Mark Date] =(select max(m3.[mark
date]) from LoanXMarks m3 where m3.[mark date] <
dateadd("d",-14,now()) ) ) AS twoweeksago ON twoweeksago.[LoanX
ID]=m.[LoanX ID] WHERE m.[mark date] = (select max([mark date])
from LoanXMarks) and m.[evaluated price] > 0 and twoweeksago.
[evaluated price] >0 and abs((m.[Evaluated Price]-twoweeksago.
[Evaluated Price])/ twoweeksago.[Evaluated Price]) > 0.1 ORDER BY
(m.[Evaluated Price]-twoweeksago.[Evaluated Price])/ twoweeksago.
[Evaluated Price];
```

这个查询有点复杂，因为它内连接了一个子查询，该子查询中包括了另一个子查询。内连接语句中的外子查询从LoanXMarks中选择LoanXMarks中距离今天（NOW函数返回今天的日期）之前14天以前最近的日期所对应的LoanX ID、Mark Date和Evaluated Price。

表别名“m”代表每笔贷款的大部分当前价格信息，因为WHERE子句用一个子查询过滤在LoanXMarks表中标记日期不是最新日期的行。表别名“twoweeksago”代表同一笔贷款两星期前的数据。由此PctChg列返回从前一个日期（见Mark Date列）到当前价格的百分比变化。

WHERE子句也使用了ABS函数，只显示绝对百分比变化超过10%的贷款。查询结果应该如图8-6所示。



LoanX ID	Deal Name	TodayPrice	TwoWeeksAgo	mark date	PctChg
LX154242	Alpha Natural Res	18.00	24.75	2/2/2017	-27.27%
LX155654	SandRidge Energy	2.50	3.38	2/2/2017	-25.93%
LX155655	SandRidge Energy	2.50	3.38	2/2/2017	-25.93%
LX154148	SunEdison 7/16 W	28.00	36.67	2/2/2017	-23.64%
LX155889	Arch Coal 10/16	32.50	39.50	2/2/2017	-17.72%
LX149022	Yellow Pages 9/15	71.50	81.50	2/2/2017	-12.27%
LX154244	Alpha Natural Res	25.50	29.00	2/2/2017	-12.07%
LX156135	TXU/TCEH 10/16	0.38	0.43	2/2/2017	-11.76%

图8-6：价格变化超过10%的贷款

8.3 本章小结

视野狭窄是分析师们常陷入的最危险的陷阱之一。分析师们很容易偏安一隅，只关注自己的角落，而不关注其他行业或者市场其他领域的发展趋势。事实上，分析师需要对整个市场有趋势性的了解，因为所有事情都不是孤立的，多少都会受到其他市场的部分影响。本章介绍如何用贷款市场的Markit数据深入挖掘潜在信息，

判断大趋势。本章是金融数据分析部分的最后一章；下一部分介绍如何在不同报告中展示分析结果。

第9章 创建报告

如果不能用图形表示，我就不能理解。

——阿尔伯特·爱因斯坦

本部分利用前面大部分章节学到的知识，展示如何针对各个公司创建两页长的分析（“活页”）报告，内容包括金融历史、比较分析和相对价值等。设计报告更有点倾向于艺术家的工作；本章会介绍多种技巧和实例，但是人们的想法和对于报告样式的偏好多种多样。活页报告指标准普尔公司以前出版的关于上市公司的一页长报告，经纪人可以从书上撕下这一页来，不过今天网上已经有大量的一页长分析报告了。彭博在Excel Template Library (XLTP<GO>) 中提供了一些报告。

然而，本书的整体目标是教你创建自己的报告，能够把来自多种来源的数据和个人观点相结合，按照个人意愿创建自己的报告是一项优势技能。你自己对于公司应当如何分类、选择什么指数进行比较才恰当的看法、你对于不准确不完整数据的补充修订和你做的备注等，都将使在本章中设计的报告成为比其他投资人使用的报告更强大的工具。此外，现实中有数不清的金融数据，还有多种计算收益的方法。所谓萝卜白菜各有所爱，哪些字段和计算是适当的没有定论，是由个人确定的。

截至目前，前面章节讲到的所有金融数据和分析都是一个企业占一行。尽管一个企业占一行有利于进行比较，但是一页长的报告更便于消化信息。与前面章节一样，本章分为三个途径：Microsoft Excel、Access和C#。每个部分分别展示如何创建一个活页报告，并根据CompanyID自动生成报告。Excel有很丰富的功能、表格，便于使用，因此是创建报告的好工具。反观Access，它的使用难度就大多了，也不直观。尽管本章介绍了用Access创建报告的方法，还是推荐用Excel或C#的SQL Server Reporting Services (SSRS)。下面还教你根据个人偏好调整版式、颜色、图表、字段等；目标是使定制报告变得更容易。

9.1 途径1：Excel

本节介绍在Excel中创建一份两页长的公司报告的步骤，该报告只需要用一个简单的下拉菜单就能转换公司。第一步是创建一个工作表，命名为CompanyReport，设置页面版式，调整页面以方便阅读报告：

1. 创建一个新的工作表，命名为CompanyReport，转换视图到Page Break Preview。
2. 在Excel函数区Page Layout上，点击Orientation按钮，然后选择Landscape。
3. 选择单元格A1至M68。然后再次在Page Layout上，点击Print Area，然后选择Set Print Area。

白色底的单元格就是放置报告内容的空间。你可以用框外命名为“Page 1”的灰色单元格部分放置计算过程。

4. 在Page Layout键的Sheet Options组中，在Gridlines部分，取消靠近View的勾选。

5. 将字号改为8号。

11号字号大于一般人所需要的字号。

报告基本结构包含六个部分，每个部分之间用小标题或单元格粗外框分隔。下面的步骤搭起报告的“骨架”。

6. 合并第三行A列至M列的单元格，改变已合并单元格的背景颜色和字体颜色，并命名为Company Description。

7. 重复第6步，合并第6行的单元格，命名为Overview。

8. 把第13行（A列至H列）的单元格合并，更换背景颜色和字体颜色，命名为Relative Value。

9.把第13行Relative Value右侧的单元格（I列至M列）合并，命名为Comments。

下面的步骤增加下拉式菜单功能，以便快速转换公司。

10.在Formula键的Defined Names组中，点击Name Manager。

11.点击New，然后在文本框中输入Ticker。在“Refers to”列表框中选择：

=Company[CompanyID]

需要为按照下面的步骤创建的下拉菜单创建一个确定的名字。

12.把单元格A7命名为Ticker，选择单元格B7，然后在Data键上点击Data Validation。

13.在Data Validation对话框中，在Settings键的Allow列表框中选择List，然后在Source框中输入=Ticker。

点击OK，然后会出现一个列表框，你可以在其中选择Company工作表中的任何CompanyID，如图9-1所示。

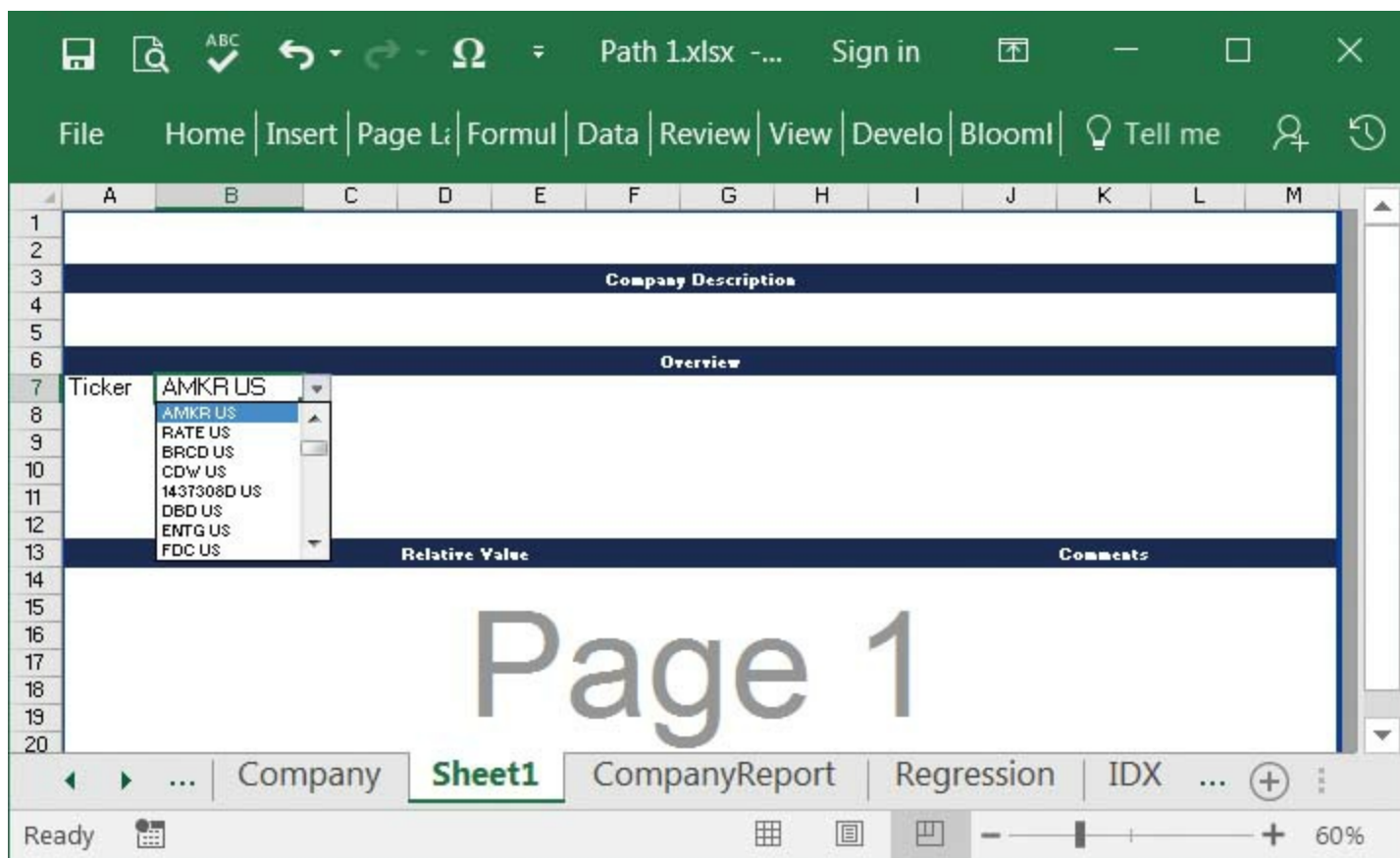


图9-1：带下拉菜单的报告骨架

下面的步骤可以基于选定的CompanyID，在报告标题上增加公司名称。标题还包括创建报告的日期以及更新财务信息的日期。这些日期十分重要，特别是在打印版上，因为它们都是报告中的重要背景信息。

14. 合并单元格A1至H2，然后输入如下公式：

`=INDEX ( Company[CompanyName] , MATCH ( $B$7 , Company[CompanyID] , 0 ) )`

用INDEX/MATCH函数从Company表中提取出CompanyID与单元格B7匹配的公司名称。在该报告上，该公式会被使用好几次，以从Company工作表中提取字段。

15. 把单元格I1命名为Financials As Of，然后在单元格J1中输入如下公式：

`=INDEX ( Company[Latest Financials] , MATCH ( $B$7 , Company[CompanyID] ) )`

该公式从Company工作表中提取最后一次金融记录的日期。

16.把I2单元格命名为Report Date，然后在单元格J2，输入如下公式：

=TODAY ()

报告日期总是今天，因为评论和证券价格每日更新。

下面的步骤将彭博对该公司的描述内容填入公司描述 (Company Description) 部分。因为描述内容可能比较长，必须用BDS函数进行提取。

17.把单元格N1命名为BBID，然后在单元格O1输入如下公式：

=INDEX (Company[BBID] , MATCH (\$B\$7 , Company[CompanyID]))

这些单元格不会显现在报告中，但从Company工作表中将BBID提取到该单元格，可以避免以后反复这样做。

18.合并单元格A4至M5，然后输入如下公式：

=BDS (\$O\$1 , "CIE_DES_BULK" , "aggregate=y")

该公式从彭博提取完整的公司描述。额外的实参使整个描述位于一个单元格中，而不能跨行。

19.调整已合并单元格的格式，然后在Home键的Alignment组中，点击Wrap Text。

下面的步骤用工作表中的信息自动填充报告的总结 (Overview) 部分。下面的步骤仅仅涉及几个字段，还可以根据个人偏好增加。此外，不同字段有不同的大小标签和值，故而可能需要进行合并以满足条件。

20.在列A的Ticker标签下，把第8~12行依次命名为：Current Price、Moody' sRating、S&P Rating、Private和EBIT。把标签加粗。

21.在列B的每个标签旁边的单元格中，用第14步中的INDEX/MATCH函数提取各列。例如，用如下公式提取Price：

=INDEX (Company[Price] , MATCH (\$B\$7 , Company[CompanyID]))

第20和21步的结果应该是列标签和来自Company工作表的对应数据。

22. 将所有单元格靠左，然后重复第20 ~ 21步，以创建多个标签和值相互对应的列。

23. 将Overview部分隔行的背景颜色调整为浅灰色，方便阅读每一行。

24. 或者在标签的值旁边写明单位，例如“Total Revenue (\$MM)”；或者调整单元格格式显示单位，例如“Short Interest/%Float”格式可以设置为0.00"%"; (0.00"%")，即在值后面增加百分号。

记住要写明单位，因为有时候单位并不明确。你还可以用传统的格式#，##0.00x；(#，##0.00x)，在一个值后面增加一个“x”，例如在Interest Coverage中增加。

上面三个部分都完成后结果如图9-2所示。

ADVANCED MICRO DEVICES										Financials As Of	12/31/2016
										Report Date	4/1/2017
Company Description											
Advanced Micro Devices, Inc. manufactures semiconductor products. The Company manufactures products that include microprocessors, embedded microprocessors, chipsets, graphics, video and multimedia products. Advanced Micro Devices, Inc. offers its products on a global basis.											
Overview											
Ticker	AMD US	Category	Electronic Component	Enterprise Value (\$MM)	13,868	Total Debt (\$MM)	1,435				
Current Price	14.55	Sub-Industry	Semiconductors	Total Revenue (\$MM)	4,272	Net Debt (\$MM)	171.0				
Moody's Rating	B3	TTM EBITDA (\$MM)	(239)	Dividend Gross Yield (%)	-	Net Debt/EBITDA	(0.72x)				
S&P Rating	B-	P/E Ratio (%)	-	Interest Coverage	(2.38x)	Country of Domicile	US				
Private	N	Market Cap (\$MM)	13,897	Free Cash Flow (\$MM)	13.00	Short Interest/% Float	13.72%				
EBIT	(372)	YoY EPS Adj. Growth	28.57%	YoY Revenue Growth	7.04%						
Relative Value						Comments					

图9-2：公司报告的上半部分

下面的步骤完成报告的相对价值 (Relative Value) 部分。该部分将公司的业绩与对等公司 (Company工作表中的Category) 以及整个市场 (Company工作表中的Index) 进行对比。根据列宽和内容多少，你可能需要合并一些单元格，甚至省略

某列。

25.把单元格A15至A21，命名为：52-Week High (%Chg)、52-Week Low (%Chg)、12-Month Total Return、YTD Price Change、3M Price Change、Total Debt/EBITDA、FCF/Total Debt。将这些单元格加粗。

26.把单元格C14命名为“Value”，将公式输入单元格C15至C21，以从Company表提取对应数据。例如，用如下公式在单元格C15提取52周以来最高价格（52-Week High）：

```
=INDEX ( Company[52Week High] , MATCH ( $B$7 ,  
Company[CompanyID] ) )
```

27.在D14中，输入如下公式：

```
= "Median" & INDEX ( Company[Category] , MATCH ( $B$7 ,  
Company[CompanyID] ) )
```

该公式显示“Median”，后面是公司的分类。

28.在标题下方的单元格（D15至D21）中，输入如下公式，以从Company表提取中位数。例如，要提取52周以来最高价格的中位数，用如下公式：

```
=INDEX ( Company[Median 52Week High] , MATCH ( $B$7 ,  
Company[CompanyID] ) )
```

每列应该在Company表中有一个对应的Median列。

29.在单元格G14，输入如下公式：

```
=INDEX ( IDX[Name] , MATCH ( INDEX ( Company[IndexID] ,  
MATCH ( $B$7 , Company[CompanyID] , 0 ) ) , IDX[IndexID] , 0 ) )
```

该公式从IDX表提取指数（Index）的全名，之前先用Company工作表单元格B7中的CompanyID提取匹配的IndexID。

30. 在Index标题下面的单元格 (G15至G19) 中，输入如下公式，以从Company工作表提取Index值。例如，要提取指数52周以来最高值 (Index' s52-Week High) ，用如下公式：

```
=INDEX(Company[Index 52 Week  
High],MATCH($B$7,Company[CompanyID]))
```

单元格G20和G21是空白的，因为对于指数来说，Total Debt/EBITDA和FCF/Total Debt是没有意义的。

31. 用第24步的方法，调整包含值的单元格的格式。

32. 将隔行的背景颜色调整为浅灰色，方便阅读每一行。

33. 第14行中的标题加粗，底下边线也加粗。用Wrap Text调整单元格标题格式，并增加第14行的行间距。

下面的步骤创建一个框，展示你对该公司的评价 (Comments) 。评价内容来自于Company工作表的Company Comments列。

34. 合并单元格I14至M21，然后将格式调整为Wrap Text。

这样就可以跨行和列填写评价内容。

35. 在新的单元格中填入如下公式：

```
=INDEX(Company[Company Comments],MATCH(B7,Company[CompanyID]))
```

Relative Value和Comments部分应该如图9-3所示。

Relative Value				Comments
	Value	Median Electronic Component	NASDAQ COMPOSITE INDEX	
52-Week High (% Chg)	(6.43%)	(5.14%)	(0.28%)	Reviewed in Q1 2017. Watching next quarter's numbers before investing
52-Week Low (% Chg)	459.62%	119.92%	29.24%	
12-Month Total Return	414.13%	93.17%	21.84%	
YTD Price Change	28.31%	28.31%	9.82%	
3M Price Change	28.31%	28.31%	8.89%	
Total Debt/EBITDA	5.24x	3.12x		
FCF/Total Debt	0.91%	10%		

图9-3：公司报告的中间部分

看上去不错的报告里不可能一张图表都没有。图表可以快速展示大量信息。下面的步骤在报告中添加了一张表，用于比较每天的公司股价、对应指数和负责研究该公司的分析师提供的共识价格目标（公平价值）。

36.把单元格N2命名为“Index BBID”，然后在单元格O2输入如下公式：

=INDEX(IDX[BBID],MATCH(INDEX(Company[IndexID],MATCH(\$B\$7,Company[CompanyID],0)),IDX[IndexID],0))

该公式从IDX工作表返回该公司对应指数的BBID。

37.把单元格N3命名为“Start Date”，然后在单元格O3输入如下公式：

=TODAY () - 365

该公式将价格历史的起始日期设为一年之前的今天。

38.把单元格N4命名为“End Date”，然后在单元格O4输入如下公式：

=TODAY ()

39.在单元格R1和W1，输入如下公式：

=01

该公式将该公司的Bloomberg ID复制到这些单元格，在后面的步骤中作为价格历史的行标题。

40. 把单元格R2命名为“Date”，然后在单元格S2输入如下公式：

=B7&" (Rt Axis) "

S2中的公式标明“(Rt Axis)”，表示它在图中坐标轴右侧。

41. 在单元格R3，输入如下公式：

=BDH (R1 , "PX_LAST" , 03 , 04 , "FX" ,
P1 , "PER=CD" , "FILL=PNA" , "CDR=US")

该公式提取从起始日期到结束日期里，每个美国日历日该公司股票的价格，用前面的价格填补空缺的日期。

42. 在单元格T4输入如下公式，然后向下复制到列R中的每一天：

= (S4-S3) /S3

该公式计算每日价格变化。

43. 在单元格U1中，输入如下公式：

=02

该公式将指数的Bloomberg ID作为价格历史上面的列标题。

44. 在单元格U2中，输入如下公式：

=INDEX (Company[IndexID] , MATCH (\$B\$7 , Company[CompanyID]))
&" (Lft Axis) "

该公式从Company工作表返回Index ID，并标明“Lft Axis”，表示它在图中坐标轴左侧。

45.在单元格U2，输入如下公式：

=BDH (U1 , "PX_LAST" , 03 , 04 , "FX" ,
P1 , "PER=CD" , "FILL=PNA" , "CDR=US" , "DTS=H")

该公式与第41步一样，除了它提取指数的历史，而且不显示日期。

46.在单元格V4中，输入如下公式，然后向下复制到每个包括价格的行：

= (U4-U3) /U3

与第42步一样，该公式计算每日指数价格变化。

47.把单元格W2命名为 “Target” 。

48.在单元格W3中，输入如下公式：

=BDH(W1,"BEST_TARGET_PRICE",03,04,"FX",P1,"PER=CD","FILL=PNA",
"CDR=US","DTS=H")

该公式与第41步相同，除了BEST_TARGET_PRICE返回该股票的共识价格目标，而且它也不显示日期。

49.添加一个线形图，大小为填满单元格A22至F43，在下面空出一行。

50.把S、U和W列的历史价格加入线形图。将价格目标和公司股票价格放在同一个坐标轴上，但是指数要放在另一个坐标轴上。根据个人偏好调整格式。

51.把单元格N5命名为Correlation，然后在单元格O5中，输入如下公式：

=CORREL (T : T , V : V)

该公式返回公司每日股票价格变动和指数之间的相关性。

52.把单元格N6命名为Beta，然后在单元格O6中，输入如下公式：

=SLOPE (T : T , V : V)

该公式返回公司每日股票价格变动和指数之间的Beta。

53.把单元格N7命名为R-Squared，然后在单元格O7中，输入如下公式：

=RSQ (T : T , V : V)

该公式返回公司每日股票价格变动和指数之间的R平方。

54.在图中第44行，从列A至F，增加标签，并将单元格连接到第51至53步得到的Correlation、Beta和R-Squared值。

下面的步骤添加一个饼状图，表示给出买入建议和卖出建议的数量。

55.把单元格N15至N17命名为：Buy、Sell和Hold。

56.在单元格O15中，输入如下公式：

=INDEX (Company[Buy Recommendations] , MATCH (\$B\$7 ,
Company[CompanyID]))

该公式从Company工作表提取Buy Recommendations。

57.在单元格O16中，输入如下公式：

=INDEX (Company[Sell Recommendations] , MATCH (\$B\$7 ,
Company[CompanyID]))

58.在单元格O17中，输入如下公式：

=INDEX (Company[Hold Recommendations] , MATCH (\$B\$7 ,
Company[CompanyID]))

59.添加一个饼状图，用N15至N17中的标签和O15至O17中的值。根据个人喜好进行排版。

报告第一页如图9-4所示。

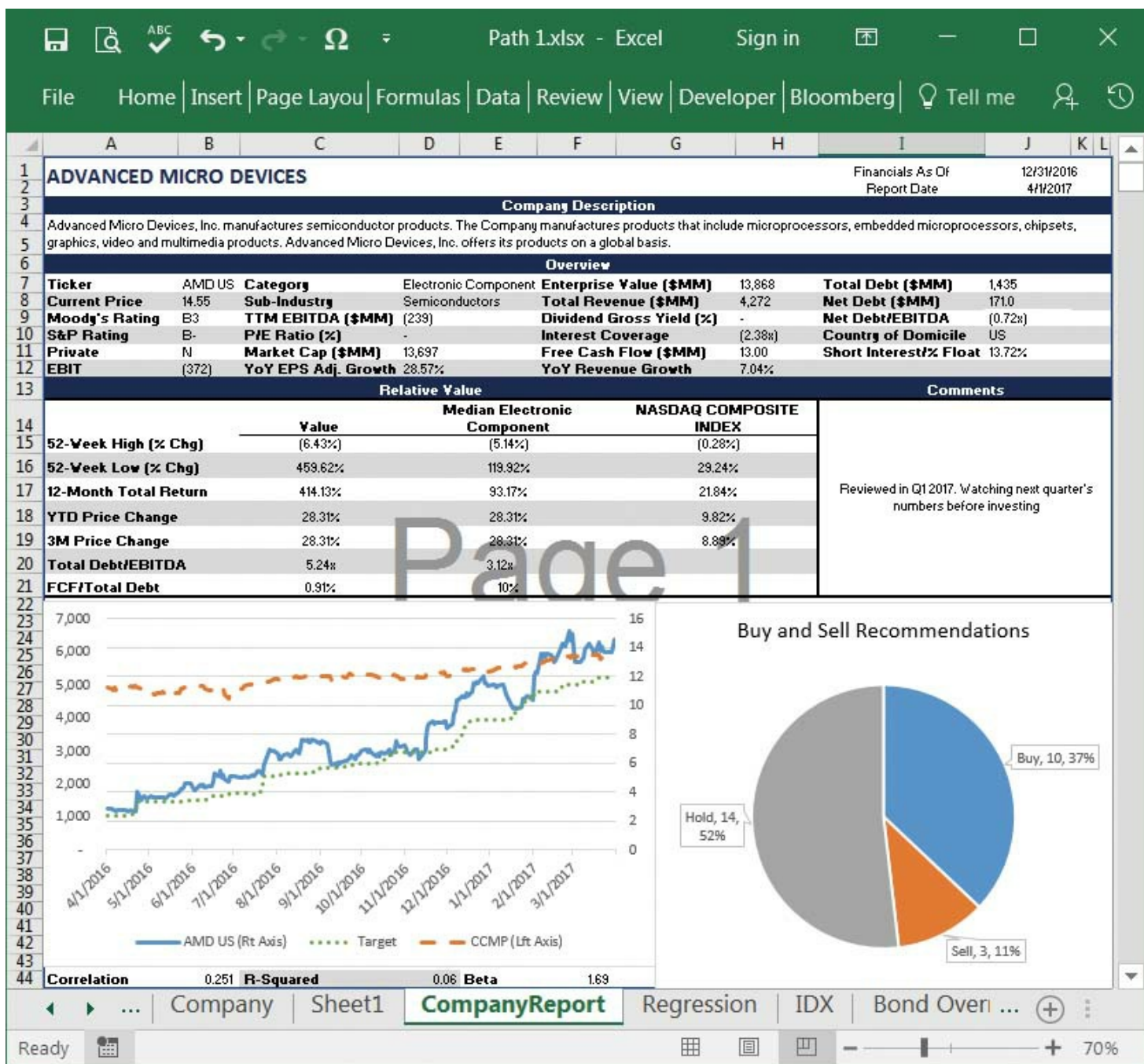


图9-4：Excel中的公司报告第一页

报告第二页将包含该公司的历史财务情况。报告需要通过下拉框自由转换年度 (Yearly)、半年度 (Semi-annually) 和季度 (Quarterly) 财务情况。下面的步骤增加下拉框，并用彭博增加与报告有关的日期。

60. 选择单元格N46 (在可打印区域外)。在函数区的Data键上，点击Data Validation按钮。

61. 在Allow列表框中选择表单，然后在Source文本框中输入Yearly、Semi-

Annual、Quarterly，然后点击Okay。

这就在单元格N46中设置了一个下拉框。现在选择“Yearly”。

62.在单元格C45中，输入如下公式：

= "-0F"&LEFT(\$N\$46,1)

结果显示“-0FY”（在单元格N46选择“Yearly”时），我们将它作为BDP函数的输入，以获取最近财年的数据。

63.在单元格C45右边的单元格中，输入相同的公式，但是把0改为1至3，这样第45行的内容就是-0FY、-1FY、-2FY、-3FY。

64.选择第45行包含第62~63步结果的单元格，然后把字体颜色调成白色，这样就看不见了。

这样做是因为尽管我们在公式中引用这些单元格，但是不需要在报告中展示它们。

65.在单元格C46中，输入如下公式：

=BDP(\$O\$1,"BEST_PERIOD_END_DATE","BEST_FPERIOD_OVERRIDE",
C45)

该公式返回在单元格C45中包含的财务时间段的最后一天彭博提供的日期。

66.将C45（第65步）中的公式复制到右边四个单元格。

根据报告版式，你可能想合并一些单元格，这样日期就能和页面契合了。

下面的步骤在报告上添加标签和相应的彭博字段。注意这些仅仅是你可以选择的数千种字段中的几种例子（见第3章）。

67.从单元格A47到A54，添加如下标签：“Market Cap战Enterprise Value战Revenue, Adj战Gross Profit, Adj战EBITDA, Adj战Net Income, Adj战EPS, Adj战Free Cashflow”。

68.从单元格A47到A54，添加带有如下包含彭博字段的标签：
HISTORICAL_MARKET_CAP、ENTERPRISE_VALUE、SALES_REV_TURN、
GROSS_PROFIT、EBITDA、EARN_FOR_COMMON、IS_DIL_EPS_CONT_OPS和
CF_FREE_CASH_FLOW。

想更多了解这些字段，请见彭博FLDS屏幕。

69.在单元格C47中，输入如下公式：

```
=BDP($O$1,$N47,"EQY_FUND_RELATIVE_PERIOD",C$45,"Fill=-  
","EQY_FUND_CRNCY", $P$1)
```

该公式从列N中提取字段（HISTORICAL_MARKET_CAP），时间段为第45行标明的时间段（-0FY），货币为在单元格P1中标明的货币。

70.将第69步创建的公式复制到每个带日期标题的列，一直到第54行。

71.添加一些有用的时间段相关数据的图表。

报告第二页如图9-5所示。将下拉式单元格N46调整为Quarterly，就能立即更新历史财务数据和图表。

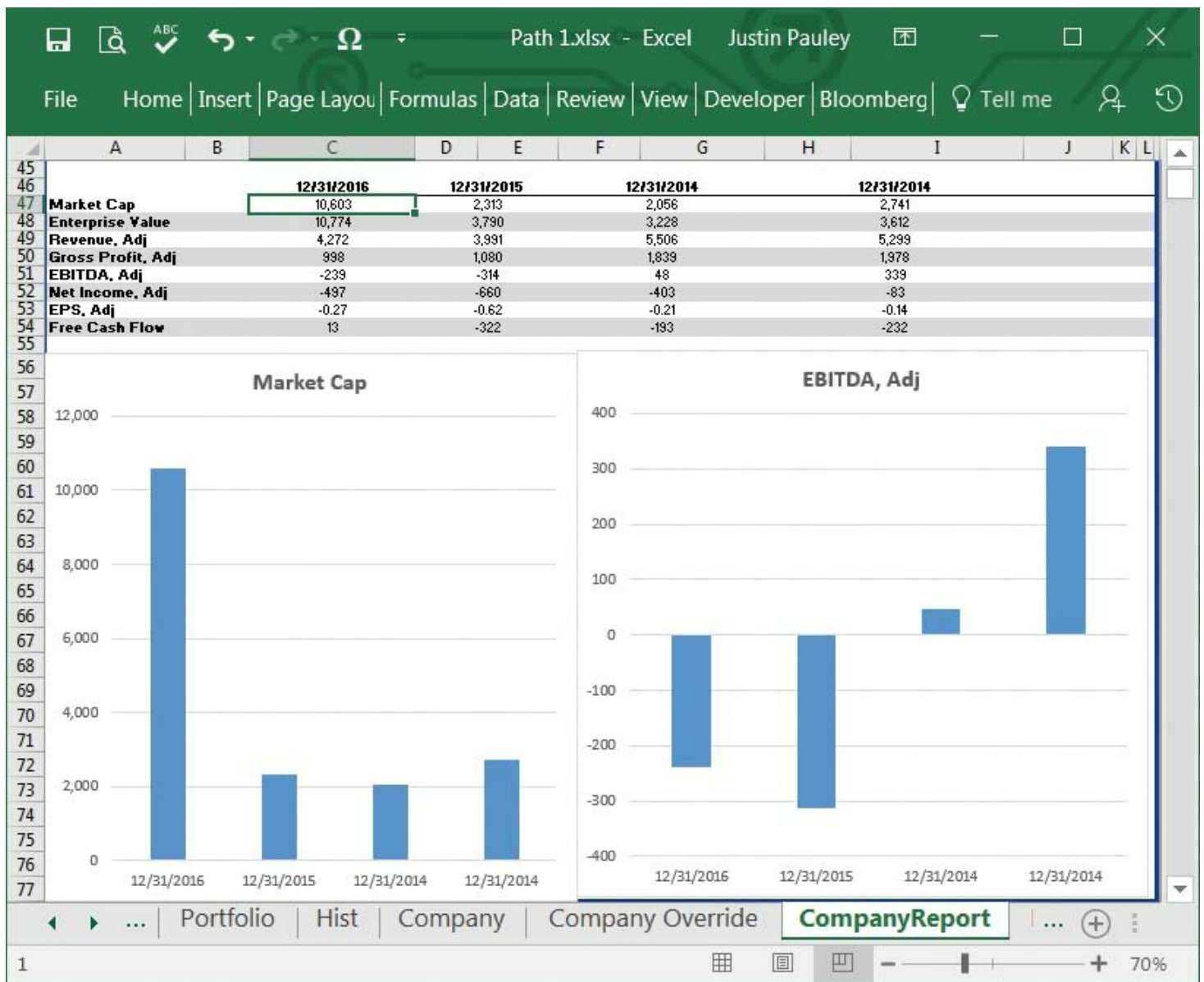


图9-5：Excel中的公司报告第二页

花点时间消化一下报告数据。我还建议，如果某个数字超出了上限或者下限，那么就对该单元格加亮，设置成醒目的格式。要转换公司，就用单元格B7中的下拉菜单，看看整个电子表格更新。如果Company工作表中的Index或者Comments部分有变化，报告会自动更新。

9.2 途径2：Microsoft Access

Access在查询大量数据时是一个好用的工具，但是作为报告工具却有些缺点。主要问题是它的界面没有Excel或其他报告工具那么直观。此外，由于Access不能直接接入彭博，一些数据（例如历史价格和企业描述）就需要在创建报告之前继续

留存在Excel工作簿中。由此，即使你用了Access进行数据分析，也推荐将Excel作为报告平台。请注意，因为不能从彭博中获取全部金融数据，或者金融数据多到难以在Excel中处理，因此了解如何在Microsoft Access中创建报告也是有用的。本节展示如何用与Excel工作表相连的Company表，创建一个一页长的报告。

第一步是添加一个查询，收集某个公司CompanyID相关的数据：

1. 创建一个新的查询，命名为Company Report Query，代码如下：

```
SELECT * FROM Company WHERE CompanyID=[CID];
```

[CID]列不是Company表中的列，因此你在运行查询或报告的时候，Access会提示一个CompanyID。查询结果是Company表中对应某个CompanyID的每一列。

下面的步骤创建报告，并在报告上增加字段。

2. 在函数区Create键上，点击Report Design创建一个新的空白报告。

3. 在Page Setup键上，点击Landscape。

4. 在Design键上，选择Property Sheet。在Data键上，点击Record Source按钮，并选择在第1步创建的查询Company Report Query。

该步将查询与报告中的数据连接在一起。

5. 然后在Design键上，选择增加Existing Fields。

显示一个Field List框，其中内容是Company表中列的表单。

6. 把Field List中的字段拖到报告中，版式按照个人偏好。

把一个字段拖到报告中时，会产生两个框：一个包含可以作为标签修改的列名，另一个显示审阅报告时的字段实际值（一般在右侧，字体加黑）。你可以删除不必要的标签。拖到页标题（Page Header）部分的字段会重复出现在每一页上。

下面的步骤介绍如何增加额外的标签和修改单元格格式。在Design键上有两种有用的控制器可以添加到报告中。一种是文本框，其中可以包含一个公式并指向另

一个字段。另一种是标签，你可以定义格式以帮助整理美化报告。要在报告中增加两种控制器之一，方式是在Design键上从Controls选择它们，然后拖到你输入控制器的区域。

7.在报告上添加一个文本框。点击文本框内部，并输入如下公式：

```
= "Report Date"&Date ( )
```

审阅报告时，该文本框会显示“Report Date”（报告日期），后面是今天的日期。

8.在报告上添加一些标签（例如Overview、Relative Value或者在9.1节创建的其他标签）。用Format键上的选项改变背景颜色、字体颜色，并把标签加粗。

9.选择报告中的所有字段、标签和文本框，然后用Format键上的选项，设置一贯使用的字号。推荐将控制器边框设为透明的（在Control Formatting组中点击Shape Outline，然后选择Transparent）。

10.Arrange键上的Align选项可以帮助排列报告上的不同字段。

11.要调整数字格式，右击一个字段，然后在快捷菜单上指向Properties，并选择一种格式。你该可以打字输入一种常见格式，例如0.00\%以显示“2.35%”这样的数字。

将某一公司的价格历史与指数的关系进行对比并画图，如图9-4所示，在Access中更困难一些。主要问题是将每个公司和指数的数据输入数据库，然后转换成Access绘图工具可以使用的格式。如果不写代码的话，其实并没有什么特别理想的办法。下面的步骤介绍如何绘制这样的图，但是并不太理想。

12.在Excel中，创建一个新的工作表，命名为PriceHistory。

13.把单元格A1命名为Date。

14.在第一行中，输入Company和IDX工作表中的每个公司和指数的BBID数据。例如，单元格B1为“ACIW US Equity”，单元格C1为“CCMP Index”。

15.在单元格A2中，输入如下公式：

```
=BDH(B$1,"PX_LAST",TODAY()-365,TODAY(),"FX=USD","PER=CD","FILL=F  
"CDR=US","DTS=0")
```

该公式在列A填入去年的日历日期。

16.在单元格B2中，输入如下公式，然后在第二行复制到各列：

```
=BDH(B$1,"PX_LAST",TODAY()-365,TODAY(),"FX=USD","PER=CD","FILL=F  
"CDR=US","DTS=H")
```

该公式对应第一行填入BBID价格历史。

17.在Access函数区External Data键上，点击Excel按钮，连接第12步创建的工作表。

18.创建一个新的查询，命名为“PriceHistQuery”，代码如下：

```
SELECT Date, switch( c.BBID='AMD US Equity',d.[AMD US Equity],  
c.BBID='ACIW US Equity',d.[ACIW US Equity], c.BBID='AMKR US  
Equity',d.[AMKR US Equity], ) AS Company, switch(  
c.IndexID='CCMP',d.[CCMP Index], c.IndexID='S5INFT',d.[S5INFT  
Index] ) AS [Index] FROM PriceHistory AS d, Company AS c WHERE  
c.CompanyID=[CID];
```

switch语句从PriceHistory表选择合适的列，对于给定CompanyID返回Company表中的BBID和IndexID。不幸的是，这个方法要求在两个switch语句中分别列出每个公司的Company BBID和Index ID。

19.在Access报告Design键上，点击Controls然后选择Chart。按照个人需求将图放在报告中某个位置上。放好后，打开Chart Wizard。

20.在Wizard第一页上，选择PriceHistQuery。在第二页上，将Date、Company和Index列复制到“Fields for Chart”中。在第三页上，选择线形图。在第四页上，将Index字段从窗口右侧拖到SumOfCompany下，然后点击Finish。

21. 双击新图，按照个人偏好调整格式，包括将第二个系列放到第二个坐标轴上。

注意当你审阅报告时，它会两次要求你填入CID（CompanyID）（这是我不喜欢 Access Reports的另一个原因）。如果用Access Form，就只需要填入一次。Access Reports结果应该如图9-6所示。

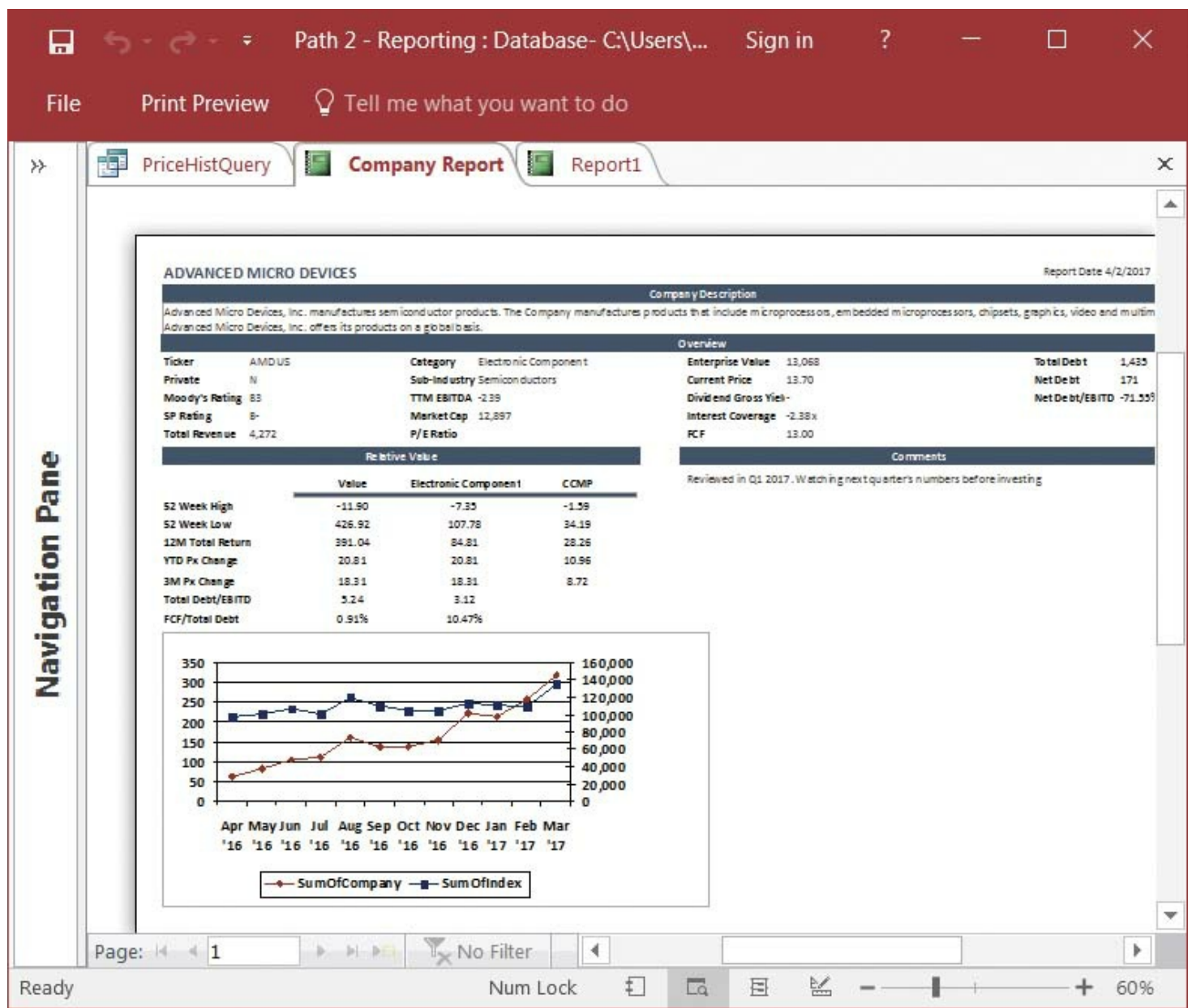


图9-6：Access中的公司报告

9.3 途径3：C#和SSRS

本节展示如何用C#和SQL Server Reporting Services (SSRS) 自动创建报

告。SSRS是一个先进的动态报告生成工具，分为服务器端和客户端。如名字中所暗示的，客户端意思是指报告（RDL文件）存储在一个远端服务器上，可以用网页浏览器、电子邮件或者从应用等访问。客户端版本并不需要SQL服务器。尽管如此，因为两种方式都用相同的机器，所以报告文件可以从RDLC转成RDL格式，反之亦然。在很多分析师无法使用SQL Server的前提下，本章仅关注客户端报告。

开始之前，你需要安装SQL Server Data Tools (SSDT)。首先在Windows控制面板上，点击Change on Microsoft Visual Studio。当Visual Studio窗口打开时，选择Modify，然后在Windows和Web Development部分，选择Microsoft SQL Server Data Tools。

SSRS是我最喜欢的报告工具，因为它功能强大，而且比Access中的report builder方便使用。本节带你学习如何创建一个C#Windows Form应用，它将结合来自Access数据库和彭博API中的数据，创建一个可以很方便地以PDF或Excel形式导出的报告。不过，由于互联网上已经有大量介绍如何建立SSRS报告的资源，故而本章将关注如何创建一个非常简单的报告，并与C#应用捆绑，而不深入介绍report builder工具。

最初几步创建新的项目，并添加一个数据集，在其中存储报告中用到的数据。

- 1.首先创建一个新的Windows Form应用，并增加对彭博API的引用。
- 2.在新项目上添加一个数据集，命名为ADS.xsd。

该数据集将Access数据库和彭博API中的数据传给报告。

3.在ADS数据集上添加一个新的TableAdapter，命名为Company，把它连接到Access数据库上。用如下代码查询：

```
SELECT c.CompanyID, c.BBID, c.CompanyName, c.IsPrivate,
c.Sector, c.Industry, c.SubIndustry, c.SPRating, c.MoodyRating,
c.MarketCap, c.TotalDebt, c.NetDebt, c.EV, c.EBITDA,
c.TotalRevenue, c.TotalDebtToEBITDA, c.InterestCoverage, c.FCF,
c.FCFToTotalDebt, c.Price, c.YrHi, c.YrLow, c.CDS5YrTicker,
c.NetDebtToEBITDA, c.CDSSpread5Yr, c.Category, c.CompanyComments,
```



```
c.PERatio, c.DVDYield, c.TotalReturn12M, c.PxChgYTD, c.PxChg3M,
c.IndexID, c.RecBuy, c.RecSell, c.RecHold, i.IndexName, i.Price AS
IdxPx, i.YrHi AS IdxHi, i.YrLow AS IdxLow, i.PxChgYTD AS
IdxPxChgYTD, i.PxChg3M AS IdxPxChg3m, i.TotalReturn12M AS
IdxTotalReturn12M, s.YrHi as MedYrHi, s.YrLow AS MedLow, s.PxChgYTD
AS MedPxChgYTD, s.PxChg3M AS MedPxChg3m, s.TotalReturn12M AS
MedTotalReturn12M, s.TotalDebtToEBITDA as MedTotalDebtToEBITDA,
s.FCFTToTotalDebt as MedFCFTToTotalDebt FROM ((Company c LEFT OUTER
JOIN [Index] i ON i.IndexID = c.IndexID) LEFT OUTER JOIN
MedianCompanyStats s ON s.Category = c.Category) WHERE (c.CompanyID
= ?)
```

该查询将Company表中的列与MedianStats和Index表相结合，以便一起提取报告所需的全部信息。用WHERE子句中的？过滤CompanyID。

4.右击Company数据表，并添加一列，命名为“FullDescription”。

该字段用彭博API填入。

5.用如下查询在ADS数据集中增加另一个TableAdapter，命名为CompanyList：

```
SELECT CompanyID, CompanyName FROM Company
```

该查询返回一个公司名称和ID表单，可以在多选框中使用，这样用户就可以用方便的下拉菜单选择要报告的公司。

6.在ADS数据集上添加一个新的数据表，命名为PriceHistory，表中包含如下列：Ticker、Date、Value、IndexName和IndexValue。将Date列的数据类型改为DateTime，并将Value和IndexValue列的数据类型改为Double。

该数据表用彭博API填入，其中存储该公司的价格历史 and 对应指数的价格。

下面的步骤在项目中增加报告，并将它连接到ADS数据集。

7.在项目上添加一个报告项，命名为CompanyReport。

8.打开新的报告，在报告数据窗口（Ctrl+Alt+D组合键）的数据集部分，右击并选择Add Dataset。在Data source下选择ADS，在Available datasets下选择Company，然后点击OK。

这样，ADS数据集中的Company数据表就被绑定为报告中的“Dataset1”。

9.重复第8步，除了在Available datasets下选择PriceHistory。

这样，ADS数据集中的PriceHistory就被绑定为报告中的“Dataset2”。

下面的步骤在报告上增加一些标签、一张表和一个图。

10.在工具栏中，选择Text Box，并拖到报告中。右击该文本框，并选择Expression。在Expression框中输入如下代码：

```
=First ( Fields ! CompanyName.Value , "DataSet1" )
```

显示Company表中的公司名称。该字段用First，并用Dataset1结束的原因是，该文本框不限于一个特定数据集，而且尽管我们知道该数据集仅仅包含一行（也就是用First（）或Last（）都一样），不过使用一个聚合函数能够让我们引用该数据集。

11.在工具栏中，选择一张表，把它拖到报告上。在表的Properties窗口上，将DatasetName设为Dataset1。

表中第一行是表头，中间是具体内容部分（在数据集的每一行中重复），最后一行是脚注（在具体内容部分之后，适合输入具体内容部分的合计数据）。

12.右击具体内容行（中间）和脚注行（最下面），并选择Delete Rows。

因为在Company数据集中只有一行，所以就不需要具体内容部分了。

13.当你的鼠标滑动过单元格时，在剩下的行中某个单元格的右侧图标上点击一下。然后，选择要在该单元格中出现的列。

14. 反复重复第13步，就能在报告中添加数据集中的多个列。你可以右击任意单元格，在报告上增加额外的列或行。

15. 从工具栏中为报告添加一个图，然后在打开的Chart窗口中选择线形图。

16. 点击图，在打开的对话框Category Groups键中，Dataset2列表框里，选择Date。

17. 在Values部分，点击绿色的加号（+），然后选择Value。再次点击加号，然后选择IndexValue。

18. 选择IndexValue旁边向下的箭头，然后选择Series Properties。在Axes和Chart部分，对于Vertical axis选择Secondary。

你可以继续增加表和图，还可以继续调整格式，直到报告如图9-7所示。

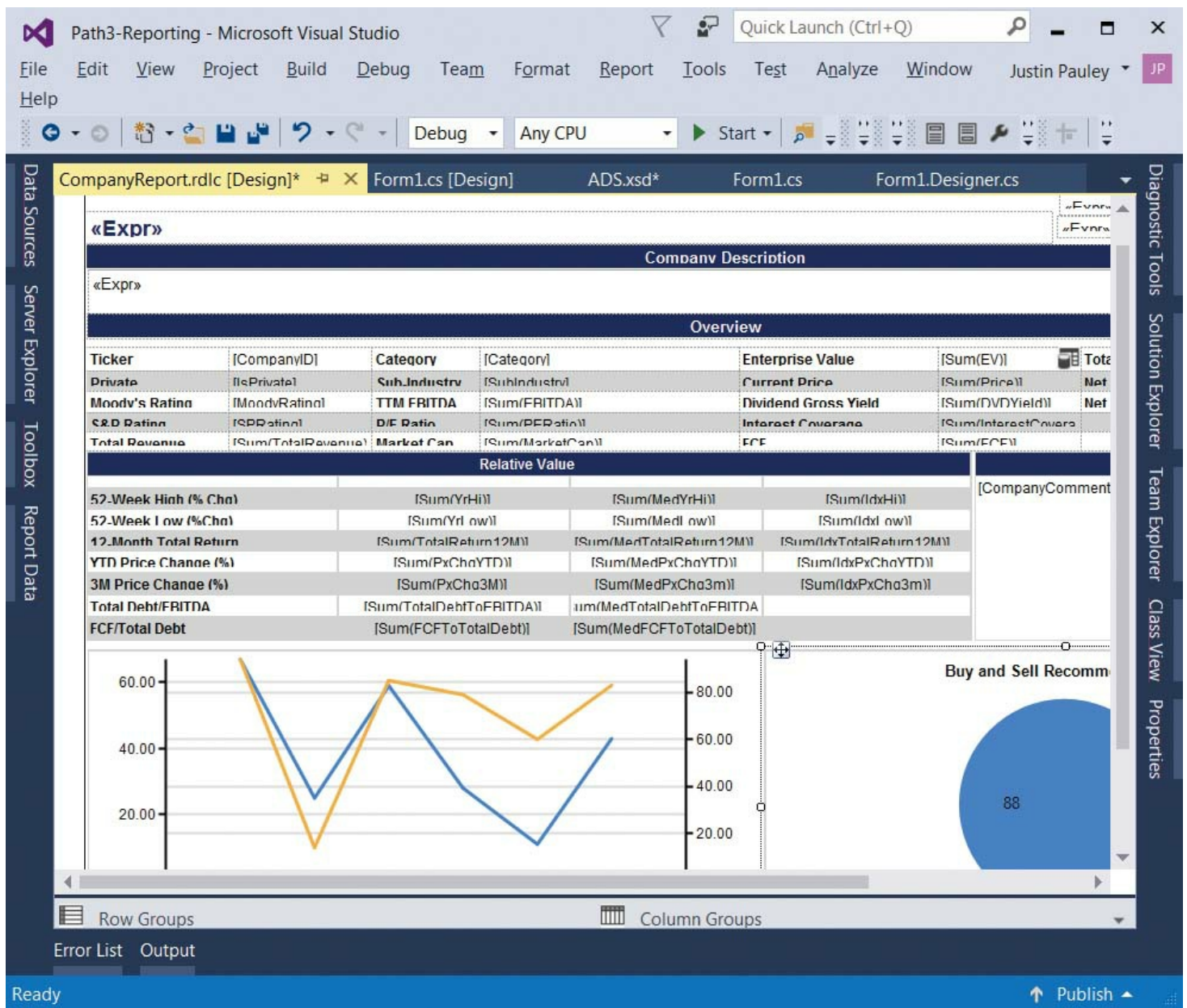


图9-7：设计模式下的SSRS公司报告

下面的步骤在Windows form Form1中增加控制器，可以用于选择公司、查看报告，以及按照PDF或者Excel形式导出。

19.在Form1的设计中，首先增加两个面板（panel），一个在上面，另一个在下面。

20.在上面的面板中，添加一个复选框（cmbCompanyList）、一个Load按钮（btnLoad）、一个PDF按钮（btnPDF）和一个Excel按钮（btnExcel）。

21.在下面的面板中，添加一个报告查看控制器（rptView）。将其dock栏设为

fill，然后在Report View任务列表框中选择CompanyReport。

该步骤会在你的表格中添加一些对象：一个ADS数据集实例、一个Company TableAdapter实例以及Company和PriceHistory表的BindingSource。

最后，进入本项目的编码部分。在Form1.cs文件中，对彭博添加如下using指令：

```
using Event = BloombergIp.Blpapi.Event; using Element =
BloombergIp.Blpapi.Element; using Message =
BloombergIp.Blpapi.Message; using Name = BloombergIp.Blpapi.Name;
using Request = BloombergIp.Blpapi.Request; using Service =
BloombergIp.Blpapi.Service; using Session =
BloombergIp.Blpapi.Session; using DataType =
BloombergIp.Blpapi.Schema.Datatype; using SessionOptions =
BloombergIp.Blpapi.SessionOptions; using InvalidRequestException =
BloombergIp.Blpapi.InvalidRequestException;
```

然后，在Form1类，增加彭博的Name实例，就像此前项目的代码：

```
private static readonly Name SECURITY_DATA = new
Name("securityData"); private static readonly Name SECURITY = new
Name("security"); private static readonly Name FIELD_DATA = new
Name("fieldData"); private static readonly Name RESPONSE_ERROR =
new Name("responseError"); private static readonly Name
SECURITY_ERROR = new Name("securityError"); private static readonly
Name FIELD_EXCEPTIONS = new Name("fieldExceptions"); private static
readonly Name FIELD_ID = new Name("fieldId"); private static
readonly Name ERROR_INFO = new Name("errorInfo"); private static
readonly Name CATEGORY = new Name("category"); private static
readonly Name MESSAGE = new Name("message"); private static
readonly Name DATE = new Name("date");
```

然后，定义Form1_Load事件，它填入ADS数据集中的CompanyList数据表（第

21步创建的实例)，并将其与复选框捆绑。程序开始时，将Access数据库中的公司名称表单填入复选框（cmbCompanyList）：

```
private void Form1_Load(object sender, EventArgs e) { //
Populate the Company List DataTable from Access database using
(ADSTableAdapters.CompanyListTableAdapter ta = new
ADSTableAdapters.CompanyListTableAdapter()) {
ta.Fill(this.ADS.CompanyList); ta.Connection.Close(); } // Bind the
CompanyList to the ComboBox. this.cmbCompanyList.DataSource =
this.ADS.CompanyList; this.cmbCompanyList.ValueMember =
"CompanyID"; this.cmbCompanyList.DisplayMember = "CompanyName"; }
```

然后，定义btnLoad_Click事件，它在点击Load按钮时启动。该方法用Company query基于在cmbCompanyList中选择的CompanyID加载某个Company。然后，它从彭博API获取公司的完整描述、股票历史价格和指数价格数据，添加到ADS数据集，并将该报告加载到Report Viewer。该报告在设计模式下与rptView对象捆绑。

```
private void btnLoad_Click(object sender, EventArgs e) { //
Populate the Company table in the Dataset using the // CompanyID
from the ComboBox this.CompanyTableAdapter.Fill(this.ADS.Company,
this.cmbCompanyList.SelectedValue.ToString()); ADS.CompanyRow
company = this.ADS.Company[0]; //Get the full description from
Bloomberg company.FullDescription =
GetFullDescription(company.BBID); this.ADS.PriceHistory.Clear();
//Get 1yr price history from Bloomberg Dictionary<DateTime, double>
history = GetHistory(company.BBID, "PX_LAST",
DateTime.Now.AddYears(-1)); //Insert price history into
PriceHistory Datatable foreach (DateTime dt in history.Keys) {
this.ADS.PriceHistory.AddPriceHistoryRow(company.CompanyID, dt,
history[dt], null, 0); } if (company.IsIndexIDNull() == false) {
//If the IndexID on the Company row is populated, get its price
history //as well. history = GetHistory(company.IndexID + "
```

```
Equity", "PX_LAST", DateTime.Now.AddYears(-1)); foreach (DateTime
dt in history.Keys) { ADS.PriceHistoryRow row =
this.ADS.PriceHistory.FindByDate(dt); if (row != null) {
row.IndexName = company.IndexID; row.IndexValue = history[dt]; } }
} //Refresh the report. this.rptView.RefreshReport(); }
```

下面两个方法发送彭博请求以获取公司的完整描述 (CIE_DES_BULK) , 然后处理反馈。代码应该看起来和本书中前面的程序一样：

```
private string GetFullDescription(string security) { string
description = ""; SessionOptions sessionOptions = new
SessionOptions(); Session session = new Session(); bool
sessionStarted = session.Start(); if (!sessionStarted) {
System.Console.Error.WriteLine("Failed to start session."); return
null; } if (!session.OpenService("//blp/refdata")) {
System.Console.Error.WriteLine("Failed to open //blp/refdata");
return null; } Service refDataService =
session.GetService("//blp/refdata"); Request request =
refDataService.CreateRequest("ReferenceDataRequest"); Element
securities = request.GetElement("securities");
securities.AppendValue(security); Element fields =
request.GetElement("fields"); //CIE_DES_BULK is the full
description of a company. fields.AppendValue("CIE_DES_BULK"); try {
session.SendRequest(request, null); } catch
(InvalidRequestException e) {
System.Console.WriteLine(e.ToString()); } bool done = false; while
(!done) { Event eventObj = session.NextEvent(); if (eventObj.Type
== Event.EventType.PARTIAL_RESPONSE) { description +=
ProcessResponse(eventObj); } else if (eventObj.Type ==
Event.EventType.RESPONSE) { description +=
ProcessResponse(eventObj); done = true; } else { foreach (Message
msg in eventObj) { System.Console.WriteLine(msg.AsElement); if
(eventObj.Type == Event.EventType.SESSION_STATUS) { if
```



```

(msg.MessageType.Equals("SessionTerminated")) { done = true; } } }
} } session.Stop(); return description; } private string
ProcessResponse(Event eventObj) { string description = ""; foreach
(Message msg in eventObj) { if (msg.HasElement(RESPONSE_ERROR)) {
Element error = msg.GetElement(RESPONSE_ERROR);
Console.WriteLine("Request failed: " +
error.GetElementAsString(CATEGORY) + " (" +
error.GetElementAsString(MESSAGE) + ")"); continue; } Element
securities = msg.GetElement(SEcurity_DATA); for (int i = 0; i <
securities.NumValues; ++i) { Element security =
securities.GetValueAsElement(i); if
(security.HasElement("securityError")) { Element error =
security.GetElement(SEcurity_ERROR); Console.WriteLine("Security
Error: " + error.GetElementAsString(CATEGORY) + " (" +
error.GetElementAsString(MESSAGE) + ")"); continue; } Element
fieldExceptions = security.GetElement(FIELD_EXCEPTIONS); if
(fieldExceptions.NumValues > 0) { for (int k = 0; k <
fieldExceptions.NumValues; ++k) { Element fieldException =
fieldExceptions.GetValueAsElement(k); Element error =
fieldException.GetElement(ERROR_INFO); Console.WriteLine("Field
Exception: " + fieldException.GetElementAsString(FIELD_ID) + " " +
error.GetElementAsString(CATEGORY) + " (" +
error.GetElementAsString(MESSAGE) + ")"); } } Element fieldElements
= security.GetElement(FIELD_DATA); if (fieldElements.NumElements >
0) { for (int j = 0; j < fieldElements.NumElements; ++j) { Element
field = fieldElements.GetElement(j); for (int k = 0; k <
field.NumValues; k++) { // Description will be broken down into
multiple values // found in the field. description +=
field.GetValueAsElement(k).GetElement(0). GetValueAsString(0) + "
"; } } } } } return description; }

```

下面两个方法是前面应用中已经使用过的，用于从彭博中提取历史价格：

```

private Dictionary<DateTime, double> GetHistory (string
security, string field, DateTime startDate) { Dictionary<DateTime,
double> date2value = new Dictionary<DateTime, double>();
SessionOptions sessionOptions = new SessionOptions(); Session
session = new Session(); bool sessionStarted = session.Start(); if
(!sessionStarted) { System.Console.Error.WriteLine("Failed to start
session."); return null; } if
(!session.OpenService("//blp/refdata")) {
System.Console.Error.WriteLine("Failed to open //blp/refdata");
return null; } Service refDataService =
session.GetService("//blp/refdata"); Request request =
refDataService.CreateRequest("HistoricalDataRequest"); Element
securities = request.GetElement("securities");
securities.AppendValue(security); Element fields =
request.GetElement("fields"); fields.AppendValue(field);
request.Set("startDate", startDate.ToString("yyyyMMdd"));
request.Set("periodicityAdjustment", "ACTUAL");
request.Set("periodicitySelection", "DAILY");
request.Set("nonTradingDayFillOption", "ALL_CALENDAR_DAYS");
request.Set("nonTradingDayFillMethod", "PREVIOUS_VALUE"); try {
session.SendRequest(request, null); } catch
(InvalidRequestException e) {
System.Console.WriteLine(e.ToString()); } bool done = false; while
(!done) { Event eventObj = session.NextEvent(); if (eventObj.Type
== Event.EventType.PARTIAL_RESPONSE) {
ProcessHistoryResponse(eventObj, date2value); } else if
(eventObj.Type == Event.EventType.RESPONSE) {
ProcessHistoryResponse(eventObj, date2value); done = true; } else {
foreach (Message msg in eventObj) {
System.Console.WriteLine(msg.AsElement); if (eventObj.Type ==
Event.EventType.SESSION_STATUS) { if
(msg.MessageType.Equals("SessionTerminated")) { done = true; } } }

```

```

} } session.Stop(); return date2value; } private void
ProcessHistoryResponse(Event eventObj, Dictionary<DateTime, double>
date2value) { foreach (Message msg in eventObj) { if
(msg.HasElement(RESPONSE_ERROR)) { Element error =
msg.GetElement(RESPONSE_ERROR); Console.WriteLine("Request failed:
" + error.GetElementAsString(CATEGORY) + " (" +
error.GetElementAsString(MESSAGE) + ")"); continue; } Element
securityData = msg.GetElement(SEcurity_DATA); string security =
securityData.GetElement(SEcurity).GetValueAsString();
Console.WriteLine(security); Element fieldData =
securityData.GetElement(FIELD_DATA); for (int i = 0; i <
fieldData.NumValues; i++) { Element element =
fieldData.GetValueAsElement(i); DateTime date =
element.GetElementAsDatetime(DATE).ToSystemDateTime(); double?
value = null; for (int f = 0; f < element.NumElements; f++) {
Element field = element.GetElement(f); if
(!field.Name.Equals(DATE)) { if (field.Datatype ==
DataType.FLOAT32) value = Convert.ToDouble(field.GetValueAs-
Float32()); else if (field.Datatype == DataType.FLOAT64) value =
field.GetValueAsFloat64(); } } if (value != null)
date2value.Add(date, value.Value); } } }

```

下一个方法是Export，它有一个数组参数ext，它可能是PDF或XLS格式，它决定了导出的文件格式。代码用Reporting Services API将Excel或PDF报告导出到一个临时文件，然后开始运行该文件：

```

private void Export(string ext) { Warning[] warnings; string[]
streamids; string mimeType; string encoding; string extension;
//Temp file location string file =
System.Environment.GetFolderPath(Environment.SpecialFolder.Internet
+ "\\\" + Guid.NewGuid().ToString() + "." + ext; string type = null;
if (ext.ToUpper() == "XLS") type = "Excel"; else type = "PDF";
//Render report as byte array byte[] bytes =

```

```
this.rptView.LocalReport.Render( type, null, out mimeType, out encoding, out extension, out streamids, out warnings); // Save byte array to a file. FileStream fs = new FileStream(file, FileMode.Create); fs.Write(bytes, 0, bytes.Length); fs.Close(); //Open file System.Diagnostics.Process.Start(file); }
```

最后两个方法将PDF和Excel按钮连接到Export方法：

```
private void btnPDF_Click(object sender, EventArgs e) { Export("PDF"); } private void btnExcel_Click(object sender, EventArgs e) { Export("XLS"); }
```

结束后，最后应该如图9-8所示。

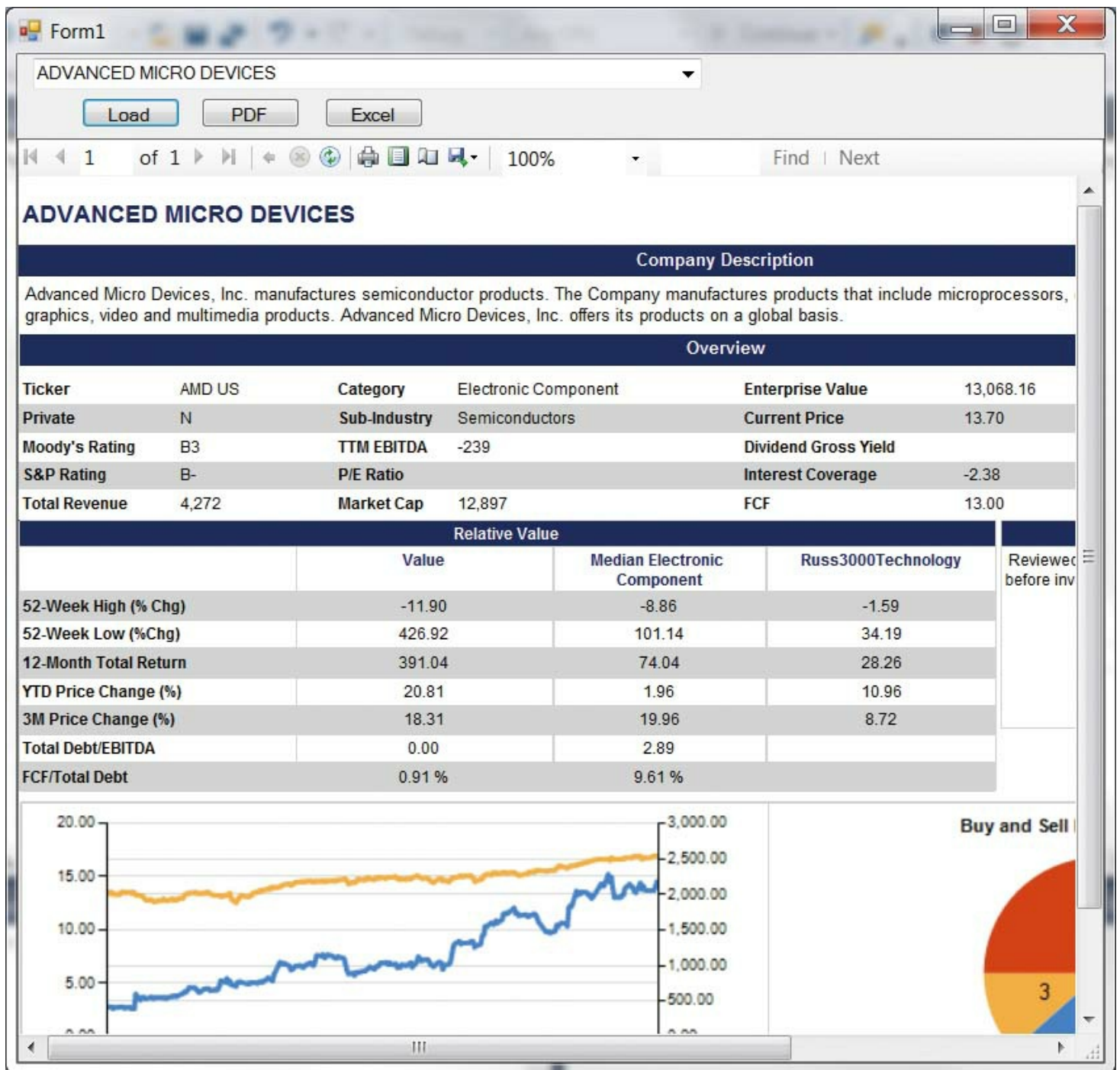


图9-8：C#报告应用

9.4 本章小结

本章介绍了三种不同的报告工具，更重要的是展示了如何结合从前面章节中学到的内容，生成一份对于做出投资决策有帮助的报告。建议你花点时间看看你为几家公司创建的报告，深入思考一下公司价值背后隐藏的信息，以及你要改进报告可以采取的措施。例如，你可以在报告中增加公司发行的基础有价值证券（债券和贷

款)的细节。第10章的内容是在本章的基础上创建投资组合级别的报告。

第10章 投资组合报告

前面的章节关注用金融数据进行投资决策，本章关注测量投资组合的业绩和风险。本章还展示如何计算一个投资组合的月度收益、年化收益、年化标准差和夏普比率。此外，本章还介绍如何创建一个两页长的活页报告，其中关注投资组合的主要指标，并将该投资组合的时间加权收益和风险调整后的收益与不同基准进行比较。尽管在本章中介绍的技巧目的是测量你的投资组合的业绩，你还可以用它们测量其他资金经理人、基金或投资组合的业绩。

本章的开头部分有两个输入变量：月度盈亏(P&L)和月度初始账户余额，并计算不同类型的收益。计算收益可能比很多人想的要复杂；在网上搜索“如何计算投资组合收益”将返回多种方法，让人产生混淆。在计算收益时，你必须考虑到用时间加权几何收益得到的复利。时间加权收益(也称为累计收益、复合收益或几何收益)是企业前期的盈亏。

10.1 监测业绩和风险

考虑下面的例子：你投入100万美元，然后第一个月损失50万美元(损失50%)。下一个月，你投入50万美元，然后挣到了上月损失的50万美元(用50万美元挣50万美元，相当于100%收益)。如果简单把两个月的月度收益相加，收益就是50%。如果把两个月的月度收益平均一下，收益就是25%。然而事实上，按照几何收益计算，收益是0% ($\$1000000 - \$500000 + \$500000 = \1000000 ，收益是0%——我们在后面讨论几何收益公式)。尽管有一些高复杂的方法可以计算时间加权收益(用到了Microsoft Excel数组函数)，不过本章要展示的是一个更简单的技巧。

然后，我们介绍了如何对收益进行年化计算，这是与不同基准或其他基金进行比较的基础。年化计算可以对复利进行反向计算，把超过一年的收益转换成12个月长的收益。换句话说，年化一个四年期的复利，会得到一个一年期收益，用该收益进行四年复利计算得到的结果就是最初的收益值。在比较收益时进行年化是很重要的，因为复利会产生重大影响。

🐭 尽管年化收益公式可以用于推算时间不满一年的收益，但是其样本规模有限，因此不建议这样做。此外，全球投资表现标准（Global Investment Performance Standards, GIPS）的规定也禁止对时间不满一年的收益进行年化计算。然而，你可以用公式年化并非整年的收益（例如16个月等）。

第7章介绍了用投资组合中每个头寸的预测收益或历史收益计算你当前持有的投资组合的夏普比率；而本章将展示如何用历史投资组合收益计算夏普比率。主要区别在于，在第7章中夏普比率是前瞻性的，是基于投资组合的当前组成的，而本章用历史业绩计算夏普比率。在此情况下，可以用夏普比率测量该投资组合相对于其他投资组合或投资的平均风险调整后的收益。

最后，本章介绍在第9章的内容基础上，在Excel和SSRS中创建一份两页活页报告，部分内容参见第7章投资组合分解部分。与其他章一样，Excel（途径1）的计算和解释比较详细。当然，你可以把途径1创建的Excel表连接到Microsoft Access。途径3部分包括如何用C#和SSRS实现。

10.2 途径1：Microsoft Excel

本节首先创建一个新的工作表（“Returns”），其中存储你的投资组合的月度损益表（P&L）、管理资产额（AUM）、夏普比率和收益计算。此外，该工作表计算标准普尔500指数（S&P 500）和彭博巴克莱美国债券综合指数（Barclays Agg）对应的收益和夏普比率。S&P 500是500种股票的指数，Barclays Agg是投资级别的固定利率债券（风险和收益都低于股票）的集合。本节后面将详细介绍如何创建一个简要的投资组合报告，展示收益工作表中算出的一些重要值。

10.2.1 计算收益

本节需要投资组合的月度损益表和每月初始市值。

创建一个新的工作表，命名为Returns，按照下面的步骤创建Excel收益表，其中包含你的投资组合和基准指数的收益信息。

1. 把单元格M1至Y1命名为：Date、Monthly PNL、Starting AUM、Monthly Return、Value of\$100、Monthly Risk Free、Return 泰F、S&P 500、S&P

500Pct、S&P 500、Barclays Aggregate、Barclays Pct、Barclays 500。

2.在Date、Monthly PNL和Starting AUM列，输入每月最后一天、该月收入或损失（以美元计）和投资组合中投资的初始市值。日期按升序排列。

例如，如果投资组合收益的第一个完整月份是2015年1月，那么在Date标题下的单元格M2输入“1/31/2015”。如果2015年1月投资组合总值为\$300000，在单元格N2输入“300000”。如果2015年1月初，投资组合的初始市值是3000万美元，在Starting AUM标题下的单元格O1输入“30000000”。

3.将使用到的整个区域，从M1至列Y的最后一个单元格，转换为Excel区域，命名为“Returns”。

4.在P（Monthly Return）列，在单元格中输入如下公式：

$$=[@[Monthly\ PNL]]/[@[Starting\ AUM]]$$

该公式计算月度收益，方法是用月度损益总额除以月初余额。

下一步在“Value of\$100”列填入公式，跟踪投资组合中的100美元的现值变化。例如，如果第一个月投资组合收益为3%，则“Value of\$100”第一行应显示\$103（ $\$100 + 3\% \text{ of } \$100 = \$103$ ）。如果投资组合下月收益为4%，“Value of\$100”的值就是107.12美元。（ $\$103 + 4\% \text{ of } \$103 = \$107.12$ ）。该列用于跟踪复利的影响和计算投资组合的时间加权收益。

5.在“Value of\$100”列输入如下公式：

$$=IF([@Date]=MIN([Date]),100*(1+[@[Monthly\ Return]]),Q1*(1+[@[Monthly\ Return]]))$$

对于现值，该公式检查当前行的日期是否是Excel表中最早的日期；如果是，用\$100；否则，用前一行的值。该公式用现值乘以（月度收益+1），相当于：现值+（现值×月度收益）。

用下面的步骤从彭博提取S&P 500、Barclays Agg的月度价格。用三个月期伦敦同业拆借利率（LIBOR）无风险利率计算夏普比率。

6.把单元格A1和B1分别命名为 “Date” 和 “SPX Index”。

7.在单元格A2中，输入如下公式：

```
=BDH(B1,"PX_LAST",EDATE(MIN>Returns[Date]),-1),"-0cm","FX=USD",  
"PER=AM","FILL=P","DAYS=A")
```

该公式提取S&P 500指数在收益表的最早日期到最后可知的月底期间内的最新月底价格。它总是包含了该月的最后一天，因为其中一个实参是"DAYS=A"。该公式的A列和B列填入的内容是S&P 500价格历史。

8.从C3开始复制如下公式：

```
= ( B3-B2 ) /B2
```

该公式计算S&P 500的月度总收益。

9.重复第6～8步，用单元格F1中的 “LBUSTRUU Index” 提取E列和F列中的 Barclays Agg，并计算G列中的月度总收益。

10.重复第6～7步，用单元格J1中的 “US0003M Index”，提取I列和J列中的三个月期LIBOR。

用与第7步相同的BDH公式，在与月度收益相同的日期区域，提取年化三个月期LIBOR利率。

11.在K列，输入如下公式，从单元格K2开始，并向下复制：

```
=J2/100/12
```

该公式用三个月期LIBOR利率，除以100变成百分比数，然后再除以12，得到月利率。

下面的步骤将前面步骤从彭博得到的月度市场数据加到Excel收益表中去。

12.在Monthly Return列，输入如下公式：

=VLOOKUP ([@Date] , I : K , 3 , FALSE)

该公式用VLOOKUP提取S&P 500某个指定月的三个月期LIBO利率。

13.在Return-RF列，输入如下公式：

=[@[Monthly Return]]-[@[Monthly Risk Free]]

该公式就是用月度收益减去无风险利率（三个月期LIBOR）。

14.在S&P 500列，输入如下公式：

=VLOOKUP ([@Date] , A : C , 2 , FALSE)

该公式用VLOOKUP提取S&P 500每月月末的收益值。

15.在S&P 500Pct列，输入如下公式：

=VLOOKUP ([@Date] , A : C , 3 , FALSE)

该公式用VLOOKUP提取S&P 500的月度收益。

16.在“S&P-RF”列，输入如下公式：

=[@[S&P 500Pct]]-[@[Monthly Risk Free]]

这步用S&P 500收益减去无风险利率。

17.重复第14~16步，在Barclays Aggregate列填入如下公式：

=VLOOKUP([@Date],E:G,2,FALSE), =VLOOKUP([@Date],E:G,3,FALSE), =
[@[Barclays Pct]]-[@[Monthly Risk Free]]

结果如图10-1所示。

	M	N	O	P	Q	R	S	T	U	V	W	X	Y
1	Date	Monthly PNL	Starting AUM	Monthly Return	Value of \$100	Monthly Risk Free	Return - RF	S&P 500	S&P 500 Pot	S&P - RF	Barclays Aggregate	Barclays Pot	Barclays - RF
2	1/31/2015	(822,079)	35,374,050	-2.32%	97.68	0.021%	-2.345%	1,995	-3.10%	-3.13%	1,955	2.10%	2.08%
3	2/28/2015	1,563,422	34,551,972	4.52%	102.10	0.022%	4.503%	2,105	5.49%	5.47%	1,937	-0.94%	-0.96%
4	3/31/2015	(508,878)	36,115,393	-1.41%	100.66	0.023%	-1.432%	2,068	-1.74%	-1.76%	1,946	0.46%	0.44%
5	4/30/2015	238,725	35,606,515	0.67%	101.33	0.023%	0.647%	2,086	0.85%	0.83%	1,939	-0.36%	-0.38%
6	5/31/2015	306,706	35,845,240	0.86%	102.20	0.024%	0.832%	2,107	1.05%	1.03%	1,934	-0.24%	-0.26%
7	6/30/2015	(704,809)	36,151,946	-1.95%	100.21	0.024%	-1.973%	2,063	-2.10%	-2.12%	1,913	-1.09%	-1.11%
8	7/31/2015	631,798	35,447,136	1.78%	101.99	0.026%	1.757%	2,104	1.97%	1.95%	1,926	0.70%	0.67%
9	8/31/2015	(1,926,954)	36,078,934	-5.34%	96.55	0.027%	-5.368%	1,972	-6.26%	-6.29%	1,923	-0.14%	-0.17%
10	9/30/2015	(732,963)	34,151,980	-2.15%	94.47	0.027%	-2.173%	1,920	-2.64%	-2.67%	1,936	0.68%	0.65%
11	10/31/2015	2,358,085	33,419,017	7.06%	101.14	0.028%	7.028%	2,079	8.30%	8.27%	1,937	0.02%	-0.01%
12	11/30/2015	1,169	35,777,102	0.00%	101.14	0.035%	-0.031%	2,080	0.05%	0.02%	1,932	-0.26%	-0.30%
13	12/31/2015	(550,457)	35,778,272	-1.54%	99.59	0.051%	-1.590%	2,044	-1.75%	-1.80%	1,925	-0.32%	-0.37%
14	1/31/2016	(1,446,500)	35,227,815	-4.11%	95.50	0.051%	-4.157%	1,940	-5.07%	-5.12%	1,952	1.38%	1.32%
15	2/29/2016	(82,587)	33,781,314	-0.24%	95.26	0.053%	-0.297%	1,932	-0.41%	-0.47%	1,966	0.71%	0.66%
16	3/31/2016	1,936,607	33,698,727	5.75%	100.74	0.052%	5.694%	2,060	6.60%	6.55%	1,984	0.92%	0.86%
17	4/30/2016	102,296	35,635,335	0.29%	101.03	0.053%	0.234%	2,065	0.27%	0.22%	1,991	0.38%	0.33%
18	5/31/2016	467,037	35,737,631	1.31%	102.35	0.057%	1.250%	2,097	1.53%	1.48%	1,992	0.03%	-0.03%
19	6/30/2016	125,461	36,204,667	0.35%	102.70	0.055%	0.292%	2,099	0.09%	0.04%	2,028	1.80%	1.74%
20	7/31/2016	1,134,107	36,330,128	3.12%	105.91	0.063%	3.058%	2,174	3.56%	3.50%	2,041	0.63%	0.57%
21	8/31/2016	(45,241)	37,464,235	-0.12%	105.78	0.070%	-0.191%	2,171	-0.12%	-0.19%	2,038	-0.11%	-0.18%
22	9/30/2016	(42,569)	37,418,994	-0.11%	105.66	0.071%	-0.185%	2,168	-0.12%	-0.19%	2,037	-0.06%	-0.13%
23	10/31/2016	(660,033)	37,376,426	-1.77%	103.79	0.074%	-1.840%	2,126	-1.94%	-2.02%	2,021	-0.76%	-0.84%
24	11/30/2016	936,286	36,716,393	2.55%	106.44	0.078%	2.472%	2,199	3.42%	3.34%	1,974	-2.37%	-2.44%
25	12/31/2016	590,467	37,652,679	1.57%	108.11	0.083%	1.485%	2,239	1.82%	1.74%	1,976	0.14%	0.06%
26	1/31/2017	592,622	38,243,145	1.55%	109.79	0.086%	1.463%	2,279	1.79%	1.70%	1,980	0.20%	0.11%
27	2/28/2017	1,267,084	38,835,768	3.26%	113.37	0.089%	3.174%	2,364	3.72%	3.63%	1,994	0.67%	0.58%
28	3/31/2017	(16,436)	40,102,852	-0.04%	113.32	0.096%	-0.137%	2,363	-0.04%	-0.13%	1,993	-0.05%	-0.15%

图10-1：Excel收益表

下面的步骤创建另一张表，总结一些关键日期的投资组合和基准价格。

18.把单元格AA1至AE1命名为：Range、Date、Portfolio、S&P、Barclays。

19.在AA列区域标题下，逐行输入如下标签：Latest、QTD、YTD、Last 12Months、ITD。

这些行标题对应一个日期；值会出现在最后三个列标题下。

20.在单元格AB2中，Date标题下，紧挨Latest，输入如下公式：

=MAX (Returns[Date])

该公式从Returns提取最近的日期。

21.在单元格AB3中，Date标题下，紧挨QTD，输入如下公式：

=EOMONTH (DATE (YEAR (AB2) , FLOOR (MONTH (AB2) -1 , 3) +1 , 1) , -1)

QTD表示本季度迄今为止 (Quarter-to-Date)。该公式基于最近的月份 (见第20步) 计算季度的起始日期，然后用EOMONTH函数得到前一个月的最后一天。我们用上一个月的日期，因为我们希望得到的是季度开始之前那个交易日的收盘价。

22.在单元格AB4中，Date标题下，紧挨YTD，输入如下公式：

=DATE (YEAR (AB12) -1 , 12 , 31)

YTD表示今年迄今为止 (Year-to-Date)。该公式返回最新收益之前那年的12月31日。

23.在单元格AB5中，Date标题下，紧挨Last 12Months，输入如下公式：

=EDATE (AB2 , -12)

该公式返回最新收益日期之间的12个月的最后一天。

24.在单元格AB6中，Date标题下，紧挨ITD，输入如下公式：

=EOMONTH (MIN (M : M) , -1)

ITD代表初期 (Inception-to-Date)。该公式返回第一次月度收益之前那个月的最后一天。注意ITD表示投资组合的开始，而非S&P 500或Barclays Agg的开始。

25.在AC列，在Portfolio列标题下，在单元格AC6输入100，然后在单元格AC2输入如下公式，并向下复制到AC5：

=INDEX (Returns[Value of\$100] , MATCH (AB2 , Returns[Date] , 0))

该公式用第20~24步的日期提取对应的“Value of\$100”。自然，该投资组合ITD的起始数值 (单元格AC6) 应当为100美元。

26.在AD列，在S&P列标题下，输入如下公式并向下复制：

=VLOOKUP (AB2 , A : B , 2 , FALSE)

该公式返回Date列指定日期的S&P值。

27.在AE列，在Barclays列标题下，输入如下公式，并向下复制：

=VLOOKUP (AB2 , E : F , 2 , FALSE)

该公式返回Date列指定日期的Barclays Agg值。

现在，我们已经有了了一张包含值的表，还需要最后一张表，其中包含给定时间区间中的收益，格式可以方便地被转换成图表。下面的步骤将在第18~27步创建的表下方创建一张新的表，其中包含每个时段的时间加权收益和年化标准差。因为第18~27步创建的表用到了“Value of\$100”，其中有复利计算，我们就可以很简单地用现值和初始值之差除以初始值来计算时间加权收益： $(\text{现值} - \text{初始值}) / \text{初始值}$ 。如果不想用“Value of\$100”列，你还可以用一个数组公式计算时间加权收益；要计算ITD时间加权收益，用数组公式（按Ctrl+Shift+Enter组合键） $=\text{PRODUCT} (1 + \text{Returns}[\text{Monthly Return}]) - 1$ 。然而，“Value of\$100”有一个特殊优势，即与S&P 500和Barclays Agg的收益计算一致。

28.从单元格AA11开始，重复第18~24步，重新创建列标题和行标题。

29.在Portfolio列标题下，在单元格AC12中，输入如下公式：

=INDEX (Returns[Monthly Return] , MATCH (AB12 , Returns[Date] , 0))

该公式用AB12中的日期返回最新月度收益。

30.在S&P列标题下，在单元格AD12中，输入如下公式：

=INDEX (Returns[S&P 500Pct] , MATCH (AB12 , Returns[Date] , 0))

该公式返回最新的S&P 500月度收益。

31.在Barclays列标题下，在单元格AE12中，输入如下公式：

=INDEX (Returns[Barclays Pct] , MATCH (AB12 , Returns[Date] , 0))

该公式返回最新的Barclays Agg月度收益。

32.在Portfolio列标题下，在第29步用的单元格下面的行（单元格AC13），输入如下公式，并向下复制到AC16：

= (AC\$2-AC3) /AC3

该公式引用前面创建的表，用Latest日期作为现值，用每行的区域作为初始值，来计算时间加权收益。

33.把第32步的公式（单元格AC13至AC16）复制到两个临近的列（AD和AE），则在单元格AC13至AE16中的公式是相同的。

计算投资组合的时间加权收益的公式同样用于计算S&P 500和Barclays Agg的时间加权收益。

下面的步骤对投资组合和两个基准指数年化Inception-to-Date收益。记住，不能用年化，除非有至少12个月收益。图10-2是对收益进行年化的基本公式。如果你在网上查询“如何年化月度收益”，会得到一长串公式，这里用到的公式是结果比较准确的公式之一。

$$\text{年化收益} = (1 + \text{总收益})^{\frac{365}{\text{\# 天数}}} - 1$$

图10-2：计算年化收益的公式

34.把单元格AA17命名为“ITD Annualized”。

35.在单元格AB17中，输入如下公式：

=EOMONTH (MIN (M : M) , -1)

该公式与紧挨ITD行标题的那个公式是一样的。

36.在单元格AC17中，在Portfolio列标题下，输入如下公式：

$$= (1+AC16) ^ (365 / (\$AB\$12-\$AB16)) -1$$

该公式用图10-2中的公式年化ITD时间加权收益。

37.把第36步用到的公式（单元格AC17）复制到单元格AD17和AE17，以年化S&P 500和Barclays Agg收益。

下面的步骤将每个时间段的年化标准差加入表中。不幸的是，使用哪个Excel标准差函数（STDEV.S或STDEV.P）最合适还没有定论。STDEV.S假设进行取样，而STDEV.P假设采用全样本。使用两种函数的情况我都听说过，根据本次情况我选择了使用STDEV.S，因为这是惯例。围绕用12的平方根来计算年化收益的标准差是不是准确有很多争论，但是它仍是一个行业标准。

38.在第11行从AF列到AH列，添加如下标签：Portfolio、S&P 500、Barclays Agg。

39.在Portfolio列标题下，在QTD行（单元格AF13），输入如下数组公式，并向下复制到单元格AF16。因为这是一个数组公式，你需要按Ctrl+Shift+Enter组合键：

$$=STDEV.S (IF (Returns[Date]>AB13 , Returns[Monthly Return])) * (12^{0.5})$$

该公式用到的是AB列内包括的日期和之后日期的月度收益的标准差。标准差用乘以12的平方根进行年化。

40.在AG和AH列重复第39步，用数组公式计算S&P 500和Barclays Agg的年化标准差：

$$=STDEV.S (IF (Returns[Date]>AB13 , Returns[S\&P 500Pct])) * (12^{0.5})$$

和

`=STDEV.S (IF (Returns[Date]>AB13 , Returns[Barclays Pct]))
* (12^0.5)`

计算年化标准差之后，可以计算ITD夏普比率。计算夏普比率的方法是用平均收益与12之积除以年化标准差。网上有不同的夏普比率计算方法，但是这个方法比较常用。

41.把单元格AA20命名为 “ITD Sharpe Ratio” 。

42.把单元格AC19至AE19，命名为：Portfolio、S&P 500、Barclays Agg。

43.在单元格AC20中，在Portfolio列标题下，输入如下公式：

`=AVERAGE ( Returns[Return-RF] ) *12/AF16`

该公式计算夏普比率的方法是对所有收益求平均值，乘以12进行年化，再除以投资组合自开始以来的年化标准差。

44.在S&P 500列标题下（单元格AD20），输入如下公式：

`=AVERAGE ( Returns[S&P-RF] ) *12/AG16`

该公式返回S&P 500的夏普比率，时间段与投资组合存续的时间段一致。

45.类似于第43～44步，在Barclays列标题下，输入如下公式：

`=AVERAGE ( Returns[Barclays-RF] ) *12/AH16`

花点时间看看结果，保证结果是讲得通的，看看有没有什么有意思的结果（结果应该如图10-3所示）。你应该可以想到，S&P 500的收益和风险（标准差）高于Barclays Agg。在图10-3这个例子中，S&P 500的年化收益较高（达到6.31%，而组合的年化收益仅为5.72%），但是因为S&P 500的标准差较大，因此夏普比率稍微低了一点（0.54对0.56）。记住，夏普比率测量风险调整后的收益，因此值越大越好。

	AA	AB	AC	AD	AE	AF	AG	AH
1	Range	Date	Portfolio	S&P	Barclays			
2	Latest	3/31/2017	113.32	2,363	1,993			
3	QTD	12/31/2016	108.11	2,239	1,976			
4	YTD	12/31/2016	108.11	2,239	1,976			
5	Last 12 Months	3/31/2016	100.74	2,060	1,984			
6	ITD	12/31/2014	100.00	2,059	1,915			
7								
8								
9								
10								
11	Range	From	Portfolio	S&P 500	Barclays Agg	Portfolio	S&P 500	Barclays Agg
12	Latest Monthly	3/31/2017	-0.04%	-0.04%	-0.05%	5.72%	6.51%	1.28%
13	QTD	12/31/2016	4.82%	5.53%	0.82%	5.72%	6.51%	1.28%
14	YTD	12/31/2016	4.82%	5.53%	0.82%	5.72%	6.51%	1.28%
15	Last 12 Months	3/31/2016	12.49%	14.71%	0.44%	5.21%	6.15%	3.38%
16	ITD	12/31/2014	13.32%	14.76%	4.05%	9.64%	11.43%	3.12%
17	ITD Annualized	12/31/2014	5.72%	6.31%	1.78%			
18								
19			Portfolio	S&P 500	Barclays Agg			
20	ITD Sharpe Ratio		0.56	0.54	0.39			
21								

图10-3：收益总结表

10.2.2 投资组合报告

本节介绍如何使用收益总结以及在第7章投资组合分解部分创建的表，创建一份两页纸长的活页报告。记住创建报告并无定法，你可以根据自身需要定制报告。报告第一页包含一些关键数据点（投资组合规模、夏普比率、最后收益日期和创建报告的日期）。此外，第一页包括四张图表，向读者介绍投资组合的业绩、增长、风险调整后的收益和组成。在第二页中，用从投资组合分解工作表得到的总结数据绘制四张图表，展示投资组合更多的细节。

下面的步骤为报告创建一张新的工作表，进行页面布局设计，使报告的观感更好。

46. 创建一张新的工作表，命名为“Portfolio Report”，然后切换到Page Break Preview。

47. 在函数区的Page Layout键上，Page Setup组中，点击Orientation键，

然后选择Landscape。

48.选择单元格A1至M96，然后在Page Layout键上，Page Setup组中，点击Print Area，然后选择Set Print Area。

这显示的是两页报告的轮廓。

49.还是在Page Layout键上，在Gridlines里清除View复选框。

50.把字号改为8号。

下面的步骤对报告中用到的部分数据进行组织整理。首先从收益（Returns）工作表中提取，以展示生成图表历史Portfolio MV。将收益工作表中的Portfolio MV链接到投资组合报告（PortfolioReport）工作表，方便显示日期。

51.把单元格O1命名为“Date”，把单元格P1命名为“PortfolioMV”。

52.在Date标题下，即单元格O2中，输入如下公式：

=MAX (Returns[Date])

该公式返回投资组合的最新收益日期。

53.在该单元格正下方，在单元格O3中，输入如下公式：

=EOMONTH (O2 , -1)

该公式返回它上面的单元格的上个月的最后一天。

54.把单元格O3的公式拖拽下来，这样就包含了整个收益历史。

55.在“Portfolio MV”标题下，在单元格P2中输入如下公式，然后将公式拖拽下来，以匹配第54步的行：

=INDEX (Returns[Starting AUM] , MATCH (O2 , Returns[Date] , 0))

用INDEX/MATCH函数从Excel Returns表提取日期相匹配的Starting AUM。结

果应为包含Portfolio MV历史的两列（O和P）。

下面的步骤创建另一张表，其中包含投资组合、S&P 500和Barclays Agg的年化收益、年化标准差和夏普比率。此外，该表包含第7章创建的投资组合的当前标准差、预测收益和夏普比率，我们之后用它们来比较投资组合预计收益和经历史风险调整后的收益。

56.在第1行，从S列到V列，输入如下标签：ITD Portfolio、Current Portfolio、S&P 500、Barclays Agg。

57.在R列，从第2行到第4行，输入如下标签：Return、Standard Deviation、Sharpe。

第56步和第57步的结果应为一张表，其中将存储的信息用下面的步骤填入。

58.在第2行ITD Portfolio标题下（单元格S2），用如下公式链接到Returns工作表的年化ITD投资组合：

=Returns ! AC17

59.分别用=Returns ! AD17和=Returns ! AE17，将在S&P 500和Barclays Agg下的第2行单元格链接到Returns工作表的年化ITD收益。

60.在第2行Current Portfolio下，用如下公式连接第7章中创建的PortfolioStats工作表的投资组合收益计算：

=PortfolioStats ! E2

61.在第3行，从S列到V列，分别用如下公式链接历史收益、现在投资组合、S&P 500和Barclays Agg的年化标准差：

=Returns ! AF16 , =PortfolioStats ! E1 , =Returns ! AG16 , =Returns ! AH16

62.在第5行，从S列到V列，用如下公式链接夏普比率：

=Returns ! AC20 , =PortfolioStats ! E3 , =Returns ! AD20 , =Returns ! AE20

在下面的步骤中，我们开始设计报告。先设计报告头部，下面的步骤是在第一页上部添加一个标题、市值、夏普比率和关键日期。

63. 合并单元格A1至C2，然后命名为“Chapter 10”，字号为18号。

64. 合并单元格H1至K1，然后在单元格中填入下面的公式：

=TEXT (P2/1000000 , "\$# , ##0.00") &"MM Market Value"

该单元格显示最新收益日期的投资组合市值，并调整数字格式。如果最新的投资组合市值是123，456，789，就显示“\$123.46MM Market Value”。

65. 合并单元格H2至K2，然后在单元格中填入下面的公式：

=TEXT (S4 , "0.00") &"Sharpe Ratio"

该公式把投资组合的夏普比率限制到小数点后两位。

66. 把单元格L1命名为“Latest Return”，然后在单元格M1，输入如下公式：

=02

该公式显示Portfolio MV列的最新日期。

67. 把单元格L2命名为“Report Date”，然后在单元格M2，输入如下公式：

=TODAY ()

68. 改变单元格A3至M3的背景颜色，使报告头部和内容区分开来。

下面的步骤在报告上增加图表，将投资组合的相对增长与两个基准指数和投资组合的AUM增长进行比较。

69. 在第一页剩下区域的左上方，用=Returns ! \$AA\$11 : \$AE\$17区域的数据增

加一个柱状图，其中包括第28~37步创建的收益总结表。在Horizontal Axis Labels下，清除ITD项复选框，因为它对列出ITD年化项才更有意义。把图命名为“Time Weighted Return”。

70.在第一页右上方，挨着“Time Weighted Return”图，用O列和P列中的数据按照第51~55步添加一个线状图。把图命名为“Portfolio MV”。

根据个人意愿进行调整；报告上部应该如图10-4所示。这两个图表能够快速展示相对业绩和增长。

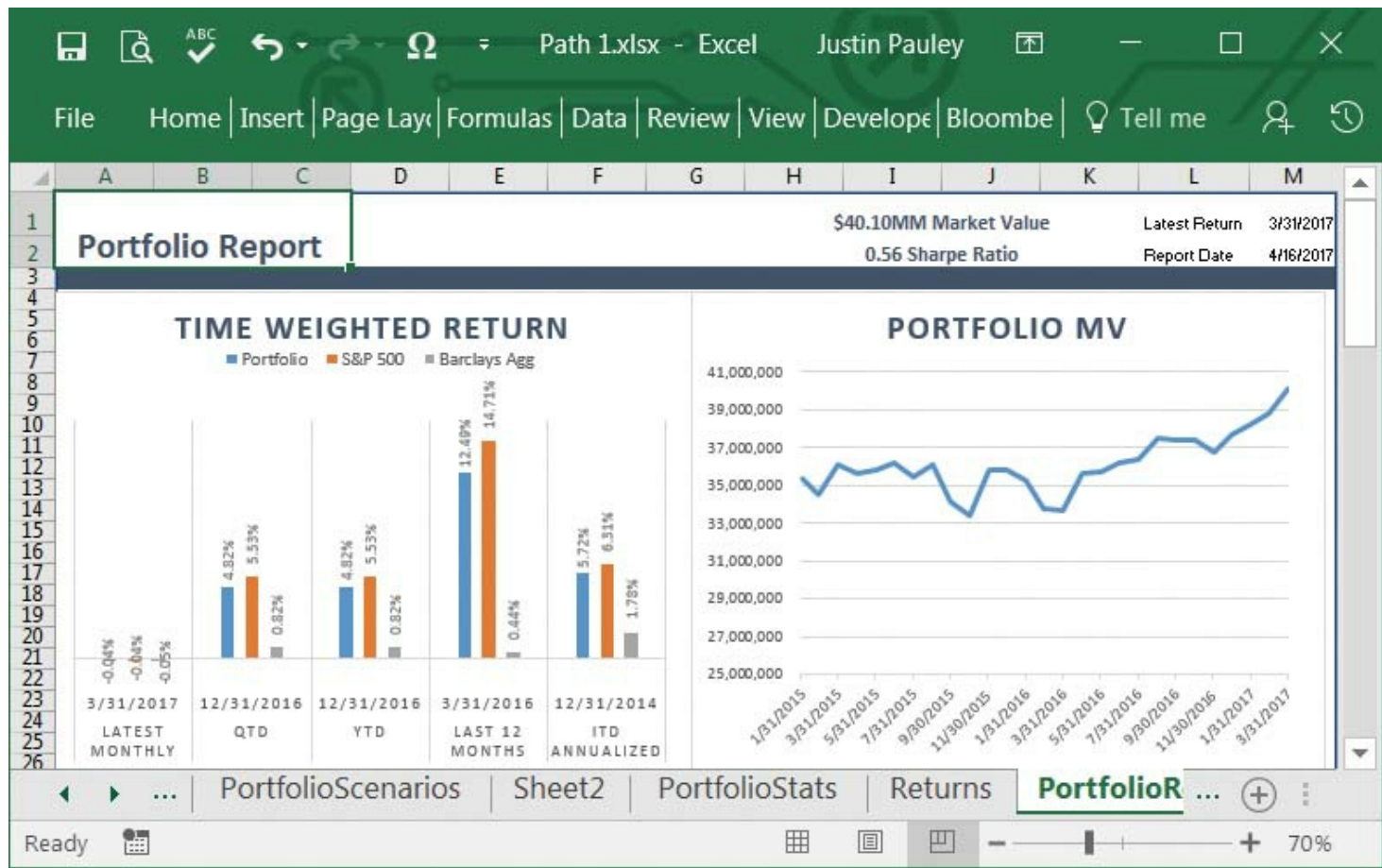


图10-4：投资组合报告第一页上部

下面的步骤是在投资组合报告的下部再增加两个图。第一个图是散点图，比较投资组合、两个基准指数和预测投资组合（见第7章）的收益风险（标准差）。第二个图按资产类别分解投资组合组成。

71.在第一页左下方，用第56~62步创造的R1至V4数据添加一个散点图。把图命名为“Risk vs Return”。

72.在第一页右下方，用PortfolioBreakdown工作表中的数据添加一个饼状图，其中按资产类型进行了分解（=PortfolioBreakdown!\$A\$1:\$C\$4）。

根据个人意愿调整格式。第一页下部应该如图10-5所示。“Risk vs Return”图能够迅速展示哪个基金的风险调整后收益最好。理想情况下，风险低、收益高的基金或投资组合处于“Risk vs Return”图的左上部。反之，风险高、收益低的基金位于图的右下部。

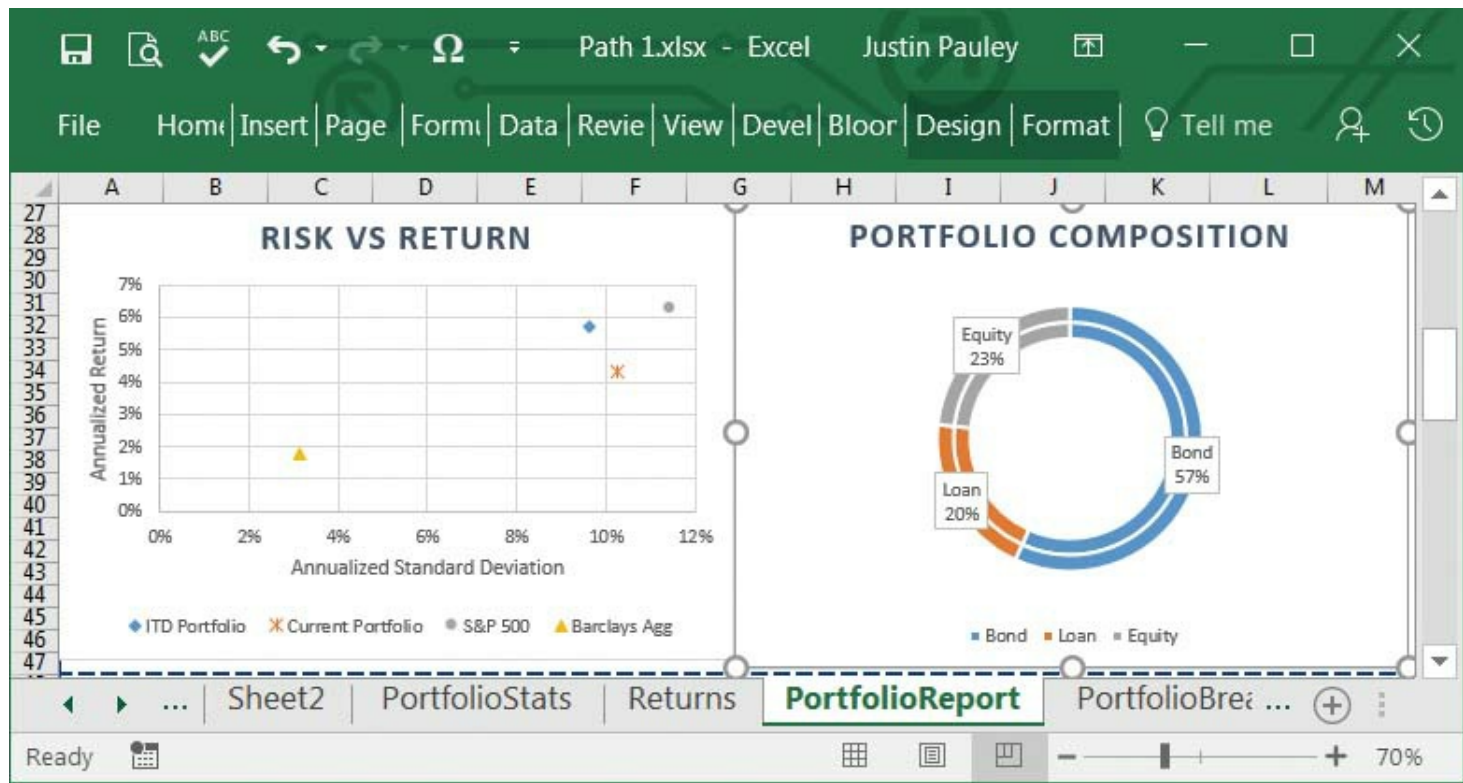


图10-5：投资组合报告第一页下部

下面几步是在第二页上部添加两个饼状图，其中投资组合的组成按照公司评级和分类进行了分解。

73.在报告分页蓝色虚线下方，在左上部添加一个饼状图。用Portfolio Breakdown工作表中投资组合公司评级分解（Portfolio Corp Rating）表里的“%of Portfolio”数据。把报告命名为“By Company Rating”。

74.在第二页右上方，用Portfolio Breakdown工作表的分类分解（Categorybreakdown）表添加一个饼状图。把图命名为“By Category”。

调整格式后，第二页上部如图10-6所示。这些图可以轻易地回答关于投资组合

组成的问题，不过你还可以再次输入更符合你需求的图。

最后两步是在报告第二页下部增加柱状和线形组合图。这些图展示某个区域的市值总额和占投资组合的百分比。

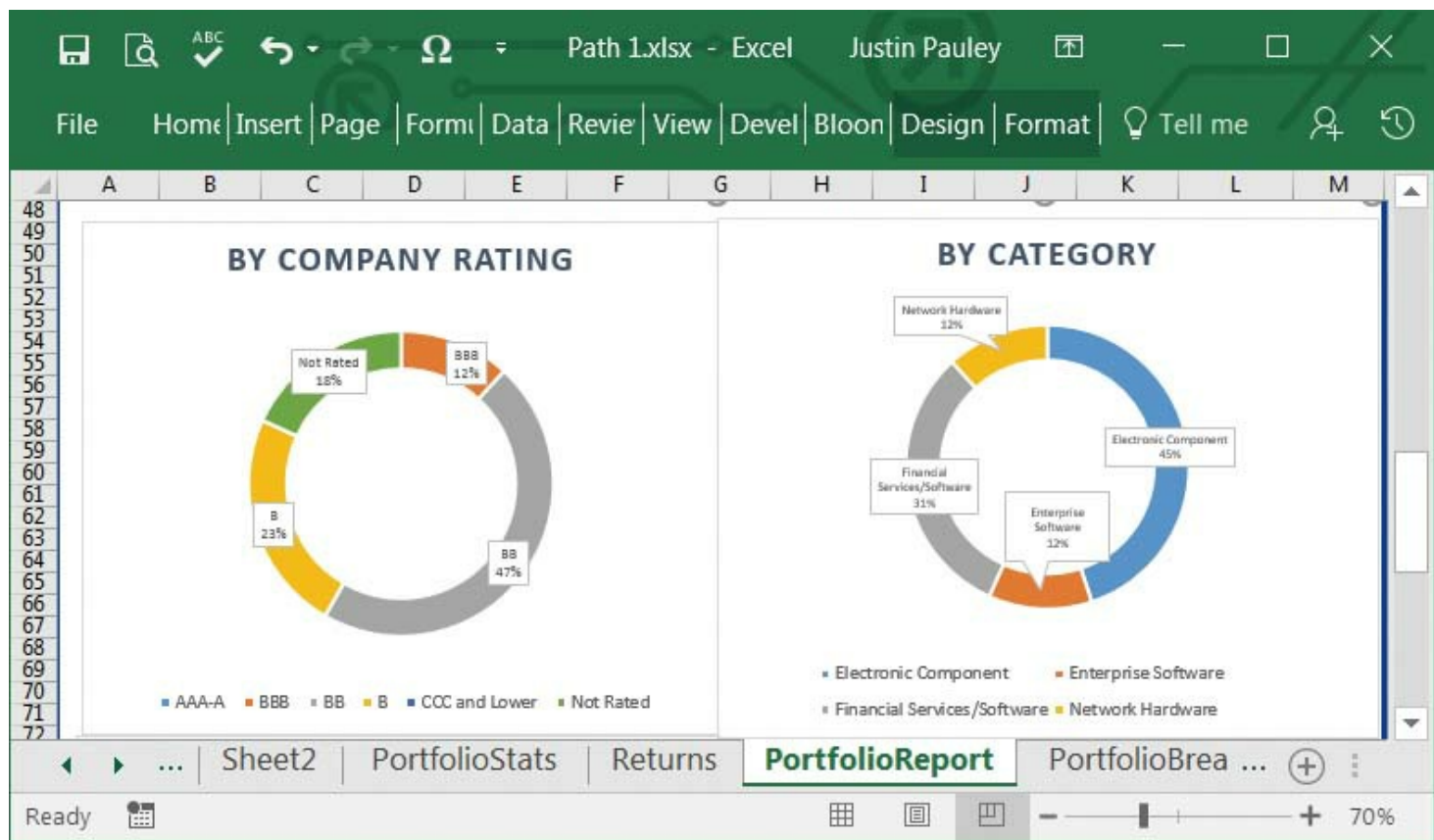


图10-6：投资组合报告第二页上部

75.在第二页左下方增加一个柱状图，命名为“Market Cap”，其中包括Portfolio Breakdown工作表中的Market Cap分布。Market Value和“%of Portfolio”是不同的两列。右击图中的Market Value列，然后选择“Change chart type”，将市值改为第二坐标轴上的线形图。

76.重复第75步，添加一个图，命名为“Fixed Income Price Distribution”，其中是Portfolio Breakdown工作表中固定收益资产的价格分布。

调整格式后，第二页下部应该如图10-7所示。这些图可以迅速地告诉读者百分比和总市值的分布情况。

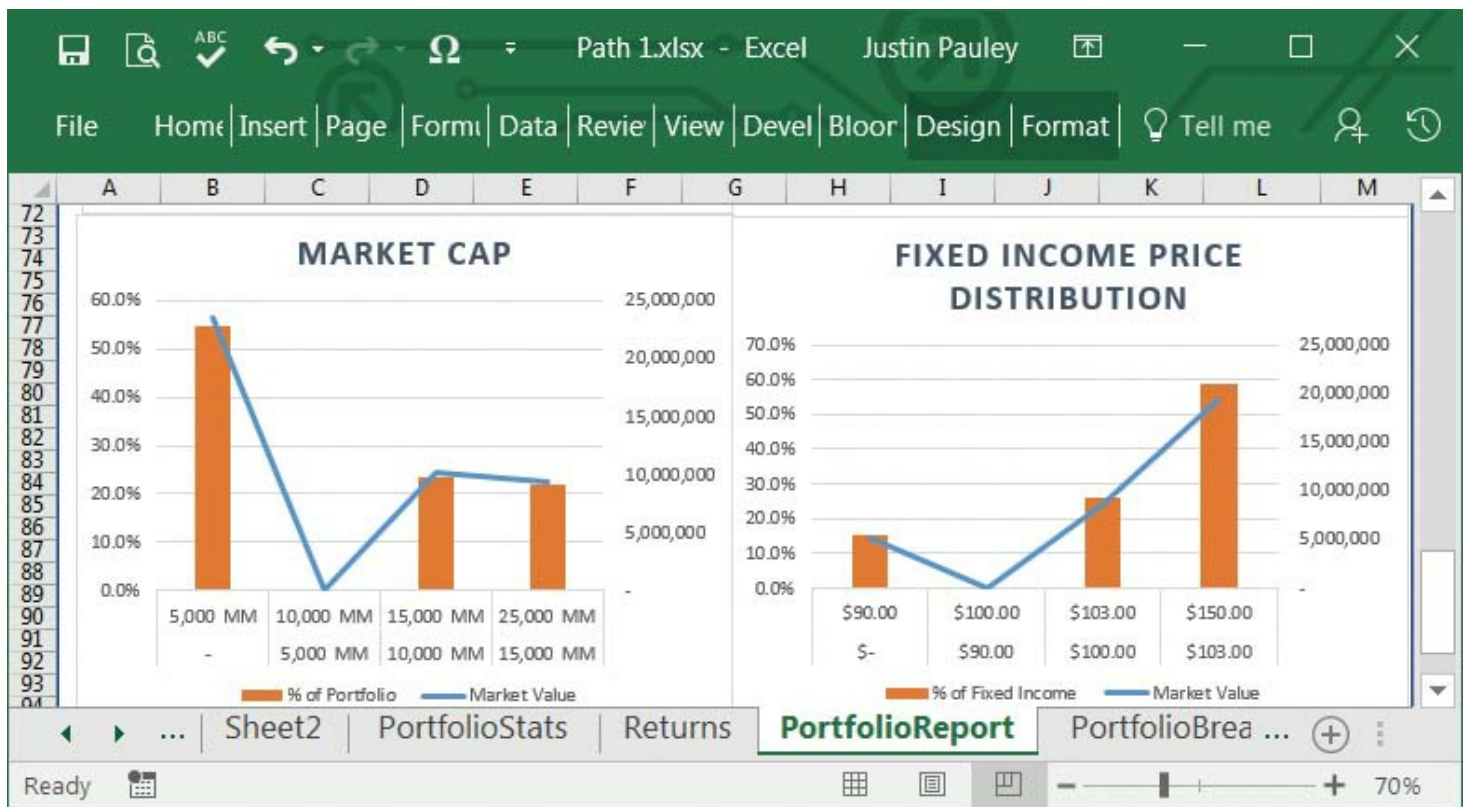


图10-7：投资组合报告第二页下部

10.3 途径3：C#和SSRS

本节介绍如何在Visual Studio中修改在第9章中创建的、用于查看报告的Windows Form应用。修改会增加一个额外的按钮，在该应用的另外一个选项卡上显示一份SSRS投资组合报告。该应用涉及的计算与本章Excel部分相同，而且大部分图也一样。因为对投资组合分解的查询在第7章中介绍过，所以本章不再包含那些图。在开始之前，一定要在本书其他项目中使用过的MathNet.Numerics中添加一个引用。本章假定你创建了一个新表（“Returns”），并填入了Access数据库中的月度P&L和起始AUM数据（按照表10-1模式）。注意Period列是主关键字。

表10-1：Returns表模式

字段名	数据类型
Period*	日期
MonthlyPNL	数字 (Double)
StartingAUM	数字 (Double)

在第9章创建的Visual Studio解决方案中，第一处修改是在ADS.xsd数据集中。除了增加Returns TableAdapter，还要增加三个额外的数据表，才能将计算结果传递给报告（RDLC）。三个数据表中的第一个是ReturnCalcs，类似于Returns Excel表，并存储来自彭博的计算后收益。第二个数据表是ReturnSummary，其中内容包括不同阶段的时间加权收益和标准差。最后一个数据表命名为Header，用于填入报告标题，内容包括投资组合的夏普比率、最后收益日期和最新市值。

1. 添加一个TableAdapter，命名为Returns，它选择Returns表的每一列。

2. 添加一个ReturnsCalc数据表，其中包括如下列：Date、MonthlyPNL、StartingAUM、MonthlyReturn、ValueOf100、MonthlyRF、ReturnRF、SP500、SPRF、Barclays和BarclaysRF。把Date作为主关键字，其数据类型设为DateTime。其他列数据类型都设为Double。

3. 添加一个ReturnSummary数据表，其中包括如下列：Range、Date、PortfolioReturn、SP500Return、BarclaysReturn、PortfolioStdDev、SP500StdDev和BarclaysStdDev。把Range作为主关键字，其数据类型设为String。把Date的数据类型设为DateTime。其他列数据类型都设为Double。

4. 添加一个Header数据表，其中包括如下列：Sharpe、LatestMV和Date。该数据表不需要主关键字。把Date的数据类型设为DateTime。其他列数据类型都设为Double。

结束后，数据集应当有一张额外的表，如图10-8所示。

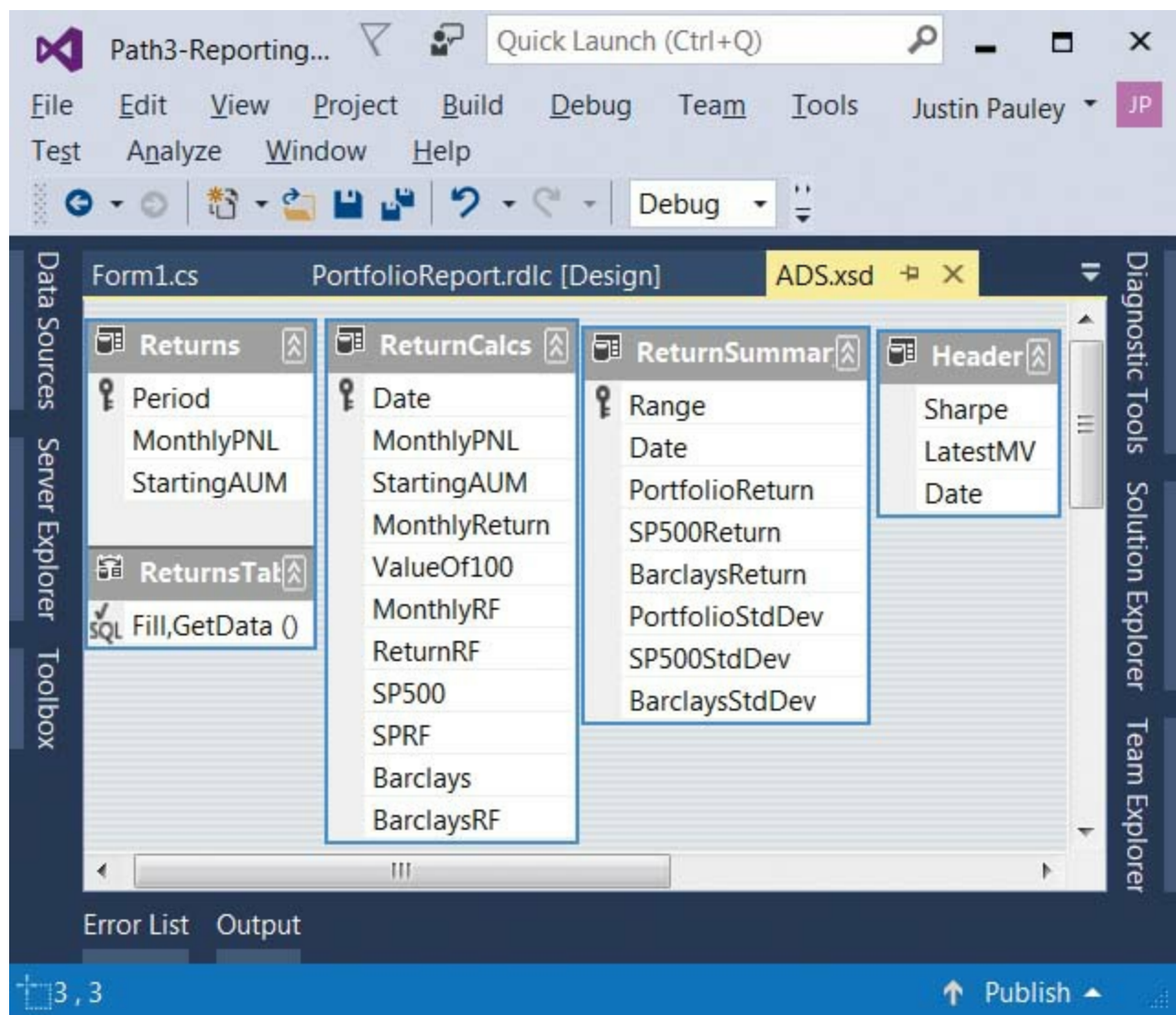


图10-8：ADS中新的数据表

下面的步骤在项目中添加一个新的SSRS报告，并在表中增加一个额外的ReportViewer：

- 5.在项目中添加一个新的SSRS报告，命名为PortfolioReport.rdlc。
- 6.在Report Data窗口，右击数据集，然后逐个添加下列数据集：Returns、Header、ReturnSummary。
- 7.在Windows Form (Form1.cs) Designer上，在panel2中添加一个选项卡控制器，将其dock设为fill。在第一个选项卡中输入初始ReportViewer控制器，然后在第二个选项卡中添加一个新的ReportView。
- 8.将新的ReportViewer (命名为rptPortfolio) 的性质设为第5步创建的

PortfolioReport。这样就为第1~4步创建的不同的表自动添加了BindingSources和TableAdapters。

9.添加一个新按钮，命名为btnPortfolio，并将标签设为Portfolio。

10.在Form1.cs中，删除自动添加到Form_Load事件中的rptPortfolio.RefreshReport()。

上面全部代码都是在btnPortfolio_Click事件中。接下来的代码开头是获取S&P 500、Barclays Agg和三个月期LIBOR等彭博历史数据。然后，它综合彭博历史数据和收益数据，填入ReturnCalcs数据表。ReturnCalcs数据表与Returns Excel表基本一致，计算也一样。

然后，代码开始填入ReturnSummary数据表的不同时间段和对应的日期。尽管需要两步才能循环ReturnSummary，但是我发现这样做比试图一步到位还要清楚明了。

只要ReturnSummary数据表有区域和日期，它就开始循环，引用ReturnCalcs数据表中给定日期的相关行，并计算不同时间段的收益，除了“ITD Annualized”。年化计算要留到下一步，因为它引用了其他时间段的计算，特别是ITD行。

最后，计算夏普比率，发送ADS数据集至投资组合报告：

```
private void btnPortfolio_Click(object sender, EventArgs e) {  
    // Populate the Returns table  
    this>ReturnsTableAdapter.Fill(this.ADS>Returns); // Find inception  
    date and latest return date DateTime inception =  
    this.ADS>Returns.OrderBy(x => x.Period).First().Period; DateTime  
    latest = this.ADS>Returns.OrderByDescending(x =>  
    x.Period).First().Period; // Get S&P 500, Barclays, and 3M Libor  
    history // The method returns daily prices, which is overkill // it  
    can be adjusted to return monthly. // The 31 days are subtracted  
    because we // need to calculate monthly return on the first month
```

```

Dictionary<DateTime, double> sp500 = GetHistory("SPX Index",
"PX_LAST", inception.AddDays(-31)); Dictionary<DateTime, double>
barclays = GetHistory("LBUSTRUU Index", "PX_LAST",
inception.AddDays(-31)); Dictionary<DateTime, double> libor =
GetHistory("US0003M Index", "PX_LAST", inception.AddDays(-31));
//Populate ReturnCalcs Table double valueof100 = 100; foreach
(ADS>ReturnsRow row in this.ADS>Returns.OrderBy(x => x.Period)) {
// Figure out the last day of the prior month // if you just
subtract 1 month, it will break when the month // is February since
it would return January 28. DateTime endOfPriorMonth = new
DateTime(row.Period.Year, row.Period.Month, 1).AddDays(-1);
ADS.ReturnCalcsRow calc = this.ADS.ReturnCalcs.NewReturnCalcsRow();
calc.Date = row.Period; calc.MonthlyPNL = row.MonthlyPNL;
calc.StartingAUM = row.StartingAUM; calc.MonthlyReturn =
(calc.MonthlyPNL / calc.StartingAUM); calc.ValueOf100 = valueof100
* (1 + calc.MonthlyReturn); valueof100 = calc.ValueOf100;
calc.MonthlyRF = libor[calc.Date] / 100 / 12; calc.ReturnRF =
calc.MonthlyReturn - calc.MonthlyRF; calc.SP500 = (sp500[calc.Date]
- sp500[endOfPriorMonth]) / sp500[endOfPriorMonth]; calc.SPRF =
calc.SP500 - calc.MonthlyRF; calc.Barclays = (barclays[calc.Date] -
barclays[endOfPriorMonth]) / barclays[endOfPriorMonth];
calc.BarclaysRF = calc.Barclays - calc.MonthlyRF;
this.ADS.ReturnCalcs.AddReturnCalcsRow(calc); } //Add Ranges to
ReturnsSummary List<string> ranges = new List<string>() { "Latest
Monthly", "QTD", "YTD", "Last 12 Months", "ITD", "ITD Annualized"
}; // This loops through and adds Ranges and their // corresponding
dates. foreach (string range in ranges) { ADS.ReturnSummaryRow srow
= this.ADS.ReturnSummary.NewReturnSummaryRow(); srow.Range = range;
switch(range) { case "Latest Monthly": srow.Date = latest; break;
case "QTD": // This gets the day before the start of the quarter //
based on the latest return. srow.Date = new DateTime(latest.Year, 3
* ((latest.Month + 2) / 3) - 2, 1). AddDays(-1); break; case "YTD":

```



```

srow.Date = new DateTime(latest.Year - 1, 12, 31); break; case
"Last 12 Months": srow.Date = latest.AddMonths(-12); break; case
"ITD": //returns the last day of the month prior to inception
srow.Date = new DateTime(inception.Year,inception.Month,
1).AddDays(-1); break; case "ITD Annualized": //returns the last
day of the month prior to inception srow.Date = new
DateTime(inception.Year,inception.Month, 1).AddDays(-1); break; }
this.ADS.ReturnSummary.AddReturnSummaryRow(srow); } //Iterate
through ReturnSummary and //Calculate Returns and StandardDev
foreach(ADS.ReturnSummaryRow srow in this.ADS.ReturnSummary) {
ADS.ReturnCalcsRow curr = this.ADS.ReturnCalcs.FindByDate(latest);
ADS.ReturnCalcsRow prev =
this.ADS.ReturnCalcs.FindByDate(srow.Date); if (srow.Range=="Latest
Monthly") { srow.PortfolioReturn = prev.MonthlyReturn;
srow.SP500Return = prev.SP500; srow.BarclaysReturn = prev.Barclays;
} else if (srow.Range== "ITD Annualized") { //ITD Annualized isn't
added here //because it is dependent on other rows continue; } else
{ //ITD always starts with a price of 100 for // ValueOf100
if(srow.Range=="ITD") srow.PortfolioReturn= (curr.ValueOf100 - 100)
/ 100; else srow.PortfolioReturn = (curr.ValueOf100 -
prev.ValueOf100) / prev.ValueOf100; srow.SP500Return =
(sp500[latest] - sp500[srow.Date]) / sp500[srow.Date];
srow.BarclaysReturn = (barclays[latest] - barclays[srow.Date]) /
barclays[srow.Date]; //Populate List Objects // with returns that
are after // the ReturnSummary starting date List<double>
monthlyReturns = new List<double>(); List<double> sp500Returns =
new List<double>(); List<double> barclaysReturns = new List<double>
(); foreach (ADS.ReturnCalcsRow calc in this.ADS.ReturnCalcs) { if
(calc.Date > srow.Date) { monthlyReturns.Add(calc.MonthlyReturn);
sp500Returns.Add(calc.SP500); barclaysReturns.Add(calc.Barclays); }
} srow.PortfolioStdDev =
Statistics.StandardDeviation(monthlyReturns) * Math.Sqrt(12);

```

```

srow.SP500StdDev = Statistics.StandardDeviation(sp500Returns) *
Math.Sqrt(12); srow.BarclaysStdDev =
Statistics.StandardDeviation(barclaysReturns) * Math.Sqrt(12); } }
ADS.ReturnSummaryRow ITDRow =
this.ADS.ReturnSummary.FindByRange("ITD"); // Once every other row
is populated, // populate the ITD Annualized row.
ADS.ReturnSummaryRow annualRow =
this.ADS.ReturnSummary.FindByRange("ITD Annualized"); double
daycount = (latest - new DateTime(inception.Year, inception.Month,
1). AddDays(-1)).TotalDays; annualRow.PortfolioReturn = Math.Pow(1
+ this.ADS.ReturnSummary.FindByRange("ITD").PortfolioReturn, (365 /
daycount)) - 1; annualRow.SP500Return = Math.Pow(1 +
this.ADS.ReturnSummary.FindByRange("ITD").SP500Return, (365 /
daycount)) - 1; annualRow.BarclaysReturn = Math.Pow(1 +
this.ADS.ReturnSummary.FindByRange("ITD").BarclaysReturn, (365 /
daycount)) - 1; //Using the same StdDev as the non-annualized rows.
annualRow.PortfolioStdDev = ITDRow.PortfolioStdDev;
annualRow.SP500StdDev = ITDRow.SP500StdDev;
annualRow.BarclaysStdDev = ITDRow.BarclaysStdDev; //Fields needed
for report header. ADS.HeaderRow header =
this.ADS.Header.NewHeaderRow(); header.Sharpe =
this.ADS.ReturnCalcs.Average(x => x.ReturnRF) *
12/ITDRow.PortfolioStdDev; header.LatestMV =
this.ADS>Returns.OrderByDescending(x =>
x.Period).First().StartingAUM; header.Date = latest;
this.ADS.Header.AddHeaderRow(header);
this.rptPortfolio.RefreshReport(); }

```

在投资组合报告中，Header数据集用于填入上部的文本框，例如Portfolio MV：

```

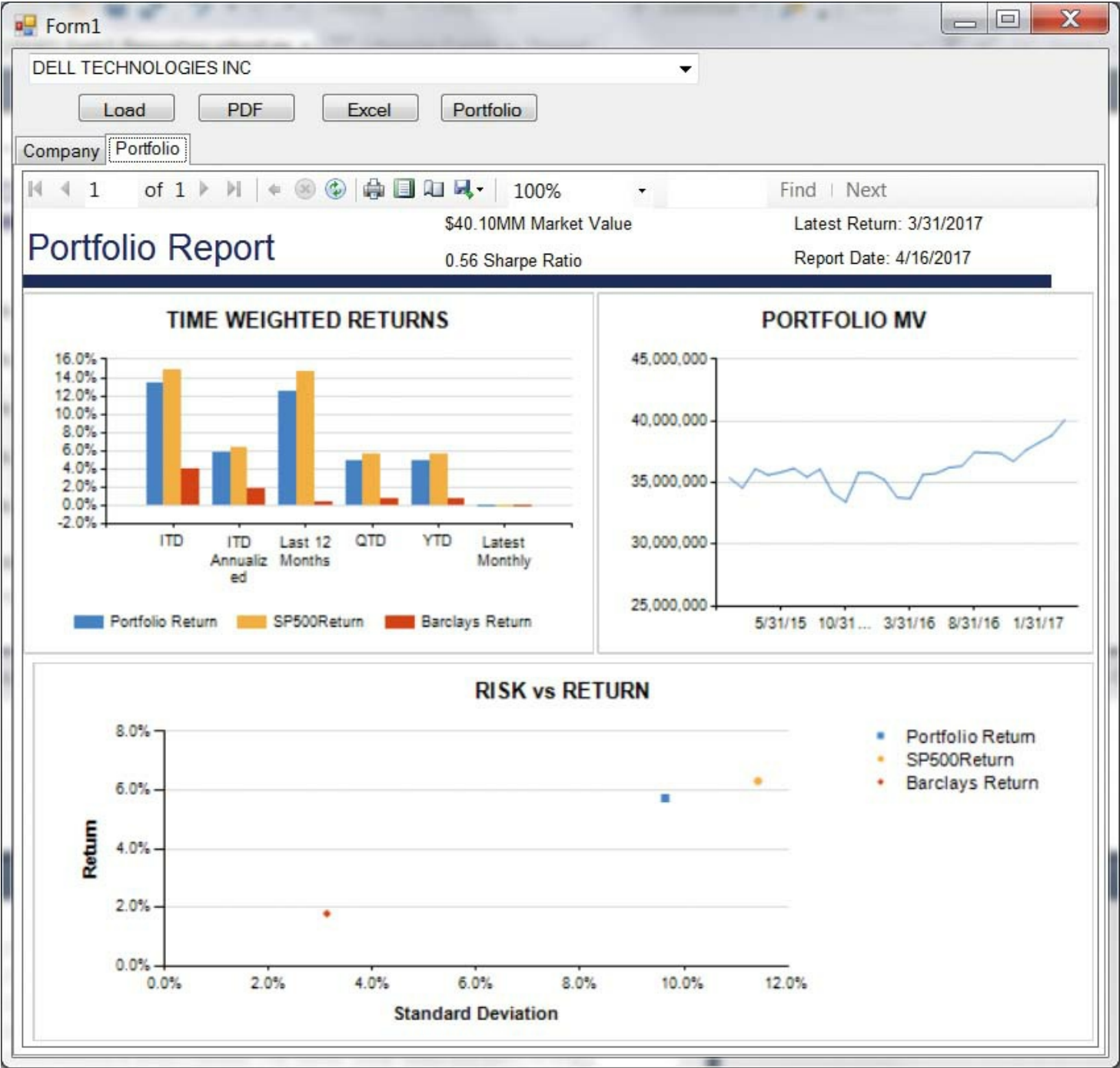
=" $"&Format(Sum(Fields!LatestMV.Value/1000000,
"Header"),"#,##0.00") & "MM Market Value"

```

ReturnSummary数据集用于填入一个带有时间加权收益的柱状图。Returns数据集用于在右上角填入一个带投资组合市值历史的线形图。

最后，再次使用ReturnSummary数据集，在底部填入一个比较年化收益和标准差的散点图。图中有一个过滤，其中只包括时间段等于ITD Annualized的行。尽管该散点图不包括第7章中计算出的、投资组合的当前标准差或者收益预测，但你仍可以用第7章的代码扩充ReturnSummary数据表。

结束后，结果如图10-9所示。



10.4 本章小结

你可以使用本章中涉及的计算和图表测算投资组合的业绩、增长和相对价值。本章还解释了考虑复利的重要性。尽管Excel和数据库表格对于分析信息是有用的，但把数字转换成图表使分析趋势和找出问题变得更加容易。

第11章 结论

本书带你学习了如何在彭博和Markit中发现并检索金融信息，分析数据，并产生定制报告。我们要强调的是，你需要在每一步有自己的思考和观点，这样才能创建真正有价值 and 独一无二的东西。本书真正的目标不是让你简单重复本书中讲过的电子表格程序、数据库或者代码，而是通过实例教你如何建立自己的分析和报告平台。毫无疑问，你们中的不少人会产生疑问，为什么书中缺少一些对于你来说很关键的域、计算或者报告内容。我希望，无论你的目标是什么，本书能够提供到达目标的路径框架。

下一步，建议你开始摘星揽月。我本人发现比较实用的办法是先把理想的电子表格程序、报告或应用勾勒出一个轮廓来，然后再一点一点地搭建。首先，确定所需的所有不同数据，找到最佳数据源。记住，有时最佳数据源或者唯一的数据源就是你自己。请确保你真的理解数据和数据之间的细微差别（准确度、来源可靠性等）。记住第2章中的第三范式，一定要注意正确保存表格数据。记住，一定要把数据放入特定的背景和语境中审视，否则就是毫无意义的。一定一定要花点时间确保一切都是有意义的、说得通的；有太多人过于相信自己的计算，而没有检查核实计算结果。最后，祝你有创造力，也能享受做报表的过程。

谢谢阅读，留言请发送至Justin.Pauley@gmail.com。

附录A 表格参考

为便于读者参考，这里汇总了书中涉及的表格。

表A-1：定价源摘要（表3-1）

定价源	描述
BGN	基于来自多个消息渠道的实时复合报价
BVAL	结合了直接市场观察和定量定价模型
MSG1	彭博从你的收件箱挖掘价格
TRAC	TRACE 是美国金融业监管局（FINRA）提供的企业和中介机构债券价格发布服务。获取实时债券价格信息需要许可并缴费，否则只能在四天之后查看价格信息

表A-2：BDS的Excel选项（见3.2.2节）

参数	描述
DIRECTION	指明返回值的显示是水平的还是垂直的
SORTASC/SORTDESC	控制返回数据的顺序
HEADEARS	显示标题
STARTROW, ENDROW, STARTCOL 和 ENDCOL	限制返回的数据
AGGREGATE/SEPARATOR	AGGREGATE 参数将 BDS 结果合并至一个单一的单元格；SEPARATOR 自定义设置数据的合并方式
PCS	改变定价来源
ARRAY	可以用于 Excel Aggregate 函数的返回值

表A-3：彭博日期类型（表3-2）

类型	每日	每周	每月	每季度	每半年	每年
Fiscal	—	—	—	FQ	FS	FY
Calendar	CD	CW	CM	CQ	CS	CY
Actual	AD	AW	AM	AQ	AS	AY

表A-4：BDH的Excel选项（见3.2.3节）

参数	描述
Fill	设置 Previous、Error、Blank、NA、PNA（前面的日期）、F（下一个可用日期）或自定义文本来指明缺失的日期
PERIOD	以一定频率而不是每日显示数据
CAPCHG	改变彭博对公司剥离、股票分拆 / 合并、股票分红和附权发行的方式
FX	调整返回值的货币
POINTS	控制返回天数的最大值
SORT	设置为 D 代表降序 (descending)
DIRECTION	指明返回值的显示是水平的还是垂直的
ARRAY	返回 Excel Aggregate 函数返回的值
PCS	改变定价源
DATES	设置为 HIDE 将仅返回数值
QUOTETYPE	当 PC_LAST 返回收益时，设置为 “P” 代表价格

表A-5：Bond工作表列和彭博映射（表3-3）

列	第一行输入	第二行输入	描述
A	[Intentionally blank]	BondID	
B	[Intentionally blank]	BBID	
C	BOND_TO_EQY_TICKER	CompanyID	债券发行人的股票代码
D	SECURITY_DES	Security Description	有价证券的彭博描述
E	CPN_TYP	Coupon Type	利息类型（固定利息、浮动利息）
F	CPN	Fixed Coupon	当前利率
G	MATURITY	Maturity	债券到期日期
H	RTG_MOODY	Moody's Rating	穆迪评级
I	RTG_SP	S&P Rating	标准普尔评级
J	QUOTED_CRNCY	Currency	报价货币
K	PAYMENT_RANK	Rank	Payment 评级（特别不安全、安全等）
L	PX_LAST	Price	最终价格
M	CHG_PCT_YTD	YTD Px Chg	今年迄今价格变化（百分比）
N	CHG_PCT_3M	3M Px Chg	三个月价格变化（百分比）
O	YAS_YLD_SPREAD	YAS Spread	当前价格，债券收益和默认基准收益之间的基准点 (bps) 差。默认设置由 YASD<GO> 屏幕控制
P	YAS_BOND_YLD	YAS Yield	在当前价格的债券收益。由 YASD<GO> 屏幕控制默认设置

表A-5：Bond工作表列和彭博映射（表3-3）（续）

列	第一行输入	第二行输入	描述
Q	CALLABLE	Callable?	债券能否被回购 / 赎回
R	NXT_CALL_DT	Next Call Date	债券的下一个提前赎回日
S	NXT_CALL_PX	Next Call Price	下一个提前赎回日的债券赎回价
T	[Intentionally blank]	Bond Comments	

表A-6：Loan工作表列和彭博映射（表3-4）

第一行输入	第二行输入	描述
[Intentionally blank]	LoanID	包含每笔贷款的唯一标识符
[Intentionally blank]	BBID	包括完整彭博标识符
ISSUER_PARENT_EQY_TICKER	CompanyID	贷款发行人的股票代码
SECURITY_DES	Security Description	有价证券的彭博描述
LN_CURRENT_MARGIN	Margin	票面上高于基准的利差或价差
INDEX_FLOOR	Floor	指数下限（如适用）
RESET_IDX	Index	票面利息的基准指数
MATURITY	Maturity	到期日
RTG_MOODY	Moody's Rating	穆迪评级
RTG_SP	S&P Rating	标准普尔评级
PX_LAST	Price	最终价格
CHG_PCT_YTD	YTD Px Chg	今年迄今价格变化（百分比）
DISC_MRGN_ASK	DM	基于当前卖出价的贴现差额 (DM)
YLD_YTM_ASK	Yield	基于卖出价的到期收益率
CALLABLE	Callable?	债券能否被回购 / 赎回
NXT_CALL_DT	Next Call Date	发行人可以赎回债券的下一个日期
NXT_CALL_PX	Next Call Price	在下一个可赎回债券日期的债券价格
[Intentionally blank]	贷款备注	

表A-7：IDX工作表列和彭博映射（表3-5）

第一行输入	第二行输入	描述
[Intentionally blank]	IndexID	包含每笔贷款的唯一标识符
[Intentionally blank]	BBID	包括完整彭博标识符
NAME	Name	指数名称
PX_LAST	Price	最终价格
CHG_PCT_HIGH_52WEEK	52 Week High	过去 52 周内当前价格和最高价格的百分比变化

表A-7：IDX工作表列和彭博映射（表3-5）（续）

第一行输入	第二行输入	描述
CHG_PCT_LOW_52WEEK	52 Week Low	过去 52 周内当前价格和最低价格的百分比变化
CHG_PCT_YTD	YTD Px Change	今年迄今价格百分比变化
CHG_PCT_3M	3M Px Change	过去三个月价格百分比变化
CURRENT_TRR_1YR	12M Total Return	一年总收益；利息进行再投资

表A-8：Company工作表列和彭博映射（表3-6）

第二行输入	第三行输入	描述
[Intentionally blank]	CompanyID	包括每个公司的唯一标识符
[Intentionally blank]	BBID	包括完整的彭博标识符
NAME	CompanyName	公司名称
COMPANY_IS_PRIVATE	Private	表示公司是非上市企业
GICS_SECTOR_NAME	Sector	Global GICS 部门分类
GICS_INDUSTRY_NAME	Industry	Global GICS 行业分类
GICS_SUB_INDUSTRY_NAME	Sub-Industry	GICS 子行业分类
RTG_SP_LT_LC_ISSUER_CREDIT	S&P Rating	标准普尔长期债务发行人评级
RTG_MDY_LT_CORP_FAMILY	Moody's Rating	穆迪长期企业家族评级
CRNCY_ADJ_MKT_CAP*	Market Cap	经过货币调整的市值
SHORT_AND_LONG_TERM_DEBT*	Total Debt	短期和长期债之和 (百万美元计)
NET_DEBT*	Net Debt	公司的净债务
CRNCY_ADJ_CURR_EV*	Enterprise Value	经过货币调整的企业价值
TRAIL_12M_EBITDA*	TTM EBITDA	过去十二个月 EBITDA
PE_RATIO	PE Ratio	市盈率, 即股票价格和公司每股收益比
EQY_DVD_YLD_IND	Dividend Gross Yield	最近宣布的年化毛息除以现价
CURRENT_TRR_1YR	12M Total Return	一年总收益。股息重新再投资
CHG_PCT_YTD	YTD Px Change	今年迄今百分比价格变化
CHG_PCT_3M	3M Px Change	过去三个月百分比价格变化
SALES_REV_TURN*	Total Revenue	公司的营业收入总额减去对销售总额的各种调整
TOT_DEBT_TO_EBITDA	Total Debt/EBITDA	总债务除以过去十二个月 EBITDA
INTEREST_COVERAGE_RATIO	Interest Coverage	息税前利润 (EBIT) 除以总利息
TRAIL_12M_FREE_CASH_FLOW*	FCF	过去十二个月净现金流
FCF_TO_TOTAL_DEBT	FCF/Total Debt	过去十二个月净现金流除以债务总额
CRNCY_ADJ_PX_LAST*	Price	经过货币调整的最终价格

表A-8：Company工作表列和彭博映射（表3-6）（续）

第二行输入	第三行输入	描述
CHG_PCT_HIGH_52WEEK	52 Week High Change	当前价格和过去 52 周中最高价格之间的百分比差
CHG_PCT_LOW_52WEEK	52 Week Low Change	当前价格和过去 52 周中最低价格之间的百分比差
CDS_SPREAD_TICKER_5Y	5yr CDS Spread Ticker	五年期信用违约互换票面价格 价差的彭博名称缩写
TOT_BUY_REC	Buy Recommendations	提出买入建议的研究分析师总数
TOT_SELL_REC	Sell Recommendations	提出卖出建议的研究分析师总数
TOT_HOLD_REC	Hold Recommendations	提出持有建议的研究分析师总数
[Intentionally blank]	Net Debt/EBITDA	
[Intentionally blank]	5yr CDS Spread	
[Intentionally blank]	Category	

表A-9：Company表设计（表3-7）

字段名	数据类型
CompanyID*	短文本
BBID	短文本
CompanyName	短文本
IsPrivate	短文本
Sector	短文本
Industry	短文本
SubIndustry	短文本
SPRating	短文本
MoodyRating	短文本
MarketCap	数字
TotalDebt	数字
NetDebt	数字
EV	数字
EBITDA	数字
TotalRevenue	数字
TotalDebtToEBITDA	数字
InterestCoverage	数字
FCF	数字
FCFToTotalDebt	数字
Price	数字

表A-9：Company表设计（表3-7）（续）

字段名	数据类型
YrHi	数字
YrLow	数字
CDS5YrTicker	短文本
NetDebtToEBITDA	数字
CDSSpread5Yr	数字
Category	短文本
CompanyComments	长文本
PERatio	数字
DVDYield	数字
TotalReturn12M	数字
PxChgYTD	数字
PxChg3M	数字
RecBuy	数字
RecSell	数字
RecHold	数字

表A-10：Bond表设计（表3-8）

字段名	数据类型
BondID*	短文本
BBID	短文本
CompanyID	短文本
SecurityDes	短文本
CpnType	短文本
FixedCpn	数字
Maturity	日期 / 时间
MoodyRating	短文本
SPRating	短文本
Currency	短文本
Rank	短文本
Price	数字
PxChgYTD	数字
PxChg3M	数字
YASSpread	数字
YASYield	数字

表A-10：Bond表设计（表3-8）（续）

字段名	数据类型
IsCallable	短文本
NextCallDate	日期 / 时间
NextCallPrice	数字
BondComments	长文本

表A-11：Loan表设计（表3-9）

字段名	数据类型
LoanID*	短文本
BBID	短文本
CompanyID	短文本
SecurityDesc	短文本
CpnType	短文本
Margin	数字
Floor	数字
Index	短文本
Maturity	日期 / 时间
MoodyRating	短文本
SPRating	短文本
Currency	短文本
Rank	短文本
Price	数字
PxChgYTD	数字
PxChg3M	数字
DM	数字
Yield	数字
IsCallable	短文本
NextCallDate	日期 / 时间
NextCallPrice	数字
LoanComments	长文本

表A-12：Index表设计（表3-10）

字段名	数据类型
IndexID*	短文本
BBID	短文本

表A-12：Index表设计（表3-10）（续）

字段名	数据类型
IndexName	短文本
Price	数字
YrHi	数字
YrLow	数字
PxChgYTD	数字
PxChg3M	数字
TotalReturn12M	数字

表A-13：Map表设计（表3-11）

字段名	数据类型
DestTable*	短文本
DestCol*	短文本
BloombergFLD	短文本

表A-14：Map表数据（表3-12）

DestTable	DestCol	BloombergFLD
Company	CompanyName	NAME
Company	IsPrivate	COMPANY_IS_PRIVATE
Company	Sector	GICS_SECTOR_NAME
Company	Industry	GICS_INDUSTRY_NAME
Company	SubIndustry	GICS_SUB_INDUSTRY_NAME
Company	SPRating	RTG_SP_LT_LC_ISSUER_CREDIT
Company	MoodyRating	RTG_MDY_LT_CORP_FAMILY
Company	MarketCap	CRNCY_ADJ_MKT_CAP
Company	TotalDebt	SHORT_AND_LONG_TERM_DEBT
Company	NetDebt	NET_DEBT
Company	EV	CRNCY_ADJ_CURR_EV
Company	EBITDA	TRAIL_12M_EBITDA
Company	TotalRevenue	SALES_REV_TURN
Company	TotalDebtToEBITDA	TOT_DEBT_TO_EBITDA
Company	InterestCoverage	INTEREST_COVERAGE_RATIO
Company	FCF	TRAIL_12M_FREE_CASH_FLOW
Company	FCFToTotalDebt	FCF_TO_TOTAL_DEBT
Company	Price	CRNCY_ADJ_PX_LAST

表A-14：Map表数据（表3-12）（续）

DestTable	DestCol	BloombergFLD
Company	YrHi	CHG_PCT_HIGH_52WEEK
Company	YrLow	CHG_PCT_LOW_52WEEK
Company	CDS5YrTicker	CDS_SPREAD_TICKER_5Y
Company	PERatio	PE_RATIO
Company	DVDYield	EQY_DVD_YLD_IND
Company	TotalReturn12M	CURRENT_TRR_1YR
Company	PxChgYTD	CHG_PCT_YTD
Company	PxChg3M	CHG_PCT_3M
Company	RecBuy	TOT_BUY_REC
Company	RecSell	TOT_SELL_REC
Company	RecHold	TOT_HOLD_REC
Bond	CompanyID	BOND_TO_EQY_TICKER
Bond	SecurityDes	SECURITY_DES
Bond	CpnType	CPN_TYP
Bond	FixedCpn	CPN
Bond	Maturity	MATURITY
Bond	MoodyRating	RTG_MOODY
Bond	SPRating	RTG_SP
Bond	Currency	QUOTED_CRNCY
Bond	Rank	PAYMENT_RANK
Bond	Price	PX_LAST
Bond	PxChgYTD	CHG_PCT_YTD
Bond	PxChg3M	CHG_PCT_3M
Bond	YASSpread	YAS_YLD_SPREAD
Bond	YASYield	YAS_BOND_YLD
Bond	IsCallable	CALLABLE
Bond	NextCallDate	NXT_CALL_DT
Bond	NextCallPrice	NXT_CALL_PX
Loan	CompanyID	ISSUER_PARENT_EQY_TICKER
Loan	SecurityDesc	SECURITY_DES
Loan	CpnType	CPN_TYP
Loan	Margin	LN_CURRENT_MARGIN
Loan	Floor	INDEX_FLOOR
Loan	Index	RESET_IDX

表A-14：Map表数据（表3-12）（续）

DestTable	DestCol	BloombergFLD
Loan	Maturity	MATURITY
Loan	MoodyRating	RTG_MOODY
Loan	SPRating	RTG_SP
Loan	Currency	QUOTED_CRNCY
Loan	Rank	PAYMENT_RANK
Loan	Price	PX_LAST
Loan	PxChgYTD	CHG_PCT_YTD
Loan	PxChg3M	CHG_PCT_3M
Loan	DM	DISC_MRGN_ASK
Loan	Yield	YLD_YTM_ASK
Loan	IsCallable	CALLABLE
Loan	NextCallDate	NXT_CALL_DT
Loan	NextCallPrice	NXT_CALL_PX
Index	IndexName	NAME
Index	Price	PX_LAST
Index	YrHi	CHG_PCT_HIGH_52WEEK
Index	YrLow	CHG_PCT_LOW_52WEEK
Index	PxChgYTD	CHG_PCT_YTD
Index	PxChg3M	CHG_PCT_3M
Index	TotalReturn12M	CURRENT_TRR_1YR

表A-15：历史请求的API实参（表3-13）

实参	BDH 对等	备注
startDate	—	起始日期 (YYYYMMDD 格式)
endDate	—	结束日期 (YYYYMMDD 格式)
periodicityAdjustment	—	ACTUAL、CALENDAR 或 FISCAL
periodicitySelection	—	DAILY, WEEKLY, MONTHLY, QUARTERLY, SEMI_ANNUALLY, YEARLY
currency	FX	三字节 ISO 代码；例如 USD, GBP
overrideOption	Quote	设为 OVERRIDE_OPTION_GPA 以便在报价计算中使用平均价格，而非收盘价格
pricingOption	QuoteType	PRICING_OPTION_PRICE (对于价格), PRICING_OPTION_YIELD (对于收益)
nonTradingDayFillOption	Days	NON_TRADING_WEEKDAYS, ALL_CALENDAR_DAYS, ACTIVE_DAYS_ONLY

表A-15：历史请求的API实参（表3-13）（续）

实参	BDH 对等	备注
nonTradingDayFillMethod	Fill	PREVIOUS_VALUE 或 NIL_VALUE (对于空白)
adjustmentNormal	CshAdjNormal	设为 true 或 false
adjustmentAbnormal	CshAdjAbnormal	设为 true 或 false
adjustmentSplit	CapChg	设为 true 或 false
adjustmentFollowDPDF	UseDPDF	设为 true 或 false
calendarCodeOverride	CDR	例如 “US” 或 “JN”

因为Markit与标准普尔有合作关系，表A-16中带星号（*）的列只能由也与标准普尔有协议的客户获取。此外，带双星号（**）的列只能通过特定请求从Markit（你可以联系support@markit.com）获得，才会出现在结果中。带三个星号（***）的列需要与标准普尔有协议，且向Markit发送特定请求。

表A-16：Facility update列（表4-1）

列	说明
LoanX ID	每笔贷款的唯一标识符
PMD ID	与某特定发行人 / 分级组合相关的唯一标识符。可以是正数、也可以是负数
PMD Trans ID*	PMD/LCD 确认一笔交易或贷款计划的唯一标识符
Issuer Name	借款人或发行人的名称
Issuer ID*	与某特定发行人相关的唯一标识符
Deal Name	借款人的名称。通常与前面的单元格一样，但是可以包含交易日期和类型
Facility Type	特定贷款类型；TLB，过桥贷款等
LoanX Facility Type	Markit 将 PMD Facility Type 合并成当前 16 个标准值中的一个
Facility Status*	特定工具类型：例如过桥贷款，364 天贷款，次级贷款，定期贷款折旧等
LoanX Facility Type Code*	LoanX Facility Type + LoanX Facility Category 的代码标志
LoanX Facility Category*	Markit 将 PMD Facility Type 简化成如下之一：Institutional, RC, TLA, Other
Industry	基于 SIC code 的行业划分
Initial Amount	机构金额（百万计）
Initial Spread	最终初始 LIBOR 价差
Maturity Date	到期日期
Ticker***	发行人的名称缩写
Currency***	贷款使用的货币
LoanX Currency Code*	标准货币缩写代码
SP Org ID***	标准普尔指定组织 ID
Commitment Fee*	承诺费

表A-16：Facility update列（表4-1）（续）

列	说明
Sponsor*	贷款担保人
LoanX Sponsor Code*	贷款担保人数字代码
Launch Date*	贷款的发行日期
Close Date*	贷款的截止日期
State*	发行人的状态
Country*	发行人的国家
LoanX Country Code*	标准国家代码缩写
Pro Rata Assignment*	按比例分配最低额
Institutional Assignment*	按机构分配最低额
Pro Rata Fee*	按比例收费
Institutional Fee*	机构费用
Facility Fee*	为一笔贷款支付的年费总额
Consent*	代理、公司、二者
Security*	贷款担保资产
LoanX Security Code*	贷款担保资产标准代码缩写
Lead Agent*	牵头代理机构
LoanX Lead Agent Code*	与牵头代理机构相关的唯一标识符
Admin Agent*	行政代理机构
LoanX Admin Agent Code*	与行政代理机构相关的唯一标识符
Document Agent*	文件代理机构
LoanX Doc Agent Code*	与文件代理机构相关的唯一标识符
Syndicate Agent*	银团代理机构
LoanX Synd Agent Code*	与银团代理机构相关的唯一标识符
Initial SP Rating*	初始标准普尔评级
Industry Code*	行业代码
SIC Code*	SIC 代码
SIC Description*	SIC 说明
Industry Segment ID*	行业分类 ID
Industry Segment Description*	行业分类说明
Status Code*	内部状态代码
Status	内部状态说明
Cancelled*	表示交易已被取消的标志
Created Time	创建贷款纪录的日期
Modified Time	修改贷款纪录的日期

表A-16：Facility update列（表4-1）（续）

列	说明
Term*	贷款期限（年）
RC Term*	RC 贷款期限（年）
TLA Term*	TLA 贷款期限（年）
TLB Term*	TLB 贷款期限（年）
TLD Term*	TLD 贷款期限（年）
OID*	受保险的贷款初始发行价格
Libor Floor*	低于特定地板价时支付的最低基准利率
Lien Type***	在借方资本结构下的留置权优先类型
Cov-Lite***	表示分级为不含借方保护条款的高风险贷款的标志

表A-17：推荐更新列（表4-2）

列	描述
LoanX ID	每笔贷款的唯一标识符
LCD ID	S&P LCD 数据的标识符
Issuer Name	借款人或发行人的名称
Dealname	借款人的名称。通常与前面的单元格一样，但是可能包括日期和交易类型
Facility Type	特定贷款类型；TLB，过桥贷款等
Industry	基于 SIC 的行业分类
Initial Amount	机构金额（百万计）
Final Maturity	最终到期日日期
Initial Spread	初始 LIBOR 价差
Facility Status	状态处于支付阶段 / 不处于支付阶段 (A 或 I)
Inactive Date	状态改变日期，总是代表过去
Inactive Reason	状态改变的原因
Replacement LoanX ID	重置贷款的 LoanX ID
Replacement PMD ID	与一个特定发行人 / 分级组合相关的重置唯一标识符。可以是正数，也可以是负数
Replacement Issuer Name	重置借款人或发行人的名称
Replacement Deal Name	重置借款人的名称；通常与上一个单元格相同，但是可以包括日期和交易类型
Replacement Facility Type	重置贷款特定类型；TLB，过桥贷款等
Replacement Industry	基于 SIC 代码的重置行业分类
Replacement Initial Amount	重置贷款机构金额（百万计）
Replacement Final Maturity	重置贷款最后到期日期
Replacement Initial Spread	重置贷款最初 LIBOR 息差
Replacement Status	重置贷款机构状态

表A-18：每日评级列（表4-3）

列	描述
As of Date	创建文件的日期
LoanX ID	每笔贷款的唯一标识符
Price Date	买卖报价日期
Moody's Rating	穆迪评级
Moody's Rating Date	穆迪评级上次更新评级的日期
Moody's Watch	来自穆迪评级的监视名单描述
Moody's Watch Date	上次更新监视名单的日期
Moody's Outlook	穆迪评级展望
Moody's Outlook Date	穆迪评级展望最后一次更新日期
S&P Rating	标准普尔评级
S&P Rating Date	标准普尔评级最后一次更新日期
S&P Watch	来自标准普尔评级的监视名单描述
S&P Watch Date	上次更新监视名单的日期
S&P Outlook	标准普尔评级展望
S&P Outlook Date	标准普尔评级展望最后一次更新日期

表A-19：LoanID Updates列（表4-4）

列	描述
Identifier	一个与某特定发行人 / 分级组合相关的行业标准唯一标识符（通常存储 CUSIP）
Identifier Type	标明标识符来源（例如“CUSIP”）
LoanX ID	每笔贷款的唯一标识符
Valid From	标识符被映射到 LoanX ID 的日期
Valid To	标识符被取消到 LoanX ID 的映射关系的日期
Modified Time	修改标识符映射的日期

表A-20：Marks列描述（表4-5）

列	描述
LoanXID	每笔贷款的唯一标识符
Mark Date	标注日期
Evaluated Price	收盘卖价 / 收盘买价之间的中间点
Bid	在标注日期对机构的平均卖价。根据交易日不同而变化
Offer	在标注日期对机构的平均买价。根据交易日不同而变化
Depth	Depth 一般表示参与的交易者的数量
Close Bid	东部时间下午 4 时的收盘卖价。不会发生改变

表A-20：Marks 列描述（表4-5）（续）

列	描述
Close Offer	东部时间下午 4 时的收盘买价。不会发生改变
Close Date	收盘卖价和买价的日期
Contributed	如果公司参与平均记分，则返回 “Yes”

表A-21：Financial Statement列（表4-6）

列	描述
SP_COMPANY_ID	标准普尔 Capital IQ 标识符
Currency	财务数据货币
Year	财务报表年份
Quarter	财务报表季度
Is_Annual	表示为年度数据
Is_Latest	表示为最新可获得信息
Total_Sr_Secured_EBITDA	有担保债务总额 /EBITDA
Sr_Debt_EBITDA	优先债务 /EBITDA
Sr_Sub_Debt_EBITDA	高级次级债务 /EBITDA
Jr_Sub_Debt_EBITDA	初级次级债务 /EBITDA
Sub_Debt_EBITDA	次级债务 /EBITDA
Total_Debt_EBITDA	总债务 /EBITDA
Net_Debt_EBITDA	(总债务－现金和短期投资)/EBITDA
Total_Assets	总资产
Revenue	收益
EBITDA	税息折旧及摊销前利润
Retained_Earnings	留存收益
EBITDA_INT	EBITDA/ 利息花费
Quick_Ratio	(总现金和短期投资 + 应收账款)/ 当前资产总额
Current_Ratio	当前资产总额 / 当前负债总额
Total_Debt_Capital	债务总额 / 资本总额
Total_Debt_Equity	债务总额 / 权益总额

表A-22：Markit表的主关键字（表4-7）

表名	主关键字
LoanIDUpdates	Identifier, Modified Time
LoanPricingAndAnalytics	PricingAsOf, LoanX ID
LoansDailyRatings	Date, LoanX ID

表A-22：Markit表的主关键字（表4-7）（续）

表名	主关键字
LoanXFacilityUpdates	LoanX ID, Modified Time
LoanXMarks	LoanX ID, Mark Date
LoanXRecUpdates	LoanX ID, Inactivation Date
FinancialStatement	SP_COMPANY_ID, Year, Quarter
FinancialStatementMap	LXID

表A-24：穆迪评级因子（来源：穆迪投资服务公司；表6-1）

字段名	数据类型
PositionID*	数字
Type	短文本
SecurityID	短文本
Size	数字
PurchasePx	数字
PurchaseDate	日期 / 时间
Position Comments	短文本

表A-24：穆迪评级因子（来源：穆迪投资服务公司；表6-1）（续）

评级	评级因子
Aaa	1
Aa1	10
Aa2	20
Aa3	40
A1	70
A2	120
A3	180
Baa1	260
Baa2	360
Baa3	610
Ba1	940
Ba2	1350
Ba3	1766
B1	2220
B2	2720
B3	3490
Caa1	4770

表A-24：穆迪评级因子(来源：穆迪投资服务公司；表6-1) (续)

评级	评级因子
Caa2	6500
Caa3	8070
Ca	10 000
C	10 000

表A-25：Comp 表模式 (表6-2)

字段名	数据类型
HistDate	日期 / 时间 (设为主关键字)
X	数字 (字段规模设为 Double)
Y	数字 (字段规模设为 Double)

表A-26：RF表模式（表6-3）

字段名	数据类型
Rating(主关键字)	短文本
Factor	数字

表A-27：MedianBondStats表模式（表6-4）

字段名	数据类型
PeerGroup(主关键字)	短文本
Count	数字
PxChgYTD	数字（字段规模设为 Double）
PxChg3M	数字（字段规模设为 Double）
YASSpread	数字（字段规模设为 Double）
YASYield	数字（字段规模设为 Double）
FixedCpn	数字（字段规模设为 Double）
MonthsUntilMaturity	数字（字段规模设为 Double）
RF	数字

表A-28：MedianLoanStats表模式（表6-5）

字段名	数据类型
PeerGroup(主关键字)	短文本
Count	数字
PxChgYTD	数字（字段规模为 Double）
PxChg3M	数字（字段规模为 Double）
DM	数字（字段规模为 Double）

表A-28：MedianLoanStats表模式（表6-5）（续）

字段名	数据类型
Yield	数字 (字段规模为 Double)
Margin	数字 (字段规模为 Double)
MonthsUntilMaturity	数字 (字段规模为 Double)
RF	数字

表A-29：MedianCompanyStats表模式 (表6-6)

字段名	数据类型
Category(主关键字)	短文本
Count	数字
MarketCap	数字 (字段规模为 Double)
TotalDebt	数字 (字段规模为 Double)
TotalDebtToEBITDA	数字 (字段规模为 Double)
NetDebtToEBITDA	数字 (字段规模为 Double)
FCFToTotalDebt	数字 (字段规模为 Double)
YrHi	数字 (字段规模为 Double)
YrLow	数字 (字段规模为 Double)
PxChgYTD	数字 (字段规模为 Double)
PxChg3M	数字 (字段规模为 Double)
TotalReturn12M	数字 (字段规模为 Double)

表A-30：BondZWeights模式 (表6-7)

字段名	数据类型
PxChgYTD	数字 (字段规模为 Double)
PxChg3M	数字 (字段规模为 Double)
YASSpread	数字 (字段规模为 Double)
YASYield	数字 (字段规模为 Double)
FixedCpn	数字 (字段规模为 Double)
MonthsUntilMaturity	数字 (字段规模为 Double)
RF	数字 (字段规模为 Double)

表A-31：投资组合情景模式 (表7-1)

字段名	数据类型
PositionID(主关键字)	数字
BestReturn	数字 (Double)
AverageReturn	数字 (Double)

表A-31：投资组合情景模式（表7-1）（续）

字段名	数据类型
WorstReturn	数字 (Double)
BestProbability	数字 (Double)
AverageProbability	数字 (Double)
WorstProbability	数字 (Double)

表A-32：Returns表模式（表10-1）

字段名	数据类型
Period*	日期
MonthlyPNL	数字 (Double)
StartingAUM	数字 (Double)